

DATA WAREHOUSING

- Data warehouse refers to a data repository that is maintained separately from an organization's operational databases
- The major task of online operational database systems is to perform online transaction and query processing
- They cover most of the day-to-day operations of an organization such as purchasing, inventory, manufacturing, banking, payroll, registration and accounting
- Data warehouse systems serve users in the role of data analysis and decision making
- They can organize and present data in various formats in order to accommodate the diverse needs of different users

Property	Database	Data Warehouse
Purpose	most useful for the small, atomic transactions	best suited for larger questions that require a higher level of analysis
Processing Method	Online Transaction Processing (OLTP)	Online Analytical Processing (OLAP)
Optimization	Deletes, inserts, updates large numbers of short online transactions quickly	Rapidly analyze massive volumes of data and provide different viewpoints for analysts
Data structure	Highly normalized data structure with many different tables containing no redundant data. Thus, data is more accurate	Denormalized data structure with few tables containing repeat data. Thus, data is potentially less accurate
Data analysis	Analysis is slow and painful due to the large number of table joins needed and the small time frame of data available	Analysis is fast and easy due to the small number of table joins needed and the extensive time frame of data available
Data timeline	Current, real-time data for one part of the business; historical queries impossible	Historical data for all parts of the business
Users	Can handle thousands of users at one time	Can only handle a smaller number

What Is a Data Warehouse?

- Data warehousing provides architectures and tools for business executives to systematically organize, understand, and use their data to make strategic decisions
- With competition mounting in every industry, data warehousing is the latest must-have marketing weapon—a way to retain customers by learning more about their needs
- Data warehouse systems allow for integration of a variety of application systems
- They support information processing by providing a solid platform of consolidated historic data for analysis

What Is a Data Warehouse?

- William H. Inmon, (leading architect in the construction of data warehouse systems) defines it as:

“A data warehouse is a subject-oriented, integrated, time-variant, and nonvolatile collection of data in support of management’s decision making process”

- Keywords—*subject-oriented*, *integrated*, *time-variant*, and *nonvolatile*—distinguish data warehouses from other data repository systems, such as relational database systems, transaction processing systems, and file systems

Subject-oriented:

- A data warehouse is organized around major subjects (e.g: customer, supplier, product, and sales).
- A data warehouse focuses on the modeling and analysis of data for decision makers (rather than concentrating on the day-to-day operations and transaction processing)
- Hence, data warehouses typically provide a simple and concise view of particular subject issues by excluding data that are not useful in the decision support process

Integrated:

- A data warehouse is usually constructed by integrating multiple heterogeneous sources, such as relational databases, flat files, and online transaction records
- Data cleaning and data integration techniques are applied to ensure consistency in naming conventions, attribute measures, and so on

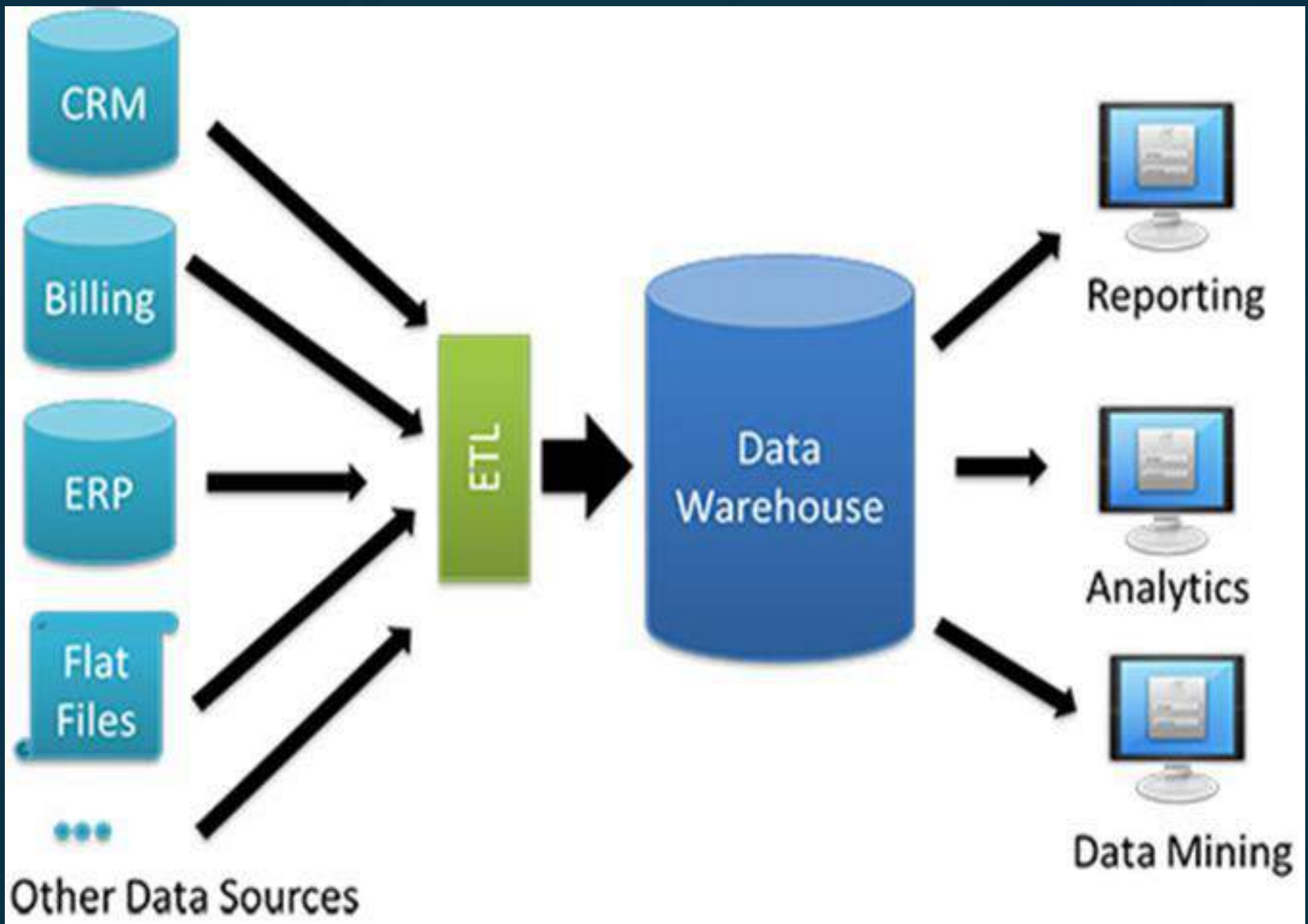
Time-variant:

- Data are stored to provide information from an historic perspective (e.g., the past 5–10 years)

Nonvolatile:

- A data warehouse is always a physically separate store of data transformed from the application data found in the operational environment
- Due to this separation, a data warehouse does not require transaction processing, recovery, and concurrency control mechanisms

- A data warehouse is a consistent data store that serves as a physical implementation of a decision support data model
- It stores the information that an enterprise needs to make strategic decisions
- A data warehouse is also often viewed as an architecture, constructed by integrating data from multiple heterogeneous sources to support structured and/or ad hoc queries, analytical reporting, and decision making.



- **Data warehousing** is the process of constructing and using data warehouses
- The **construction** of a data warehouse requires data cleaning, data integration, and data consolidation
- The **utilization** of a data warehouse often necessitates a collection of decision support technologies
- This allows “knowledge workers” (e.g., managers, analysts, and executives) to use the warehouse to quickly and conveniently obtain an overview of the data, and to make sound decisions based on information in the warehouse

How are organizations using the information from data warehouses?

Organizations use information to support business decision-making activities:

- Increasing customer focus, which includes the analysis of customer buying patterns
- Repositioning products and managing product portfolios by comparing the performance of sales by quarter, by year, and by geographic regions in order to fine-tune production strategies
- Analyzing operations and looking for sources of profit
- Managing customer relationships, making environmental corrections, and managing the cost of corporate assets

Differences between OLTP and OLAP

System orientation and Users:

- OLTP: *customer-oriented*; used for transaction and query processing by clerks, clients, and information technology professionals
- OLAP: *market-oriented*; used for data analysis by knowledge workers, including managers, executives, and analysts

Data contents:

- OLTP : manages current data that, typically, are too detailed to be easily used for decision making
- OLAP: manages large amounts of historic data, provides facilities for summarization and aggregation, and stores and manages information at different levels of granularity. These features make the data easier to use for informed decision making.

Database design:

- OLTP: usually adopts an entity-relationship (ER) data model and an application-oriented database design
- OLAP: typically adopts either a *star* or a *snowflake* model and a subject-oriented database design

Access patterns:

- OLTP: accesses consist mainly of short, atomic transactions. Such a system requires concurrency control and recovery mechanisms
- OLAP: accesses to OLAP systems are mostly read-only operations

View:

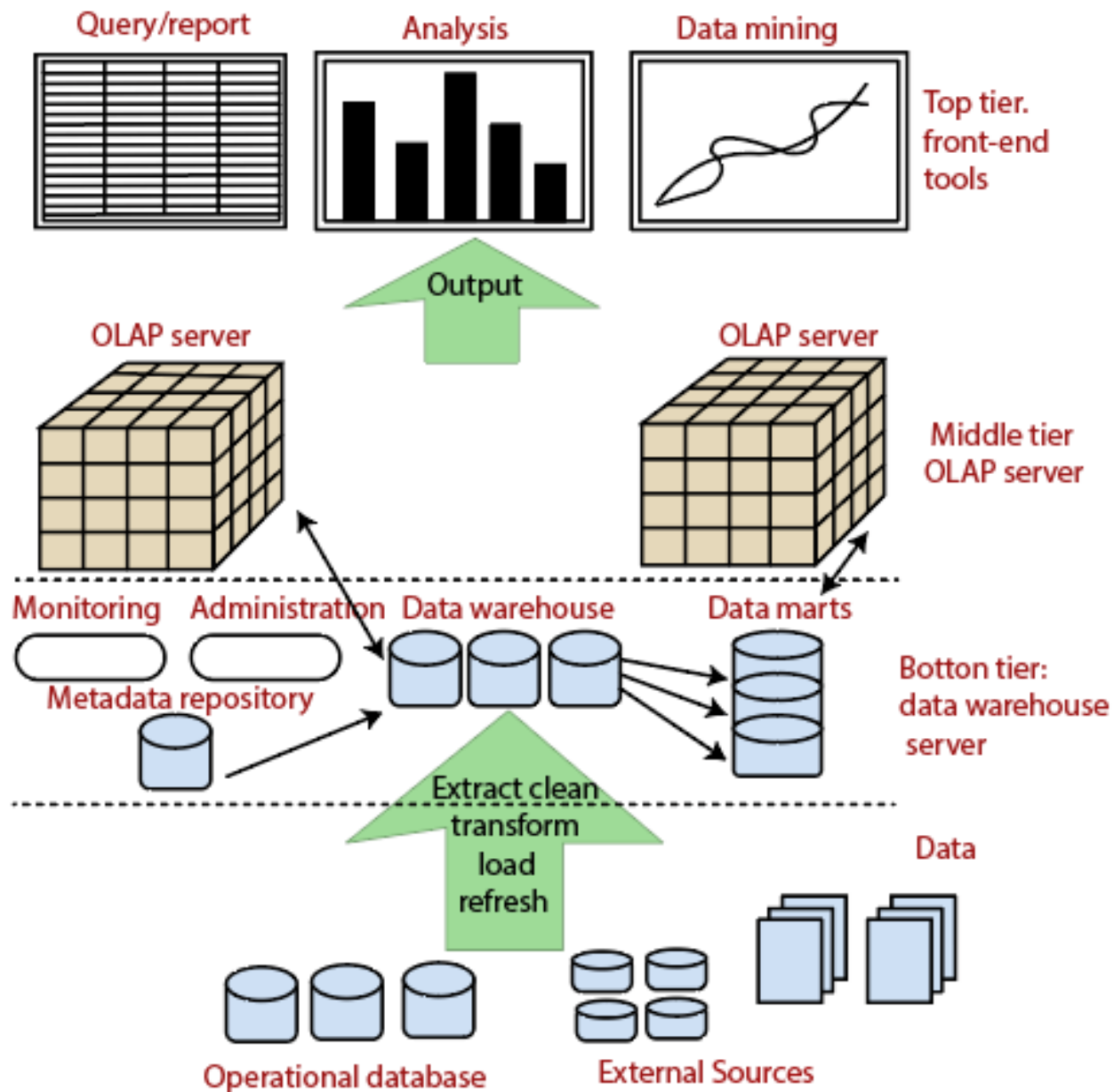
- OLTP: focuses mainly on the current data within an enterprise or department, without referring to historic data or data in different organizations
- OLAP: often spans multiple versions of a database schema, due to the evolutionary process of an organization. OLAP systems also deal with information that originates from different organizations, integrating information from many data stores

<i>Feature</i>	<i>OLTP</i>	<i>OLAP</i>
Characteristic	operational processing	informational processing
Orientation	transaction	analysis
User	clerk, DBA, database professional	knowledge worker (e.g., manager, executive, analyst)
Function	day-to-day operations	long-term informational requirements decision support
DB design	ER-based, application-oriented	star/snowflake, subject-oriented
Data	current, guaranteed up-to-date	historic, accuracy maintained over time
Summarization	primitive, highly detailed	summarized, consolidated
View	detailed, flat relational	summarized, multidimensional
Unit of work	short, simple transaction	complex query
Access	read/write	mostly read
Focus	data in	information out
Operations	index/hash on primary key	lots of scans
Number of records accessed	tens	millions
Number of users	thousands	hundreds
DB size	GB to high-order GB	$\geq$ TB
Priority	high performance, high availability	high flexibility, end-user autonomy
Metric	transaction throughput	query throughput, response time

Data Warehousing: A Multitiered Architecture

Data Warehouses usually have a three-level (tier) architecture that includes:

1. Bottom Tier (Data Warehouse Server)
2. Middle Tier (OLAP Server)
3. Top Tier (Front end Tools)



Three-Tier Data Warehouse Architecture

Bottom tier: Data warehouse Server

- Back-end tools and utilities are used to feed data into the bottom tier from operational databases or other external sources
- These tools and utilities perform data extraction, cleaning, and transformation as well as load and refresh functions to update the data warehouse
- The data are extracted using application program interfaces known as **Gateways**
 - A gateway is supported by the underlying DBMS
 - It allows client programs to generate SQL code to be executed at a server
 - Examples: ODBC (Open Database Connection), OLEDB (Object Linking and Embedding Database), JDBC (Java Database Connection)

- This tier also contains a metadata repository, which stores information about the data warehouse and its contents
- It includes the following parameters and information for the middle and the top-tier applications:
- A description of the DW structure, including the warehouse schema, dimension, hierarchies, data mart locations, and contents, etc.
- Operational metadata, which usually describes the currency level of the stored data, i.e., active, archived or purged, and warehouse monitoring information, i.e., usage statistics, error reports, audit, etc.
- System performance data, which includes indices, used to improve data access and retrieval performance.
- Information about the mapping from operational databases, which provides source RDBMSs and their contents, cleaning and transformation rules, etc.

Middle tier: OLAP server

- Data Warehouse can have more than one OLAP server, and it can have more than one type of OLAP server model as well, which depends on the volume of the data to be processed and the type of data held in the bottom tier
- OLAP server is typically implemented using either:
- **Relational OLAP(ROLAP) model**
 - An extended relational DBMS that maps operations on multidimensional data to standard relational operations
- **Multidimensional OLAP (MOLAP) model**
 - A special-purpose server that directly implements multidimensional data and operations

Top tier: front-end client layer

- Includes Tools and APIs that connects and get data out from the data warehouse and present the same to the client
- Contains query and reporting tools, analysis tools, data mining tools (e.g., trend analysis, prediction etc.)
- The type of tool depends purely on the form of outcome expected

Data Warehouse Models

From the architecture point of view, there are three data warehouse models:

- Enterprise warehouse
- Data mart
- Virtual warehouse

Enterprise warehouse

- It collects all the information about subjects spanning the entire organization
- It provides corporate-wide data integration, usually from one or more operational systems or external information providers
- It is cross-functional in scope
- It typically contains detailed data as well as summarized data, and can range in size from a few gigabytes to hundreds of gigabytes, terabytes, or beyond
- An enterprise data warehouse may be implemented on traditional mainframes, computer superservers, or parallel architecture platforms
- It requires extensive business modeling and may take years to design and build

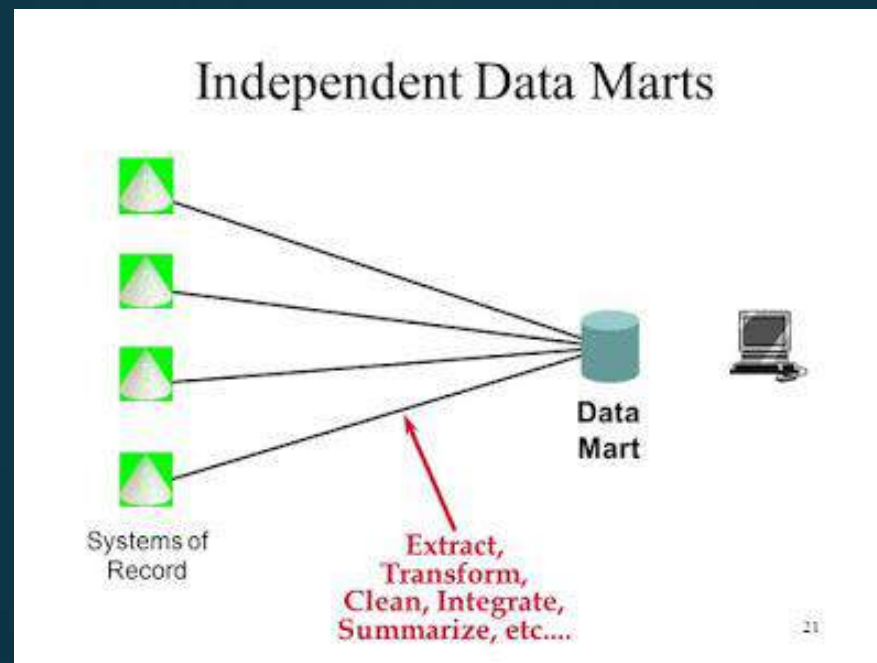


Data mart

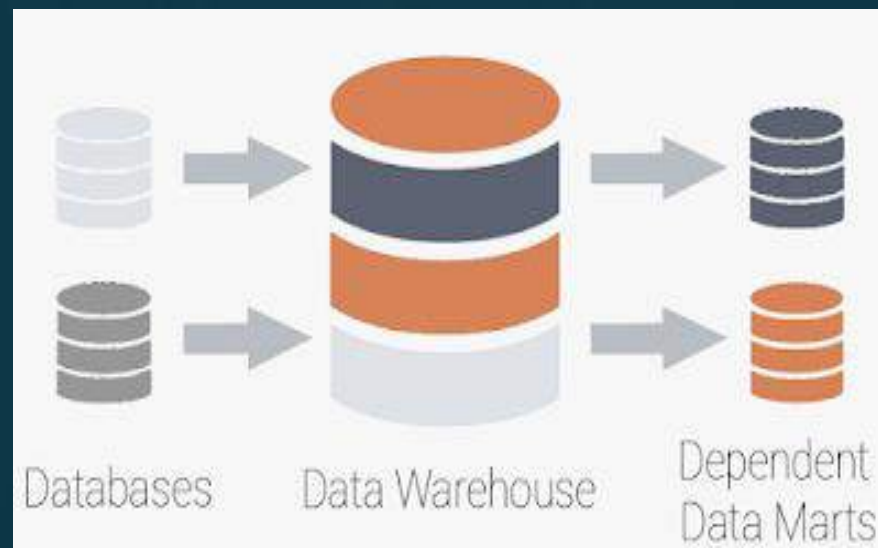
- A data mart contains a subset of corporate-wide data that is of value to a specific group of users
- The scope is confined to specific selected subjects
- Example: A marketing data mart may confine its subjects to customer, item, and sales.
- The data contained in data marts tend to be summarized
- Data marts are usually implemented on low-cost departmental servers that are Unix/Linux or Windows based
- The implementation cycle of a data mart is more likely to be measured in weeks rather than months or years

- Independent data marts:

- Sourced from data captured from one or more operational systems or external information providers, or from data generated locally within a particular department or geographic area



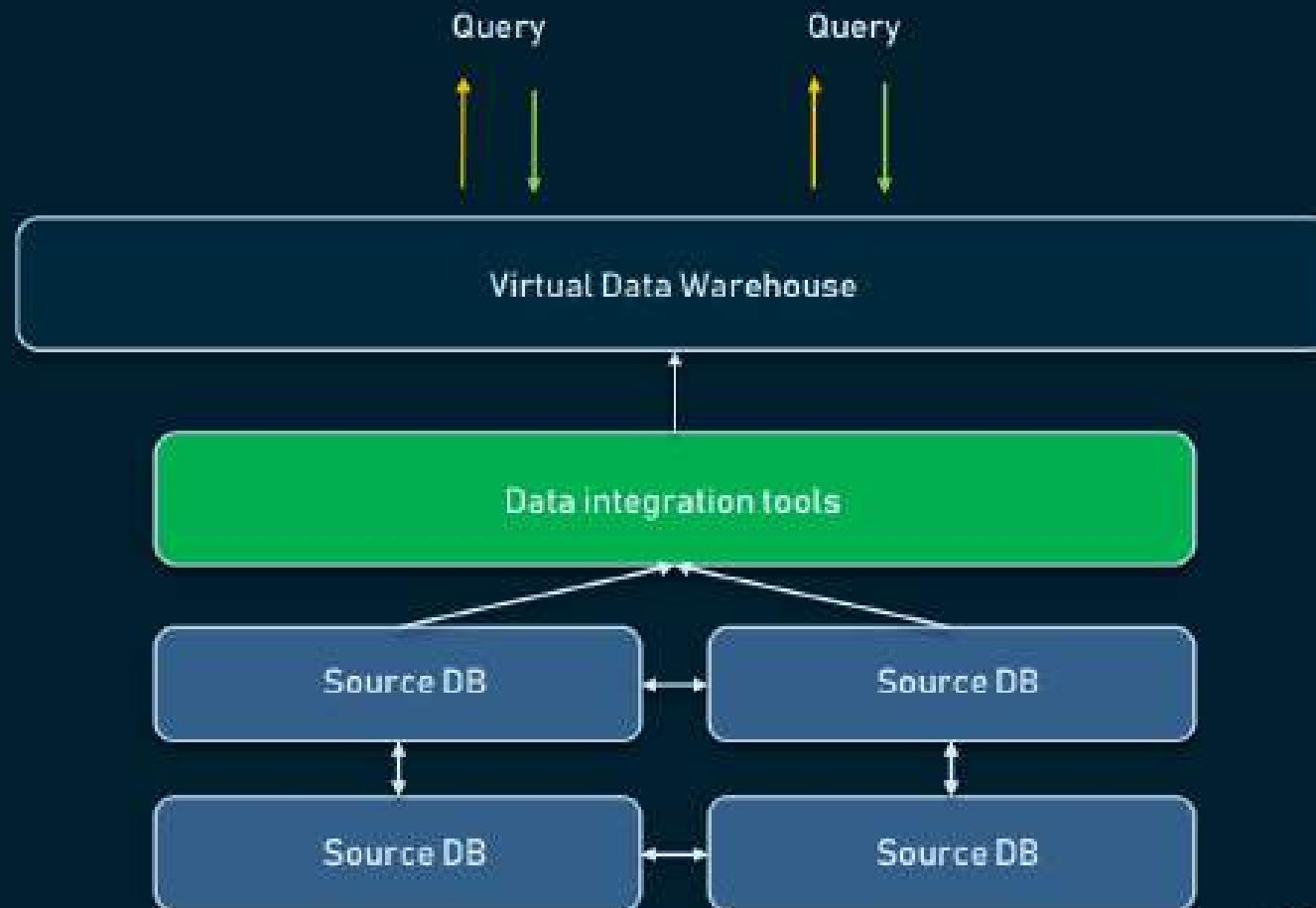
- Dependent data marts:
 - Sourced directly from enterprise data warehouses



Virtual Warehouse

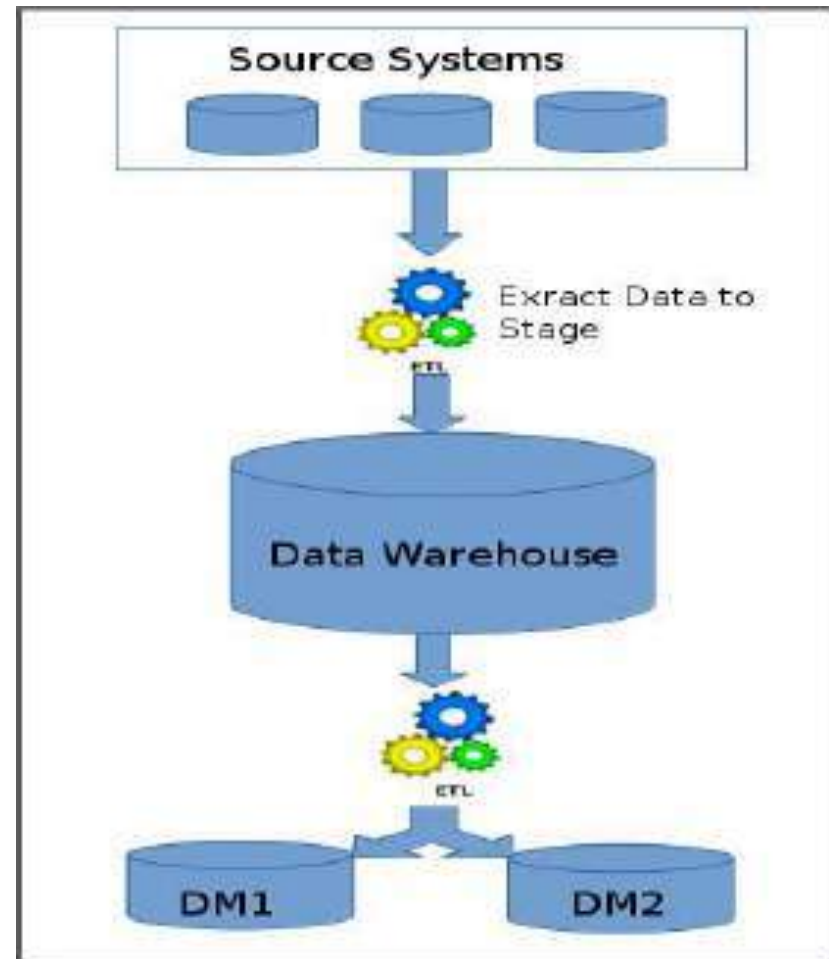
- A virtual warehouse is a set of views over operational databases
- For efficient query processing, only some of the possible summary views may be materialized
- It uses middleware to build connections to different data sources
- Set of separate databases, which can be queried together, so a user can effectively access all the data as if it was stored in one data warehouse- **A layer of virtuality is created**
- They can be fast as they allow users to filter the most important pieces of data from different legacy applications
- A virtual warehouse is easy to build but requires excess capacity on operational database servers

VIRTUAL DATA WAREHOUSE



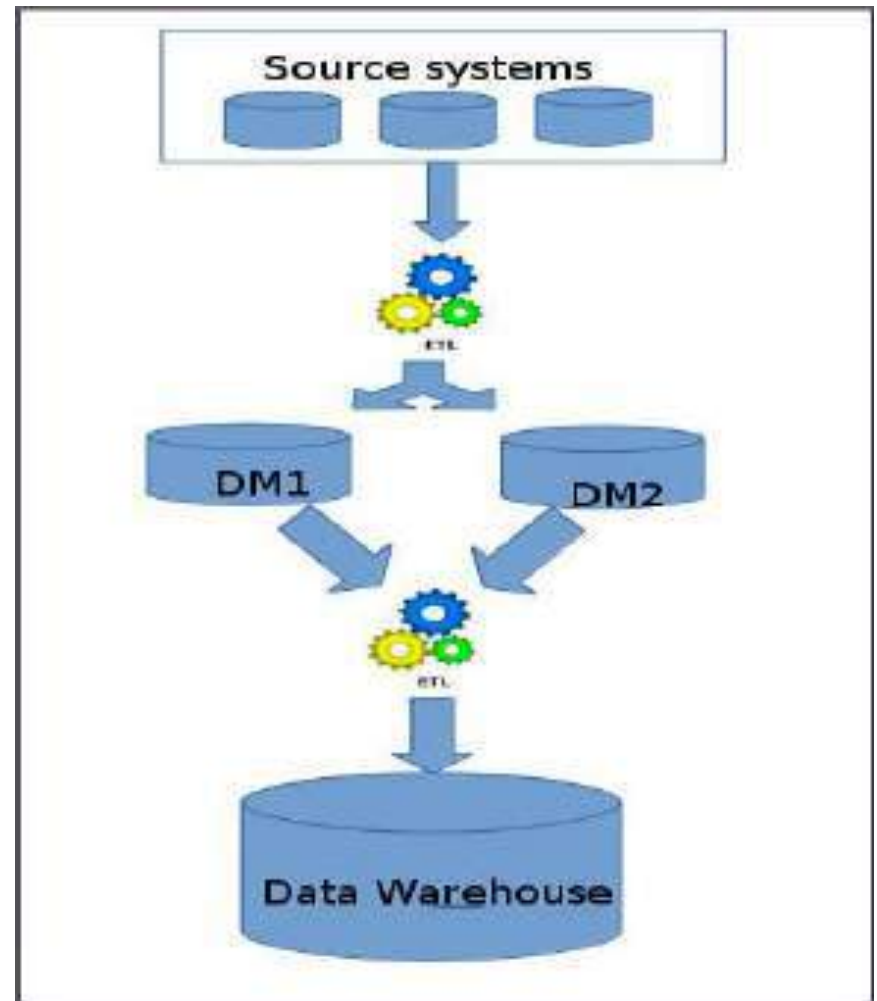
Approaches to the DW development

- ▶ **Top-down Approach(Bill Inmon)**
 - Data warehouse is designed first and then data mart are built on top of data warehouse
 - Serves as a systematic solution
 - Minimizes integration problems
 - Expensive
 - Takes a long time to develop
 - Lacks flexibility due to the difficulty in achieving consistency and consensus for a common data model for the entire organization



► Bottom-up Data Approach(Ralph Kimball)

- Data marts are first created to provide the reporting and analytics capability for specific business process, later with these data marts enterprise data warehouse is created
- Development and deployment of independent data marts provides flexibility, low cost, and rapid return of investment
- It can lead to problems when integrating various disparate data marts into a consistent enterprise data warehouse



(Kimball Approach)

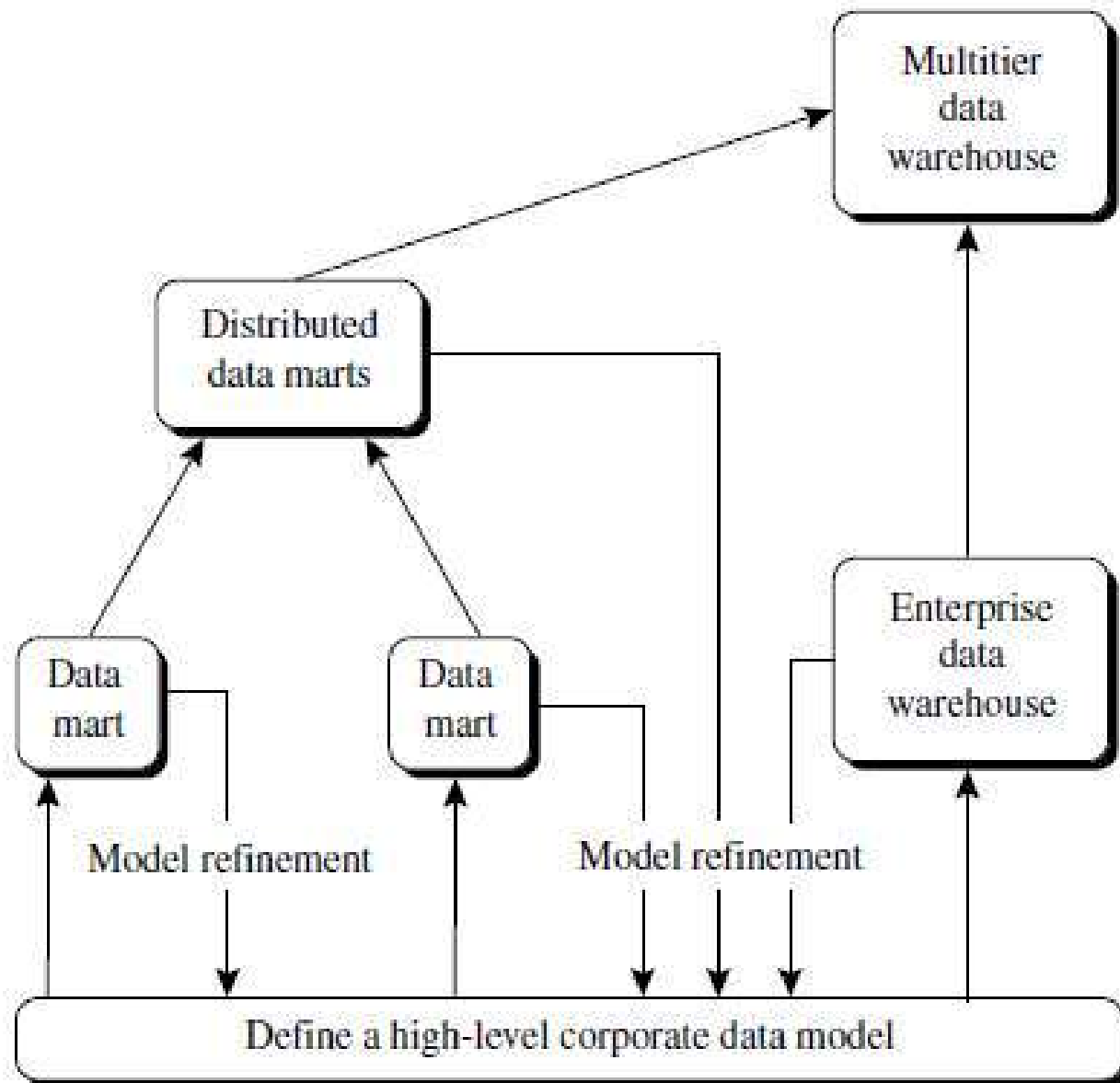
(Inmon Approach)

	Functional Data Mart	Enterprise Data Warehouse
Scope	One subject or functional area	Complete enterprise data needs
Value	Functional area reporting and insights	Deeper insights connecting multiple functional areas
Target organization	Decentralized management	Centralized management
Time	Low to medium	High
Cost	Low	High
Size	Small to medium	Medium to large
Approach	Bottom up	Top down
Complexity	Low (fewer data transformations)	High (data standardization)
Technology	Smaller scale servers and databases	Industrial strength

Recommended method for the development of DW

Implement the warehouse in an incremental and evolutionary manner

- A high-level corporate data model is defined within a reasonably short period
 - provides a corporate-wide, consistent, integrated view of data among different subjects and potential usages
 - need to be refined in the further development of enterprise data warehouses and departmental data marts
 - greatly reduce future integration problems
- Independent data marts can be implemented in parallel with EDW based on the corporate data model
- Distributed data marts can be constructed to integrate different data marts via hub servers
- A multitier data warehouse is constructed
 - where the enterprise warehouse is the sole custodian of all warehouse data, which is then distributed to the various dependent data marts

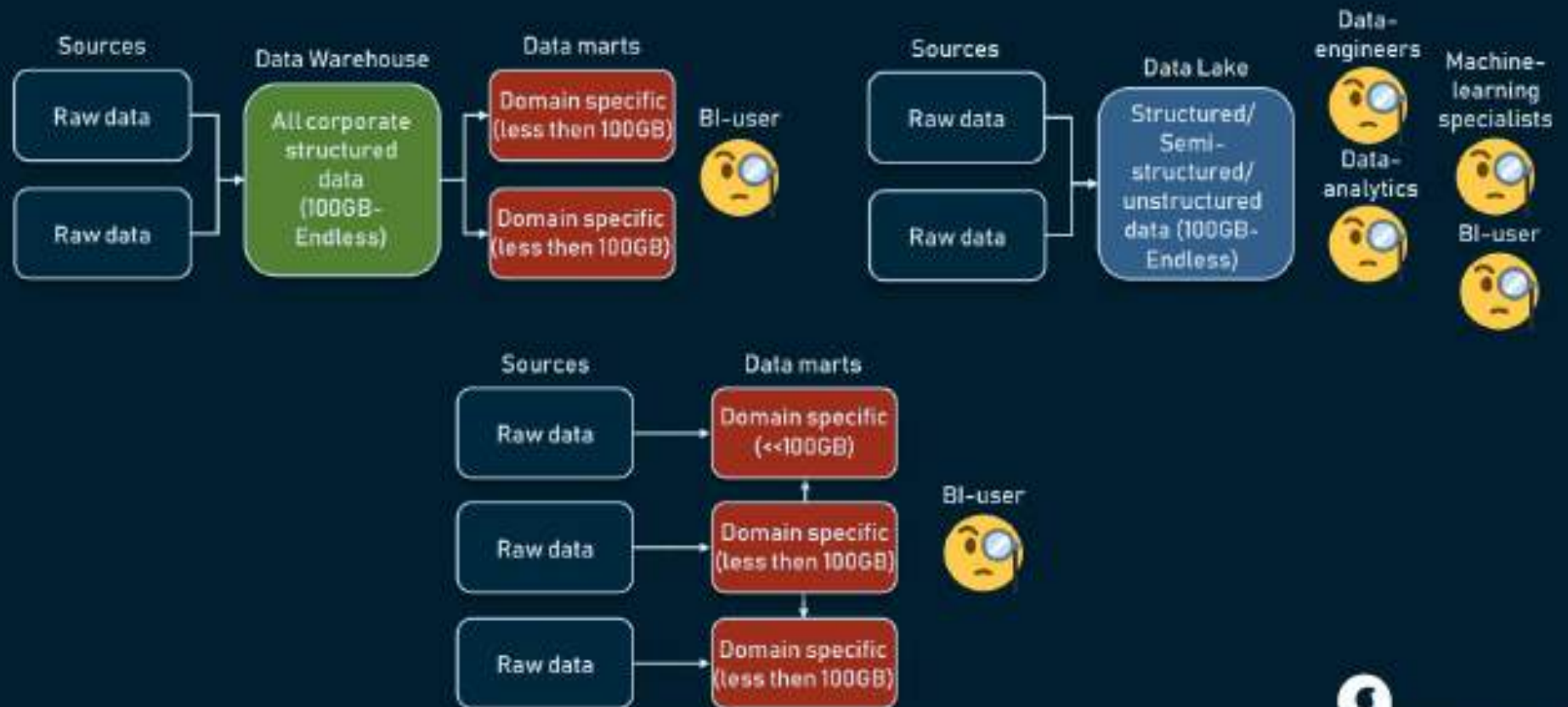


Data Lakes

- A data lake is a storage repository that holds a vast amount of raw data in its native format until it is needed
- It is usually a single store of data including raw copies of source system data, sensor data, social data etc., and transformed data used for tasks such as reporting, visualization, advanced analytics and machine learning.
- A data lake can include:
 - structured data from relational databases (rows and columns),
 - semi-structured data (CSV, logs, XML, JSON),
 - unstructured data (emails, documents, PDFs) and
 - binary data (images, audio, video)
- A data lake can be established "on premises" or "in the cloud"

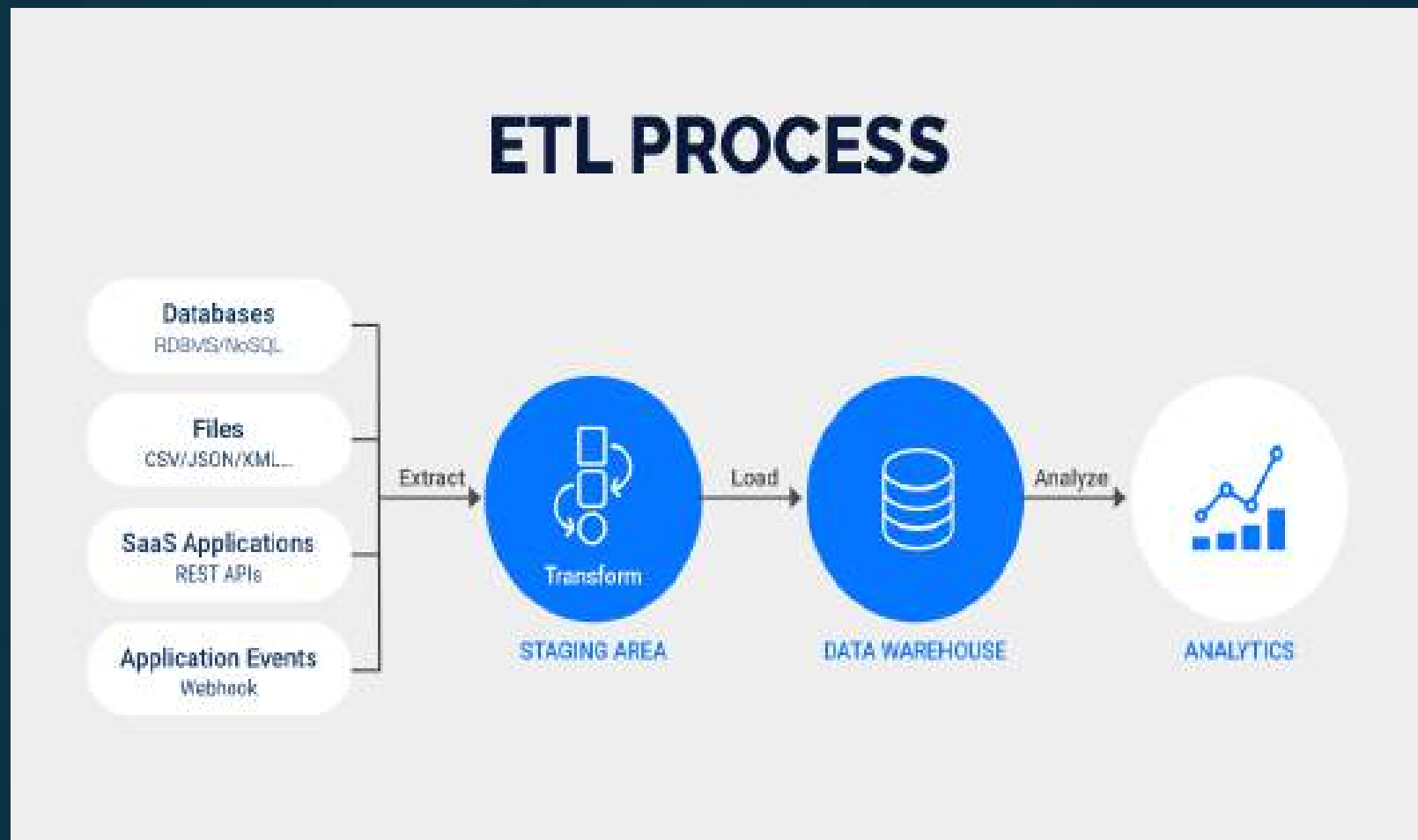
Parameters	Data Lake	Data Warehouse
Data Structure	Data is raw and all types—structured, semi-structured, or unstructured—is captured in its original form.	Data is processed and only structured information is captured and organized in schemas.
Users	Ideal for users who carry out deep analysis such as data scientists and need advanced analytical tools.	Ideal for operational users such as business professionals and moguls since the data is structured and easy to use.
Storage Costs	Storing data is relatively inexpensive.	Storing data is time-consuming and costly.
Accessibility	Updates can be made quickly thus making it highly accessible	Costly to make changes, thereby quite complicated
Position of Schema	Schema is defined after data is stored, thus making it highly agile.	Schema is defined before data is stored, thus offering performance and security.
Data Processing	Uses ELT (Extract Load Transform) process.	Uses ETL (Extract Transform Load) process.

DATA WAREHOUSE VS DATA LAKE VS DATA MART



Extraction, Transformation, and Loading

- ETL is a process that extracts the data from different source systems, then transforms the and finally loads the data into the Data Warehouse system



Extraction

- Process of extracting the data from the source system(s)
- Extracting data correctly sets the stage for the success of subsequent processes
- Most data-warehousing projects combine data from different source systems- relational databases, XML, JSON and flat files
- Data warehouse needs to integrate systems that have different
- DBMS, Hardware, Operating Systems and Communication Protocols
- Sources could include legacy applications like Mainframes, customized applications, Point of contact devices like ATM, Call switches, text files, spreadsheets, ERP, data from vendors, partners amongst others

Examples for validations done during Extraction:

- Make sure that no spam/unwanted data loaded
- Data type check
- Remove all types of duplicate/fragmented data

Transformation

- In the data transformation stage, a series of rules or functions are applied to the extracted data in order to prepare it for loading into the end target
- Data extracted from source server is raw and not usable in its original form. Therefore it needs to be cleansed, mapped and transformed
- This step adds value and changes data such that insightful BI reports can be generated
- Transformations if any are done in staging area so that performance of source system is not degraded
- Also, if corrupted data is copied directly from the source into Data warehouse database, rollback will be a challenge.
- Staging area gives an opportunity to validate extracted data before it moves into the Data warehouse

Examples for transformations:

- Selecting only certain columns to load:
- Translating coded value(ssource system codes male as "1" and female as "2", but the warehouse codes male as "M" and female as "F")
- Encoding free-form values: (e.g., mapping "Male" to "M")
- Deriving a new calculated value: (e.g., $\text{sale_amount} = \text{qty} * \text{unit_price}$)
- Sorting or ordering the data based on a list of columns to improve search performance
- Aggregating
- Splitting a column into multiple columns
- Applying any form of data validation

Loading

- Process of Loading data into the target datawarehouse database
- In a typical Data warehouse, huge volume of data needs to be loaded in a relatively short period. Hence, load process should be optimized for performance.
- In case of load failure, recover mechanisms should be configured to restart from the point of failure without data integrity loss
- Updating extracted data is frequently done on a daily, weekly, or monthly basis
- Some data warehouses may overwrite existing information with cumulative information
- Other data warehouses (or even other parts of the same data warehouse) may add new data in a historical form at regular intervals — for example, hourly

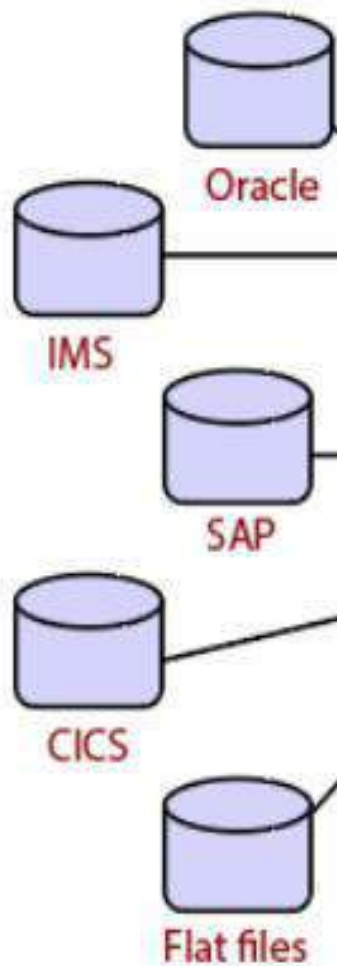
- **Data cleaning:** detects errors in the data and rectifies them when possible.
- **Refresh:** propagates the updates from the data sources to the warehouse
- Examples for ETL tools: Xplenty, Talend, Stitch, Informatica PowerCenter, Oracle Data Integrator, Skyvia etc

OPERATIONAL DATA STORES(ODS)

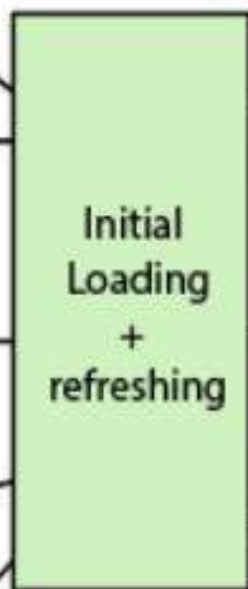
- A subject-oriented, integrated, volatile, current valued data store, containing only corporate detailed data
- **Volatile:** data in the ODS changes frequently as new information refreshes the ODS
- **Current valued:** an ODS is up-to-date and reflects the current status of the information
- Since the OLTP systems data is changing all the time, data from underlying sources refresh the ODS as regularly and frequently as possible
- **Detailed:** ODS is detailed enough to serve the needs of the operational management staff in the enterprise

Operational Data Stores	Data Warehouse
ODS means for operational reporting and supports current or near real-time reporting requirements.	A data warehouse is intended for historical and trend analysis, usually reporting on a large volume of data.
An ODS consist of only a short window of data .	A data warehouse includes the entire history of data .
It is typically detailed data only.	It contains summarized and detailed data.
It is used for detailed decision making and operational reporting.	It is used for long term decision making and management reporting.
It is used at the operational level.	It is used at the managerial level.
It is updated often as the transactions system generates new data.	It is usually updated in batch processing mode on a set schedule.

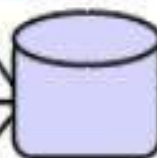
Source Systems
End Users



Extraction
Transformation
Loading



Operational Data
Source

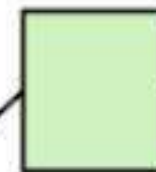


ETL

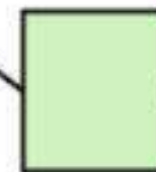
ODS



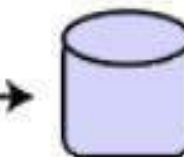
Management
reports



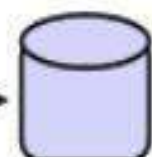
Web based
Applications



Other
Applications



Data
Warehouse



Data
Marts

Operational Data Store Structure

DATA WAREHOUSE MODELING

Data Warehouse Modeling

- Data warehouses and OLAP tools are based on a **multidimensional data model**
- This model views data in the form of a **data cube**

Data Cube: A Multidimensional Data Model

- A data cube allows data to be modeled and viewed in multiple dimensions
- It is defined by dimensions and facts

Dimensions

- Perspectives or entities with respect to which an organization wants to keep records
- E.g.: *AllElectronics* may create a *sales* data warehouse in order to keep records of the store's sales with respect to the dimensions *time*, *item*, *branch*, and *location*
- These dimensions allow the store to keep track of things like monthly sales of items and the branches and locations at which the items were sold
- Each dimension may have a table associated with it, called a **dimension table**, which further describes the dimension
- E.g: dimension table for *item* may contain the attributes *item name*, *brand*, and *type*

Facts

- A multidimensional data model is typically organized around a central theme, such as *sales*
- This theme is represented by a fact table
- Facts are numeric measures/quantities by which we can analyze relationships between dimensions
- E.g: for a sales data warehouse facts could be *dollars sold* (sales amount in dollars), *units sold* (number of units sold), and *amount budgeted*
- **fact table** contains the names of the *facts*, or measures, as well as keys to each of the related dimension tables

- In data warehousing the data cube is n -dimensional

2-D View of Sales Data for *AllElectronics* According to *time* and *item*

<i>location</i> = "Vancouver"				
<i>time</i> (quarter)	<i>item</i> (type)			
	<i>home entertainment</i>	<i>computer</i>	<i>phone</i>	<i>security</i>
Q1	605	825	14	400
Q2	680	952	31	512
Q3	812	1023	30	501
Q4	927	1038	38	580

Note: The sales are from branches located in the city of Vancouver. The measure displayed is *dollars_sold* (in thousands).

- In this 2-D representation, the sales for Vancouver are shown with respect to the
 - *time* dimension (organized in quarters) and the *item* dimension (organized according to the types of items sold).
 - The fact or measure displayed is *dollars sold* (in thousands)

- To view the sales data according to time and item, as well as location, for the cities Chicago, New York, Toronto, and Vancouver:

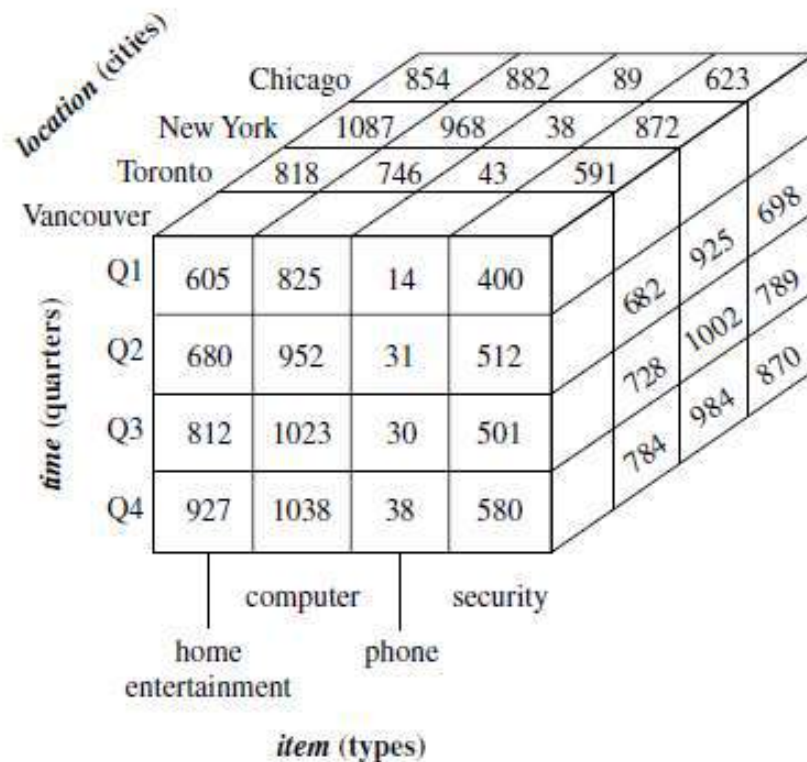
Table 4.3 3-D View of Sales Data for *AllElectronics* According to *time*, *item*, and *location*

<i>location</i> = "Chicago"					<i>location</i> = "New York"				<i>location</i> = "Toronto"				<i>location</i> = "Vancouver"			
<i>item</i>					<i>item</i>				<i>item</i>				<i>item</i>			
<i>home</i>					<i>home</i>				<i>home</i>				<i>home</i>			
<i>time</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>
Q1	854	882	89	623	1087	968	38	872	818	746	43	591	605	825	14	400
Q2	943	890	64	698	1130	1024	41	925	894	769	52	682	680	952	31	512
Q3	1032	924	59	789	1034	1048	45	1002	940	795	58	728	812	1023	30	501
Q4	1129	992	63	870	1142	1091	54	984	978	864	59	784	927	1038	38	580

Note: The measure displayed is *dollars_sold* (in thousands).

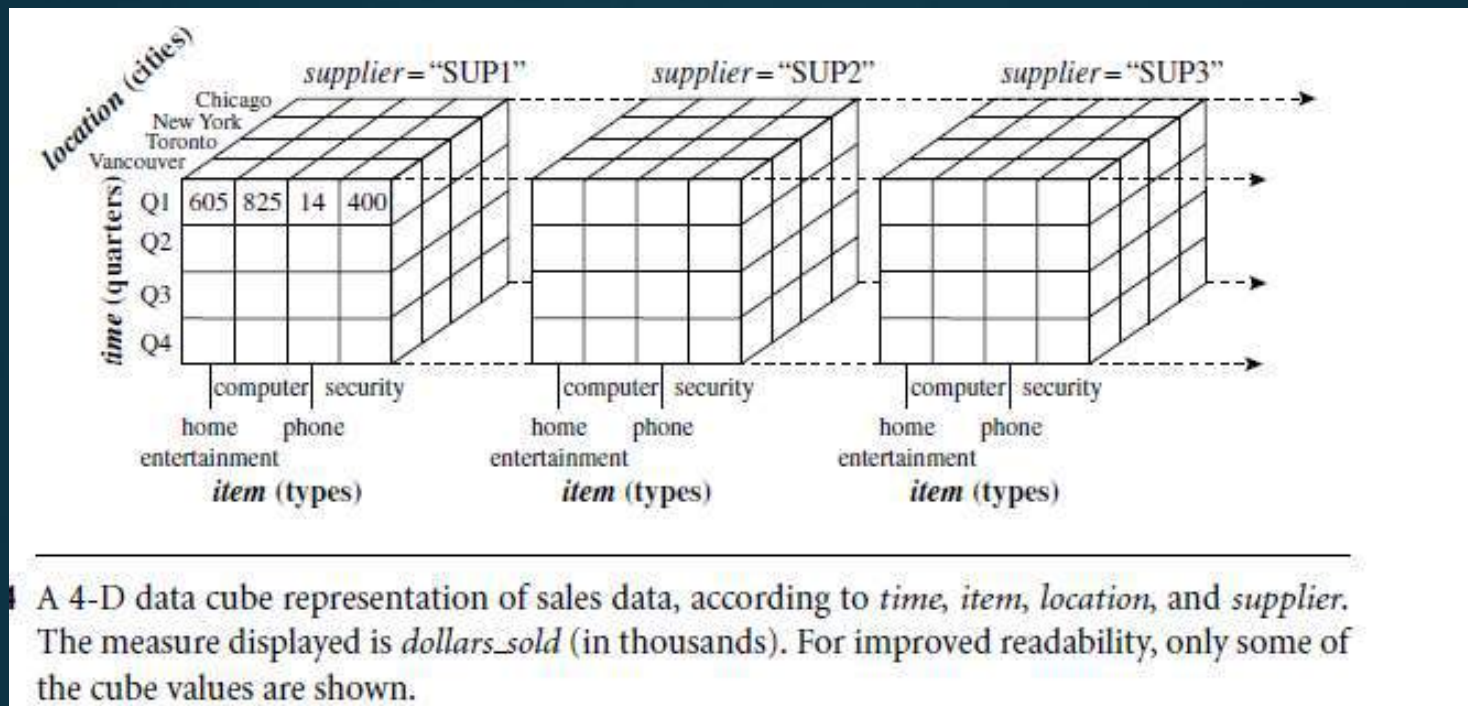
- The 3-D data in the table are represented as a series of 2-D tables

- Conceptually, we may also represent the same data in the form of a 3-D data cube

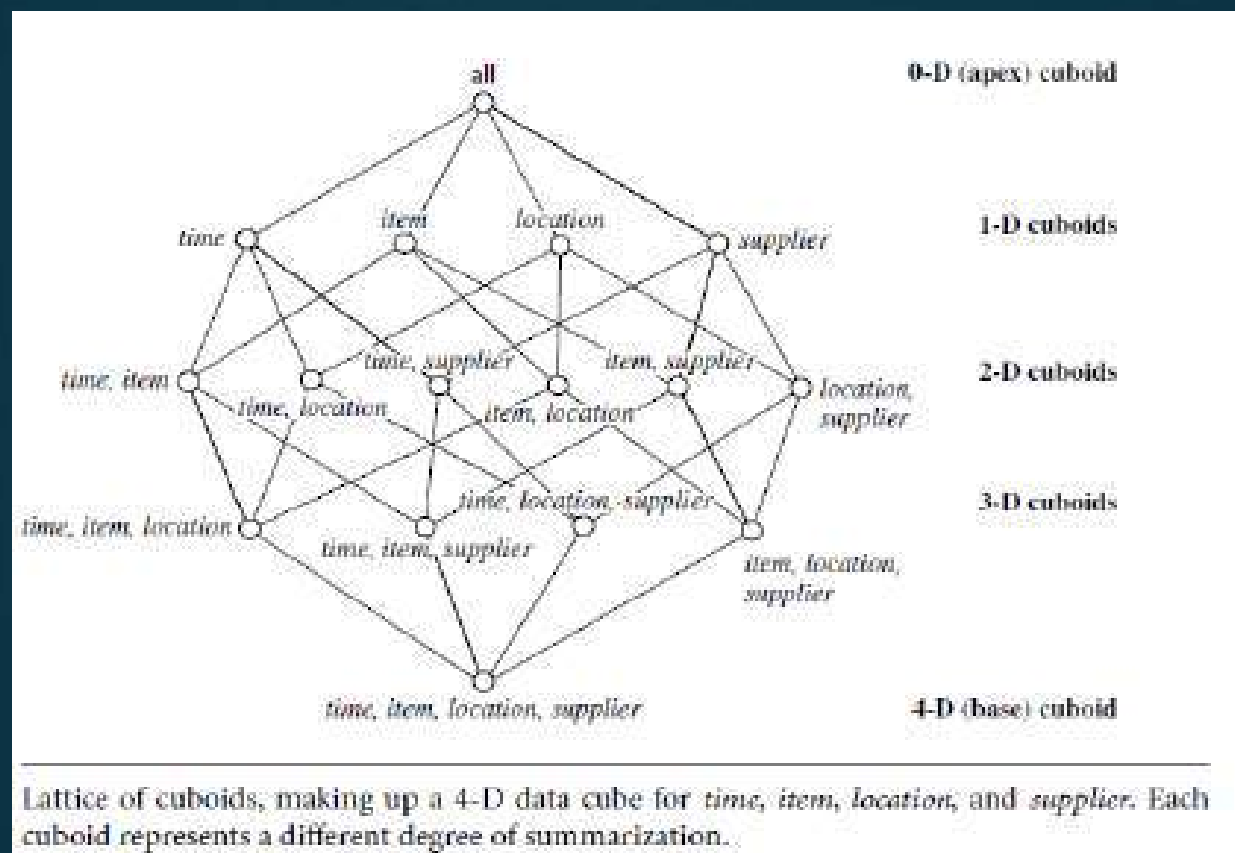


A 3-D data cube representation of the data in Table 4.3, according to *time*, *item*, and *location*. The measure displayed is *dollars_sold* (in thousands).

- To view sales data with an additional fourth dimension such as *supplier*:
- We can think of a 4-D cube as being a series of 3-D cubes



- Given a set of dimensions, we can generate a cuboid for each of the possible subsets of the given dimensions.
- The result would form a *lattice* of cuboids, each showing the data at a different level of summarization, or group-by.
- The lattice of cuboids is then referred to as a data cube



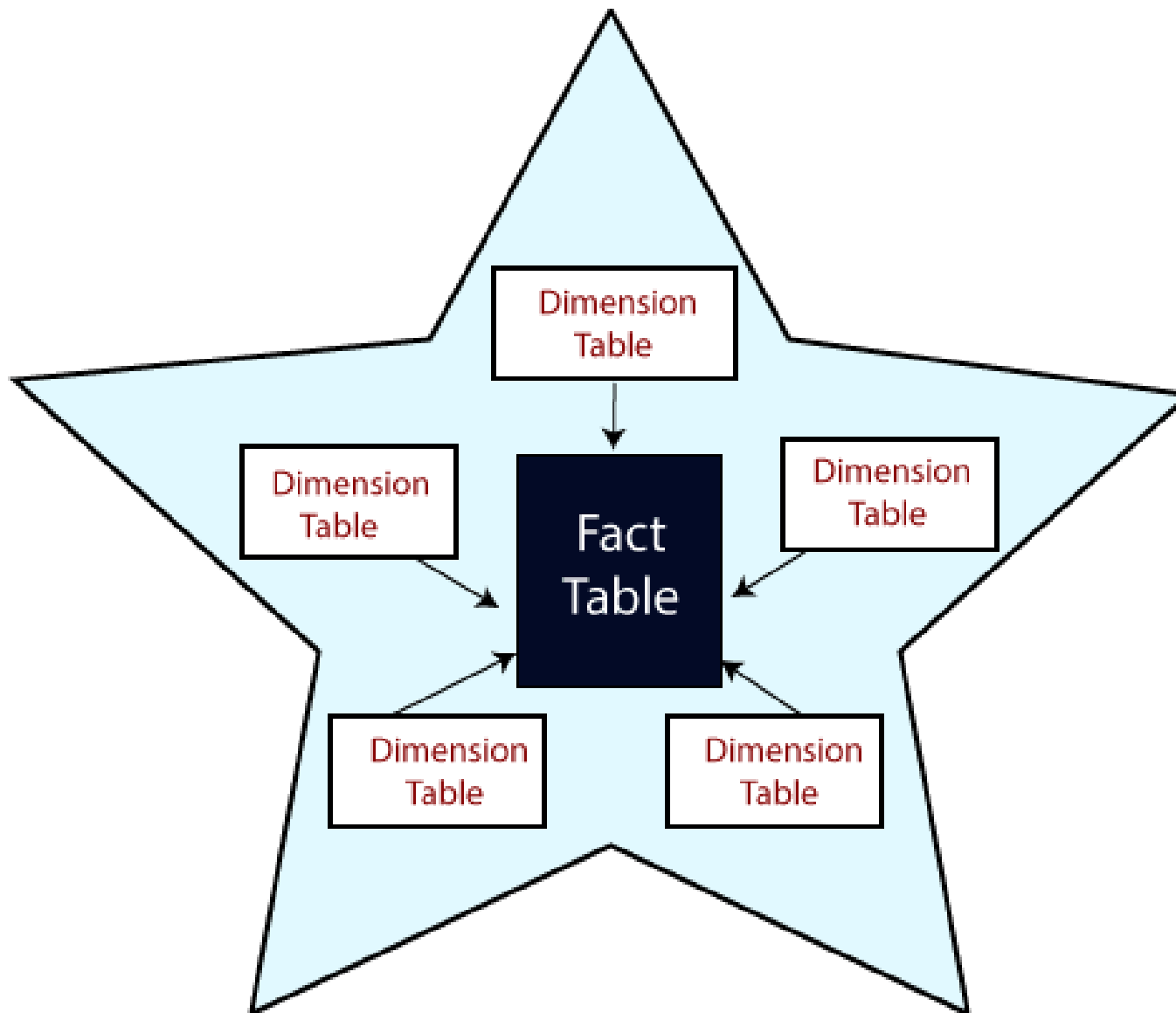
- The cuboid that holds the lowest level of summarization is called the **base cuboid**
- The 0-D cuboid, which holds the highest level of summarization, is called the **apex cuboid**.
- In our example, this is the total sales, or *dollars sold*, summarized over all four dimensions. The apex cuboid is typically denoted by all.

Schemas for Multidimensional Data Models

- A data warehouse requires a concise, subject-oriented schema that facilitates online data analysis
- The most popular data model for a data warehouse is a multidimensional model
- It exist in the form of a **star schema**, a **snowflake schema**, or a **fact constellation schema**

Star schema

- The most common modeling paradigm
- In this, the data warehouse contains
- A large central table (**fact table**) containing the bulk of the data, with no redundancy
- A set of smaller attendant tables (**dimension tables**), one for each dimension
- The schema graph resembles a starburst, with the dimension tables displayed in a radial pattern around the central fact table
- Star schemas are denormalized

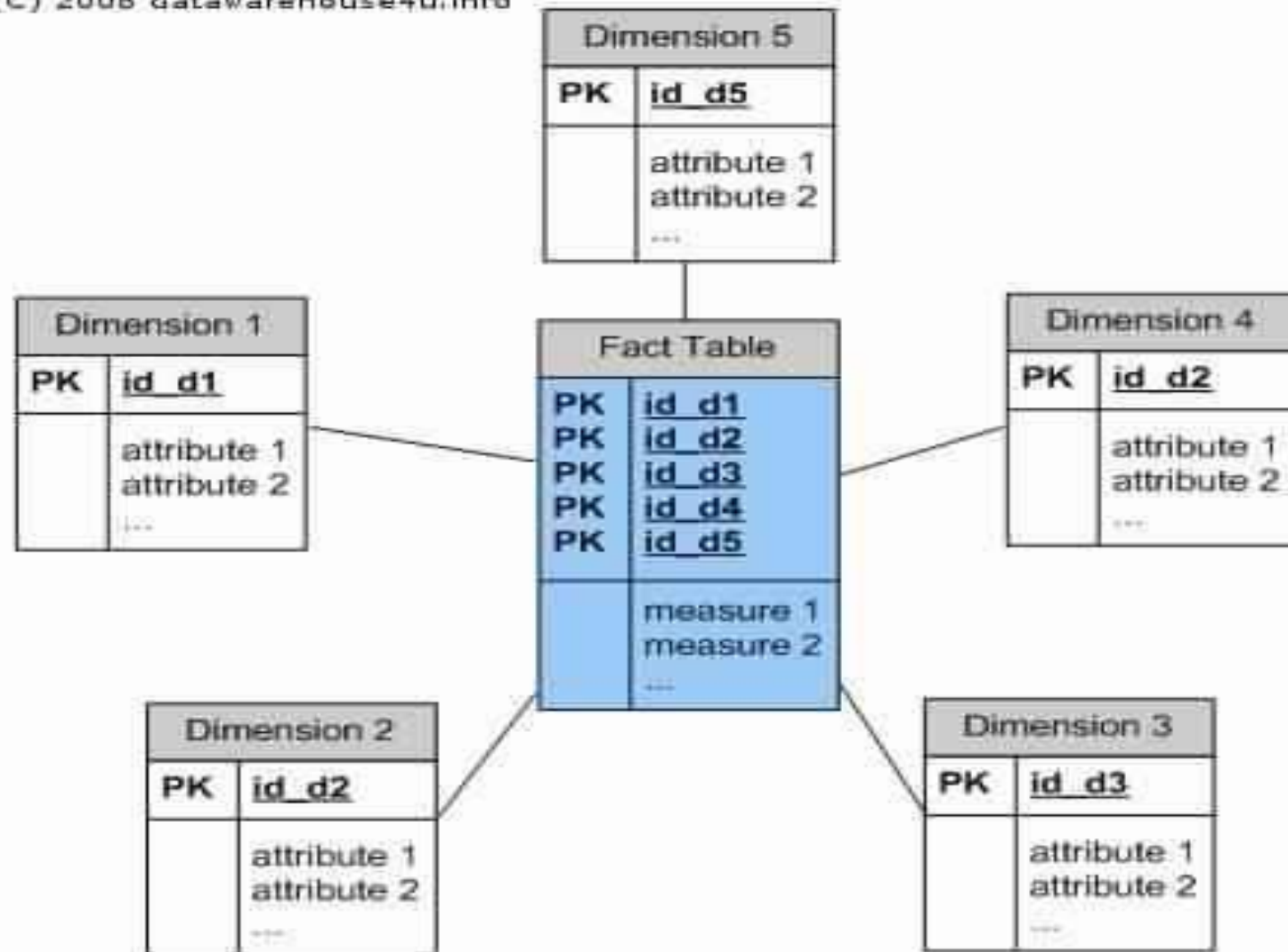


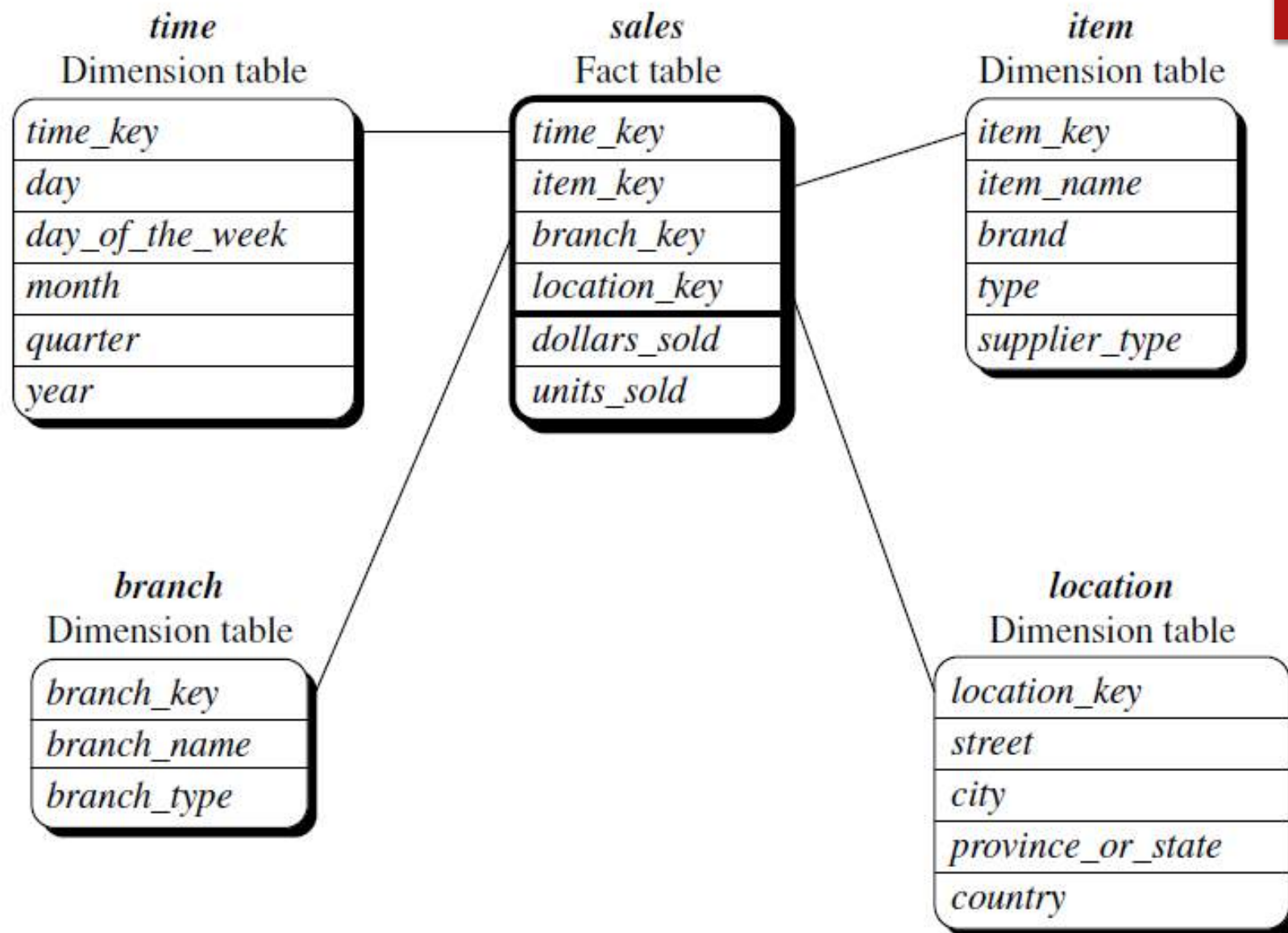
Star Schema

Star schema for *AllElectronics* sales

- Sales are considered along four dimensions: *time*, *item*, *branch*, and *location*
- The schema contains a central fact table for *sales* that contains keys to each of the four dimensions, along with two measures: *dollars sold* and *units sold*
- each dimension is represented by only one table, and each table contains a set of attributes
- Example: the *location* dimension table contains the attribute set {*location key*, *street*, *city*, *province_or_state*, *country*}

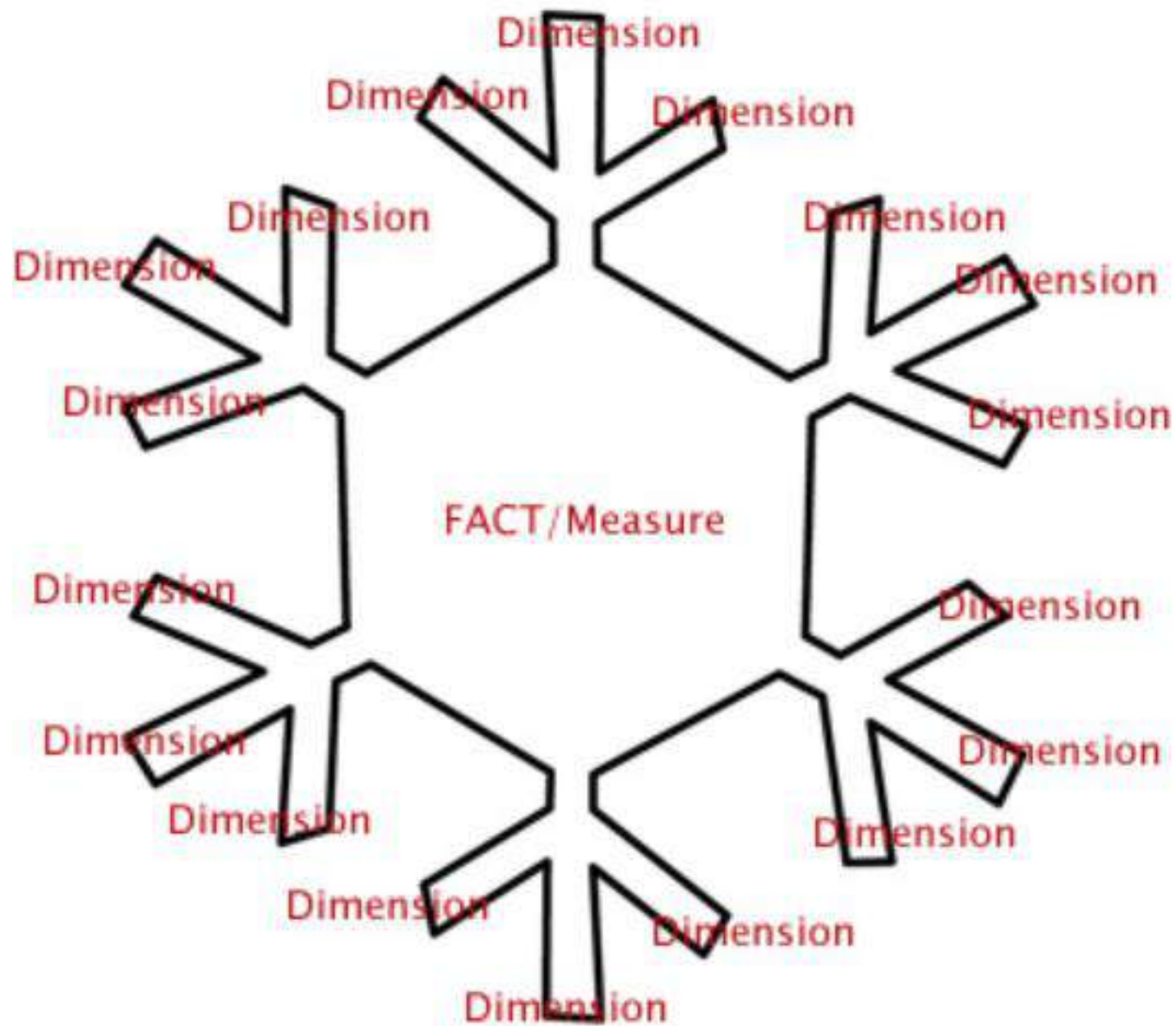
(C) 2008 datawarehouse4u.info





Snowflake schema

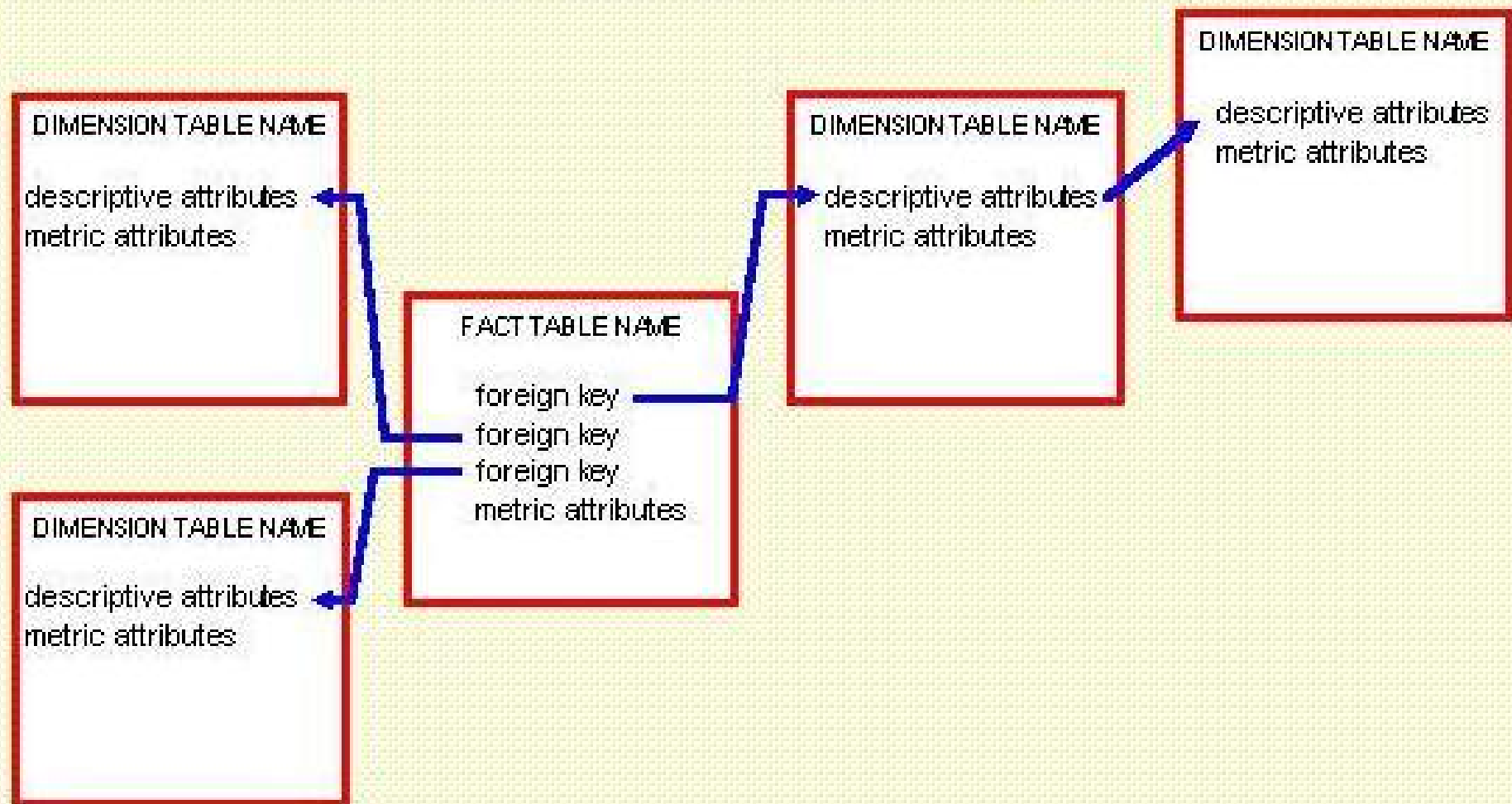
- The snowflake schema is a variant of the star schema model
- Here some dimension tables are *normalized*, thereby further splitting the data into additional tables
- The resulting schema graph forms a shape similar to a snowflake
- Dimension tables of the snowflake model are kept in normalized form to reduce redundancies
- Such tables are easy to maintain and saves storage space
- However, this space savings is negligible in comparison to the typical magnitude of the fact table
- Also, the snowflake structure can reduce the effectiveness of browsing, since more joins will be needed to execute a query => system performance may be adversely impacted
- Hence, although the snowflake schema reduces redundancy, it is not as popular as the star schema in data warehouse design

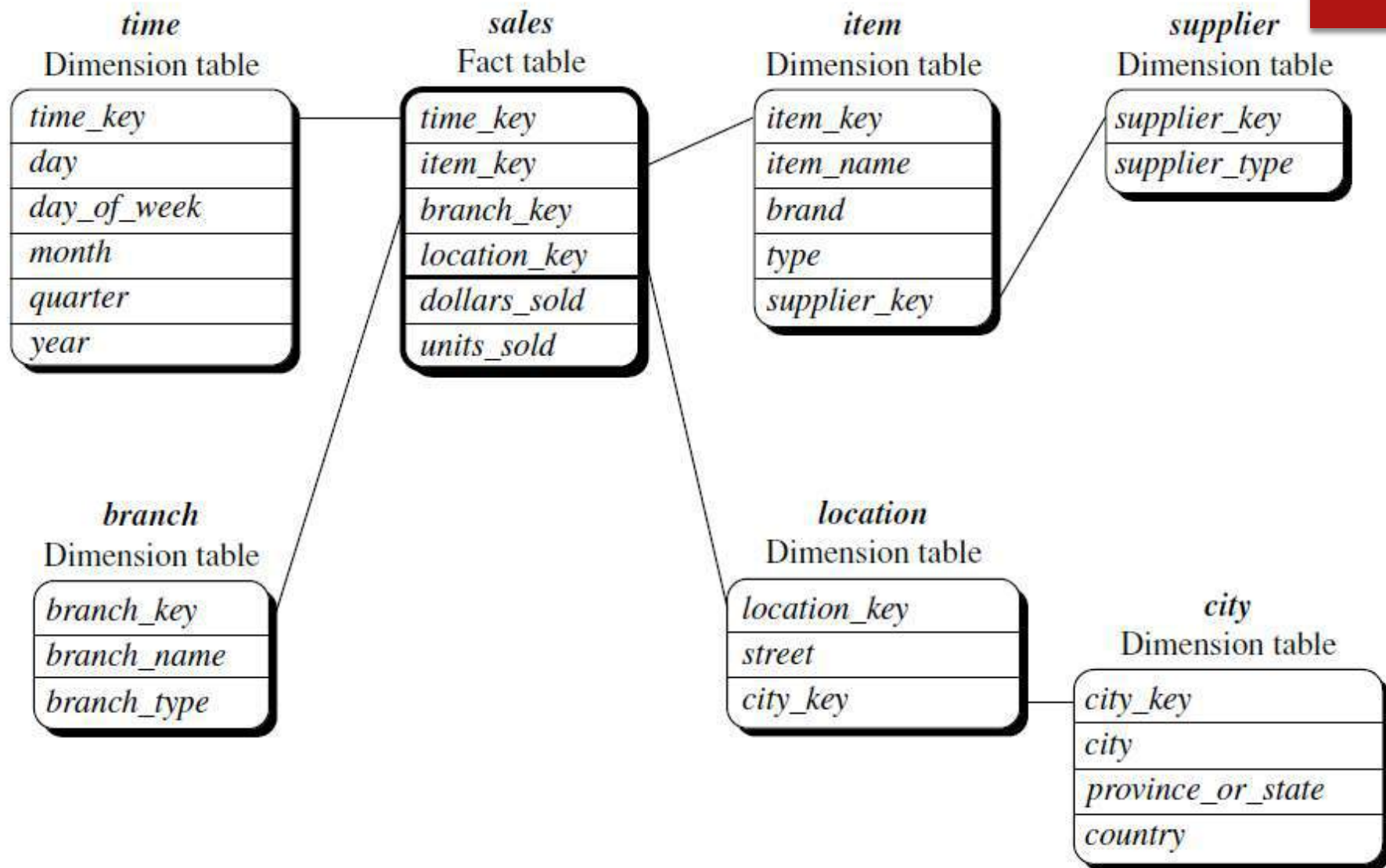


Example:

- The *sales* fact table is identical to that of the star schema
- The single dimension table for *item* in the star schema is normalized in the snowflake schema, resulting in new *item* and *supplier* tables
- For example, the *item* dimension table now contains the attributes *item key*, *item name*, *brand*, *type*, and *supplier key*, where *supplier key* is linked to the *supplier* dimension table, containing *supplier key* and *supplier type* information
- single dimension table for *location* in the star schema can be normalized into two new tables: *location* and *city*
- The *city key* in the new *location* table links to the *city* dimension.

Snowflake Schema Template



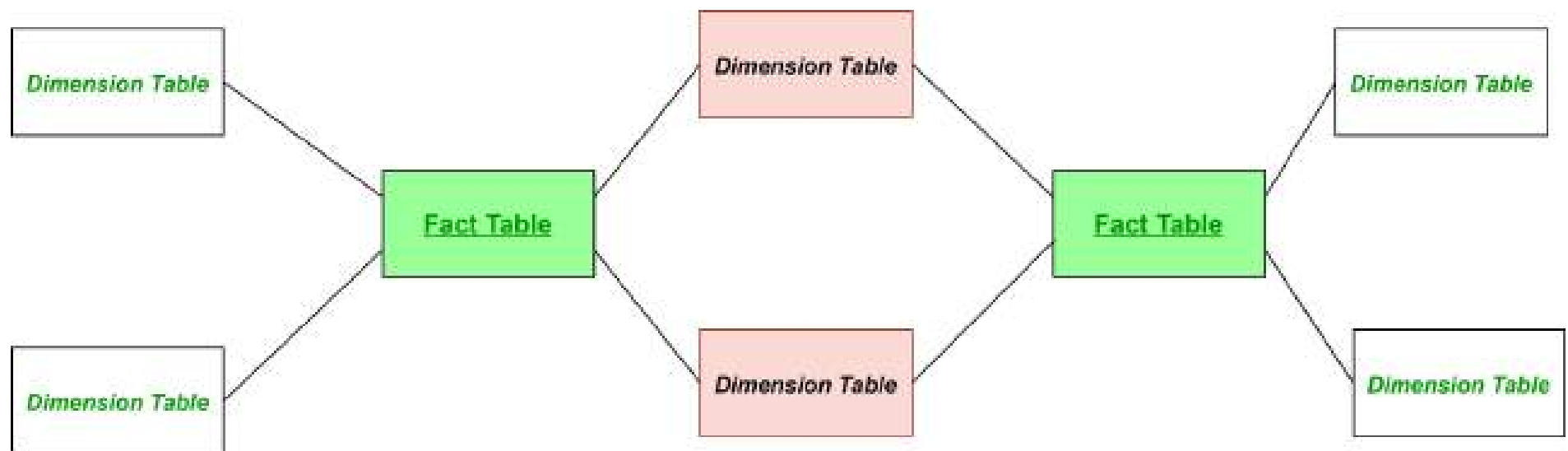


Difference between Snow Flake and Star Schema

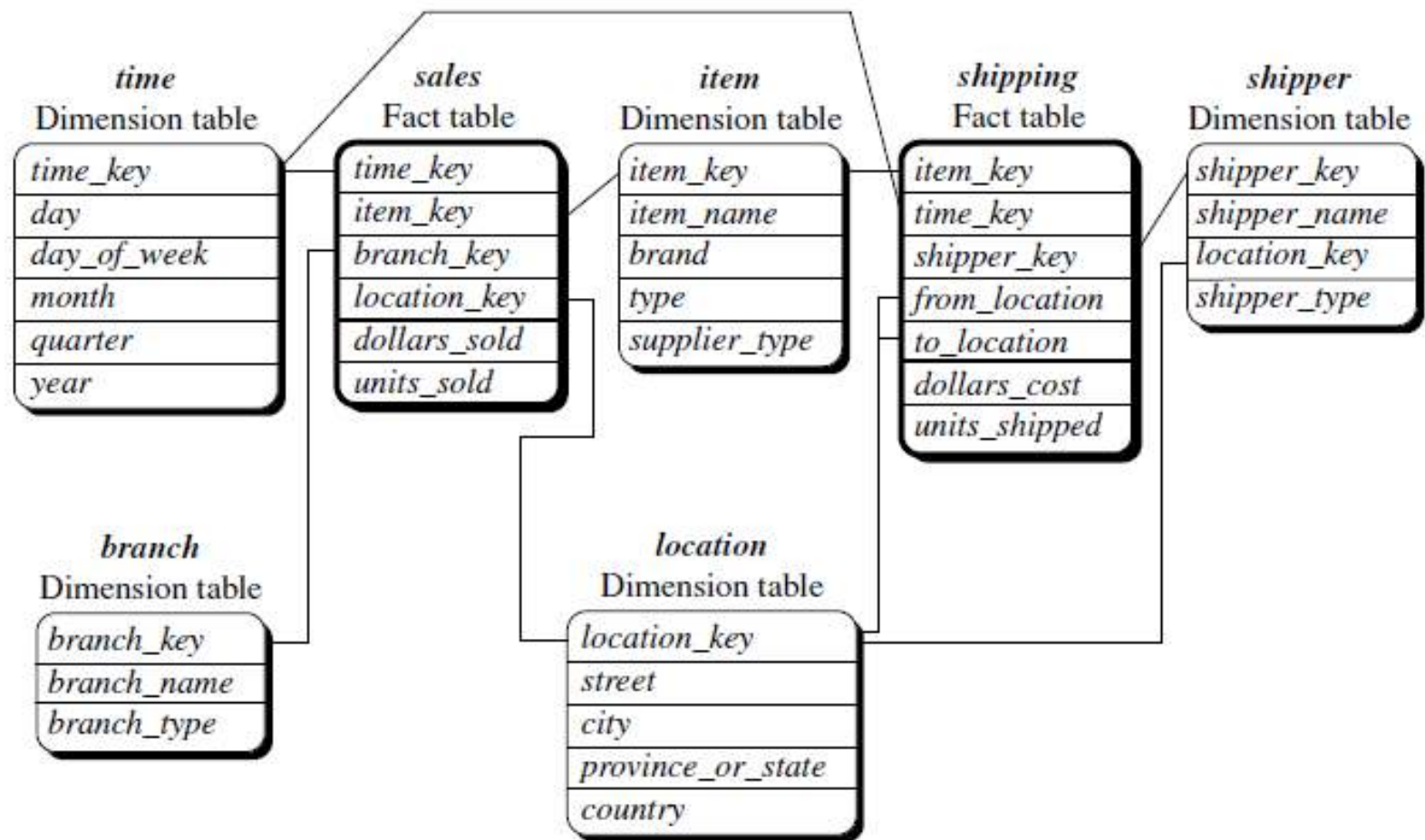
Star Schema	Snow Flake Schema
The star schema is the simplest data warehouse scheme.	Snowflake schema is a more complex data warehouse model than a star schema.
In star schema each of the dimensions is represented in a single table .It should not have any hierarchies between dims.	In snow flake schema at least one hierarchy should exists between dimension tables.
It contains a fact table surrounded by dimension tables. If the dimensions are de-normalized, we say it is a star schema design.	It contains a fact table surrounded by dimension tables. If a dimension is normalized, we say it is a snow flaked design.
In star schema only one join establishes the relationship between the fact table and any one of the dimension tables.	In snow flake schema since there is relationship between the dimensions tables it has to do many joins to fetch the data.
A star schema optimizes the performance by keeping queries simple and providing fast response time. All the information about the each level is stored in one row.	Snowflake schemas normalize dimensions to eliminated redundancy. The result is more complex queries and reduced query performance.
It is called a star schema because the diagram resembles a star.	It is called a snowflake schema because the diagram resembles a snowflake.

Fact constellation/ Galaxy schema

- Sophisticated applications may require multiple fact tables to *share* dimension tables
- It is a collection of multiple fact tables having some common dimension tables
- It can be viewed as a collection of several star schemas



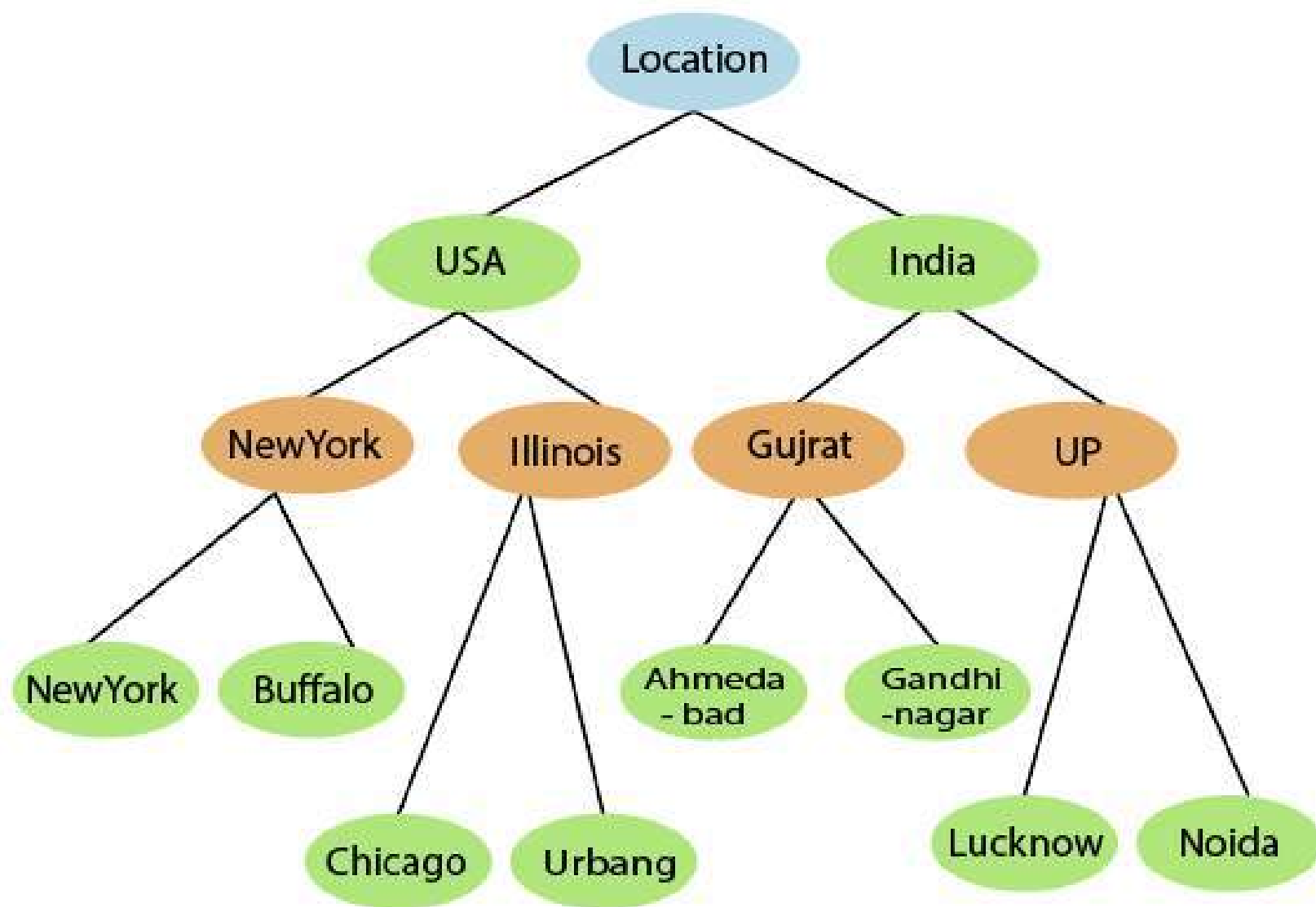
- Consider a Schema which specifies two fact tables, *sales* and *shipping*
- The *shipping* table has dimensions: *item*, *time*, *shipper*, *location* and two measures—*dollars cost* and *units shipped*
- The dimensions tables for *time*, *item*, and *location* are shared between the *sales* and *shipping* fact tables



Dimensions: The role of concept Hierarchies

- It defines a sequence of mappings from a set of low-level concepts to higher-level, more general concepts
- A **hierarchy** is an organizational structure in which items are ranked according to levels of importance
- Concept hierarchy formation: Concept Hierarchy recursively reduce the data by collecting and replacing low level concepts (such as numeric values for the attribute age) by higher level concepts (such as young, middle-aged, or senior)
- Can be explicitly specified by domain experts and/or datawarehouse designers

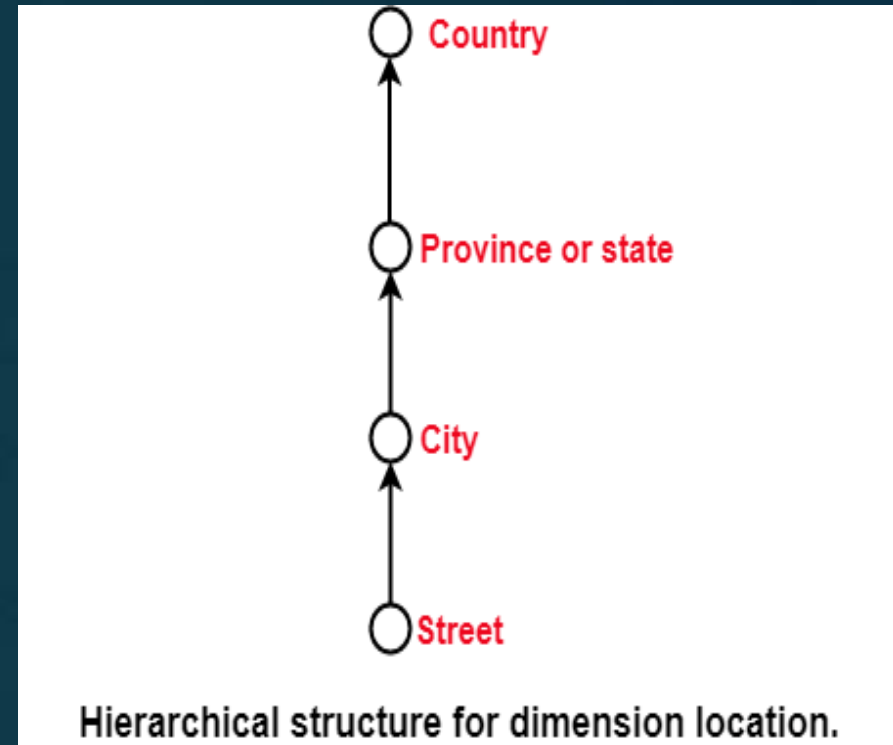
- Example: Consider a concept hierarchy for the dimension *location*
- City values for *location* can be mapped to the province or state to which it belongs
- The provinces and states can in turn be mapped to the country to which they belong
- Set of low-level concepts (i.e., cities) *are mapped to* higher-level, more general concepts (i.e., countries)



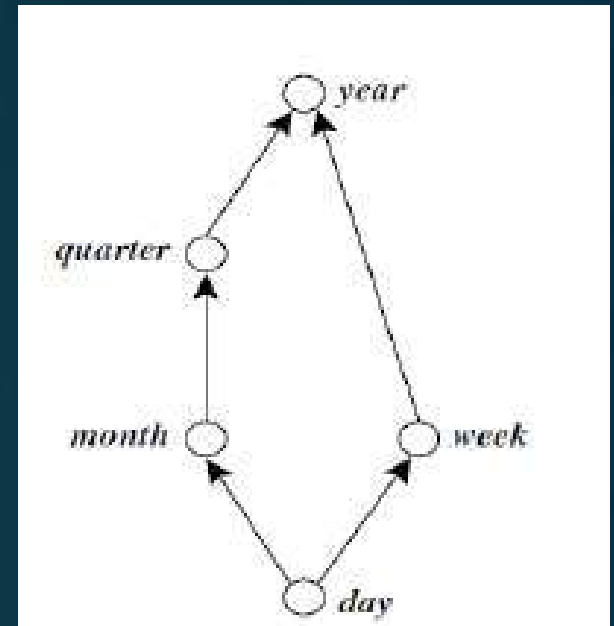
Concept hierarchy for the dimension location

Schema hierarchy

- Many concept hierarchies are implicit within the database schema
- This hierarchy may formally express semantic relation between attributes
- Example: dimension *location* is described by the attributes *number*, *street*, *city*, *province or state*, *zip code*, and *country*
- These attributes are related by a total order, forming a concept hierarchy such as “*street* < *city* < *province or state* < *country*” i.e street is conceptually lower than city which is lower than state which is lower than country

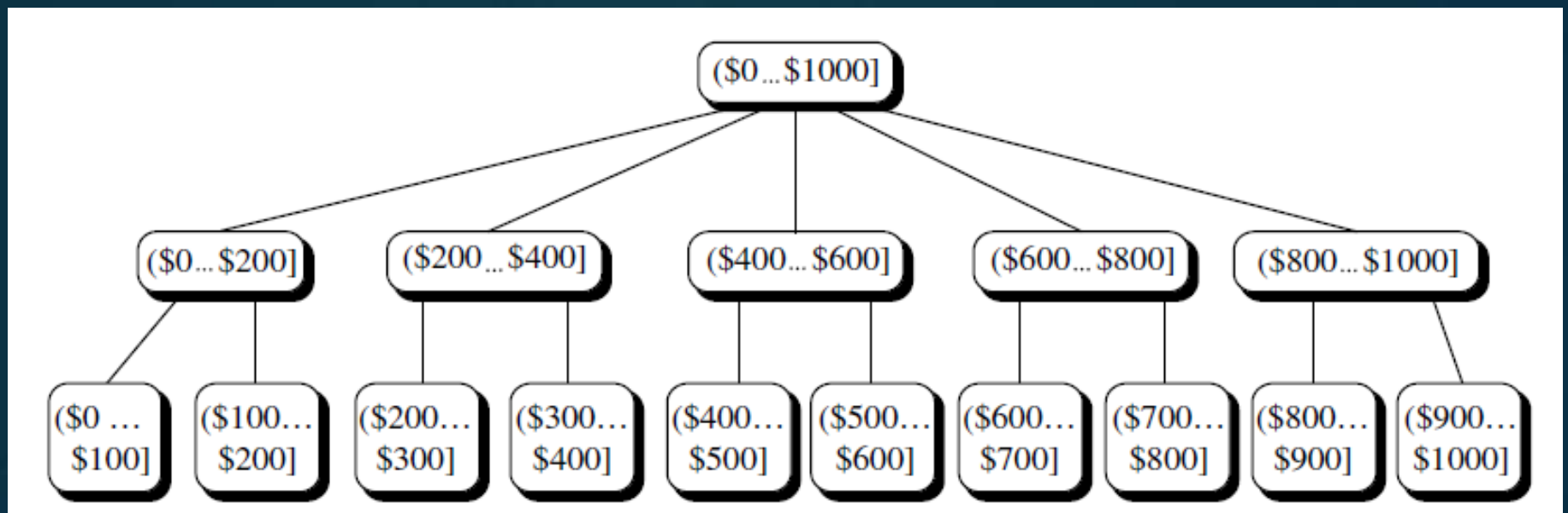


- The attributes of a dimension may be organized in a partial order, forming a lattice
- Partial order for the *time* dimension is described by the attributes *day*, *week*, *month*, *quarter*, and *year*
- These attributes are related by a Partial order, forming a concept hierarchy as
“*day* < {*month* < *quarter*; *week*} < *year*”
- A concept hierarchy that is a total or partial order among attributes in a database schema is called a **schema hierarchy**
- Concept hierarchies may be predefined in the data mining system

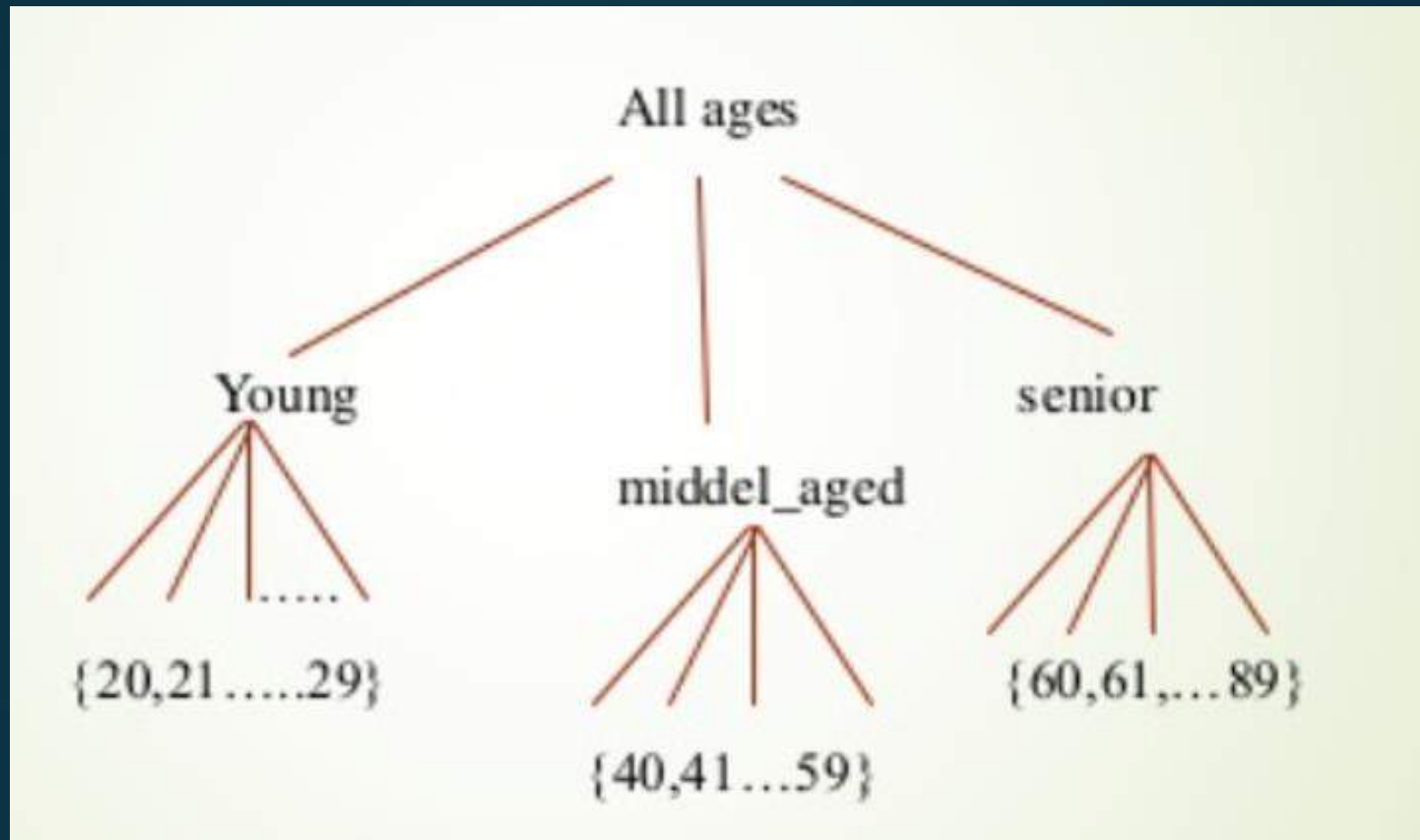


Set-grouping hierarchy

- Here, Concept hierarchies are defined by discretizing or grouping values for a given dimension or attribute
- A total or partial order can be defined among groups of values
- Example: for the dimension *price*, where an interval $(\$X \dots \$Y]$ denotes the range from $\$X$ (exclusive) to $\$Y$ (inclusive)



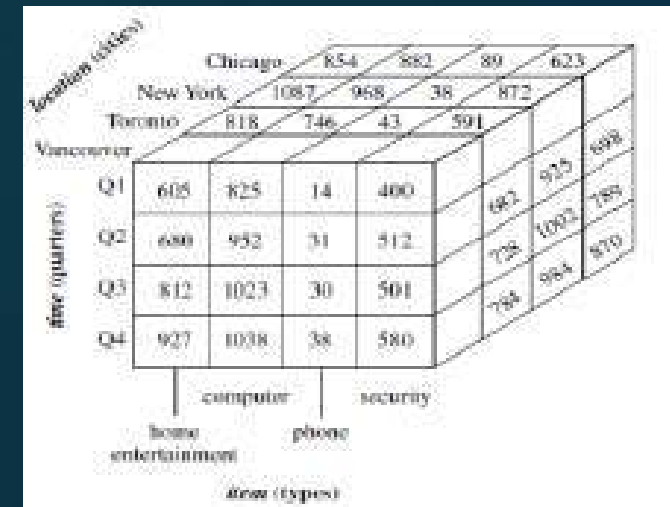
- Example2:



- There may be more than one concept hierarchy for a given attribute or dimension based on different user viewpoints
- Example: a user may prefer to organize *price* by defining ranges for *inexpensive*, *moderately priced*, and *expensive*
- Concept hierarchies may be provided manually by system users, domain experts, or knowledge engineers, or may be automatically generated based on statistical analysis of the data distribution
- Concept hierarchies allow data to be handled at varying levels of abstraction

Measures: Their Categorization and Computation

- *Multidimensional point* in the data cube space can be defined by a set of dimension–value pairs
 $\langle \text{time} = \text{"Q1"}, \text{location} = \text{"Vancouver"}, \text{item} = \text{"computer"} \rangle$
- A data cube **measure** is a numeric function that can be evaluated at each point in the data cube space
- A measure value is computed for a given point by aggregating the data corresponding to the respective dimension–value pairs defining the given point



- Measures can be organized into three categories based on the kind of aggregate functions used
 - Distributive
 - Algebraic
 - Holistic

Distributive Measures

- An aggregate function is *distributive* if it can be computed in a distributed manner
- Suppose the data are partitioned into n sets.
- We apply the function to each partition, resulting in n aggregate values.
- If the result derived by applying the function to the n aggregate values is the same as that derived by applying the function to the entire data set (without partitioning), the function can be computed in a distributed manner
- `sum()` can be computed for a data cube by first partitioning the cube into a set of subcubes, computing `sum()` for each subcube, and then summing up the counts obtained for each subcube
- A measure is *distributive* if it is obtained by applying a distributive aggregate function.
- Distributive measures can be computed efficiently because of the way the computation can be partitioned

Algebraic Measures

- An aggregate function is *algebraic* if it can be computed by an algebraic function with M arguments (where M is a bounded positive integer), each of which is obtained by applying a distributive aggregate function
- Example: `avg()` can be computed by `sum()/count()`, where both `sum()` and `count()` are distributive aggregate functions
- A measure is *algebraic* if it is obtained by applying an algebraic aggregate function

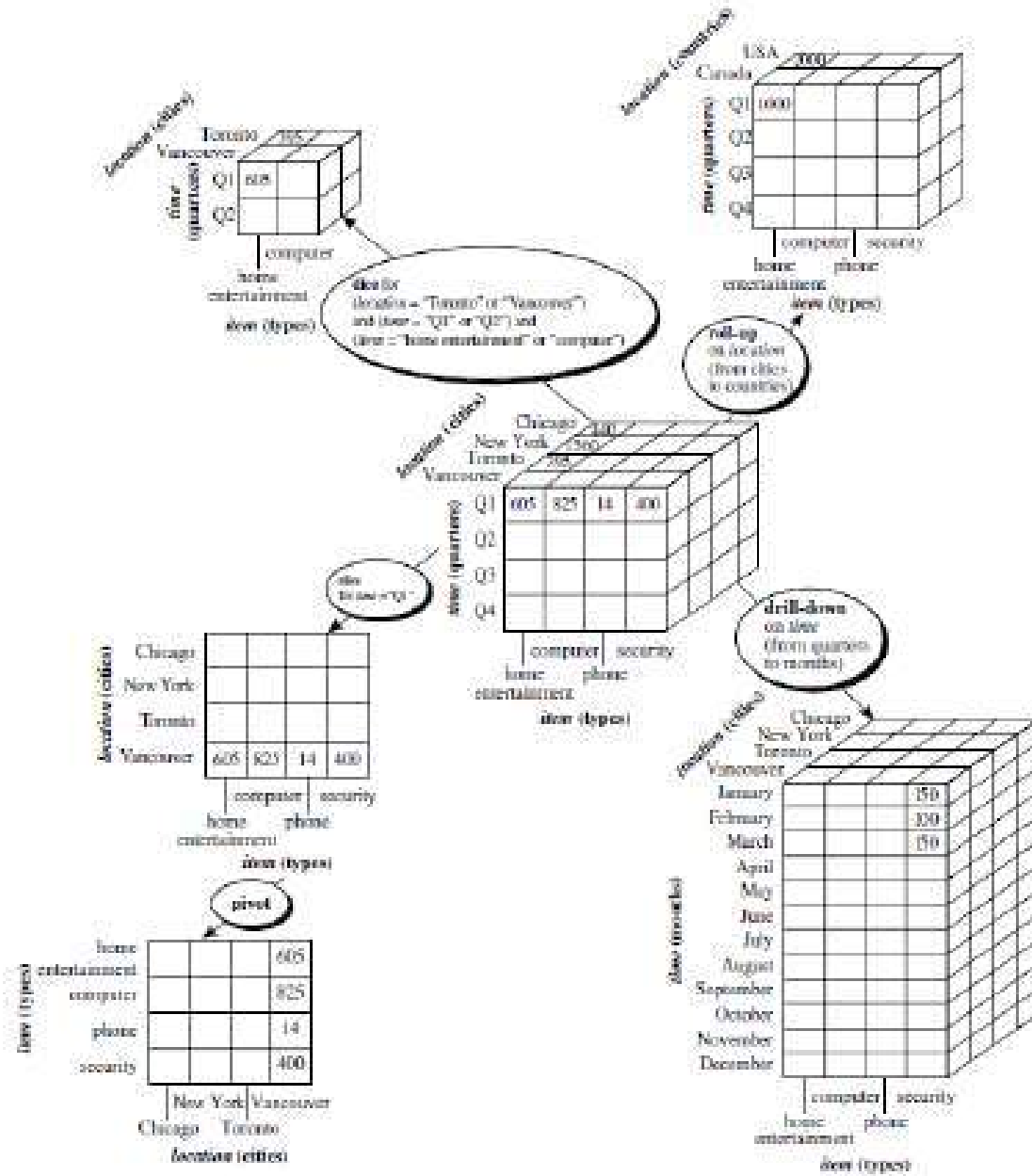
Holistic Measures

- An aggregate function is *holistic* if there is no constant bound on the storage size needed to describe a subaggregate.
- That is, there does not exist an algebraic function with M arguments (where M is a constant) that characterizes the computation
- Examples: median(), mode(), and rank() etc
- A measure is *holistic* if it is obtained by applying a holistic aggregate function

Typical OLAP Operations

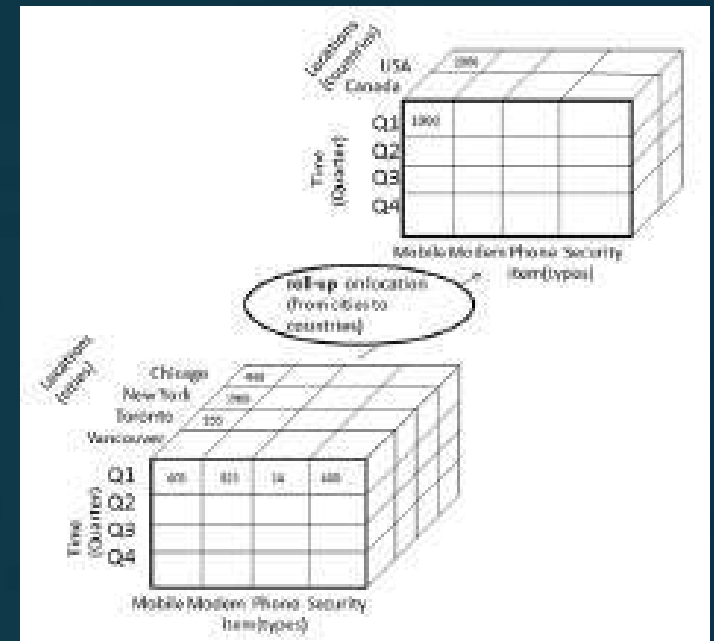
- In the multidimensional model, each dimension contains multiple levels of abstraction defined by concept hierarchies
- This organization provides users with the flexibility to view data from different perspectives
- A number of OLAP data cube operations exist to materialize these different views, allowing interactive querying and analysis of the data at hand

- Consider a data cube for *AllElectronics* sales
- Cube contains the dimensions: *location*, *time*, and *item*
- *location* is aggregated with respect to city values
- *time* is aggregated with respect to quarters
- *item* is aggregated with respect to item types
- Measure: *dollars sold* (in thousands).



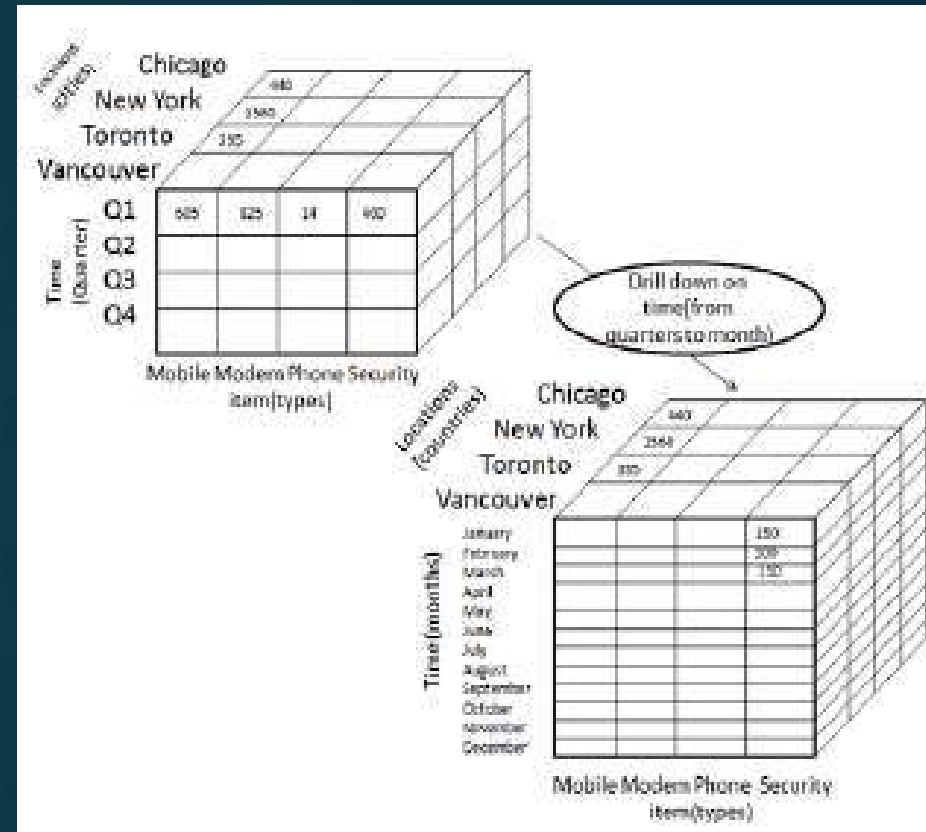
Roll-up/drill-up

- Performs aggregation on a data cube, either by **climbing up a concept hierarchy for a dimension** or by **dimension reduction**
- Concept hierarchy: total order “street < city < province or state < country”
- Roll-up operation aggregates the data by ascending the location hierarchy from the level of city to the level of country
- Rather than grouping the data by city, the resulting cube groups the data by country
- When roll-up is performed by dimension reduction, one or more dimensions are removed from the given cube
- Exmple: consider a sales data cube containing only the location and time dimensions
- Roll-up may be performed by removing, say, the time dimension, resulting in an aggregation of the total sales by location, rather than by location and by time



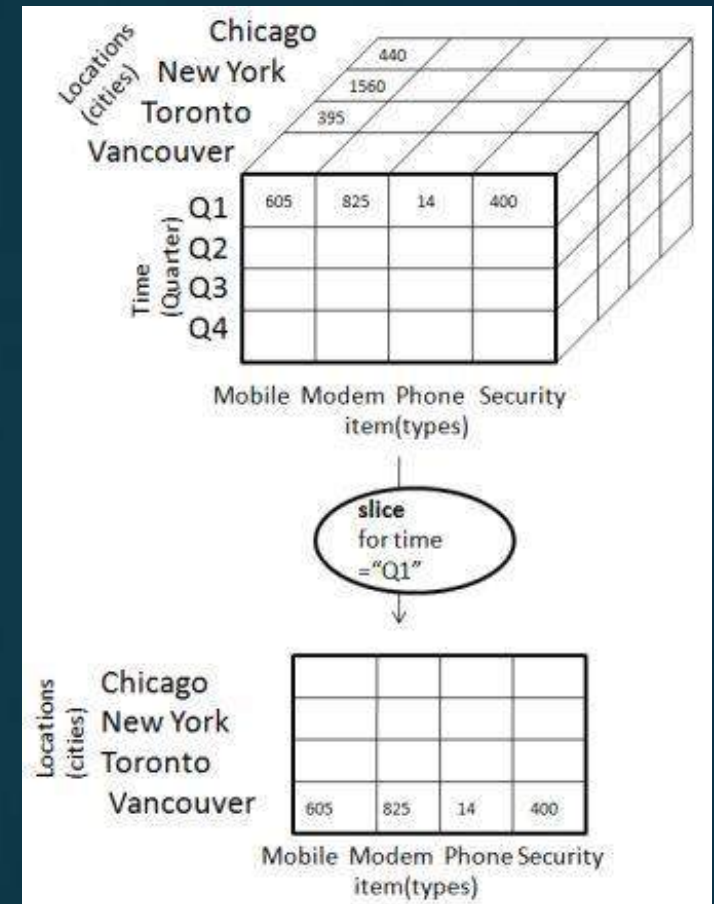
Drill-down

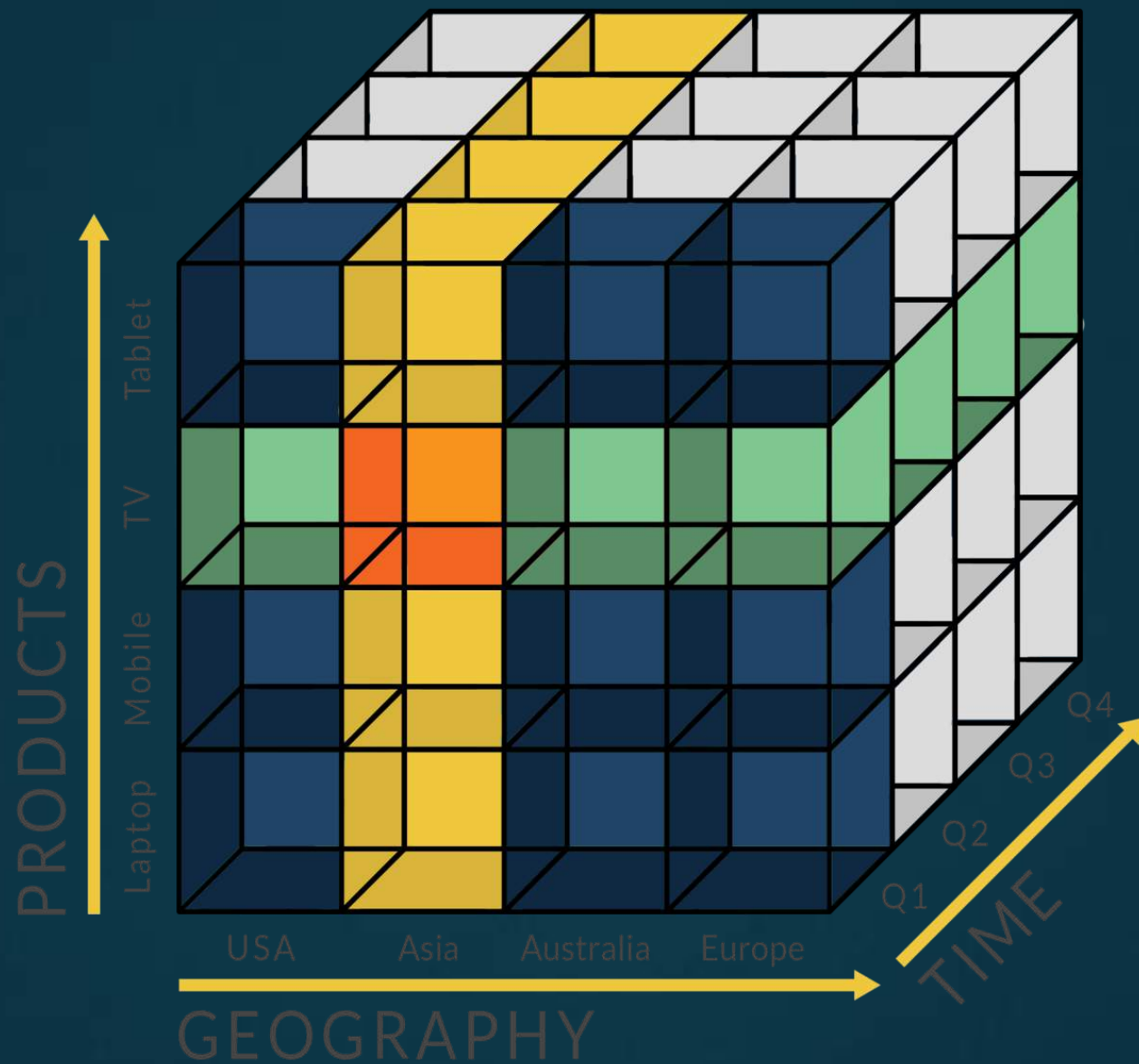
- Reverse of roll-up
- It navigates from less detailed data to more detailed data.
- Can be realized by either *stepping down a concept hierarchy for a dimension* or *introducing additional dimensions*
- Example: drill-down operation performed on the central cube by stepping down a concept hierarchy for *time* defined as
“*day < month < quarter < year.*”
- Drill-down occurs by descending the *time* hierarchy from the level of *quarter* to the more detailed level of *month*
- The resulting data cube details the total sales per month rather than summarizing them by quarter
- Because a drill-down adds more detail to the given data, it can also be performed by adding new dimensions to a cube



Slice

- This operation performs a selection on one dimension of the given cube, resulting in a subcube
- Sales data are selected from the central cube for the dimension *time* using the criterion *time* = “Q1”





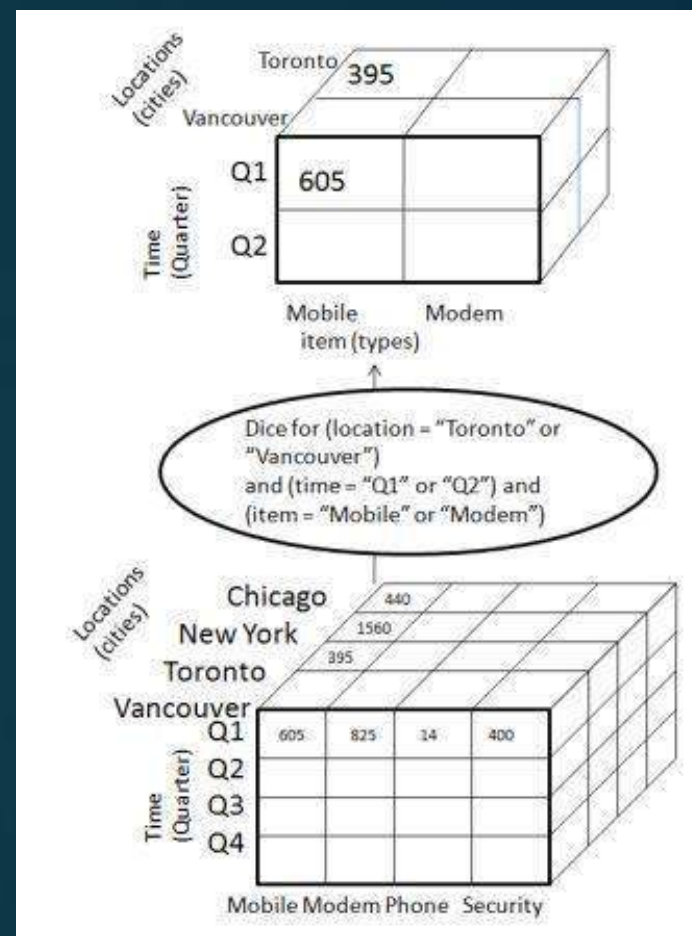
Dice

- This operation defines a subcube by performing a selection on two or more dimensions
- Dice operation on the central cube based on the following selection criteria that involve three dimensions:

(*location* = “Toronto” or “Vancouver”),

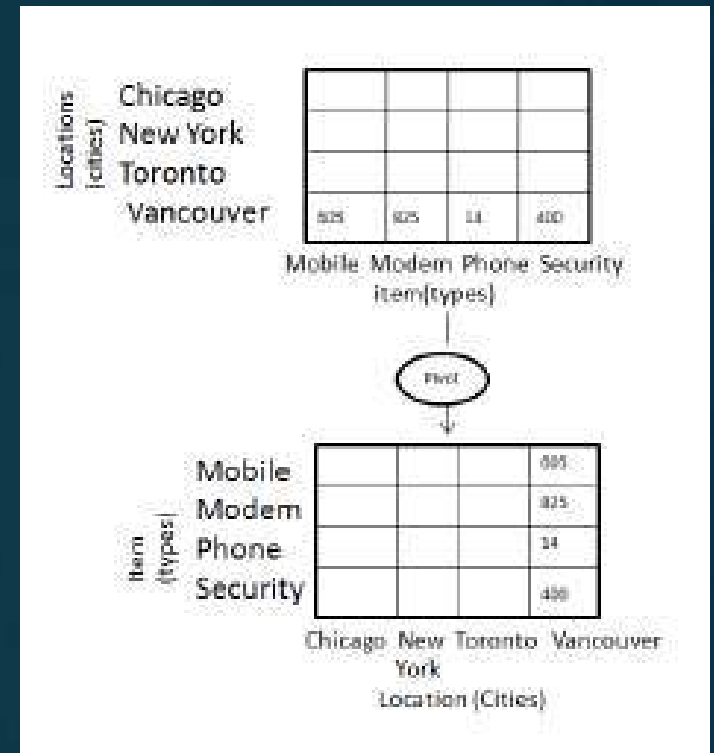
(*time* = “Q1” or “Q2”) and

(*item* = “home entertainment” or “computer”)



Pivot (rotate)

- It is a visualization operation that rotates the data axes in view to provide an alternative data presentation
- The *item* and *location* axes in a 2-D slice are rotated
- Examples: rotating the axes in a 3-D cube, transforming a 3-D cube into a series of 2-D planes



DATA WAREHOUSE IMPLEMENTATION

- Data warehouses contain huge volumes of data
- OLAP servers demand that decision support queries be answered in the order of seconds
- It is crucial for data warehouse systems to support
 - highly efficient cube computation techniques,
 - access methods, and
 - query processing techniques

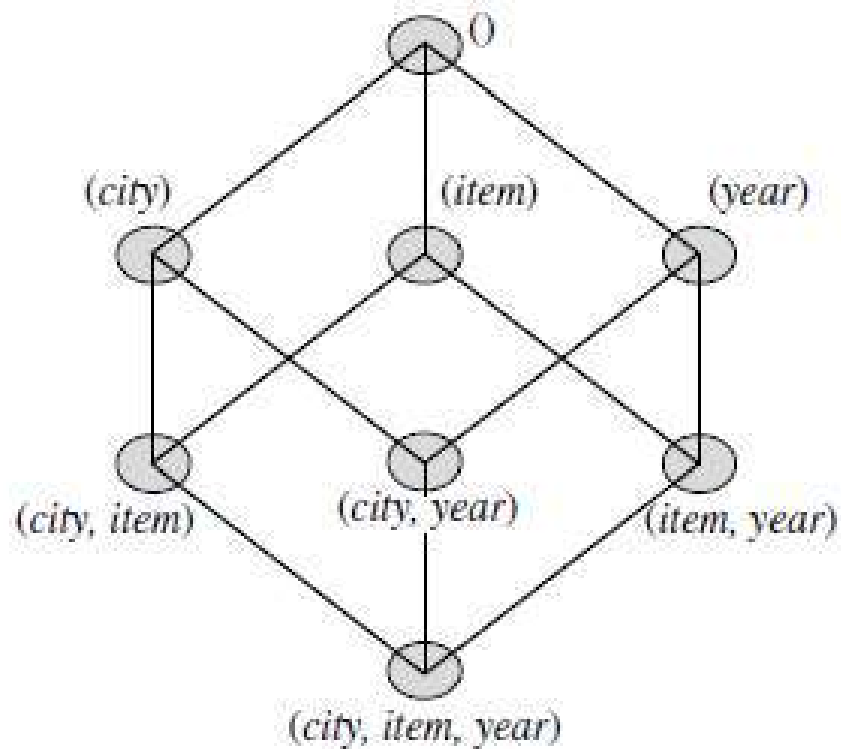
Efficient Data Cube Computation: An Overview

- At the core of multidimensional data analysis is the efficient computation of aggregations across many sets of dimensions
- In SQL terms, the aggregations are referred to as group-by's
- Each group-by can be represented by a *cuboid*, where the set of group-by's forms a lattice of cuboids defining a data cube

The compute cube Operator and the Curse of Dimensionality

- One approach to cube computation extends SQL so as to include a compute cube operator
- The compute cube operator computes aggregates over all subsets of the dimensions specified in the operation
- This can require excessive storage space, especially for large numbers of dimensions

- A data cube is a lattice of cuboids
- to create a data cube for *AllElectronics* sales that contains the *city*, *item*, *year*, and *sales in dollars*:
- Total number of cuboids, or groupby's, that can be computed for this data cube is $2^3 = 8$
- The possible group-by's are:
 $\{(city, item, year), (city, item), (city, year), (item, year), (city), (item), (year), ()\}$, where $()$ means that the group-by is empty (i.e., the dimensions are not grouped)



0-D (apex) cuboid

1-D cuboids

2-D cuboids

3-D (base) cuboid

Lattice of cuboids, making up a 3-D data cube. Each cuboid represents a different group-by. The base cuboid contains *city*, *item*, and *year* dimensions.

- The **base cuboid** contains all three dimensions, *city*, *item*, and *year*.
 - It can return the total sales for any combination of the three dimensions
 - The base cuboid is the least generalized (most specific) of the cuboids
-
- The **apex cuboid**, or 0-D cuboid, refers to the case where the group-by is empty
 - It contains the total sum of all sales
 - The Apex cuboid is the most generalized (least specific) of the cuboids, and is often denoted as all
-
- If we start at the apex cuboid and explore downward in the lattice, this is equivalent to drilling down within the data cube
 - If we start at the base cuboid and explore upward, this is akin to rolling up.

- An SQL query containing no group-by is a *zero dimensional operation*
 - e.g., “*compute the sum of total sales*”
- An SQL query containing one group-by is a *one-dimensional operation*
 - e.g., “*compute the sum of sales, group-by city*”
- A cube operator on n dimensions is equivalent to a collection of group-by statements, one for each subset of the n dimensions
- cube operator is the n -dimensional generalization of the group-by operator

- Data cube definition:
- `define cube sales cube [city, item, year]: sum(sales in dollars)`
- For a cube with n dimensions, there are a total of 2^n cuboids, including the base cuboid

`compute cube sales_cube`

- Statement explicitly instruct the system to compute the sales aggregate cuboids for all eight subsets of the set $\{city, item, year\}$, including the empty subset

- Online analytical processing may need to access different cuboids for different queries
- It is good idea to compute in advance all or at least some of the cuboids in a data cube
- Precomputation leads to fast response time and avoids some redundant computation
- **Challenge:** required storage space may explode if all the cuboids in a data cube are precomputed, especially when the cube has many dimensions
- Storage requirements are even more excessive when many of the dimensions have associated concept hierarchies, each with multiple levels
- This problem is referred to as the **curse of dimensionality**

No. of cuboids in an n-dimensional data cube:

- If there are no hierarchies associated with each dimension then it is 2^n
- If hierarchies are associated with each dimension, (e.g: *time: day < month < quarter < year*)

$$\text{Total number of cuboids} = \prod_{i=1}^n (L_i + 1)$$

- where L_i is the number of levels associated with dimension i .
- 1 is added to L_i to include the *virtual* top level, all

- Example:
- time dimension has five levels (including all)
- If the cube has 10 dimensions and each dimension has five levels (including all),
- The total number of cuboids that can be generated is 5^{10}

- It is unrealistic to precompute and materialize all of the cuboids that can possibly be generated for a data
- If there are many cuboids, and these cuboids are large in size, a more reasonable option is *partial materialization*
 - to materialize only *some* of the possible cuboids that can be generated

Partial Materialization: Selected Computation of Cuboids

There are three choices for data cube materialization given a base cuboid:

No materialization:

- Do not precompute any of the cuboids
- This leads to computing expensive multidimensional aggregates on-the-fly, which can be extremely slow

Full materialization:

- Precompute all of the cuboids
- The resulting lattice of computed cuboids is referred to as the *full cube*.
- This choice typically requires huge amounts of memory space in order to store all of the precomputed cuboids

Partial materialization:

- Selectively compute a proper subset of the whole set of possible Cuboids
- Alternatively, we may compute a subset of the cube, which contains only those cells that satisfy some user-specified criterion, such as where the tuple count of each cell is above some threshold
- Represents an interesting trade-off between storage space and response time

Partial materialization of cuboids should consider three factors:

- Identify the subset of cuboids or subcubes to materialize
- Exploit the materialized cuboids or subcubes during query processing
- Efficiently update the materialized cuboids or subcubes during load and refresh.

Indexing OLAP Data

- To facilitate efficient data accessing, most data warehouse systems support index structures
- OLAP data can be indexed using
 - **Bitmap Index**
 - **Join Index**

Bitmap Index

- Popular method because it allows quick searching in data cubes
- It is an alternative representation of the *record ID (RID)* list
- In the bitmap index for a given attribute, there is a distinct bit vector, B_v , for each value v in the attribute's domain
- If a given attribute's domain consists of n values, then n bits are needed for each entry in the bitmap index
- If the attribute has the value v for a given row in the data table, then the bit representing that value is set to 1 in the corresponding row of the bitmap index. All other bits for that row are set to 0

- **Example:**
- Consider AllElectronics data warehouse
- Let dimension *item* at the top level has four values- “home entertainment,” “computer,” “phone,” and “security.”
- Each value (e.g., “computer”) is represented by a bit vector in the *item* bitmap index table
- If the cube is stored as a relation table with 100,000 rows, because the domain of *item* consists of four values, the bitmap index table requires four bit vectors (or lists), each with 100,000 bits
- The diagram shows a base (data) table containing the dimensions *item* and *city*, and its mapping to bitmap index tables for each of the dimensions

Base table

<i>RID</i>	<i>item</i>	<i>city</i>
R1	H	V
R2	C	V
R3	P	V
R4	S	V
R5	H	T
R6	C	T
R7	P	T
R8	S	T

item bitmap index table

<i>RID</i>	H	C	P	S
R1	1	0	0	0
R2	0	1	0	0
R3	0	0	1	0
R4	0	0	0	1
R5	1	0	0	0
R6	0	1	0	0
R7	0	0	1	0
R8	0	0	0	1

city bitmap index table

<i>RID</i>	V	T
R1	1	0
R2	1	0
R3	1	0
R4	1	0
R5	0	1
R6	0	1
R7	0	1
R8	0	1

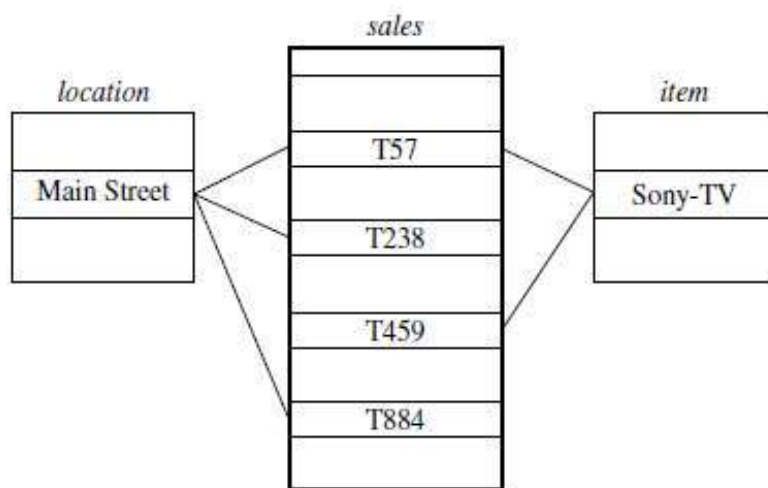
Note: H for “home entertainment,” C for “computer,” P for “phone,” S for “security,”
V for “Vancouver,” T for “Toronto.”

- It is especially useful for low-cardinality domains
- Comparison, join, and aggregation operations are reduced to bit arithmetic, which substantially reduces the processing time
- leads to significant reductions in space and input/output (I/O) since a string of characters can be represented by a single bit
- For higher-cardinality domains, the method can be adapted using compression techniques

Join index

- Gained popularity from its use in relational database query processing
- Join indexing registers the joinable rows of two relations from a relational database
- If two relations $R(RID, A)$ and $S(B, SID)$ join on the attributes A and B , then the join index record contains the pair (RID, SID) , where RID and SID are record identifiers from the R and S relations, respectively
- The join index records can identify joinable tuples without performing costly join operations
- It is especially useful for maintaining the relationship between a foreign key and its matching primary keys, from the joinable relation
- In data warehouses, join index relates the values of the dimensions of a star schema to rows in the fact table
- Join indices may span multiple dimensions to form **composite join indices**

- Consider a star schema for *AllElectronics* of the form
- “*sales_star* [*time*, *item*, *branch*, *location*]: dollars sold D sum (*sales in dollars*).”
- Example for join index relationship between the *sales* fact table and the *location* and *item* dimension tables
- Example: the “Main Street” value in the *location* dimension table joins with tuples T57, T238, and T884 of the *sales* fact table
- Similarly, the “Sony-TV” value in the *item* dimension table joins with tuples T57 and T459 of the *sales* fact table
- To further speed up query processing, the join indexing and the bitmap indexing methods can be integrated to form **bitmapped join indices**



Linkages between a *sales* fact table and *location* and *item* dimension tables.

Join index table for
location/sales

<i>location</i>	<i>sales_key</i>
...	...
Main Street	T57
Main Street	T238
Main Street	T884
...	...

Join index table for
item/sales

<i>item</i>	<i>sales_key</i>
...	...
Sony-TV	T57
Sony-TV	T459
...	...

Join index table linking
location and *item* to *sales*

<i>location</i>	<i>item</i>	<i>sales_key</i>
...	...	...
Main Street	Sony-TV	T57
...	...	...

Join index tables based on the linkages between the *sales* fact table and the *location* and *item*

Efficient Processing of OLAP Queries

- The purpose of materializing cuboids and constructing OLAP index structures is to speed up query processing in data cubes
- Given materialized views, query processing should proceed as follows:
- **Determine which operations should be performed on the available cuboids**
 - Involves transforming any selection, projection, roll-up (group-by), and drill-down operations specified in the query into corresponding SQL and/or OLAP operations
- **Determine to which materialized cuboid(s) the relevant operations should be applied**
 - Involves identifying all of the materialized cuboids that may potentially be used to answer the query, pruning the set using knowledge of “dominance” relationships among the cuboids, estimating the costs of using the remaining materialized cuboids, and selecting the cuboid with the least cost

- Consider a data cube for *AllElectronics* of the form
“*sales cube [time, item, location]: sum(sales in dollars).*”
- dimension hierarchies:
 - *day < month < quarter < year* for time
 - *item name < brand < type* for item;
 - *street < city < province or state < country* for location
- Query to be processed is on $\{brand, province_or_state\}$, with the selection constant “*year = 2010.*”
- cuboid 1: $\{year, item\ name, city\}$
- cuboid 2: $\{year, brand, country\}$
- cuboid 3: $\{year, brand, province\ or\ state\}$
- cuboid 4: $\{item\ name, province\ or\ state\}$, where *year = 2010*
- “Which of these four cuboids should be selected to process the query?”

- Cuboid 2 cannot be used because *country* is a more general concept than *province or state*
- Cuboids 1, 3, and 4 can be used to process the query because
- They have the same set or a superset of the dimensions in the query,
- The selection clause in the query can imply the selection in the cuboid, and
- The abstraction levels for the *item* and *location* dimensions in these cuboids are at a finer level than *brand* and *province or state*, respectively

- Cost Factors:
- It is likely that using cuboid 1 would cost the most because both *item name* and *city* are at a lower level than the *brand* and *province or state* concepts specified in the query.
- If there are not many *year* values associated with *items* in the cube, but there are several *item names* for each *brand*, then cuboid 3 will be smaller than cuboid 4, and thus cuboid 3 should be chosen to process the query.
- However, if efficient indices are available for cuboid 4, then cuboid 4 may be a better choice. Therefore, some cost-based estimation is required to decide which set of cuboids should be selected for query processing

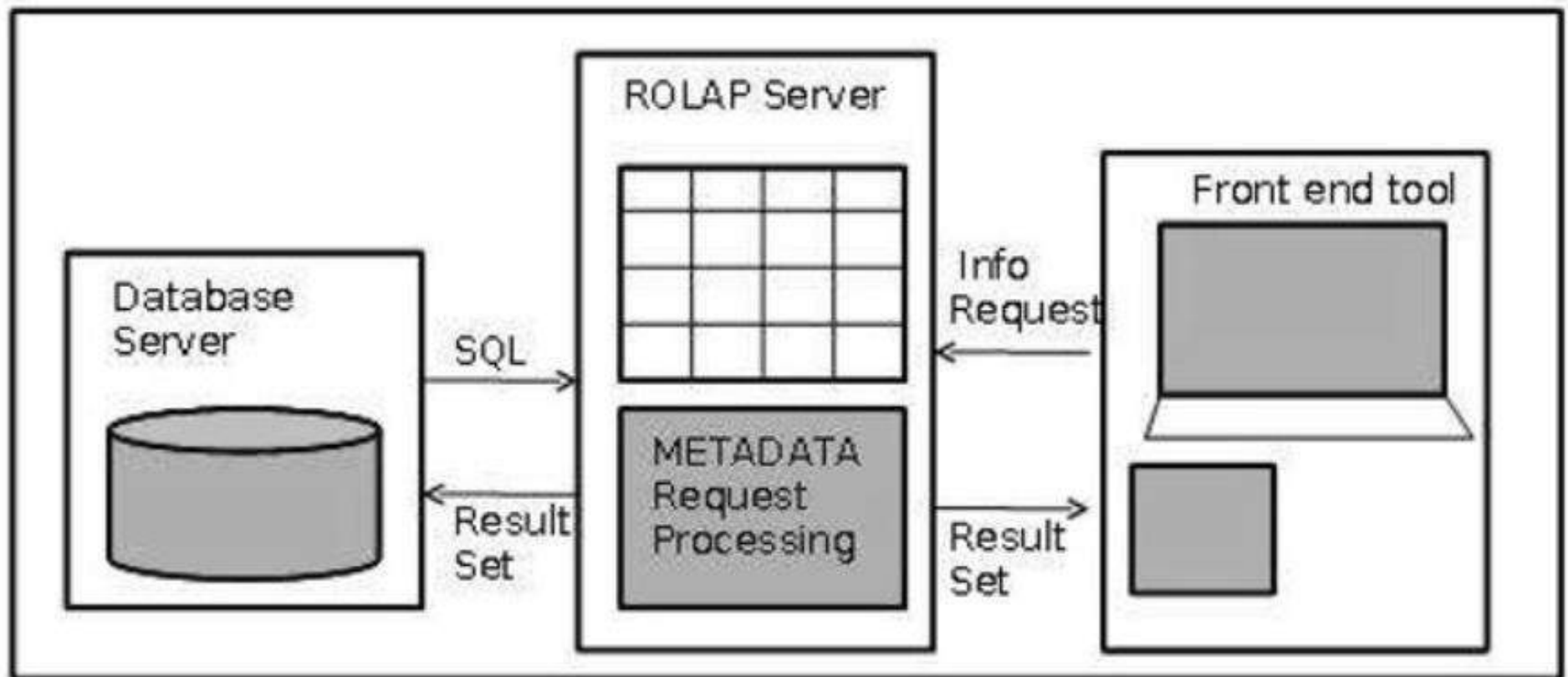
OLAP Server Architectures

- OLAP servers present business users with multidimensional data from data warehouses or data marts, without concerns regarding how or where the data are stored
- However, the physical architecture and implementation of OLAP servers must consider data storage issues
- Implementations of a warehouse server for OLAP processing include the following architectures:
 - Relational OLAP (ROLAP) servers
 - Multidimensional OLAP (MOLAP) servers
 - Hybrid OLAP (HOLAP) servers

Relational OLAP (ROLAP) servers

- Relational OLAP servers are placed between relational back-end server and client front-end tools
- To store and manage the warehouse data, the relational OLAP uses relational or extended-relational DBMS
- ROLAP includes the following :
 - Implementation of aggregation navigation logic
 - Optimization for each DBMS back-end
 - Additional tools and services
- A ROLAP database can be accessed through complex SQL queries to calculate information

- **Advantages:**
- ROLAP servers can be easily used with existing RDBMS
- ROLAP can handle large data volumes, but the larger the data, the slower the processing times
- Because queries are made on-demand, ROLAP does not require the storage and pre-computation of information
- **Disadvantages:**
- Potential performance constraints and scalability limitations that result from large and inefficient join operations between large tables



A ROLAP data store

- ROLAP stores data in columns and rows and retrieves the information on demand through user submitted queries
- Consider a summary fact table that contains both base fact data and aggregated data
- The schema is “*<record identifier (RID), item, ..., day, month, quarter, year, dollars sold>*”

Single Table for Base and Summary Facts

<i>RID</i>	<i>item</i>	<i>...</i>	<i>day</i>	<i>month</i>	<i>quarter</i>	<i>year</i>	<i>dollars_sold</i>
1001	TV	...	15	10	Q4	2010	250.60
1002	TV	...	23	10	Q4	2010	175.00
...	...	...	...	...	...	...	...
5001	TV	...	all	10	Q4	2010	45,786.08
...	...	...	...	...	...	...	...

- Tuples with an *RID* of 1001 and 1002 data are at the base fact level, where the sales dates are October 15, 2010, and October 23, 2010, respectively
- Tuple with an *RID* of 5001 is at a more general level of abstraction
- *day* value has been generalized to all, so that the corresponding *time* value is October 2010 i.e. the *dollars sold* amount shown is an aggregation representing the entire month of October 2010, rather than just October 15 or 23, 2010.
- The special value all is used to represent subtotals in summarized data

Multidimensional OLAP (MOLAP) servers

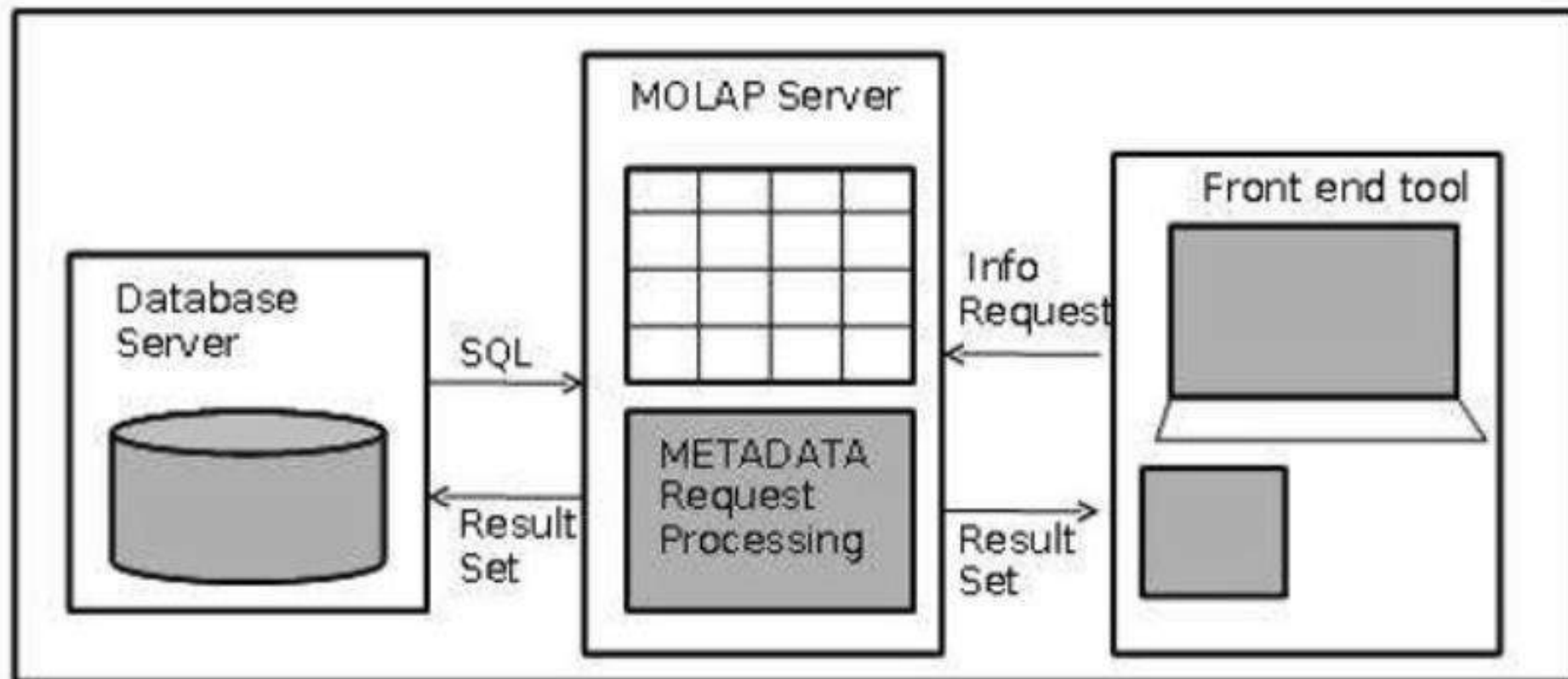
- MOLAP uses array-based multidimensional storage engines for multidimensional views of data
- They map multidimensional views directly to data cube array structures
- With multidimensional data stores, the storage utilization may be low if the dataset is sparse
- Therefore, many MOLAP servers use two levels of data storage representation to handle dense and sparse datasets
- Denser subcubes are identified and stored as array structures, whereas sparse subcubes employ compression technology for efficient storage utilization

Advantages:

- Data is pre-computed, pre-summarized, and stored
- Its simple interface makes MOLAP easy to use, even for inexperienced users
- Its speedy data retrieval makes it the best for “slicing and dicing” operations

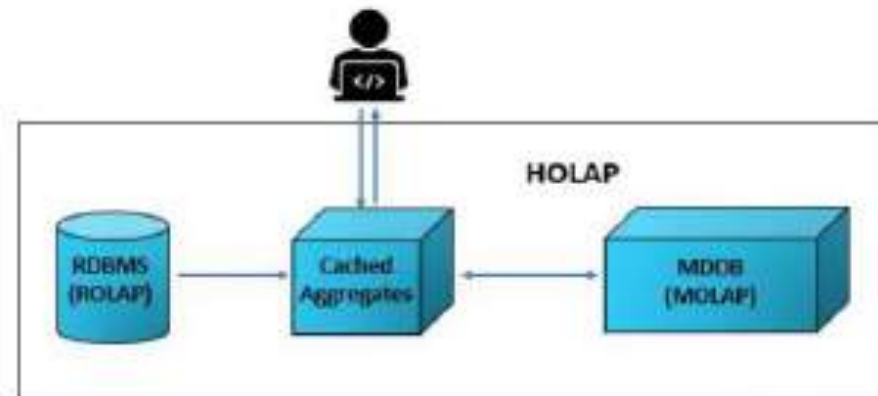
Disadvantages:

- Less scalable than ROLAP, as it can handle a limited amount of data
- Storage utilization may be low if the data set is sparse



Hybrid OLAP (HOLAP) servers

- Combines ROLAP and MOLAP technology
- involves storing part of data in a ROLAP store and another part in a MOLAP store, thus developers get the benefits of both
- Gets benefited from the greater scalability of ROLAP and the faster computation of MOLAP
- HOLAP server may allow large volumes of detailed data to be stored in a relational database, while aggregations are kept in a separate MOLAP store
- This setup allows much more flexibility for handling data



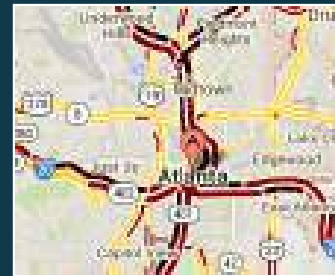
DATA MINING INTRODUCTION

Large-scale Data is Everywhere!

- There has been enormous data growth in both commercial and scientific databases due to advances in data generation and collection technologies
- New mantra
 - Gather whatever data you can whenever and wherever possible.
- Expectations
 - Gathered data will have value either for the purpose collected or for a purpose not envisioned.



E-Commerce



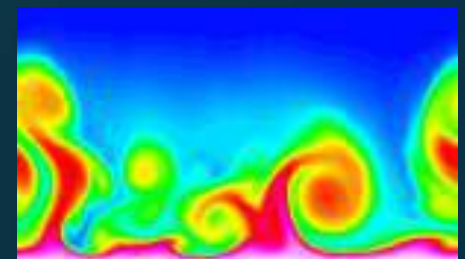
Traffic Patterns



Social Networking: Twitter



Sensor Networks



Computational Simulations

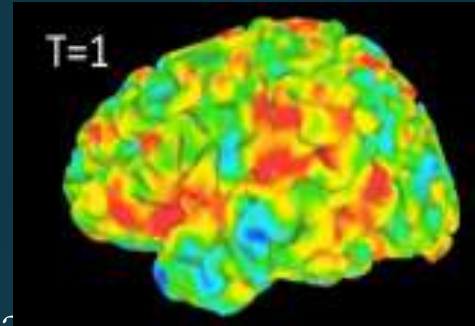
Why Data Mining? Commercial Viewpoint

- Lots of data is being collected and warehoused
 - Web data
 - Yahoo has Peta Bytes of web data
 - Facebook has billions of active users
 - purchases at department/grocery stores, e-commerce
 - Amazon handles millions of visits/day
 - Bank/Credit Card transactions
- Computers have become cheaper and more powerful
- Competitive Pressure is Strong
 - Provide better, customized services for an edge (e.g. in Customer Relationship Management)



Why Data Mining? Scientific Viewpoint

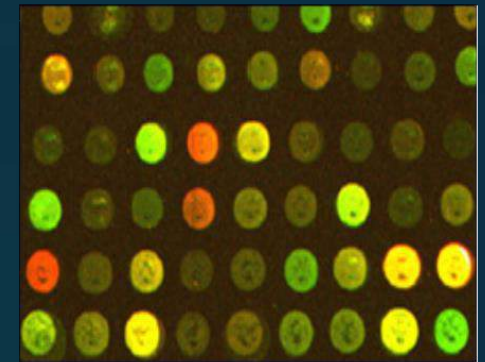
- Data collected and stored at enormous speeds
 - remote sensors on a satellite
 - NASA EOSDIS archives over petabytes of earth science data / year
 - telescopes scanning the skies
 - Sky survey data
 - High-throughput biological data
 - Scientific simulations
 - terabytes of data generated in a few hours
- Data mining helps scientists
 - in automated analysis of massive datasets
 - in hypothesis formation



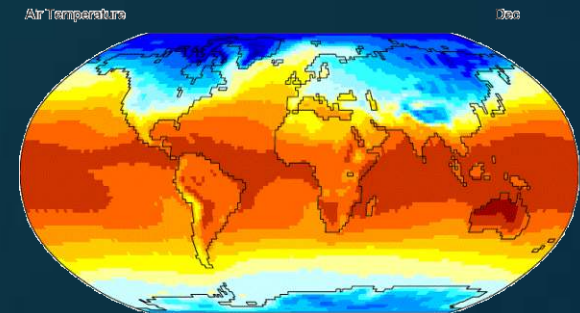
fMRI Data from Brain



Sky Survey Data



Gene Expression Data

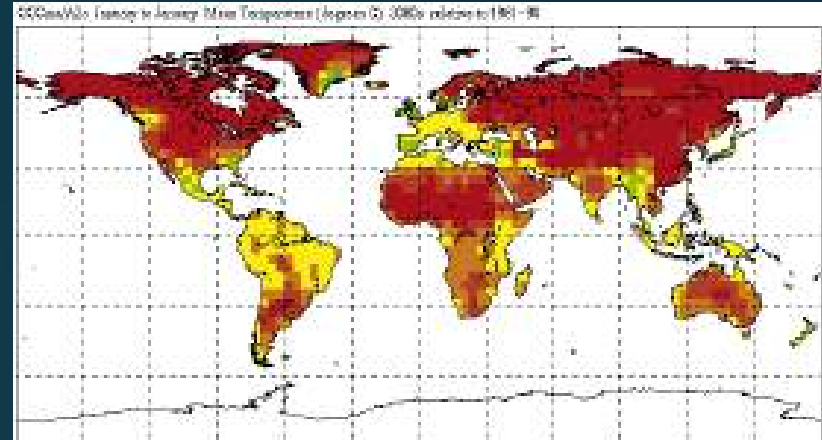


Surface Temperature of Earth

Great Opportunities to Solve Society's Major Problems



Improving health care and reducing costs



Predicting the impact of climate change



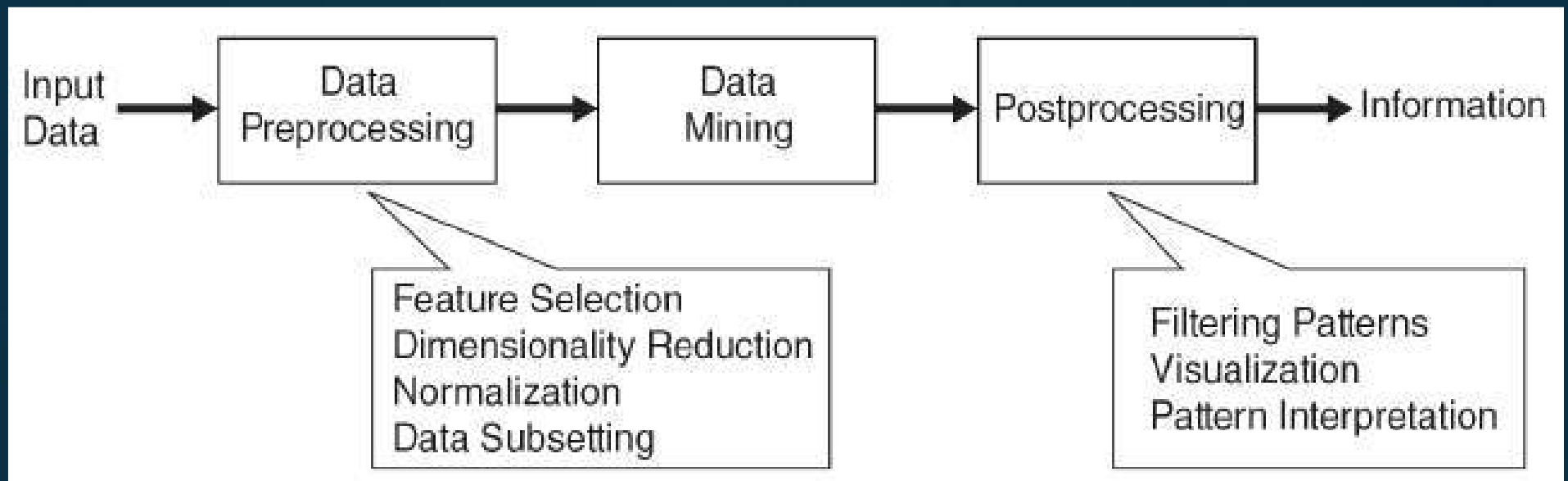
Finding alternative/ green energy sources



Reducing hunger and poverty by increasing agriculture production

What is Data Mining?

- Non-trivial extraction of implicit, previously unknown and potentially useful information from data
- Exploration & analysis, by automatic or semi-automatic means, of large quantities of data in order to discover meaningful patterns



What is (not) Data Mining?

□ What is not Data Mining?

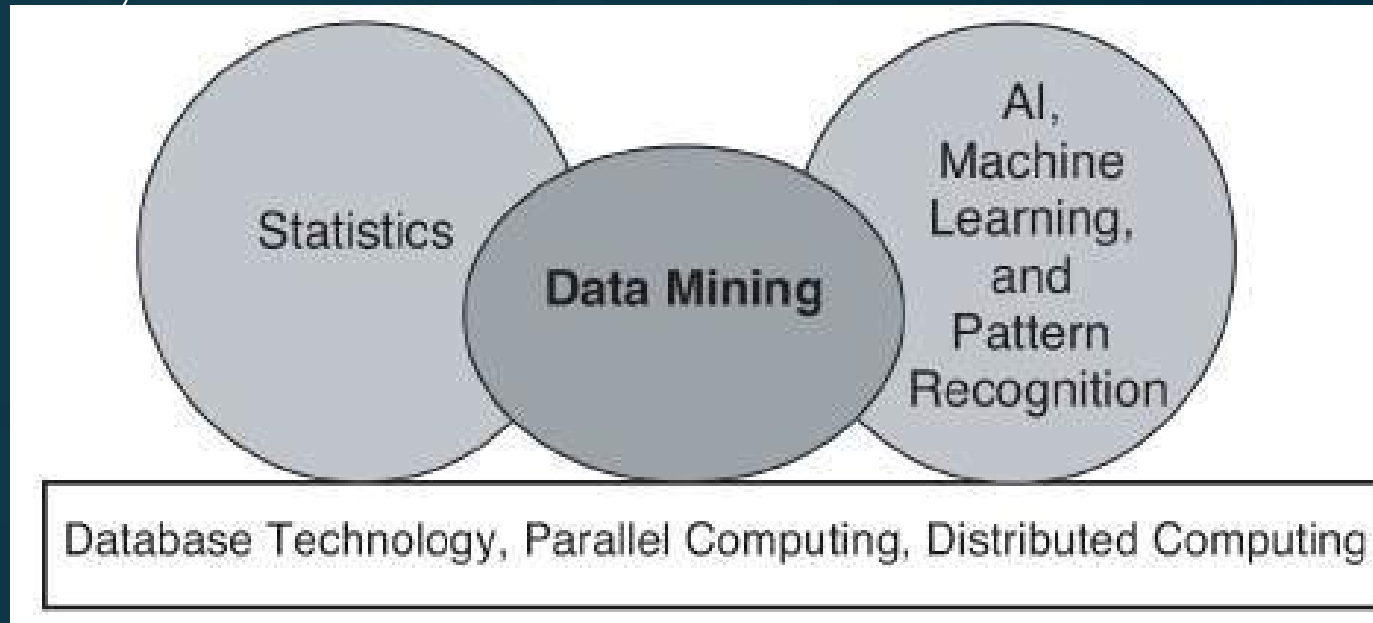
- Look up phone number in phone directory
- Query a Web search engine for information about “Amazon”

□ What is Data Mining?

- Certain names are more prevalent in certain US locations (O’Brien, O’Rourke, O’Reilly... in Boston area)
- Group together similar documents returned by search engine according to their context (e.g., Amazon rainforest, Amazon.com)

Origins of Data Mining

- Draws ideas from machine learning/AI, pattern recognition, statistics, and database systems
- Traditional techniques may be unsuitable due to data that is
 - Large-scale
 - High dimensional
 - Heterogeneous
 - Complex
 - Distributed
- A key component of the emerging field of data science and data-driven discovery



Motivating Challenges

- Scalability
- High Dimensionality
- Heterogeneous and Complex Data
- Data Ownership and Distribution
- Non-traditional Analysis

Scalability

- Data sets with sizes of gigabytes, terabytes, or even petabytes are becoming common
- Data mining algorithms are scalable
- DM algorithms employ special search strategies to handle exponential search problems
- Implements novel data structures to access individual records in an efficient manner
- Can be improved by using sampling or developing parallel and distributed algorithms

High Dimensionality

- Data sets with large number of attributes are common
- E.g: In bioinformatics, progress in microarray technology has produced gene expression data involving thousands of features
- Data sets with temporal or spatial components also tend to have high dimensionality

Heterogeneous and Complex Data

- Traditional data analysis methods often deal with data sets containing attributes of the same type, either continuous or categorical
- But need for techniques that can handle heterogeneous attributes has grown
- Emergence of more complex data objects
- Examples non-traditional types of data include
 - collections of Web pages containing semi-structured text and hyperlinks;
 - DNA data with sequential and three-dimensional structure
 - climate data

Data ownership and Distribution

- Data is geographically distributed among resources belonging to multiple entities
- Distributed data mining algorithms challenges:
 - how to reduce the amount of communication needed to perform the distributed computation
 - how to effectively consolidate the data mining results obtained from multiple source
 - how to address data security issues

Non-traditional Analysis

- Hypothesis is proposed, an experiment is designed to gather the data, and then the data is analyzed with respect to the hypothesis- labor intensive
- Current data analysis tasks often require the generation and evaluation of thousands of hypotheses
- Development of some data mining techniques has been motivated by the desire to automate the process of hypothesis generation and evaluation
- Data sets analyzed in data mining are typically not the result of a carefully designed experiment
- Often represent opportunistic samples of the data
- Data sets frequently involve non-traditional types of data and data distributions

Data Mining Tasks

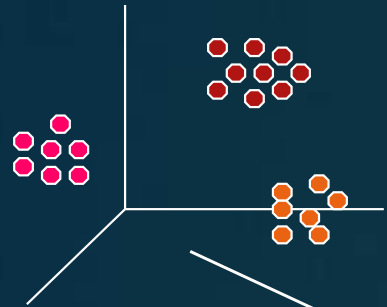
► Predictive Methods

- The objective of these tasks is to predict the value of a particular attribute based on the values of other attributes
- Use some variables to predict unknown or future values of other variables
- The attribute to be predicted is commonly known as the target or dependent variable
- The attributes used for making the prediction are known as the explanatory or independent variables

► Descriptive Methods

- objective is to derive patterns (correlations, trends, clusters, trajectories, and anomalies) that summarize the underlying relationships in data
- Often exploratory in nature and frequently require postprocessing techniques to validate and explain the results
- Find human-interpretable patterns that describe the data

Data Mining Tasks

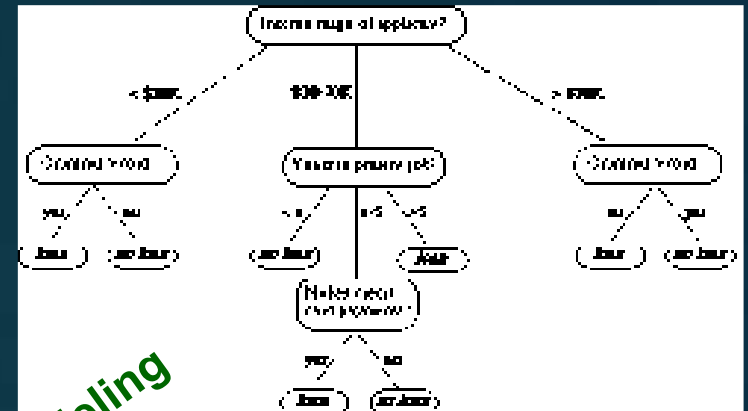


Clustering

Data

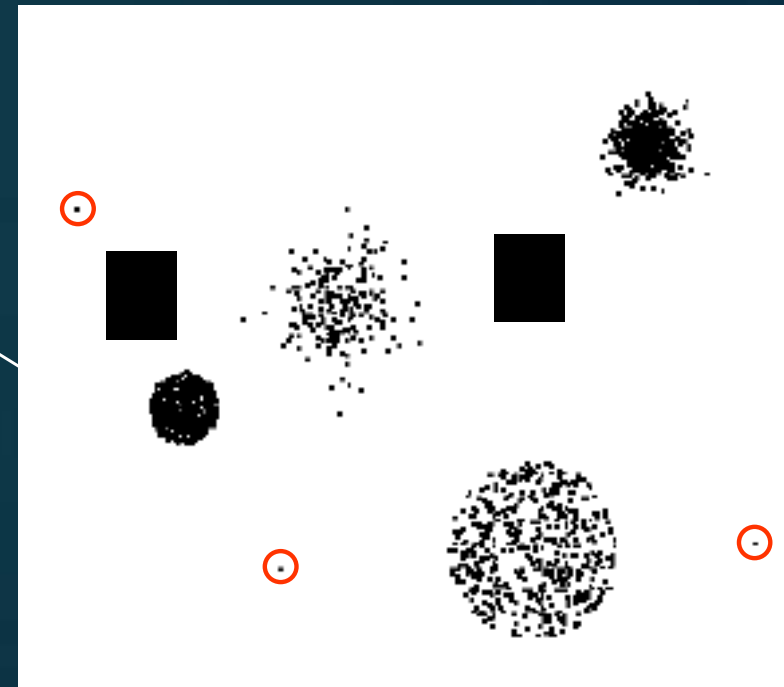
Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes
11	No	Married	60K	No
12	Yes	Divorced	220K	No
13	No	Single	85K	Yes
14	No	Married	75K	No
15	No	Single	90K	Yes

Association Rules



Predictive Modeling

Anomaly Detection



Predictive Modeling

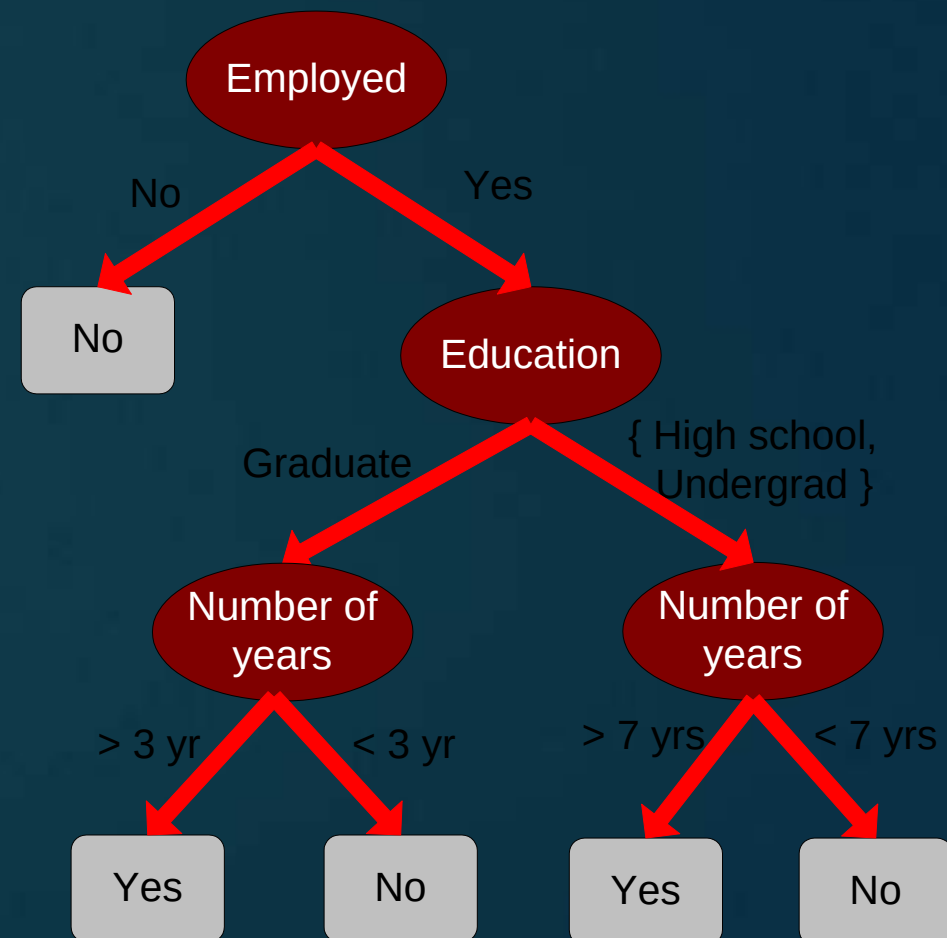
- ▶ Task of building a model for the target variable as a function of the explanatory variables
- ▶ **Classification**
 - Used for discrete target variables
 - Predicting whether a Web user will make a purchase at an online bookstore is a classification task because the target variable is binary-valued
- ▶ **Regression**
 - used for continuous target variables
 - forecasting the future price of a stock is a regression task because price is a continuous-valued attribute.

Predictive Modeling: Classification

- Find a model for class attribute as a function of the values of other attributes

				Class
<i>Tid</i>	Employed	Level of Education	# years at present address	Credit Worthy
1	Yes	Graduate	5	Yes
2	Yes	High School	2	No
3	No	Undergrad	1	No
4	Yes	High School	10	Yes
...	...	...	...	...

Model for predicting credit worthiness

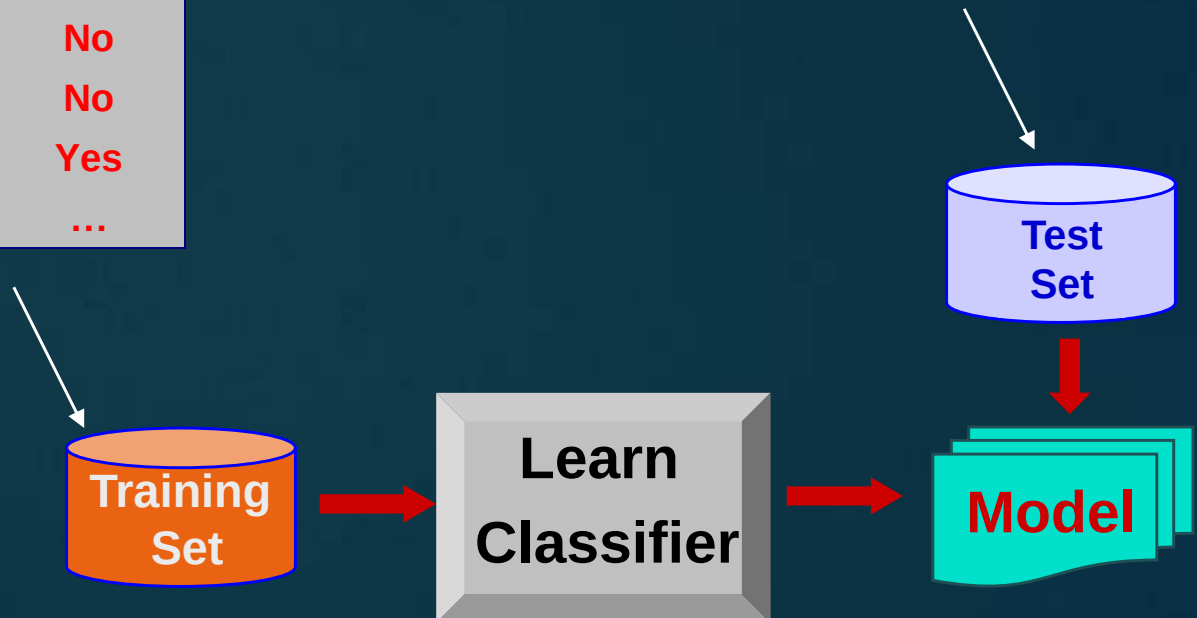


Classification Example

categorical categorical quantitative class

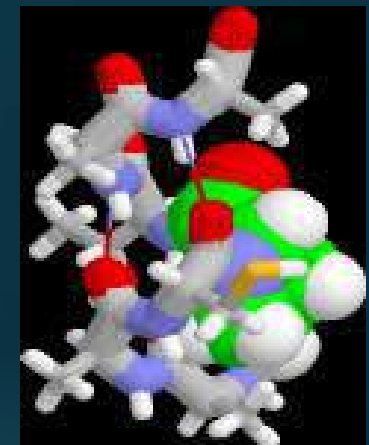
<i>Tid</i>	Employed	Level of Education	# years at present address	Credit Worthy
1	Yes	Graduate	5	Yes
2	Yes	High School	2	No
3	No	Undergrad	1	No
4	Yes	High School	10	Yes
...	...	...	...	...

<i>Tid</i>	Employed	Level of Education	# years at present address	Credit Worthy
1	Yes	Undergrad	7	?
2	No	Graduate	3	?
3	Yes	High School	2	?
...	...	...	...	...



Examples of Classification Task

- Classifying credit card transactions as legitimate or fraudulent
- Classifying land covers (water bodies, urban areas, forests, etc.) using satellite data
- Categorizing news stories as finance, weather, entertainment, sports, etc
- Identifying intruders in the cyberspace
- Predicting tumor cells as benign or malignant
- Classifying secondary structures of protein as alpha-helix, beta-sheet, or random coil

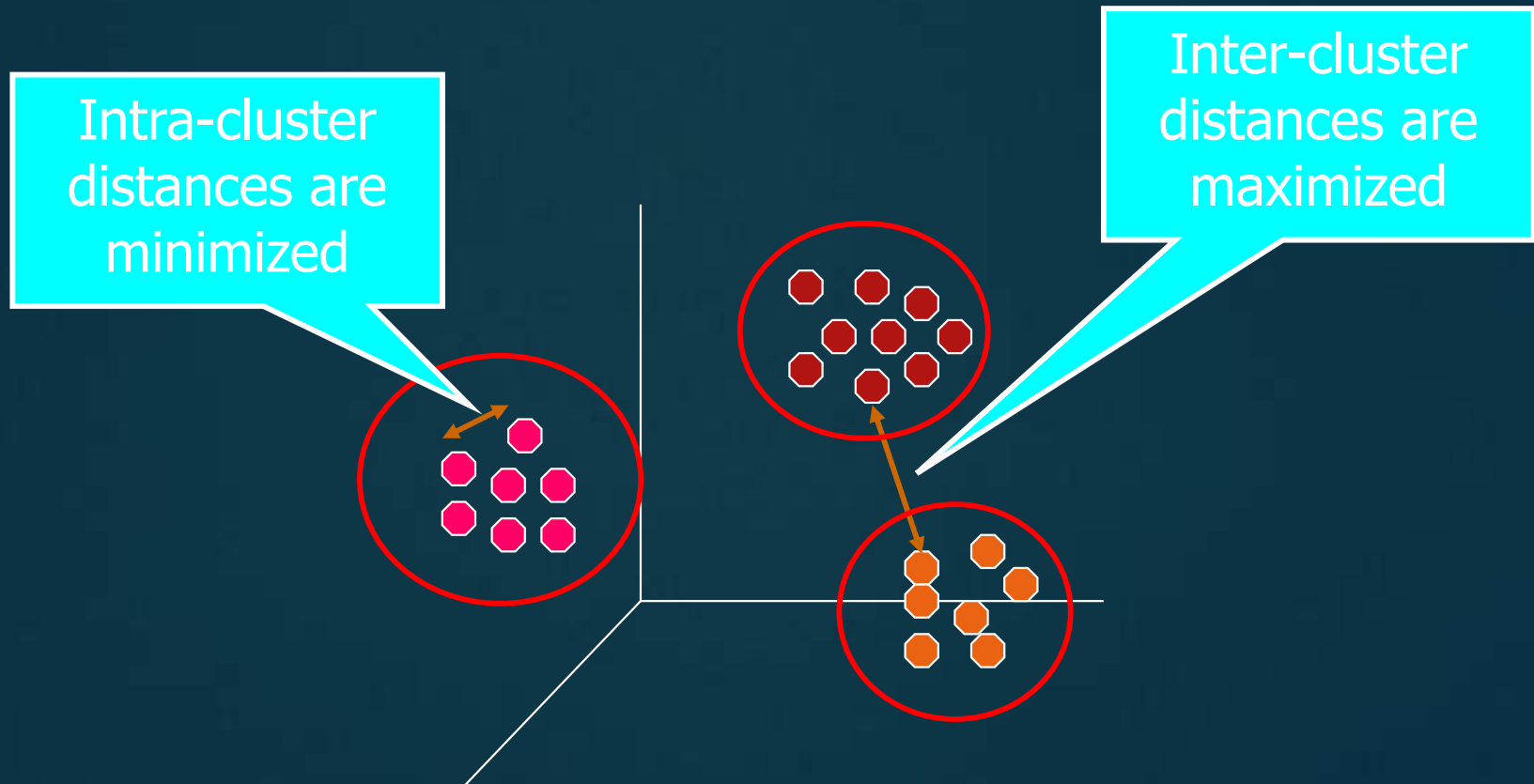


Regression

- Predict a value of a given continuous valued variable based on the values of other variables, assuming a linear or nonlinear model of dependency.
- Extensively studied in statistics, neural network fields.
- Examples:
 - Predicting sales amounts of new product based on advertising expenditure.
 - Predicting wind velocities as a function of temperature, humidity, air pressure, etc.
 - Time series prediction of stock market indices.

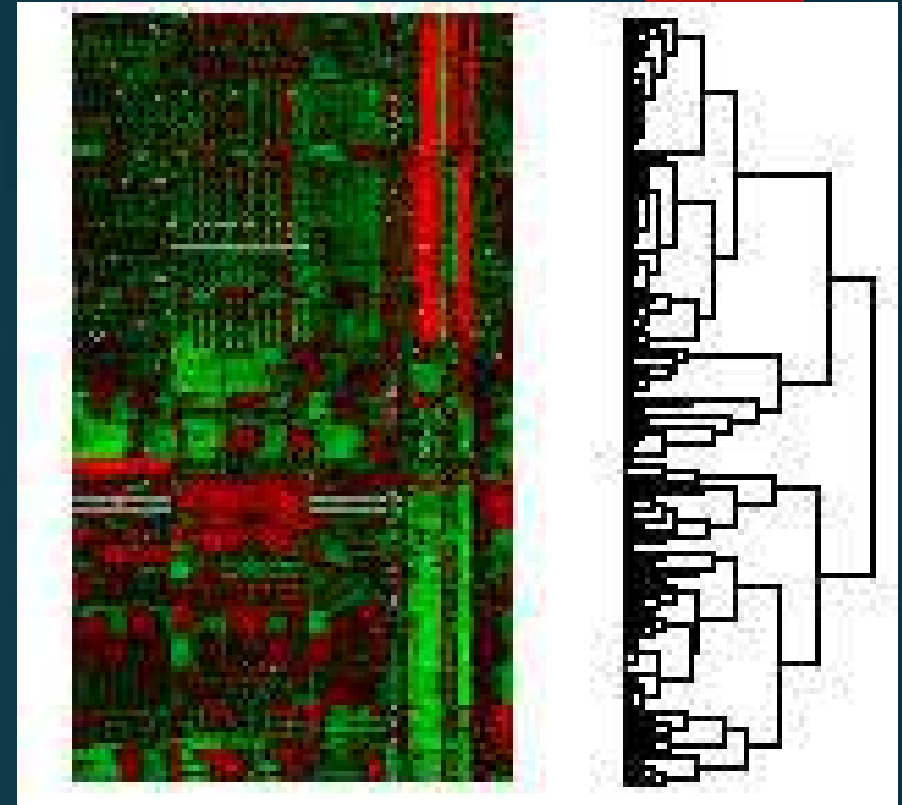
Clustering

- Finding groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in other groups

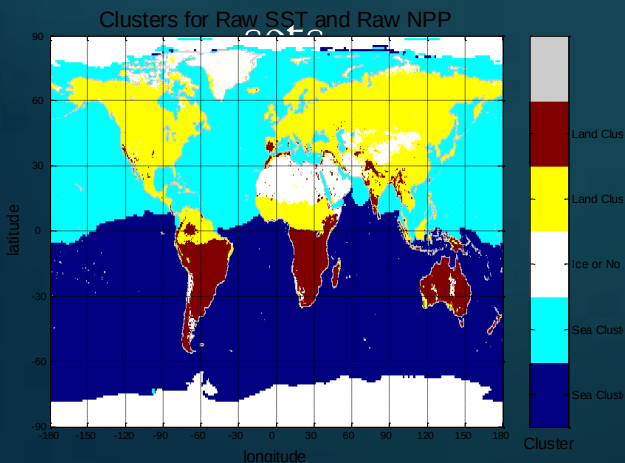


Applications of Cluster Analysis

- Understanding
 - Custom profiling for targeted marketing
 - Group related documents for browsing
 - Group genes and proteins that have similar functionality
 - Group stocks with similar price fluctuations
- Summarization
 - Reduce the size of large data



Courtesy: Michael Eisen



Use of K-means to partition Sea Surface Temperature (SST) and Net Primary Production (NPP) into clusters that reflect the Northern and Southern Hemispheres.



Association analysis

- Used to discover patterns that describe strongly associated features in the data.
- The discovered patterns are typically represented in the form of implication rules or feature subsets
- Because of the exponential size of its search space, the goal of association analysis is to extract the most interesting patterns in an efficient manner

Association Rule Discovery: Definition

- Given a set of records each of which contain some number of items from a given collection
 - Produce dependency rules which will predict occurrence of an item based on occurrences of other items.

<i>TID</i>	<i>Items</i>
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Rules Discovered:

$\{\text{Milk}\} \rightarrow \{\text{Coke}\}$

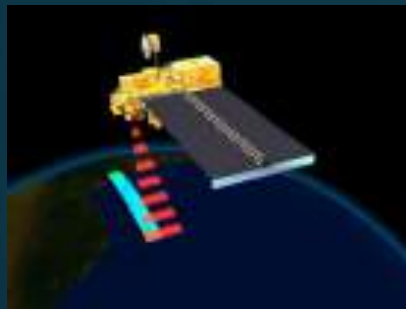
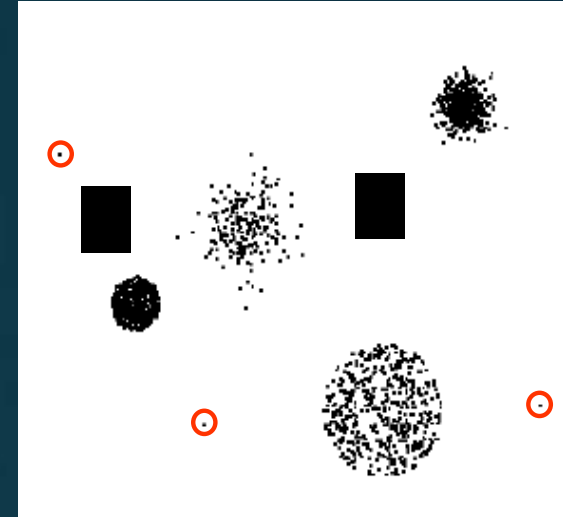
$\{\text{Diaper, Milk}\} \rightarrow \{\text{Beer}\}$

Association Analysis: Applications

- Market-basket analysis
 - Rules are used for sales promotion, shelf management, and inventory management
- Telecommunication alarm diagnosis
 - Rules are used to find combination of alarms that occur together frequently in the same time period
- Medical Informatics
 - Rules are used to find combination of patient symptoms and test results associated with certain diseases

Deviation/Anomaly/Change Detection

- Detect significant deviations from normal behavior
- Applications:
 - Credit Card Fraud Detection
 - Network Intrusion Detection
 - Identify anomalous behavior from sensor networks for monitoring and surveillance.
 - Detecting changes in the global forest cover.



DATA MINING: DATA

What is Data?

- Collection of *data objects*- *data sets*
 - Object is also known as record, point, case, sample, entity, or instance
- Data objects are described by a number of attributes
- An **attribute** is a property or characteristic of an object that may vary either from one object to another or from one time to another
 - Examples: eye color of a person, temperature, etc.
 - Attribute is also known as variable, field, characteristic, dimension, or feature

Attributes

Objects

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

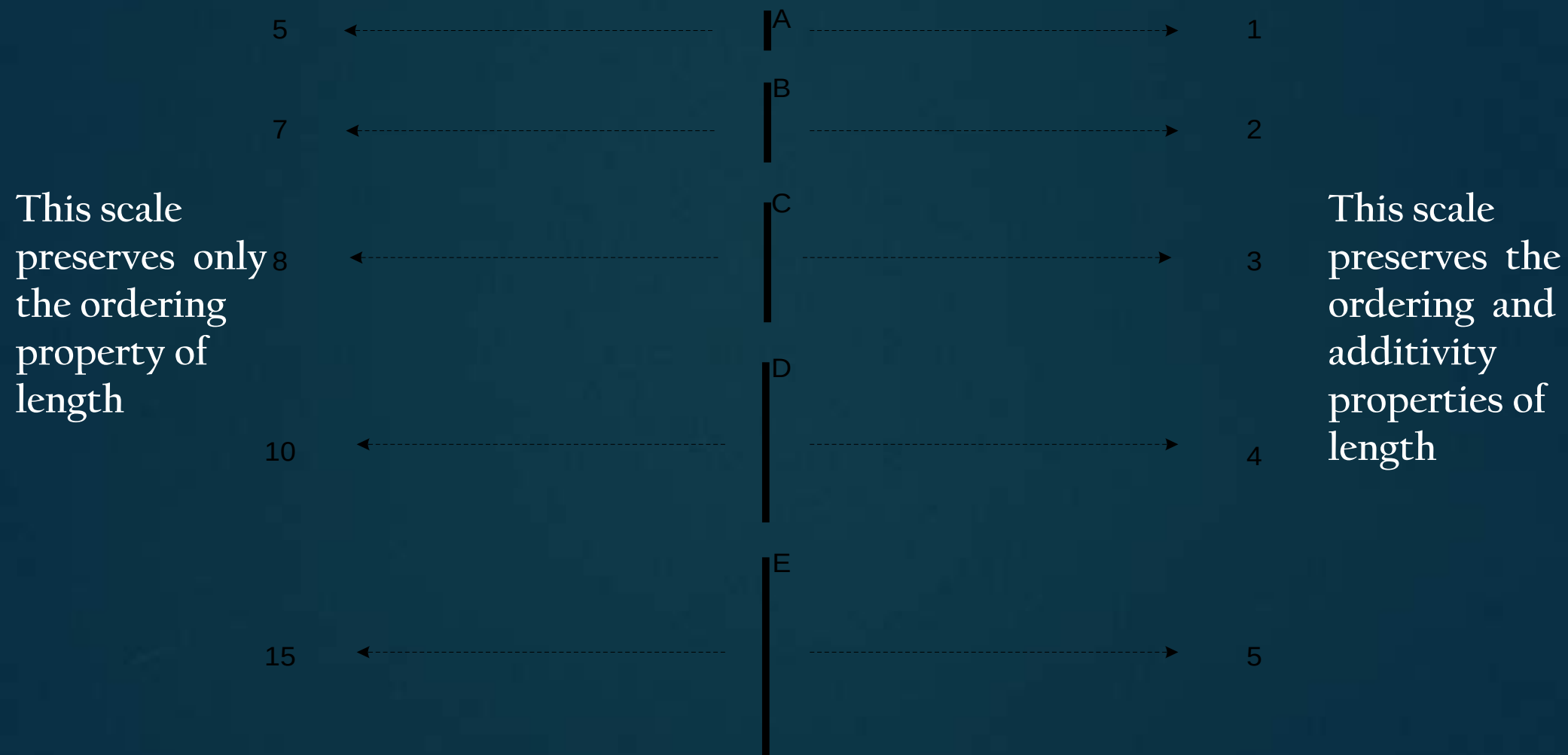
- A measurement scale is a rule (function) that associates a numerical or symbolic value with an attribute of an object
- Process of measurement is the application of a measurement scale to associate a value with a particular attribute of a specific object

Attribute Values

- **Attribute values** are numbers or symbols assigned to an attribute for a particular object
- **Distinction between attributes and attribute values**
 - Same attribute can be mapped to different attribute values
 - Example: height can be measured in feet or meters
 - Different attributes can be mapped to the same set of values
 - Example: Attribute values for **ID** and **age** are **integers**
 - But properties of attribute values can be different
 - (Average of ID is meaningless)
- **Properties of an attribute need not be the same as the properties of the values used to measure it**

Measurement of Length

- The way you measure an attribute may not match the attributes properties.



Types of Attributes

There are different types of attributes

- Nominal
- Ordinal
- Interval
- Ratio

Types of Attributes

Nominal

- Values of a nominal attribute are just different names
- Nominal values provide only enough information to distinguish one object from another ($=$, $\neq$)
- Examples: ID numbers, eye color, zip codes

Types of Attributes

Ordinal

- The values of an ordinal attribute provide enough information to order objects($\prec, \succ$)
- Ordinal Attributes contains values that have a meaningful sequence or ranking(order) between them
- The magnitude between values is not actually known- the order of values shows what is important but don't indicate how important it is
- Examples: rankings (e.g., taste of potato chips on a scale from 1-10), grades, height {tall, medium, short}

Types of Attributes

Interval

- The differences between values are meaningful(+,-)
- A unit of measurement exists
- Intervals between each value are equally split
- No true “zero value” (0 deg Celsius doesn't mean “No temperature”)
- Examples: calendar dates, temperature in Celsius or Fahrenheit
- If temperature in Delhi is 40 ° Celsius and that in Shimla is 20 ° Celsius, then Delhi is 20 ° Celsius hotter than Shimla (taking difference)
- But you cannot say Delhi is twice as hot as Shimla

Types of Attributes

Ratio

- Meaningful ratio can be constructed(/,*)
- There is an absolute zero
- Examples: monetary quantities, counts, age, mass, length, electrical current, temperature in Kelvin
- Examples:
 - 40 kg is twice as heavy as 20 kg
 - No. of clients may be 0 in last six months or may be twice as that of previous six months

Types of Attributes

Categorical or Qualitative attributes: Nominal and Ordinal attributes

- Attribute where the values correspond to discrete categories- Deals with characteristics and descriptors that can't be easily measured, but can be observed subjectively
- Lack most of the properties of numbers
- Even if they are represented by numbers, i.e., integers, they should be treated more like symbols

Types of Attributes

Quantitative or numeric attributes-Interval and Ratio attributes

- Represented by numbers
- Have most of the properties of numbers
- Can be integer-valued or continuous

Properties of Attribute Values

- The type of an attribute depends on which of the following properties/operations it possesses:
 - Distinctness: $=$ $+$
 - Order: $<$ $>$
 - Differences $+$ $-$
 - Ratios $*$ $/$
- Nominal attribute: distinctness
- Ordinal attribute: distinctness & order
- Interval attribute: distinctness, order & meaningful differences
- Ratio attribute: all 4 properties/operations

- Is it physically meaningful to say that a temperature of 10° is twice that of 5° on
 - the Celsius scale?
 - the Fahrenheit scale?
 - the Kelvin scale?
- Consider measuring the height above average
 - If Bill's height is three inches above average and Bob's height is six inches above average, then would we say that Bob is twice as tall as Bill?
 - Is this situation analogous to that of temperature?

		Attribute Type	Transformation	Comments
Categorical Qualitative		Nominal	Any permutation of values	If all employee ID numbers were reassigned, would it make any difference?
		Ordinal	An order preserving change of values, i.e., $new_value = f(old_value)$ where f is a monotonic function	An attribute encompassing the notion of good, better best can be represented equally well by the values {1, 2, 3} or by { 0.5, 1, 10}.
Numeric Quantitative		Interval	$new_value = a * old_value + b$ where a and b are constants	Thus, the Fahrenheit and Celsius temperature scales differ in terms of where their zero value is and the size of a unit (degree).
		Ratio	$new_value = a * old_value$	Length can be measured in meters or feet.

This categorization of attributes is due to S. S. Stevens

Describing Attributes by the Number of Values

Discrete Attribute

- Has only a finite or countably infinite set of values
- Examples: zip codes, counts, or the set of words in a collection of documents
- Often represented as integer variables
- **Binary attributes**
 - Special case of discrete attributes
 - Assume only two values, e.g., true/false, yes/no, male/female, or 0/1
 - Often represented as Boolean variables, or as integer variables that only take the values 0 or 1

Continuous Attribute

- Has real numbers as attribute values
- Examples: temperature, height, or weight.
- Practically, real values can only be measured and represented using a finite number of digits
- Continuous attributes are typically represented as floating-point variables

Asymmetric Attributes

- Only presence (a non-zero attribute value) is regarded as important
- Binary attributes where only non-zero values are important are called asymmetric binary attributes
- A binary variable is said to be symmetric if both of its states are of the same value and carry the same weight, and there is no preference
- Example: attribute of gender
- An asymmetric binary attribute is one in which outcomes are not of equal importance

Important Characteristics of Data sets

Dimensionality

- Number of attributes that the objects in the data set possess
- High dimensional data brings a number of challenges
- Difficulties associated with analyzing high-dimensional data-

Curse of dimensionality

Sparsity

- For some data sets, most attributes of an object have values of 0
- Advantage because usually only the non-zero values need to be stored and manipulated
- Results in significant savings with respect to computation time and storage
- Some data mining algorithms work well only for sparse data

Resolution

- It is possible to obtain data at different levels of resolution
- Properties of the data are different at different resolutions
- Patterns in the data depend on the level of resolution
- If the resolution is too fine, a pattern may not be visible or may be buried in noise; if the resolution is too coarse, the pattern may disappear
- Example: Variations in atmospheric pressure on a scale of hours reflect the movement of storms and other weather systems. On a scale of months, such phenomena are not detectable

Types of data sets

- Record
 - Data Matrix
 - Document Data
 - Transaction Data
- Graph
 - World Wide Web
 - Molecular Structures
- Ordered
 - Spatial Data
 - Temporal Data
 - Sequential Data
 - Genetic Sequence Data

Record Data

- Data set is a collection of records(data objects), which consists of a fixed set of data fields (attributes)
- Record data is usually stored either in flat files or in relational databases

<i>Tid</i>	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Transaction Data(Market Basket Data)

- A special type of record data, where each record (transaction) involves a set of items
- Example: Grocery store (Market basket) data
 - The set of products purchased by a customer during one shopping trip constitute a transaction, while the individual products that were purchased are the items

<i>TID</i>	<i>Items</i>
1	Bread Coke, Milk
2	Ber Bread
3	Ber Coke, Diaper, Milk
4	Ber Bread, Diaper, Milk
5	Coke, Diaper, Milk

Data Matrix

- Data objects have the same fixed set of numeric attributes
- Such a data set can be represented by an m by n matrix, where there are m rows, one for each object, and n columns, one for each attribute
- Because it consists of numeric attributes, standard matrix operation can be applied to transform and manipulate the data

Projection of x Load	Projection of y Load	Distance	Load	Thickness
10.23	5.27	15.22	27	1.2
12.65	6.25	16.22	22	1.1
13.54	7.23	17.34	23	1.2
14.27	8.43	18.45	25	0.9

Sparse Data Matrix

- Special case of a data matrix in which the attributes are of the same type and are asymmetric; i.e., only non-zero values are important
- Example: Transaction data

TID	Bread	Milk	Diapers	Beer	Eggs	Cola
1	1	1	0	0	0	0
2	1	0	1	1	1	0
3	0	1	1	1	0	1
4	1	1	1	1	0	0
5	1	1	1	0	0	1

Example: Document Data

- Each document becomes a 'term' vector
- Each term is a component (attribute) of the vector
- The value of each component is the number of times the corresponding term occurs in the document

	team	coach	play	ball	score	game	win	lost	timeout	season
Document 1	3	0	5	0	2	6	0	2	0	2
Document 2	0	7	0	2	1	0	0	3	0	0
Document 3	0	1	0	0	1	2	2	0	3	0

Graph based data

- A graph can sometimes be a convenient and powerful representation for data
- Types:
 - Graph capturing relationships among data objects
 - Data objects themselves are represented as graph

Data with Relationships among Objects

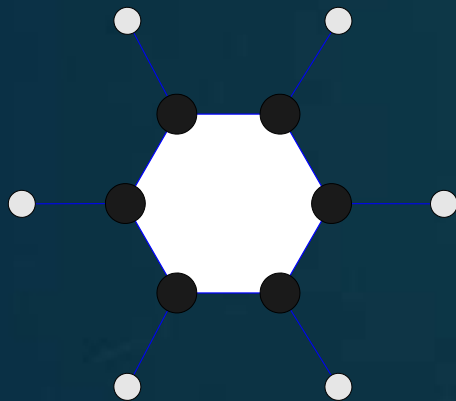
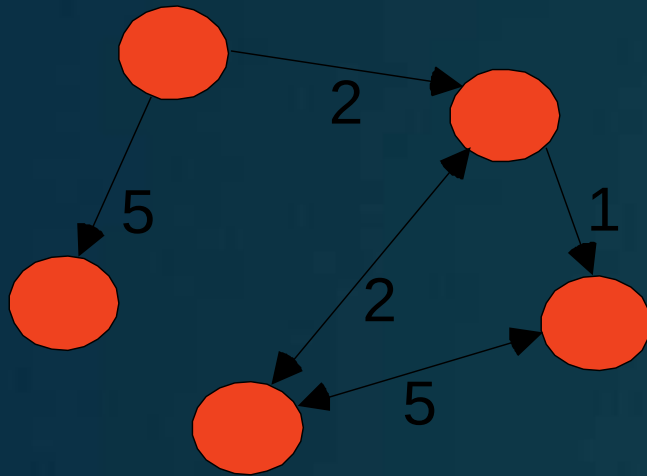
- Relationships among objects frequently convey important information
- Data **objects** are mapped to **nodes** of the graph
- The **relationships among objects** are captured by the **links between objects and link properties** (such as direction and weight)
- Example: Web pages on the World Wide Web

Data with Objects That Are Graphs

- If objects contain subobjects that have relationships, then such objects are frequently represented as graphs
- structure of chemical compounds can be represented by a graph, where the nodes are atoms and the links between nodes are chemical bonds
- Graph representation makes it possible to
 - determine which substructures occur frequently in a set of compounds
 - ascertain whether the presence of any of these substructures is associated with the presence or absence of certain chemical properties, such as melting point or heat of formation.

Graph Data

- Examples: Generic graph, a molecule, and webpages



Benzene Molecule: C₆H₆

Useful Links:

- [Bibliography](#)
- Other Useful Web sites
 - [ACM SIGKDD](#)
 - [KDMuggets](#)
 - [The Data Mine](#)

Knowledge Discovery and Data Mining Bibliography

(Gets updated frequently, so visit often!)

- [Books](#)
- [General Data Mining](#)

Book References in Data Mining and Knowledge Discovery

Usama Fayyad, Gregory Piatetsky-Shapiro, Padhraic Smyth, and Ramasamy Uthurasamy, "Advances in Knowledge Discovery and Data Mining", AAAI Press/the MIT Press, 1996.

J. Ross Quinlan, "C4.5: Programs for Machine Learning", Morgan Kaufmann Publishers, 1993.
Michael Berry and Gordon Linoff, "Data Mining Techniques (For Marketing, Sales, and Customer Support)", John Wiley & Sons, 1997.

General Data Mining

Usama Fayyad, "Mining Databases: Towards Algorithms for Knowledge Discovery", Bulletin of the IEEE Computer Society Technical Committee on data Engineering, vol. 21, no. 1, March 1998.

Christopher Matheus, Philip Chan, and Gregory Piatetsky-Shapiro, "Systems for knowledge Discovery in databases", IEEE Transactions on Knowledge and Data Engineering, 5(6):903-913, December 1993.

Ordered Data

- Attributes may have relationships that involve order in time or space

Sequential Data/Temporal data

- Extension of record data, where each record has a time associated with it
- Example: retail transaction data set that also stores the time at which the transaction took place

Time	Customer	Items Purchased
t1	C1	A, B
t2	C3	A, C
t2	C1	C, D
t3	C2	A, D
t4	C2	E
t5	C1	A, E

Customer	Time and Items Purchased
C1	(t1: A,B) (t2:C,D) (t5:A,E)
C2	(t3: A, D) (t4: E)
C3	(t2: A, C)

Ordered Data

Sequence Data

- Consists of a data set that is a sequence of individual entities, such as a sequence of words or letters
- Quite similar to sequential data, except that there are no time stamps; instead, there are positions in an ordered sequence

Ordered Data

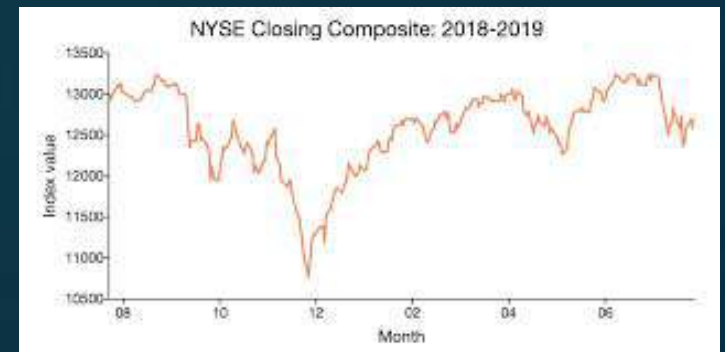
- Example: Genomic sequence data

```
GGTTCCGCGCTTCAGCCCCGCGCC  
CGCAGGGGCCCGCCCCGCGCCGTC  
GAGAAGGGGCCCGCCTGGCGGGCG  
GGGGGAGGCGGGGGCCGCCCGAGC  
CCAACCGAGTCCGACCAAGGTGCC  
CCCTCTGCTCGGCCTAGACCTGA  
GCTCATTAGGCGGCAGCGGACAG  
GCCAAGTAGAACACGCGAAGCGC  
TGGGCTGCCTGCTGCGACCAAGG
```

Ordered Data

Time Series Data

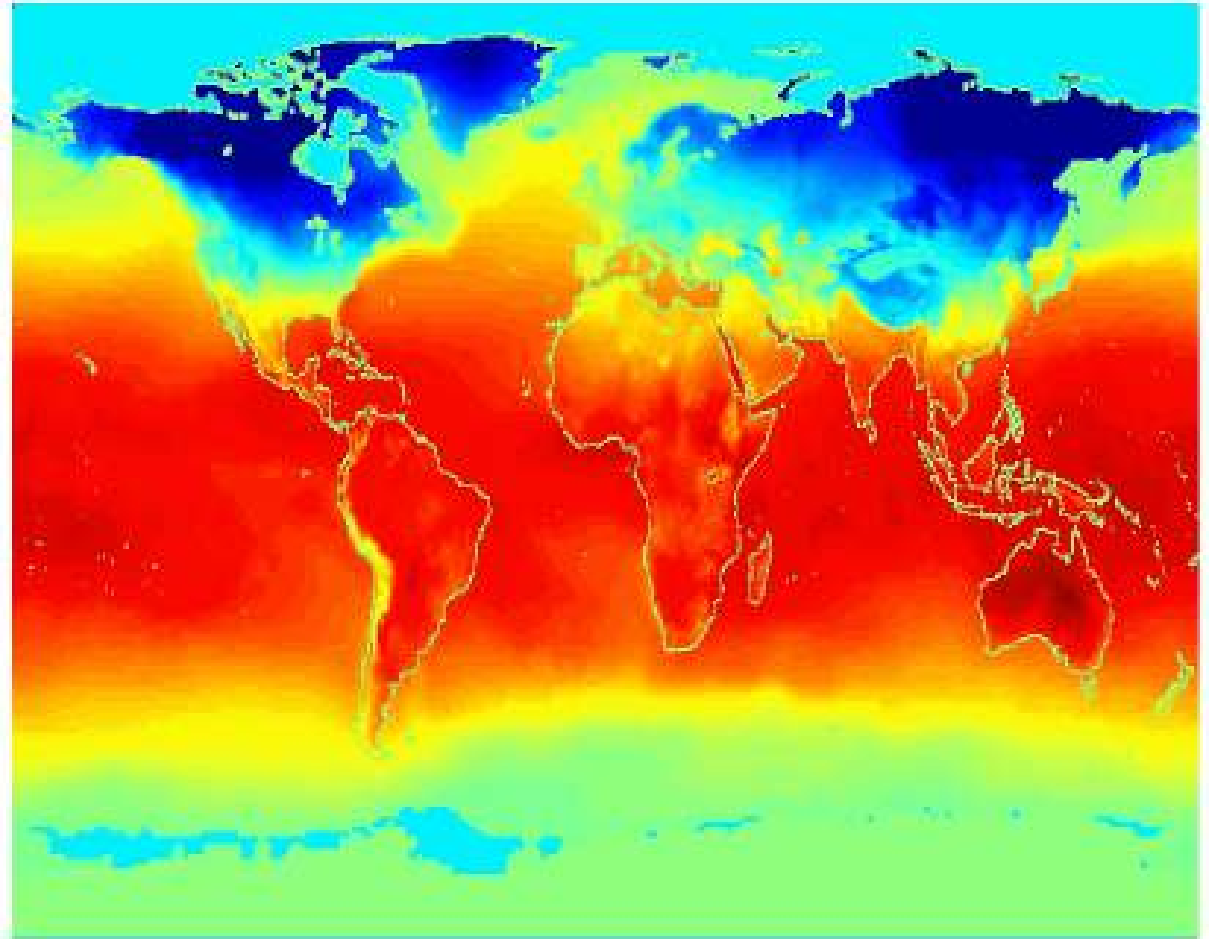
- Time-series data contain values that are typically generated by continuous measurement over time
- If two measurements are close in time, then the values of those measurements are often very similar
- Financial data set might contain objects that are time series of the daily prices of various stocks



Ordered Data

**Average
Monthly
Temperature**

Jan



Ordered Data

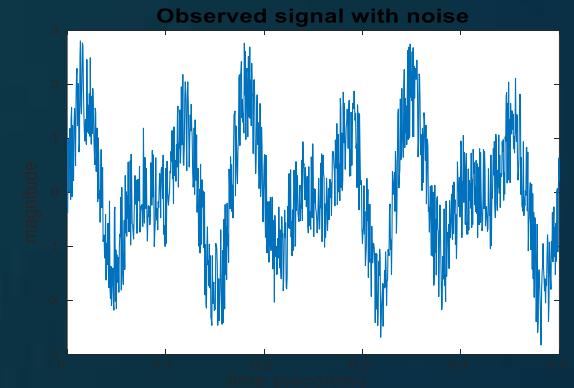
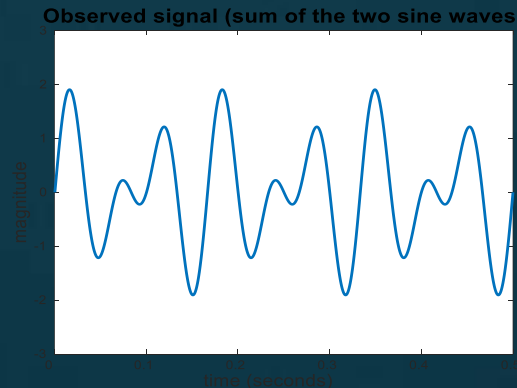
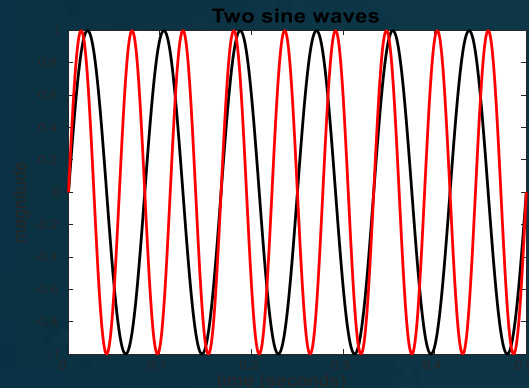
- Spatial Data
- Some objects have spatial attributes, such as positions or areas
- Examples: Weather data (precipitation, temperature, pressure) that is collected for a variety of geographical locations
- Objects that are physically close tend to be similar in other ways also
- Example: Data sets that are the result of measurements or model output taken at regularly or irregularly distributed points on a two- or three-dimensional grid or mesh

DATA QUALITY

- It is unrealistic to expect that data will be perfect
- There may be problems due to human error, limitations of measuring devices, flaws in the data collection process, application related issues
- **Measurement and Data Collection Errors:**
- Measurement Errors: Any problem resulting from the measurement process- value recorded differs from the true value to some extent
- Data Collection Errors: Omitting data objects/attribute values, inappropriately including a data object

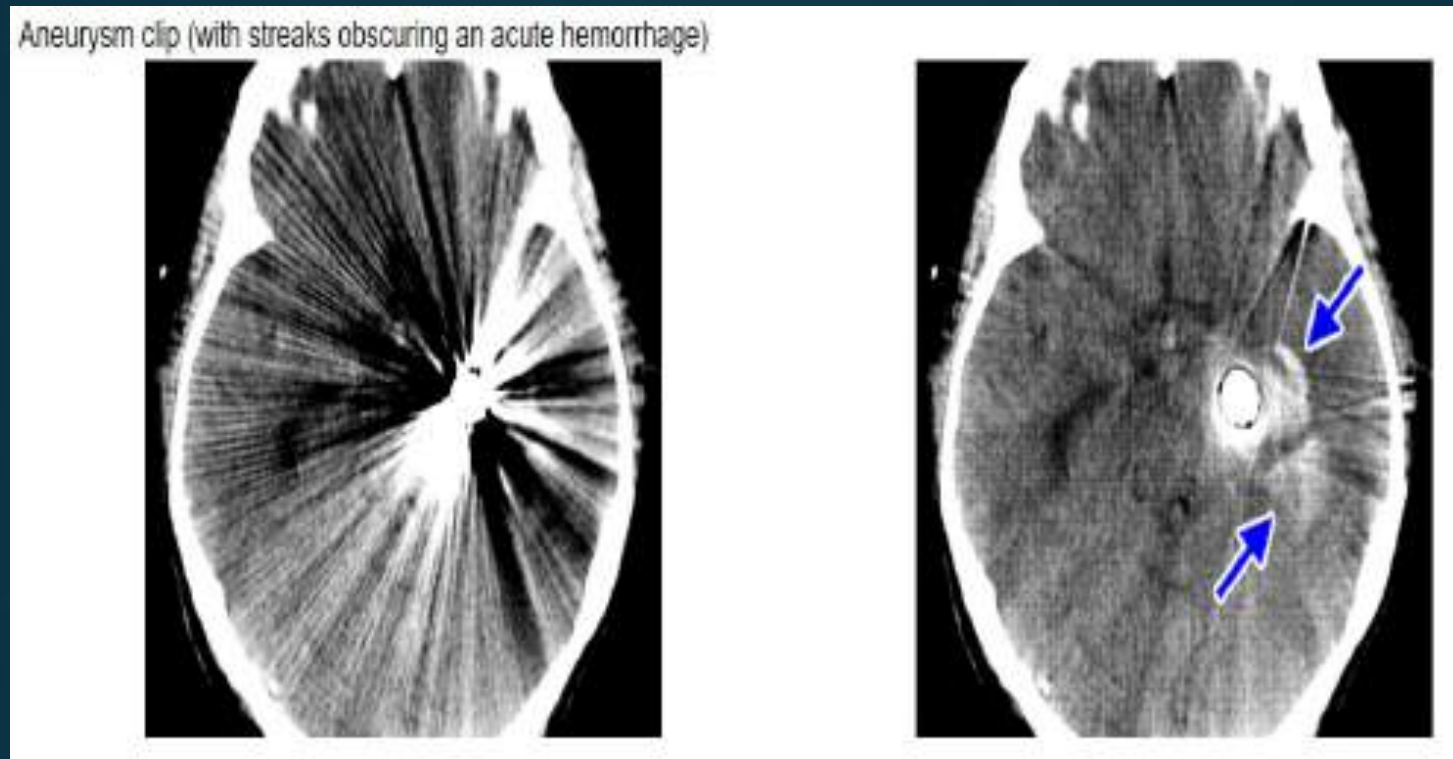
Noise

- May involve the distortion of a value or the addition of spurious objects
- Often used with data that has spatial or temporal component
- For objects, noise is an extraneous object
- For attributes, noise refers to modification of original values
- Examples: distortion of a person's voice when talking on a phone
- The figures below show two sine waves of the same magnitude and different frequencies, the waves combined, and the two sine waves with random noise
- The magnitude and shape of the original signal is distorted



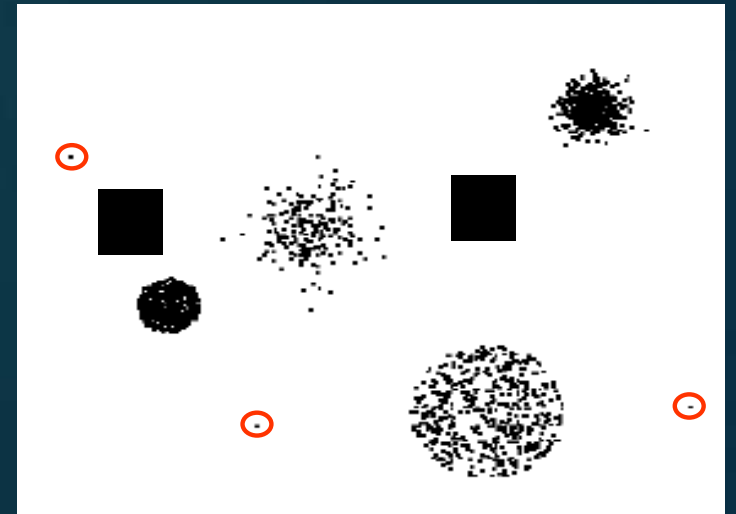
Artifact

- Deterministic distortions of the data
- Streak in the same place on a set of photographs, an image that becomes visibly distorted after compression



Outliers

- Outliers are either:
 - i) Data objects with characteristics that are considerably different than most of the other data objects in the data set
 - ii) Values of an attribute that are unusual with respect to the typical values for that attribute
- Outliers can be legitimate data objects
- Example: Fraud and network intrusion- Goal is to find unusual objects or events from among large number of normal ones



Missing Values

- Objects missing one or more attribute values
- Causes: Information not collected, attribute not applicable to data object
- There are strategies for dealing with missing data

i. Eliminate Data Objects or Attributes

- Simple and effective strategy is to eliminate objects with missing values
- Issues: If **many** objects have missing values, reliable analysis can be difficult if data objects are eliminated; also partially specified data will be lost
- If a dataset has only few objects, that have missing values, removing objects is not a good idea
- As an alternative, attributes that have missing values can be eliminated
- Should be done with caution, as attributes may be ones that are critical to the analysis

ii. Estimate Missing Values

- Sometimes, missing data can be reliably be estimated by using the remaining values
- If we consider a data set that ha many similar data points, attribute values of the points closest to the point with missing value are often used to estimate the missing value
- For continuous attribute, average attribute value of the nearest neighbor is used; for categorical attribute, most commonly occurring attribute value can be taken
- Example: for areas not containing a ground station, the precipitation can be estimated using values observed at nearby ground stations

iii. Ignore the Missing value during Analysis

- Many data mining approaches can be modified to ignore missing values
- Example: if objects are clustered, and the similarity between pairs of data objects to be calculated-
- If one or both objects of a pair have missing values, for some attributes, similarity can be calculated using only the attributes that do not have missing values

Inconsistent values

- Data can contain inconsistent values
- In address field, both zipcode and city are listed, but specified zipcode area is not contained in the city
- Some types of inconsistencies are easy to detect; in other cases, it may be necessary to consult an external source of information
- It is sometimes possible to correct the data; however, the correction of inconsistencies requires additional or redundant information

Duplicate Data

- A data set may include data objects that are duplicates, or almost duplicates of one another
- Two objects that actually represent a single object; data objects that are similar but not duplicates
- Deduplication: Process of dealing with duplication issues

Issues related to Applications

- Data quality issues can be considered from an application viewpoint
- Data is of high quality if it is suitable for its intended use

Timeliness:

- If data provides a snapshot of some ongoing phenomenon or process, it represents reality for only a limited time
- If data is out of date, then so are the methods and patterns that are based on it

Relevance:

- The available data must contain the information necessary for the application

Issues related to Applications

- Knowledge about the data:
- Ideally data sets are accompanied by documentation that describes different aspects of the data
- The quality of the documentation can either aid or hinder the subsequent analysis

Precision and Bias

- The quality of measurement process is often measured by precision and bias

Precision:

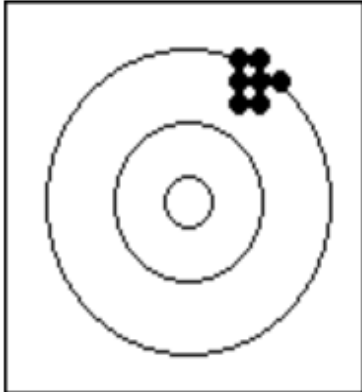
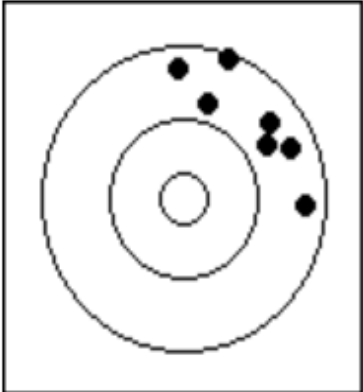
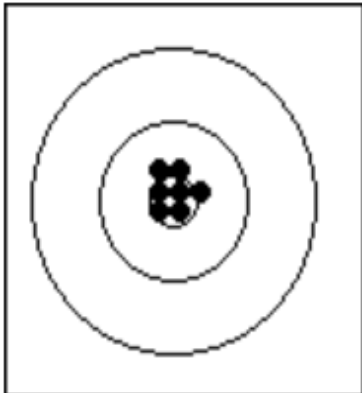
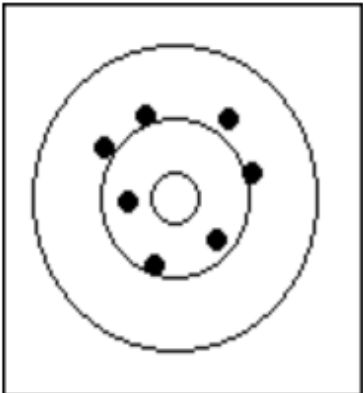
- The closeness of repeated measurements(of the same quantity) to one another
- Often measured by the standard deviation of a set of values

Bias:

- A systematic variation of measurements from the quantity being measured
- Measured by taking the difference between the mean of the set of values and the known value of the quantity being measured

Example:

- Consider a standard lab. weight with a mass of 1g
- Mass is measured 5 times and values be {1.015, 0.990, 1.013, 1.001, 0.986}
- Mean=1.001
- Bias=0.001
- 0.013

	Precise	Imprecise
Biased		
Unbiased		

Accuracy

- The closeness of measurements to the true value of the quantity being measured

DATA PREPROCESSING

Data Preprocessing

- Makes the data more suitable for data mining
- Broad area consisting of a number of different strategies and techniques that are interrelated in complex ways
- Categories:
 - Selecting data objects and attributes for the analysis
 - Creating/changing the attributes

- Aggregation

Sampling

- Dimensionality Reduction
- Feature subset selection
- Feature creation
- Discretization and Binarization
- Attribute Transformation

Aggregation

Combining two or more objects into a single object

Purpose

Data reduction

Smaller data sets resulting from data reduction require less memory and processing time, and hence, aggregation may permit the use of more expensive data mining algorithms

Change of scale

By providing high level view of the data instead of a low level

Cities aggregated into regions, states, countries, etc.

Days aggregated into weeks, months, or years

More “stable” Data

Aggregated data tends to have less variability

Disadvantage:

- Potential loss of interesting details.
- Example: aggregating over months loses information about which day of the week has the highest sales



TID	Item	Store Location	Date	Price	...
..	..	..	..	..	..
101	Watch	Delhi	01/10/20	5000	
102	Batthey	Mumbai	01/10/20	2000	
103	Shoes	Delhi	02/10/20	2500	

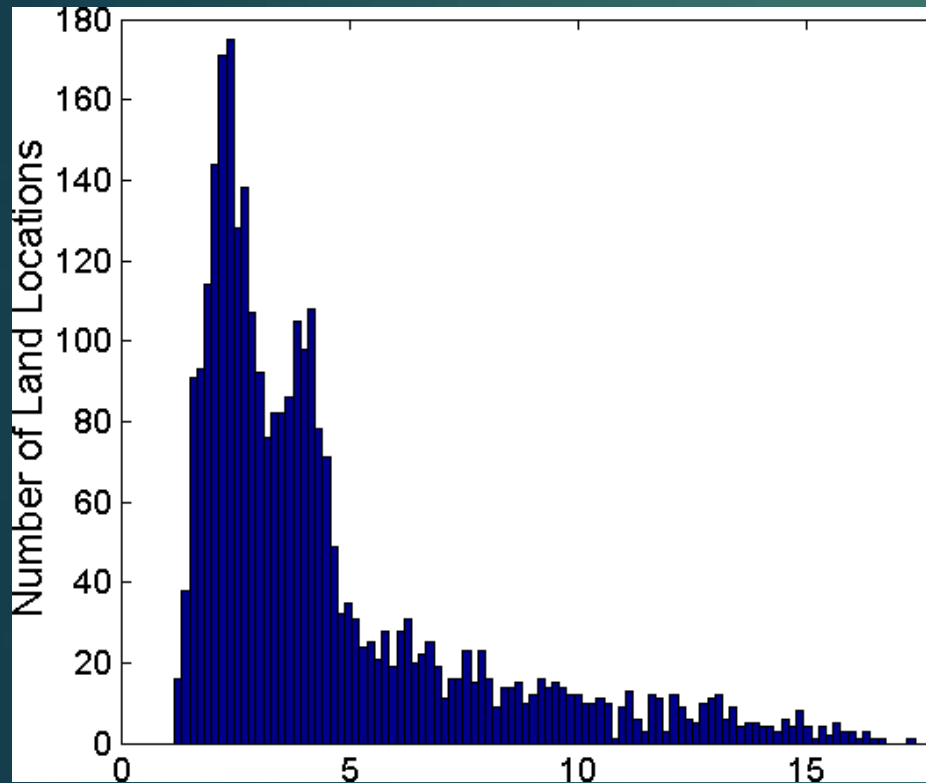
Year 2010		Q1	Q2	Q3	Q4
Quarter	Sales				
Year 2009					
Quarter	Sales				
Year 2008					
Quarter	Sales				
Q1	\$124,000				
Q2	\$408,000				
Q3	\$350,000				
Q4	\$586,000				

→

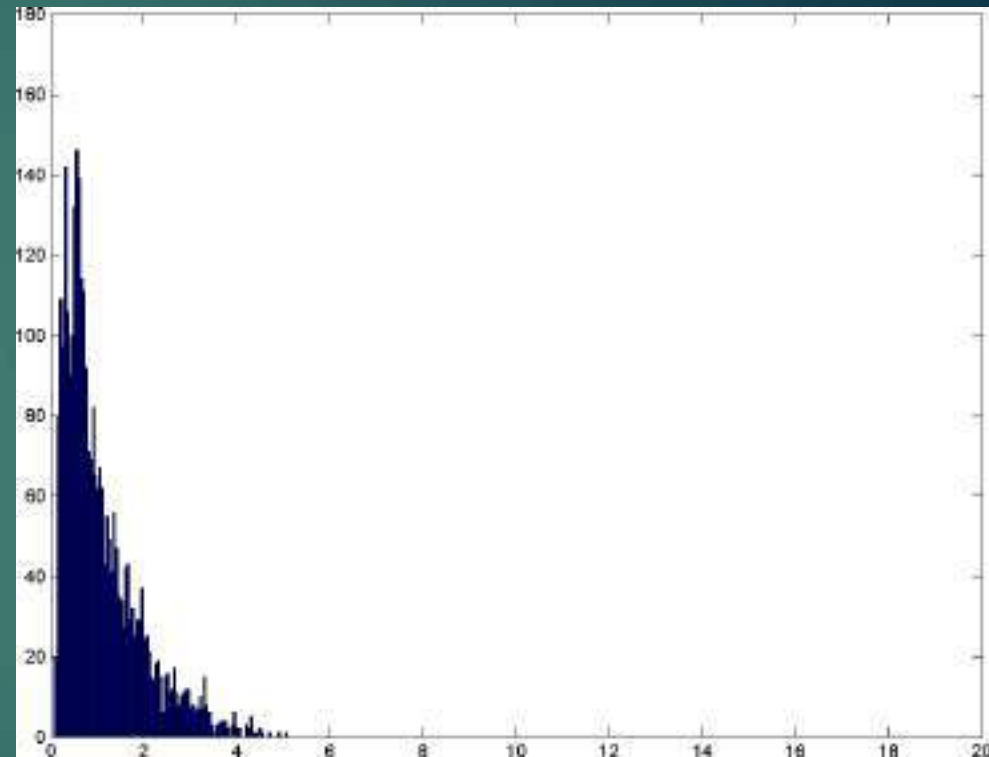
Year	Sales
2008	\$1,568,000
2009	\$2,356,000
2010	\$3,594,000

Example: Precipitation in Australia

Variation of Precipitation in Australia



Standard Deviation of Average
Monthly Precipitation



Standard Deviation of Average
Yearly Precipitation

Sampling

- Approach for selecting a subset of the data objects to be analyzed
- Statisticians use sampling because obtaining the entire set of data of interest is too expensive or time consuming
- Data miners sample because it is too expensive or time consuming to process all the data

The key principle for effective sampling :

- Using a sample will work almost as well as using the entire data set, if the sample is **representative**
- A sample is **representative** if it has approximately the same properties (of interest) as the original set of data
- **“Choose a sampling scheme that guarantees a high probability of getting a representative sample”**

Sampling Approaches

Simple Random Sampling

- There is an equal probability of selecting any particular item
- Sampling without replacement
 - As each item is selected, it is removed from the population
- Sampling with replacement
 - Objects are not removed from the population as they are selected for the sample
 - In sampling with replacement, the same object can be picked up more than once

- When the population consists of different types of objects, with widely different numbers of objects method can fail to adequately represent those types of objects that are less frequent
- Can cause problems when the analysis requires proper representation of all object types
- Example: when building classification models for rare classes, it is critical that the rare classes be adequately represented in the sample

Stratified Sampling

- Split the data into several partitions; then draw random samples from each partition

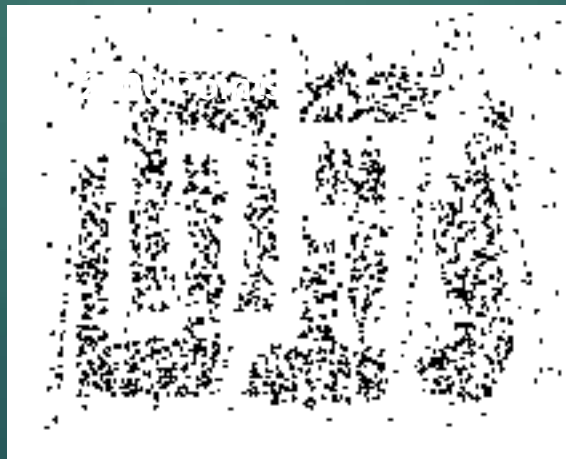
Variations:

- Equal numbers of objects are drawn from each group even though the groups are of different size
- Number of objects drawn from each group is proportional to the size of that group

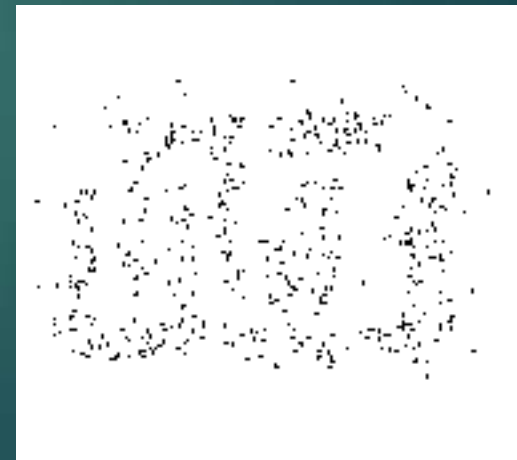
- Once a sampling technique has been selected, it is still necessary to choose the sample size
- Larger sample sizes increase the probability that a sample will be representative, but they also eliminate much of the advantage of sampling
- With smaller sample sizes, patterns may be missed or erroneous patterns can be detected



8000 points



2000 Points



500 Points

Progressive(Adaptive) Sampling

- Proper sample size can be difficult to determine
- **Solution:**
- Start with a small sample, and then increase the sample size until a sample of sufficient size has been obtained
- It requires that there be a way to evaluate the sample to judge if it is large enough

- Although the accuracy of predictive models increases as the sample size increases, at some point the increase in accuracy levels off
- We want to stop increasing the sample size at this leveling-off point
- by keeping track of the change in accuracy of the model as we take progressively larger samples
- by taking other samples close to the size of the current one

Dimensionality Reduction

- Data sets can have a large number of features
- Example: Word count in a document
- **Benefits of Dimensionality reduction**
 - Data mining algorithms work better if the dimensionality is lower
 - We can eliminate irrelevant features and reduce noise
 - Can lead to a more understandable model because the model may involve fewer attributes
 - May allow the data to be more easily visualized
 - Amount of time and memory required by the data mining algorithm is reduced with a reduction in dimensionality

- Techniques that reduce the dimensionality of a data set by creating new attributes that are a combination of the old attributes - Dimensionality reduction

The reduction of dimensionality by selecting new attributes that are a subset of the old - Feature subset selection or Feature selection

Curse of Dimensionality

- Phenomenon that many types of data analysis become significantly harder as the dimensionality of the data increase
- As dimensionality increases, the data becomes increasingly sparse in the space that it occupies

Curse of dimensionality Problems:

- **Classification**
 - There are not enough data objects to allow the creation of a model that reliably assigns a class to all possible objects
- **Clustering**
 - Definitions of density and the distance between points, which are critical for clustering, become less meaningful

Linear Algebra Techniques for Dimensionality Reduction

- **Principal Components Analysis (PCA)**

For continuous attributes that finds new attributes (principal components) that

- are linear combinations of the original attributes
- are orthogonal (perpendicular) to each other
- capture the maximum amount of variation in the data

- **Singular Value Decomposition (SVD)**

Feature Subset Selection

- Reducing dimensionality by using only a subset of features
- Involves removing:
 - **Redundant features** (duplication of much or all of the information contained in one or more other attributes)
 - **Irrelevant features** (contain almost no useful information for the data mining task at hand) are removed
- **Ideal approach:** Try all possible subsets of features as input to the data mining algorithm of interest, and then take the subset that produces the best results
- Number of subsets involving n attributes is 2^n , such an approach is impractical in most situations

Standard approaches to Feature Selection:

Embedded Approach

- Feature selection occurs naturally as part of the data mining algorithm
- During the operation of the data mining algorithm, the algorithm itself decides which attributes to use and which to ignore
- Example: Algorithms for building decision tree classifier

Filter approaches

- Features are selected before the data mining algorithm is run, using some approach that is independent of the data mining task
- Example: selecting sets of attributes whose pairwise correlation is as low as possible

Wrapper approaches

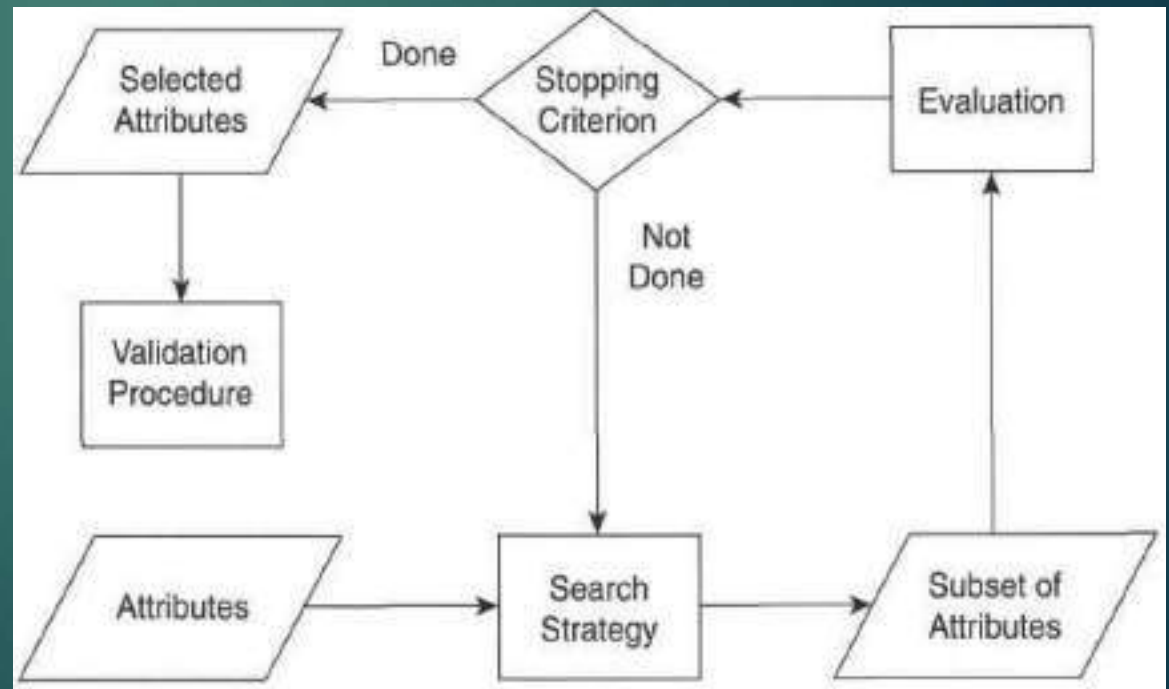
- Use the target data mining algorithm as a black box to find the best subset of attributes

An Architecture for Feature Subset Selection

It is possible to encompass both the filter and wrapper approaches within a common architecture

The Feature Selection process consists of four parts:

- Measure for evaluating a subset
- Search strategy that controls the generation of a new subset of features
- Stopping criterion
- Validation procedure



Search strategy

- Feature subset selection is a search over all possible subsets of features
- Many different types of search strategies can be used
 - should be computationally inexpensive
 - should find optimal or near optimal sets of features
- Usually not possible to satisfy both requirements, and thus, tradeoffs are necessary

Evaluating a subset

- Step to judge how the current subset of features compares to others that have been considered
- Requires an **evaluation measure** that attempts to determine the goodness of a subset of attributes with respect to a particular data mining task, such as classification or clustering
- For a wrapper method, subset evaluation uses the target data mining algorithm
- For a filter approach, the evaluation technique is distinct from the target data mining algorithm

Stopping criterion

- Because the number of subsets can be enormous and it is impractical to examine them all, some sort of stopping criterion is necessary

Based on one or more conditions such as-

- Number of iterations
- Whether the value of the subset evaluation measure is optimal or exceeds a certain threshold
- Whether a subset of a certain size has been obtained
- Whether simultaneous size and evaluation criteria have been achieved
- Whether any improvement can be achieved by the options available to the search strategy

Validation Procedure

- Results of the target data mining algorithm on the selected subset should be validated

Approach 1:

- Run the algorithm with the full set of features and compare the full results to results obtained using the subset of features
- Hopefully, the subset of features will produce results that are better than or almost as good as those produced when using all features

Approach 2:

- Use a number of different feature selection algorithms to obtain subsets of features and then compare the results of running the data mining algorithm on each subset

Feature weighting

- Alternative to keeping or eliminating feature
- More important features are assigned a higher weight, while less important features are given a lower weight
- Weights are sometimes assigned based on domain knowledge about the relative importance of features
- Alternatively, they may be determined automatically
- Example: some classification schemes, such as support vector machines produce classification models in which each feature is given a weight
- Features with larger weights play a more important role in the model

Feature Creation

Creating a new set of attributes from the original attributes, that captures the important information in a data set much more effectively

Methodologies for creating new attributes:

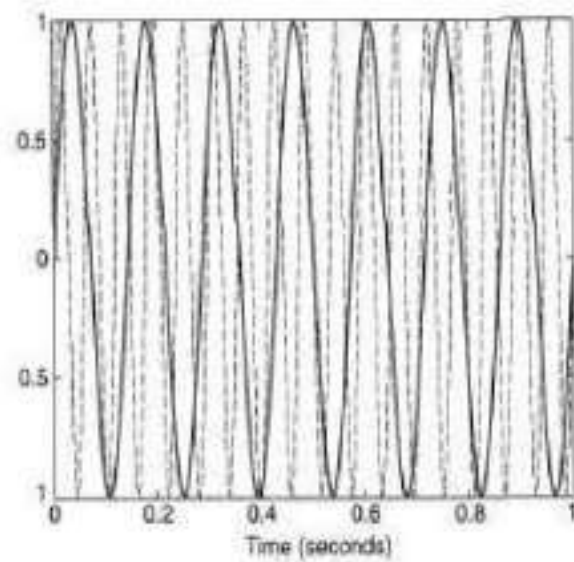
- Feature extraction
- Mapping the data to a new space
- Feature construction

Feature Extraction

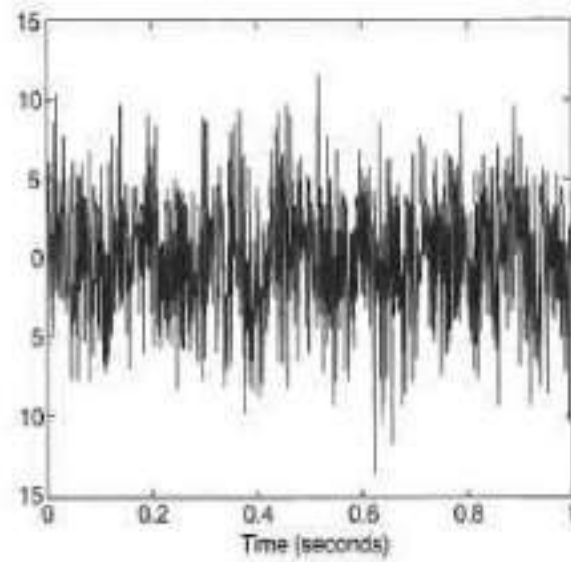
- Creation of a new set of features from the original raw data
- Photographs are classified according to whether or not it contains a human face
- Highly domain-specific
- Whenever data mining is applied to a relatively new area, a key task is the development of new features and feature extraction methods

Mapping the Data to a New Space

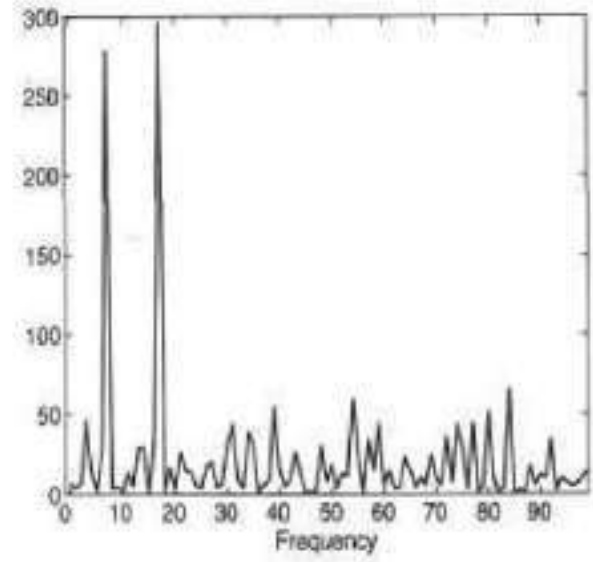
- Totally different view of the data can reveal important and interesting features
- Example: time series data, which often contains periodic patterns
- If there is only a single periodic pattern and not much noise then the pattern is easily detected
- If there are a number of periodic patterns and a significant amount of noise is present, then these patterns are hard to detect
- Such patterns can often be detected by applying a Fourier transform to the time series
- Other transformations: wavelet transformation



(a) Two time series.



(b) Noisy time series.



(c) Power spectrum

Feature Construction

- Sometimes the features in the original data sets have the necessary information, but it is not in a form suitable for the data mining algorithm
- One or more new features constructed out of the original features can be more useful than the original features
- Example: density feature constructed from the mass and volume features

$$\text{density} = \text{mass} / \text{volume}$$

Binarization and Discretization

- Some data mining algorithms require that the data be in the form of binary attributes or categorical attributes

Binarization

Transforming continuous and discrete attributes into binary attributes

Simple Technique:

- If there are m categorical values, then uniquely assign each original value to an integer in the interval $[0, m - 1]$
- If the attribute is ordinal, then order must be maintained by the assignment
- Convert each of these m integers to a binary number
- Example: categorical variable with 5 values {awful, poor, OK, good, great} would require three binary variables x_1, x_2, x_3

Table 2.5. Conversion of a categorical attribute to three binary attributes.

Categorical Value	Integer Value	x_1	x_2	x_3
<i>awful</i>	0	0	0	0
<i>poor</i>	1	0	0	1
<i>OK</i>	2	0	1	0
<i>good</i>	3	0	1	1
<i>great</i>	4	1	0	0

Table 2.6. Conversion of a categorical attribute to five asymmetric binary attributes.

Categorical Value	Integer Value	x_1	x_2	x_3	x_4	x_5
<i>awful</i>	0	1	0	0	0	0
<i>poor</i>	1	0	1	0	0	0
<i>OK</i>	2	0	0	1	0	0
<i>good</i>	3	0	0	0	1	0
<i>great</i>	4	0	0	0	0	1

- Creates unintended relationships among the transformed attributes
attributes **x2 and x3 are correlated** because information about the **good** value is encoded using both attributes
- Association analysis requires asymmetric binary attributes, where only the presence of the attribute (value : 1) is important.
- For association problems, it is therefore necessary to introduce one binary attribute for each categorical value

Discretization

- Transformation of a continuous attribute to a categorical attribute

Involves two subtasks:

- Deciding how many categories to have:

After the values of the continuous attribute are sorted, they are then divided into n intervals by specifying $n - 1$ split points

- Determining how to map the values of the continuous attribute to these categories:

All the values in one interval are mapped to the same categorical value

Problem of discretization is one of deciding how many split points to choose and where to place them

Unsupervised Discretization

- Class information is not used

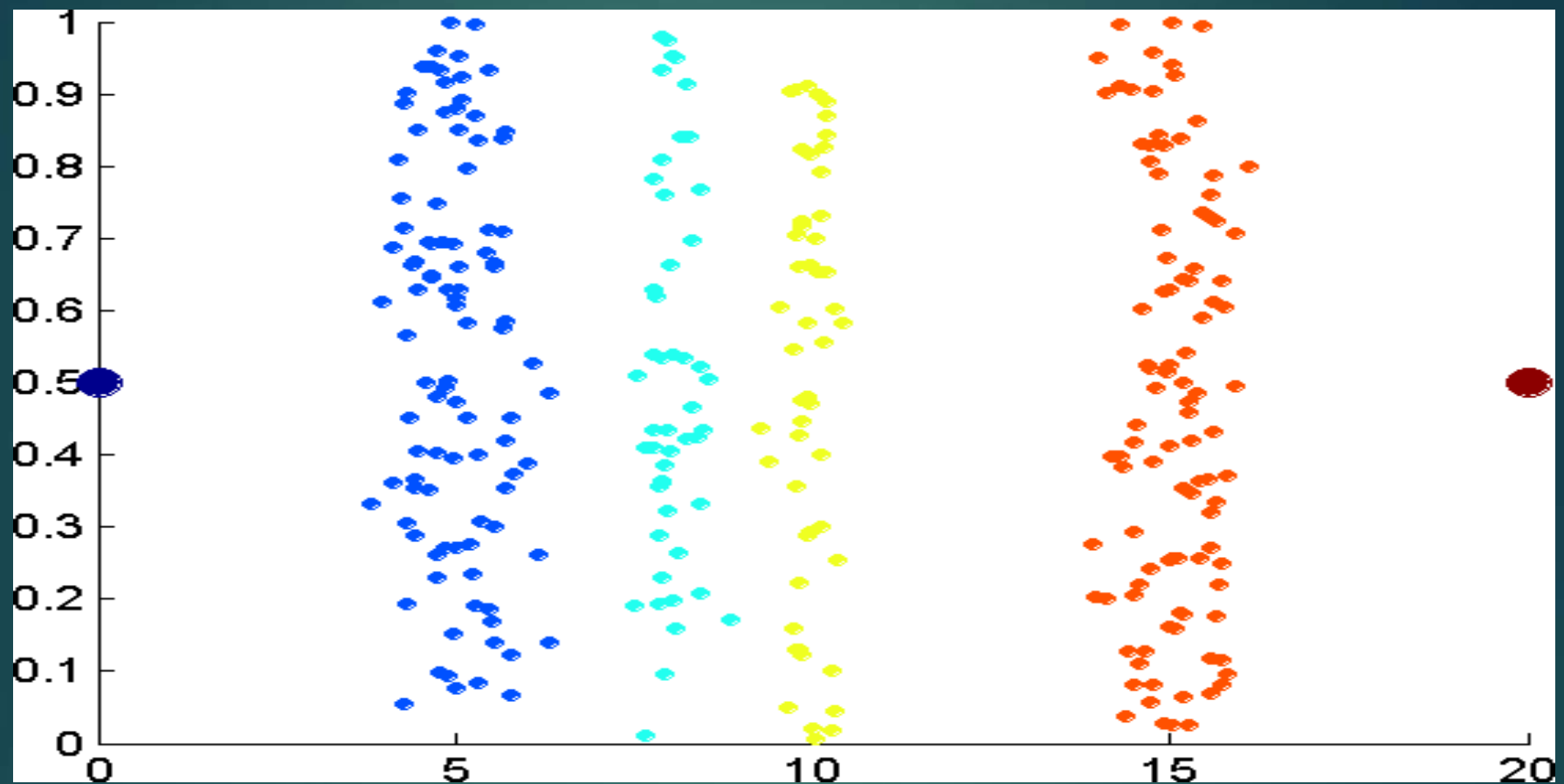
Equal width approach:

- divides the range of the attribute into a user-specified number of intervals each having the same width
- Can be badly affected by outliers

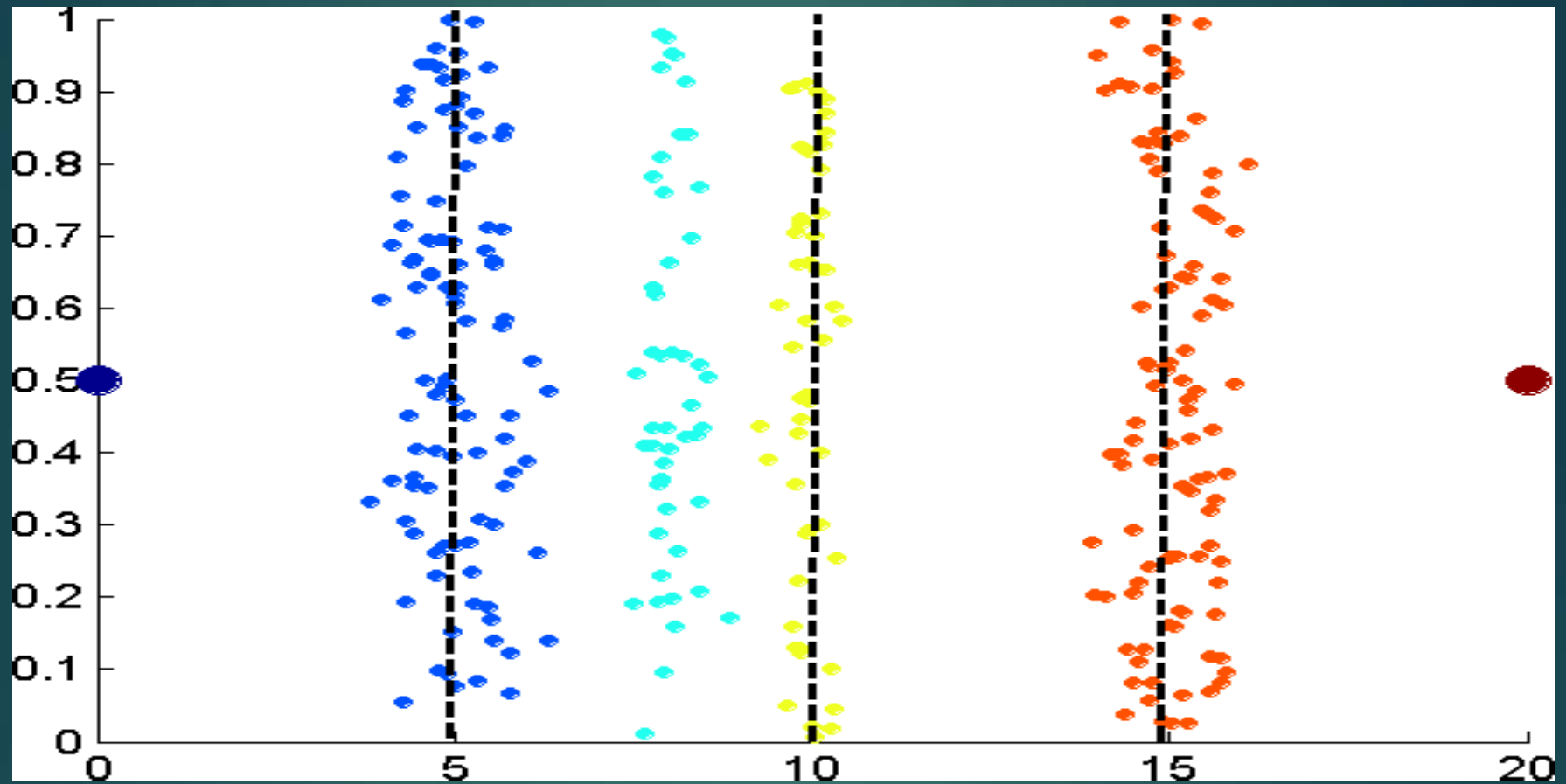


Equal frequency (equal depth) approach

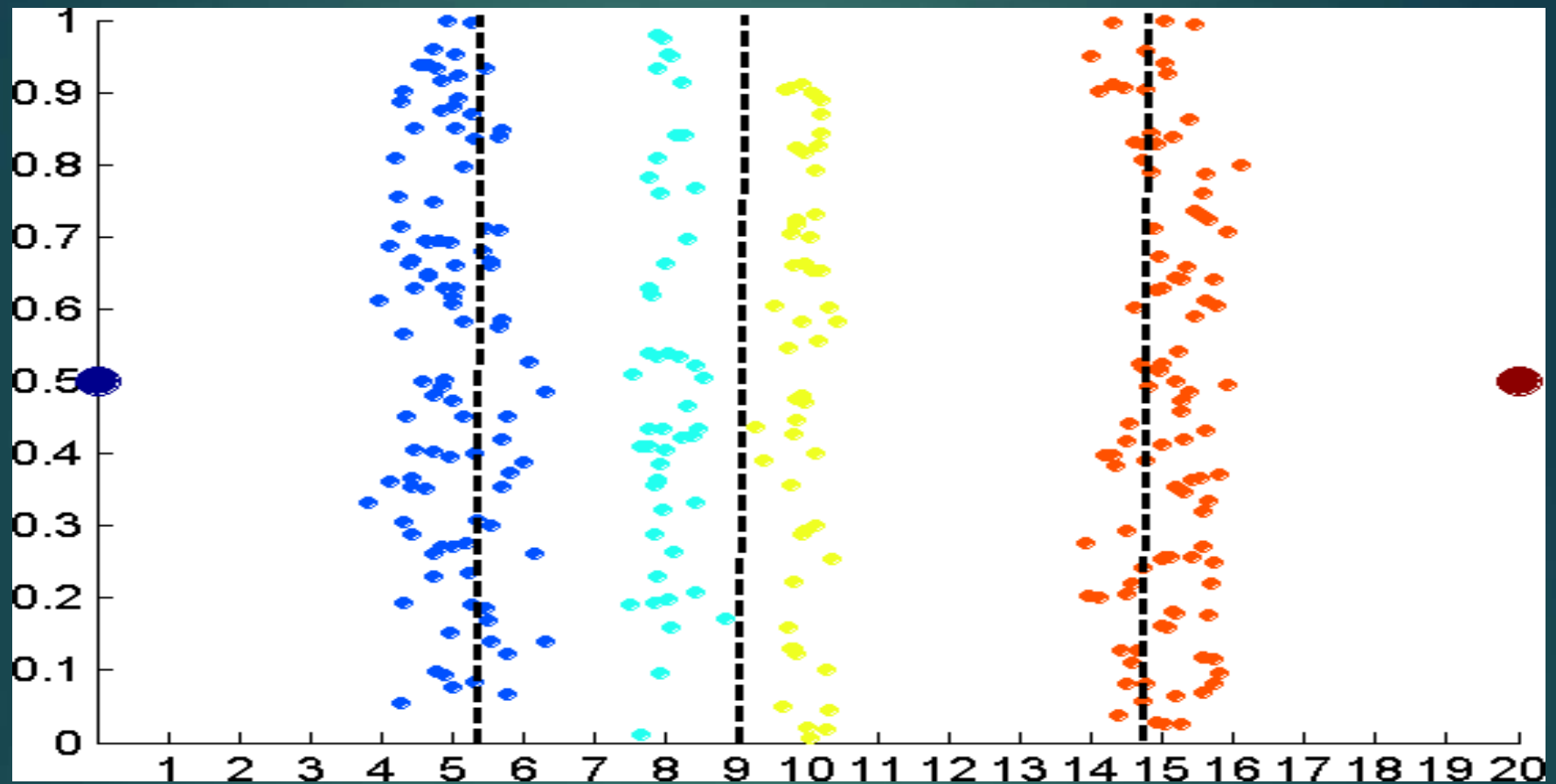
- Tries to put the same number of objects into each interval



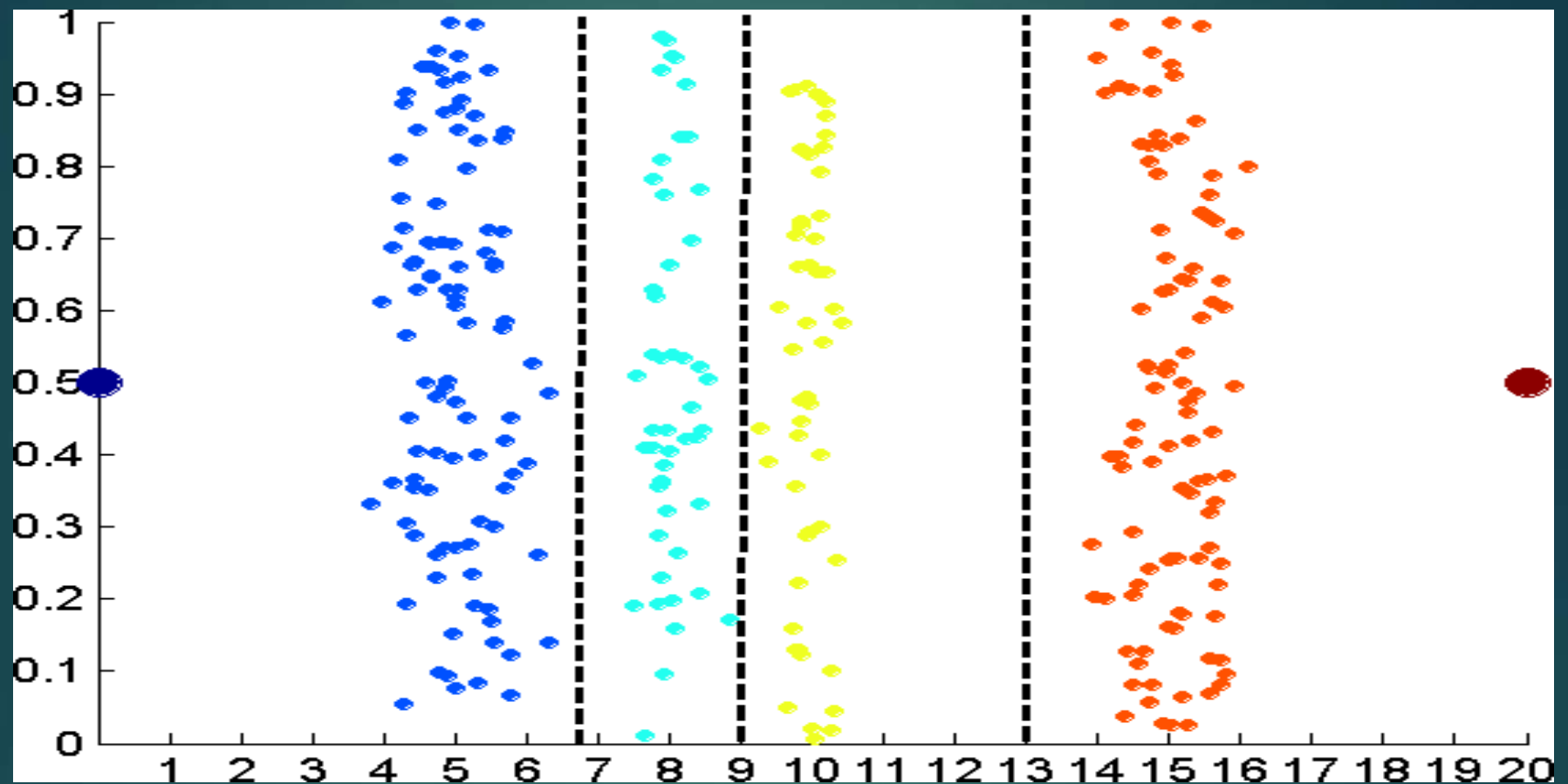
Data consists of four groups of points and two outliers. Data is one-dimensional, but a random y component is added to reduce overlap.



Equal interval width approach used to obtain 4 values.



Equal frequency approach used to obtain 4 values.



K-means approach to obtain 4 values

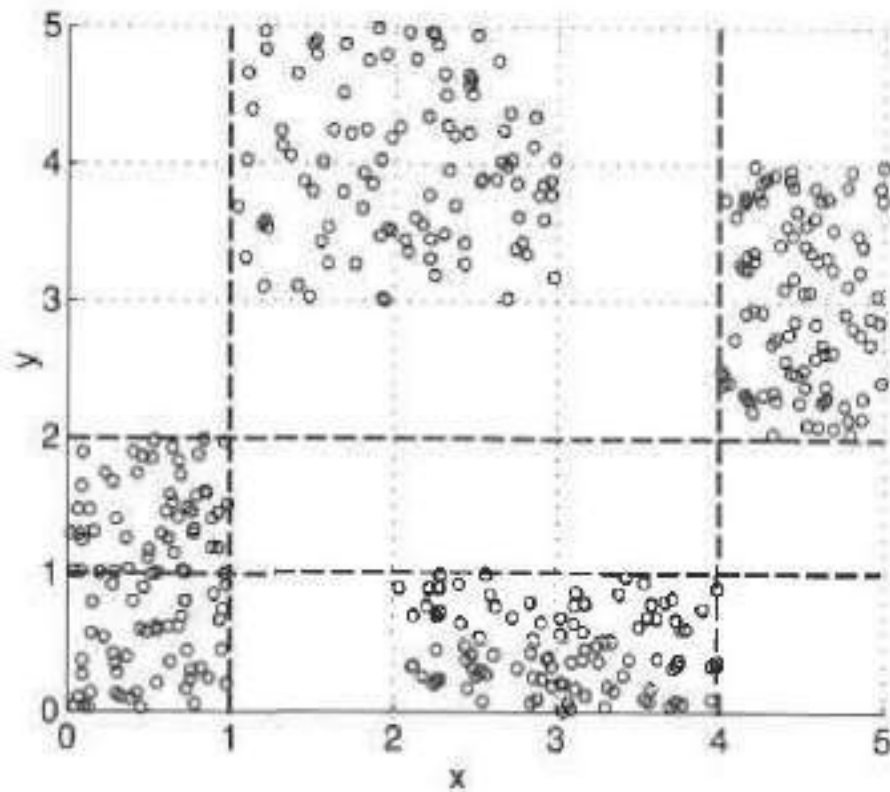
Supervised Discretization

- Class information is used
- An interval constructed with no knowledge of class labels often contains a mixture of class labels
- A conceptually simple approach is to place the splits in a way that maximizes the purity of the intervals
- Entropy based approaches are one of the most promising approaches to discretization

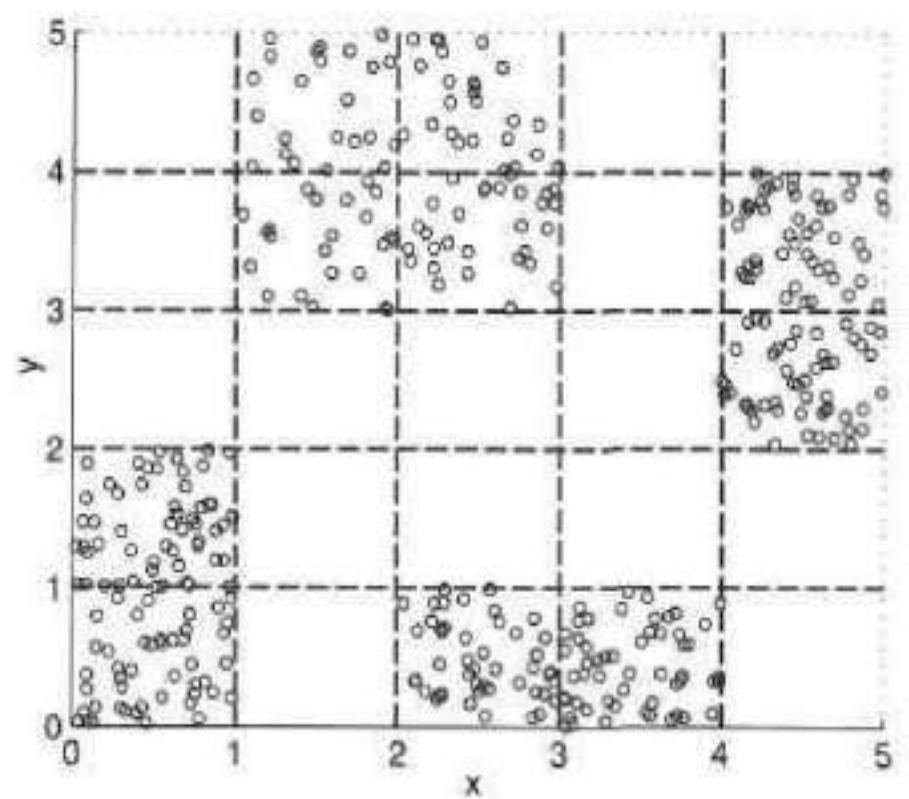
- Entropy of an interval is a measure of the purity of an interval
- If an interval contains only values of one class (is perfectly pure), then the entropy is 0
- If the classes of values in an interval occur equally often (the interval is as impure as possible), then the entropy is a maximum

Approach for partitioning a continuous attribute

- Bisecting the initial values so that the resulting two intervals give minimum entropy
- This technique only needs to consider each value as a possible split point, because it is assumed that intervals contain ordered sets of values
- Splitting process is then repeated with another interval, typically choosing the interval with the worst (highest) entropy, until a user-specified number of intervals is reached, or a stopping criterion is satisfied



(a) Three intervals



(b) Five intervals

Variable Transformation

- Transformation that is applied to all the values of a variable
- Variable transformations should be applied with caution since they change the nature of the data

Simple Functions

- Simple mathematical function is applied to each value individually
- If x is a variable, then examples of such transformations include x^k , $\log x$, e^x , square root of x , $1/x$, $\sin x$, or $|x|$ etc.

Normalization or Standardization

- Goal of standardization or normalization is to make an entire set of values have a particular property
- Min-max normalization: Suppose that the minimum and maximum values for the attribute *income* are \$12,000 and \$98,000, respectively.
- We would like to map *income* to the range [0.0, 1.0]. By min-max normalization, a value of \$73,600 for *income* is

$$\frac{73,600 - 12,000}{98,000 - 12,000} (1.0 - 0) + 0 = 0.716.$$

MEASURES OF SIMILARITY AND DISSIMILARITY

- Similarity and Dissimilarity are important because they are used by a number of data mining algorithms-clustering, nearest neighbour classification, clustering
- In many cases, initial data set is not needed once these similarities or dissimilarities have been computed
- Term 'Proximity' is used to refer to either similarity or dissimilarity

Similarity

- Numerical measure of the degree to which the two objects are alike
- Higher for pairs of objects that are more alike
- Usually non-negative and are often between 0 (no similarity) and 1 (complete similarity)

Dissimilarity

- Numerical measure of the degree to which the two objects are different
- Lower for more similar pairs of objects
- Sometimes fall in the interval $[0,1]$, but it is also common for them to range from 0 to ∞

Transformations

- Applied to convert a similarity to dissimilarity or vice versa
 - Example: We may have similarities ranging from 1 to 10, but an algorithm may work only with dissimilarities
- To transform a proximity measure to fall within a particular range
 - We may have similarities ranging from 1 to 10, but an algorithm may work only with similarities in the interval $[0,1]$

Example:

1.

- If the similarity falls in the interval $[0,1]$, then the dissimilarity can be defined as $d = 1 - s$

2.

- If similarities between objects range from 1(not at all similar) to 10 (completely similar), we can make them to fall within the range $[0,1]$ by using transformation $s' = (s - 1) / 9$
- In general, the transformation of similarities to the interval $[0,1]$ is given by $s' = (s - \min_s) / (\max_s - \min_s)$
- Similarly, the transformation of dissimilarities to the interval $[0,1]$ is given by $d' = (d - \min_d) / (\max_d - \min_d)$

Similarity and Dissimilarity between Simple Attributes

Table 2.7. Similarity and dissimilarity for simple attributes

Attribute Type	Dissimilarity	Similarity
Nominal	$d = \begin{cases} 0 & \text{if } x = y \\ 1 & \text{if } x \neq y \end{cases}$	$s = \begin{cases} 1 & \text{if } x = y \\ 0 & \text{if } x \neq y \end{cases}$
Ordinal	$d = x - y / (n - 1)$ (values mapped to integers 0 to $n-1$, where n is the number of values)	$s = 1 - d$
Interval or Ratio	$d = x - y $	$s = -d, s = \frac{1}{1+d}, s = e^{-d},$ $s = 1 - \frac{d - \min_d}{\max_d - \min_d}$

Dissimilarity between data objects

Distances

- Dissimilarities with certain properties

Euclidean distance

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{k=1}^n (x_k - y_k)^2},$$

- $\mathbf{x}$ and $\mathbf{y}$ are two points
- n is the number of dimensions and
- x_k and y_k are k^{th} attributes (components) of $\mathbf{x}$ and $\mathbf{y}$

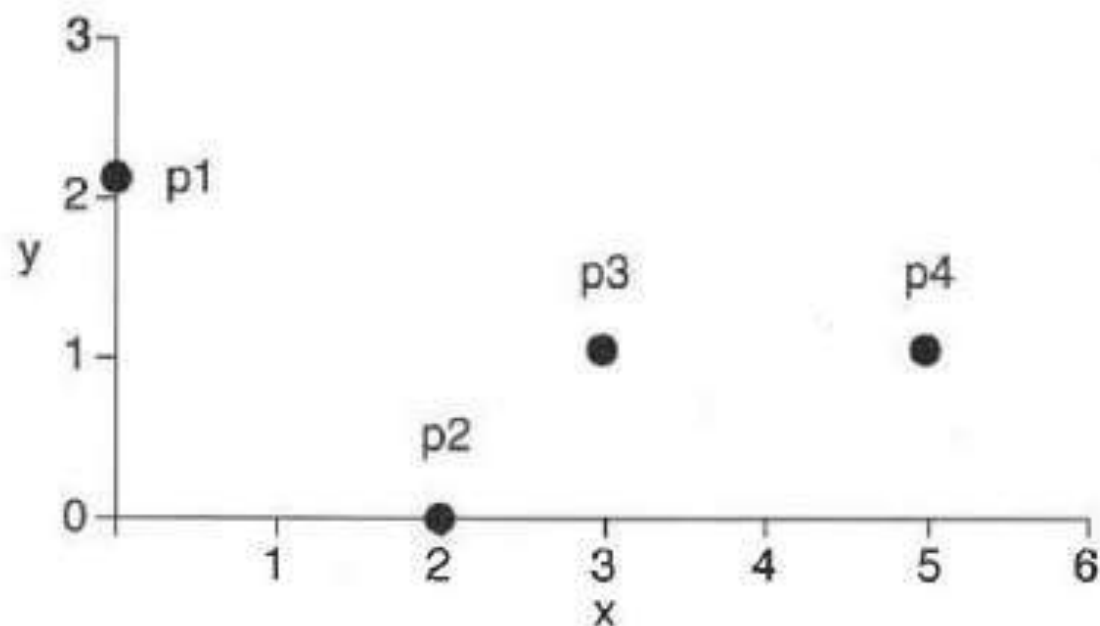


Figure 2.15. Four two-dimensional points.

Table 2.8. x and y coordinates of four points.

point	x coordinate	y coordinate
p1	0	2
p2	2	0
p3	3	1
p4	5	1

Table 2.9. Euclidean distance matrix for Table 2.8.

	p1	p2	p3	p4
p1	0.0	2.8	3.2	5.1
p2	2.8	0.0	1.4	3.2
p3	3.2	1.4	0.0	2.0
p4	5.1	3.2	2.0	0.0

The Euclidean distance measure is generalized by the Minkowski distance metric

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{k=1}^n |x_k - y_k|^r \right)^{1/r},$$

where r is a parameter

Common examples of Minkowski distances

- $r = 1$. City block (Manhattan, taxicab, L_1 norm) distance. A common example is the **Hamming distance**, which is the number of bits that are different between two objects that have only binary attributes, i.e., between two binary vectors.
- $r = 2$. Euclidean distance (L_2 norm).
- $r = \infty$. Supremum (L_{max} or L_∞ norm) distance. This is the maximum difference between any attribute of the objects. More formally, the L_∞ distance is defined by Equation 2.3

$$d(\mathbf{x}, \mathbf{y}) = \lim_{r \rightarrow \infty} \left(\sum_{k=1}^n |x_k - y_k|^r \right)^{1/r}. \quad (2.3)$$

Table 2.10. L_1 distance matrix for Table 2.8.

L_1	p1	p2	p3	p4
p1	0.0	4.0	4.0	6.0
p2	4.0	0.0	2.0	4.0
p3	4.0	2.0	0.0	2.0
p4	6.0	4.0	2.0	0.0

Table 2.11. L_∞ distance matrix for Table 2.8.

L_∞	p1	p2	p3	p4
p1	0.0	2.0	3.0	5.0
p2	2.0	0.0	1.0	3.0
p3	3.0	1.0	0.0	2.0
p4	5.0	3.0	2.0	0.0

Properties of distances

If $d(x, y)$ is the distance between two points, x and y , then the following properties hold

1. Positivity

(a) $d(x, y) \geq 0$ for all x and y

(b) $d(x, y) = 0$ only if $x=y$

2. Symmetry

$d(x, y) = d(y, x)$ for all x and y

3. Triangle Inequality

$d(x, z) \leq d(x, y) + d(y, z)$ for all points x, y and z

- Measures that satisfy all three properties are known as **metrics**
- Many dissimilarities do not satisfy one or more of the metric properties
- Example: Non-metric Dissimilarities: Time

$$d(t_1, t_2) = \begin{cases} t_2 - t_1 & \text{if } t_1 \leq t_2 \\ 24 + (t_2 - t_1) & \text{if } t_1 \geq t_2 \end{cases}$$

$d(1\text{PM}, 2\text{PM}) = 1$ hour, while $d(2\text{PM}, 1\text{PM}) = 23$ hours

Similarities between Data Objects

- If $s(x, y)$ is the similarity between points x and y , then the typical properties of similarities are:

1. $s(\mathbf{x}, \mathbf{y}) = 1$ only if $\mathbf{x} = \mathbf{y}$. ($0 \leq s \leq 1$)

2. $s(\mathbf{x}, \mathbf{y}) = s(\mathbf{y}, \mathbf{x})$ for all $\mathbf{x}$ and $\mathbf{y}$. (Symmetry)

Similarity Measures for Binary Data

Similarity coefficients

- Similarity measures between objects that contain only binary attributes
- typically have values between 0 and 1
- 1 indicates that the two objects are completely similar; 0 indicates that the objects are not at all similar
- Let x and y be two objects that consist of n binary attributes. The comparison of two such objects, i.e., two binary vectors, leads to the following four quantities (frequencies):

f_{00} = the number of attributes where x is 0 and y is 0

f_{01} = the number of attributes where x is 0 and y is 1

f_{10} = the number of attributes where x is 1 and y is 0

f_{11} = the number of attributes where x is 1 and y is 1

Simple Matching Coefficient

$$SMC = \frac{\text{number of matching attribute values}}{\text{number of attributes}} = \frac{f_{11} + f_{00}}{f_{01} + f_{10} + f_{11} + f_{00}}.$$

- Counts both presences and absences equally

Jaccard Coefficient

- Used to handle objects consisting of asymmetric binary attributes

$$J = \frac{\text{number of matching presences}}{\text{number of attributes not involved in 00 matches}} = \frac{f_{11}}{f_{01} + f_{10} + f_{11}}.$$

$$\mathbf{x} = (1, 0, 0, 0, 0, 0, 0, 0, 0, 0)$$

$$\mathbf{y} = (0, 0, 0, 0, 0, 0, 1, 0, 0, 1)$$

$f_{01} = 2$ the number of attributes where $\mathbf{x}$ was 0 and $\mathbf{y}$ was 1

$f_{10} = 1$ the number of attributes where $\mathbf{x}$ was 1 and $\mathbf{y}$ was 0

$f_{00} = 7$ the number of attributes where $\mathbf{x}$ was 0 and $\mathbf{y}$ was 0

$f_{11} = 0$ the number of attributes where $\mathbf{x}$ was 1 and $\mathbf{y}$ was 1

$$SMC = \frac{f_{11} + f_{00}}{f_{01} + f_{10} + f_{11} + f_{00}} = \frac{0 + 7}{2 + 1 + 0 + 7} = 0.7$$

$$J = \frac{f_{11}}{f_{01} + f_{10} + f_{11}} = \frac{0}{2 + 1 + 0} = 0$$

Cosine Similarity

- One of the most common measure of document similarity
- Can handle non binary vectors
- If $\mathbf{x}$ and $\mathbf{y}$ are two document vectors, then

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|},$$

- Where $\cdot$ Indicates the vector dot product
- $\|\mathbf{x}\|$ is the length of vector $\mathbf{x}$,

$$\mathbf{x} \cdot \mathbf{y} = \sum_{k=1}^n x_k y_k$$

$$\|\mathbf{x}\| = \sqrt{\sum_{k=1}^n x_k^2} = \sqrt{\mathbf{x} \cdot \mathbf{x}}$$

$$\mathbf{x} = (3, 2, 0, 5, 0, 0, 0, 2, 0, 0)$$

$$\mathbf{y} = (1, 0, 0, 0, 0, 0, 0, 1, 0, 2)$$

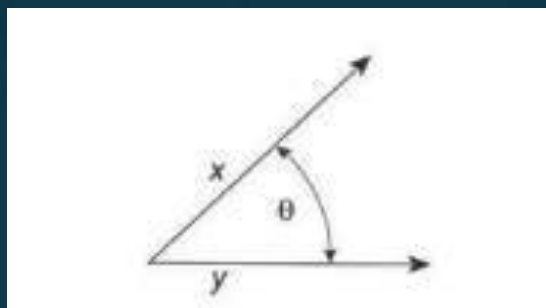
$$\mathbf{x} \cdot \mathbf{y} = 3 * 1 + 2 * 0 + 0 * 0 + 5 * 0 + 0 * 0 + 0 * 0 + 0 * 0 + 2 * 1 + 0 * 0 + 0 * 2 = 5$$

$$\|\mathbf{x}\| = \sqrt{3 * 3 + 2 * 2 + 0 * 0 + 5 * 5 + 0 * 0 + 0 * 0 + 0 * 0 + 2 * 2 + 0 * 0 + 0 * 0} = 6.48$$

$$\|\mathbf{y}\| = \sqrt{1 * 1 + 0 * 0 + 0 * 0 + 0 * 0 + 0 * 0 + 0 * 0 + 0 * 0 + 1 * 1 + 0 * 0 + 2 * 2} = 2.24$$

$$\cos(\mathbf{x}, \mathbf{y}) = 0.31$$

- cosine similarity really is a measure of the (cosine of the) angle between x and y



- if the cosine similarity is 1, the angle between x and y is 0 degree, and x and y are the same except for magnitude (length)
- If the cosine similarity is 0, then the angle between x and y is 90 degree, and they do not share any terms (words)

- Dividing $\mathbf{x}$ and $\mathbf{y}$ by their lengths normalizes them to have a length of 1

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}}{\|\mathbf{x}\|} \cdot \frac{\mathbf{y}}{\|\mathbf{y}\|} = \mathbf{x}' \cdot \mathbf{y}'$$

- For vectors with a length of 1, the cosine measure can be calculated by taking a simple dot product
- when many cosine similarities between objects are being computed, normalizing the objects to have unit length can reduce the time required

Difference between SMC and Jaccard index

The SMC is very similar to the more popular Jaccard index. The main difference is that the SMC has the term f_{00} in its numerator and denominator, whereas the Jaccard index does not. Thus, the SMC counts both mutual presences (when an attribute is present in both sets) and mutual absence (when an attribute is absent in both sets) as matches and compares it to the total number of attributes in the universe, whereas the Jaccard index only counts mutual presence as matches and compares it to the number of attributes that have been chosen by at least one of the two sets

In market basket analysis, for example, the basket of two consumers who we wish to compare might only contain a small fraction of all the available products in the store, so the SMC will usually return very high values of similarities even when the baskets bear very little resemblance, thus making the Jaccard index a more appropriate measure of similarity in that context. For example, consider a supermarket with 1000 products and two customers. The basket of the first customer contains salt and pepper and the basket of the second contains salt and sugar. In this scenario, the similarity between the two baskets as measured by the Jaccard index would be $1/3$, but the similarity becomes 0.998 using the SMC

ASSOCIATION ANALYSIS

Association analysis

- Methodology useful for discovering interesting relationships hidden in large data sets
- Uncovered relationships can be represented in the form of association rules

Association Rule Mining

- Given a set of transactions, find rules that will predict the occurrence of an item based on the occurrences of other items in the transaction

Market-Basket transactions

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Example of Association Rules:

{Diaper} → {Beer}

(strong relationship exists between the sale of diapers and beer)

- Discovering patterns from a large transaction data set can be computationally expensive
- Some of the discovered patterns are potentially spurious because they may happen simply by chance
- Association analysis is also applicable to other application domains such as bioinformatics, medical diagnosis, Web mining and scientific data analysis

Basic Terminology

Binary representation:

TID	Bread	Milk	Diapers	Beer	Eggs	Cola
1	1	1	0	0	0	0
2	1	0	1	1	1	0
3	0	1	1	1	0	1
4	1	1	1	1	0	0
5	1	1	1	0	0	1

- Let $I = \{i_1, i_2, i_3, \dots, i_d\}$ be the set of all items in a market basket data
- $T = \{t_1, t_2, t_3, \dots, t_N\}$ be the set of all transactions
- Each transaction t_i contains a subset of items chosen from I

- Itemset

- Collection of zero or more items
- Example: {Milk, Bread, Diaper}

- k-itemset

- An itemset that contains k items

- **Transaction width**

- Number of items present in a transaction

- **Support count ()**

- Number of transactions that contain a particular itemset

$$\sigma(X) = |\{t_i | X \subseteq t_i, t_i \in T\}|.$$

- E.g. $\sigma(\{\text{Milk, Bread, Diaper}\}) = 2$

Association Rule

- An implication expression of the form $X \rightarrow Y$, where X and Y are disjoint itemsets
- Strength of an association rule can be measured in terms of its Support and Confidence

Support

- Determines how often a rule is applicable to a given data set
- Fraction of transactions that contain both X and Y

Confidence

- Determines how frequently items in Y appear in transactions that contain X

$$\begin{aligned}\text{Support, } s(X \rightarrow Y) &= \frac{\sigma(X \cup Y)}{N} \\ \text{Confidence, } c(X \rightarrow Y) &= \frac{\sigma(X \cup Y)}{\sigma(X)}\end{aligned}$$

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Consider the rule: $\{\text{Milk, Diaper}\} \Rightarrow \{\text{Beer}\}$

$$s = \frac{\sigma(\text{Milk, Diaper, Beer})}{N} = \frac{2}{5} = 0.4$$

$$c = \frac{\sigma(\text{Milk, Diaper, Beer})}{\sigma(\text{Milk, Diaper})} = \frac{2}{3} = 0.67$$

Why Use Support and Confidence?

- A rule that has very low support may occur simply by chance
- A low support rule is also likely to be uninteresting from a business perspective because it may not be profitable to promote items that customers seldom buy together
- For these reasons, support is often used to eliminate uninteresting rules
- Support also has a desirable property that can be exploited for the efficient discovery of association rules

- **Confidence** measures the reliability of the inference made by a rule
- For a given rule $X \rightarrow Y$, the higher the confidence, the more likely it is for Y to be present in transactions that contain X
- Confidence also provides an estimate of the conditional probability of Y given X

Formulation of Association Rule Mining Problem

- Definition (Association Rule Discovery):

Given a set of transactions T , find all the rules having support $\geq \text{minsup}$ and confidence $\geq \text{minconf}$

(where minsup and minconf are the corresponding support and confidence thresholds)

- A brute-force approach for mining association rules is to compute the support and confidence for every possible rule
- This approach is prohibitively expensive because there are exponentially many rules that can be extracted from a data set
- Total number of possible rules extracted from a data set that contains d items is

$$R = 3^d - 2^{d+1} + 1$$

- Even for the small data set, this approach requires us to compute the support and confidence for large number of rules
- More than 80% of the rules are discarded after applying $\text{minsup} = 20\%$ and $\text{minconf} = 50\%$, thus making most of the computations become wasted
- To avoid performing needless computations, it would be useful to prune the rules early without having to compute their support and confidence value

Solution:

- Decouple the support and confidence requirements
- Support of a rule $X \rightarrow Y$ depends only on the support of its corresponding itemset, $X \cup Y$

```
{Beer, Diapers} → {Milk}, {Beer, Milk} → {Diapers},  
{Diapers, Milk} → {Beer}, {Beer} → {Diapers, Milk},  
{Milk} → {Beer, Diapers}, {Diapers} → {Beer, Milk}.
```

- These rules have identical support because they involve items from the same itemset {Beer, Diapers, Milk}

Mining Association Rules

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Example of Rules:

$\{\text{Milk, Diaper}\} \rightarrow \{\text{Beer}\}$ ($s=0.4$, $c=0.67$)

$\{\text{Milk, Beer}\} \rightarrow \{\text{Diaper}\}$ ($s=0.4$, $c=1.0$)

$\{\text{Diaper, Beer}\} \rightarrow \{\text{Milk}\}$ ($s=0.4$, $c=0.67$)

$\{\text{Beer}\} \rightarrow \{\text{Milk, Diaper}\}$ ($s=0.4$, $c=0.67$)

$\{\text{Diaper}\} \rightarrow \{\text{Milk, Beer}\}$ ($s=0.4$, $c=0.5$)

$\{\text{Milk}\} \rightarrow \{\text{Diaper, Beer}\}$ ($s=0.4$, $c=0.5$)

Observations:

- All the above rules are binary partitions of the same itemset: {Milk, Diaper, Beer}
- Rules originating from the same itemset have identical support but can have different confidence
- Thus, we may decouple the support and confidence requirements

- Common strategy adopted by many association rule mining algorithms is to decompose the problem into two major subtasks:
- **Frequent Itemset Generation**
 - whose objective is to find all the itemsets that satisfy the minsup threshold. These itemsets are called frequent itemsets
- **Rule Generation**
 - whose objective is to extract all the high-confidence rules from the frequent itemsets found in the previous step. These rules are called strong rules

The computational requirements for frequent itemset generation are generally more expensive than those of rule generation

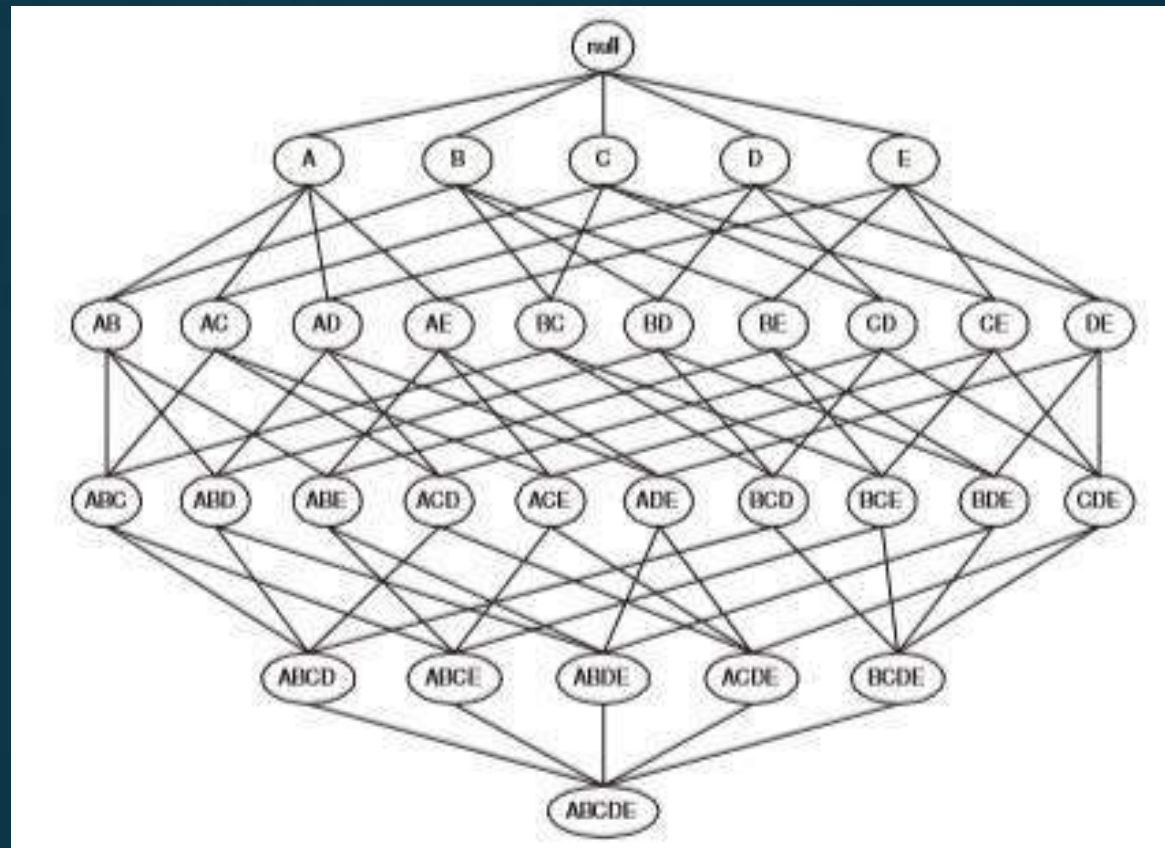
Frequent Itemset Generation

A lattice structure can be used to enumerate the list of all possible itemsets

Let $I = \{A, B, C, D, E\}$

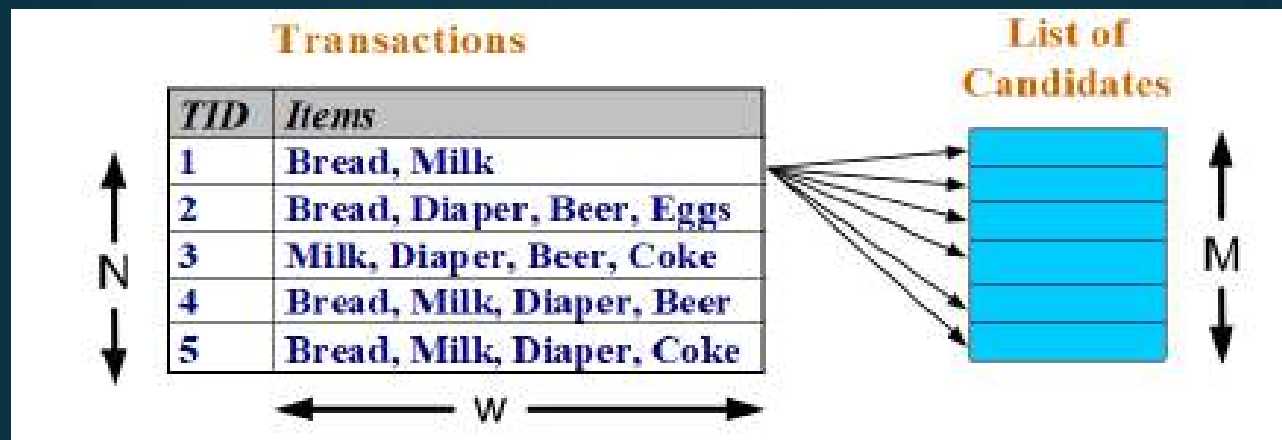
Given d items, there
are 2^d

possible candidate
itemsets



Brute-force approach:

- Each itemset in the lattice is a **candidate** frequent itemset
- Count the support count of each candidate by scanning the database



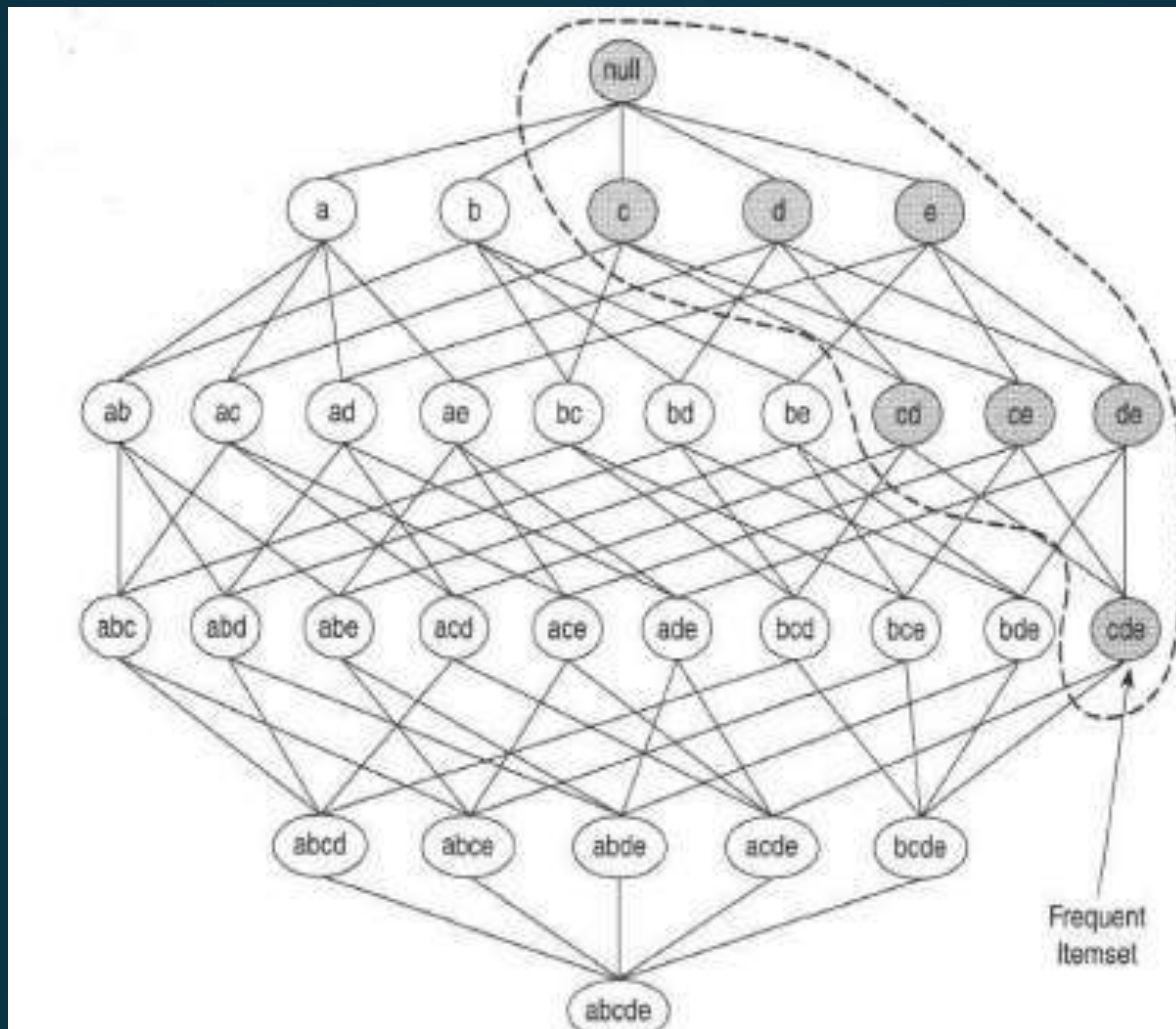
- Match each transaction against every candidate
- Complexity $\sim O(NMw) \Rightarrow$ **Expensive** since $M = 2^d$!!!

Ways to reduce computational complexity

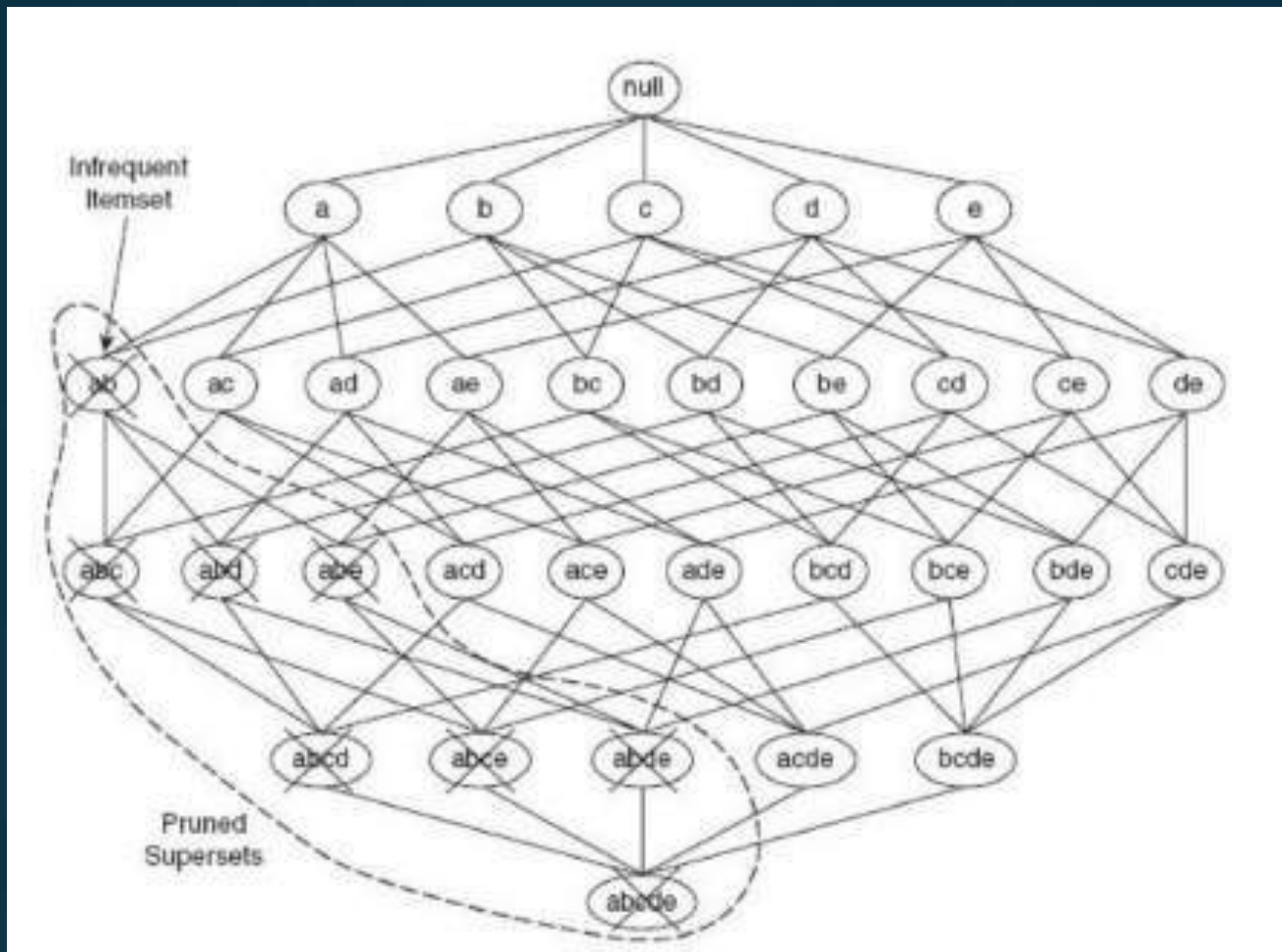
- Reduce the number of candidate itemsets(M)
- The Apriori Principle is an effective way to eliminate some of the candidate itemsets
- Reduce the number of comparisons
- Use more advanced data structures either to store the candidate itemsets or to compress the data set

The Apriori Principle

- Used to Reduce the number of candidate itemsets
- If an itemset is frequent, then all of its subsets must also be frequent
- Suppose $\{c, d, e\}$ is a frequent itemset
- Any transaction that contains $\{c, d, e\}$ must also contain its subsets, $\{c, d\}, \{c, e\}, \{d, e\}, \{c\}, \{d\}$, and $\{e\}$
- As a result, if $\{c, d, e\}$ is frequent, then all subsets of $\{c, d, e\}$ must also be frequent



- Conversely, if an itemset such as $\{a, b\}$ is infrequent, then all of its supersets must be infrequent too
- the entire subgraph containing the supersets of $\{a, b\}$ can be pruned immediately once $\{a, b\}$ is found to be infrequent
- This strategy of trimming the exponential search space based on the support measure is known as **Support-based pruning**



- Such a pruning strategy is made possible by a key property of the support measure
- Anti-monotone property of the support measure:
The support for an itemset never exceeds the support for its subsets

Monotonicity and Antimonotonicity Property

- Let I be a set of items, and $J = 2^I$ be the power set of I
- A measure f is **monotone** (or upward closed) if

$$\forall X, Y \in J : (X \subseteq Y) \longrightarrow f(X) \leq f(Y)$$

i.e. if X is a subset of Y , then $f(X)$ must not exceed $f(Y)$

- f is **anti-monotone** (or downward closed) if

$$\forall X, Y \in J : (X \subseteq Y) \longrightarrow f(Y) \leq f(X)$$

i.e. if X is a subset of Y , then $f(Y)$ must not exceed $f(X)$

Illustrating Apriori Principle

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk



Items (1-itemsets)

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3$$

$$6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$

Illustrating Apriori Principle

TID	Items
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset
{Bread, Milk}
{Bread, Beer }
{Bread, Diaper}
{Beer, Milk}
{Diaper, Milk}
{Beer, Diaper}

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3$$

$$6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$

Illustrating Apriori Principle

TID	Items
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread, Milk}	3
{Beer, Bread}	2
{Bread, Diaper}	3
{Beer, Milk}	2
{Diaper, Milk}	3
{Beer, Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$

Illustrating Apriori Principle

TID	Items
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread, Milk}	3
{Bread, Beer}	2
{Bread, Diaper}	3
{Milk, Beer}	2
{Milk, Diaper}	3
{Beer, Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,
 ${}^6C_1 + {}^6C_2 + {}^6C_3$
 $6 + 15 + 20 = 41$
 With support-based pruning,
 $6 + 6 + 4 = 16$



Itemset
{ Beer, Diaper, Milk}
{ Beer, Bread, Diaper}
{Bread, Diaper, Milk}
{ Beer, Bread, Milk}

Triplets (3-itemsets)

Illustrating Apriori Principle

TID	Items
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread, Milk}	3
{Bread, Beer}	2
{Bread, Diaper}	3
{Milk, Beer}	2
{Milk, Diaper}	3
{Beer, Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$



Triplets (3-itemsets)

Itemset	Count
{ Beer, Diaper, Milk}	2
{ Beer, Bread, Diaper}	2
{Bread, Diaper, Milk}	2
{Beer, Bread, Milk}	1

Illustrating Apriori Principle

TID	Items
1	Bread, Milk
2	Beer, Bread, Diaper, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread, Milk}	3
{Bread, Beer}	2
{Bread, Diaper}	3
{Milk, Beer}	2
{Milk, Diaper}	3
{Beer, Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16 \\ 6 + 6 + 1 = 13$$



Triplets (3-itemsets)

Itemset	Count
{ Beer, Diaper, Milk }	2
{ Beer, Bread, Diaper }	2
{ Bread, Diaper, Milk }	2
{ Beer, Bread, Milk }	1

Algorithm 6.1 Frequent itemset generation of the *Apriori* algorithm.

```
1:  $k = 1$ .
2:  $F_k = \{ i \mid i \in I \wedge \sigma(\{i\}) \geq N \times \text{minsup} \}$ .    {Find all frequent 1-itemsets}
3: repeat
4:    $k = k + 1$ .
5:    $C_k = \text{apriori-gen}(F_{k-1})$ .    {Generate candidate itemsets}
6:   for each transaction  $t \in T$  do
7:      $C_t = \text{subset}(C_k, t)$ .    {Identify all candidates that belong to  $t$ }
8:     for each candidate itemset  $c \in C_t$  do
9:        $\sigma(c) = \sigma(c) + 1$ .    {Increment support count}
10:    end for
11:  end for
12:   $F_k = \{ c \mid c \in C_k \wedge \sigma(c) \geq N \times \text{minsup} \}$ .    {Extract the frequent  $k$ -itemsets}
13: until  $F_k = \emptyset$ 
14:  $\text{Result} = \bigcup F_k$ .
```

- F_k : frequent k -itemsets
- C_k : candidate k -itemsets

Algorithm

- Let $k=1$
- Generate $F_1 = \{\text{frequent 1-itemsets}\}$
- Repeat until F_k is empty
 - ◆ **Candidate Generation:** Generate C_{k+1} from F_k
 - ◆ **Candidate Pruning:** Prune candidate itemsets in C_{k+1} containing subsets of length k that are infrequent
 - ◆ **Support Counting:** Count the support of each candidate in C_{k+1} by scanning the DB
 - ◆ **Candidate Elimination:** Eliminate candidates in C_{k+1} that are infrequent, leaving only those that are frequent $\Rightarrow F_{k+1}$

Candidate Generation

List of requirements for an effective candidate generation procedure:

- It should avoid generating too many unnecessary candidates(atleast one of its subsets is infrequent)
- It must ensure that the candidate set is complete(no frequent itemsets are left out)
- It should not generate same candidate itemset more than once

Candidate Generation: Brute-force method

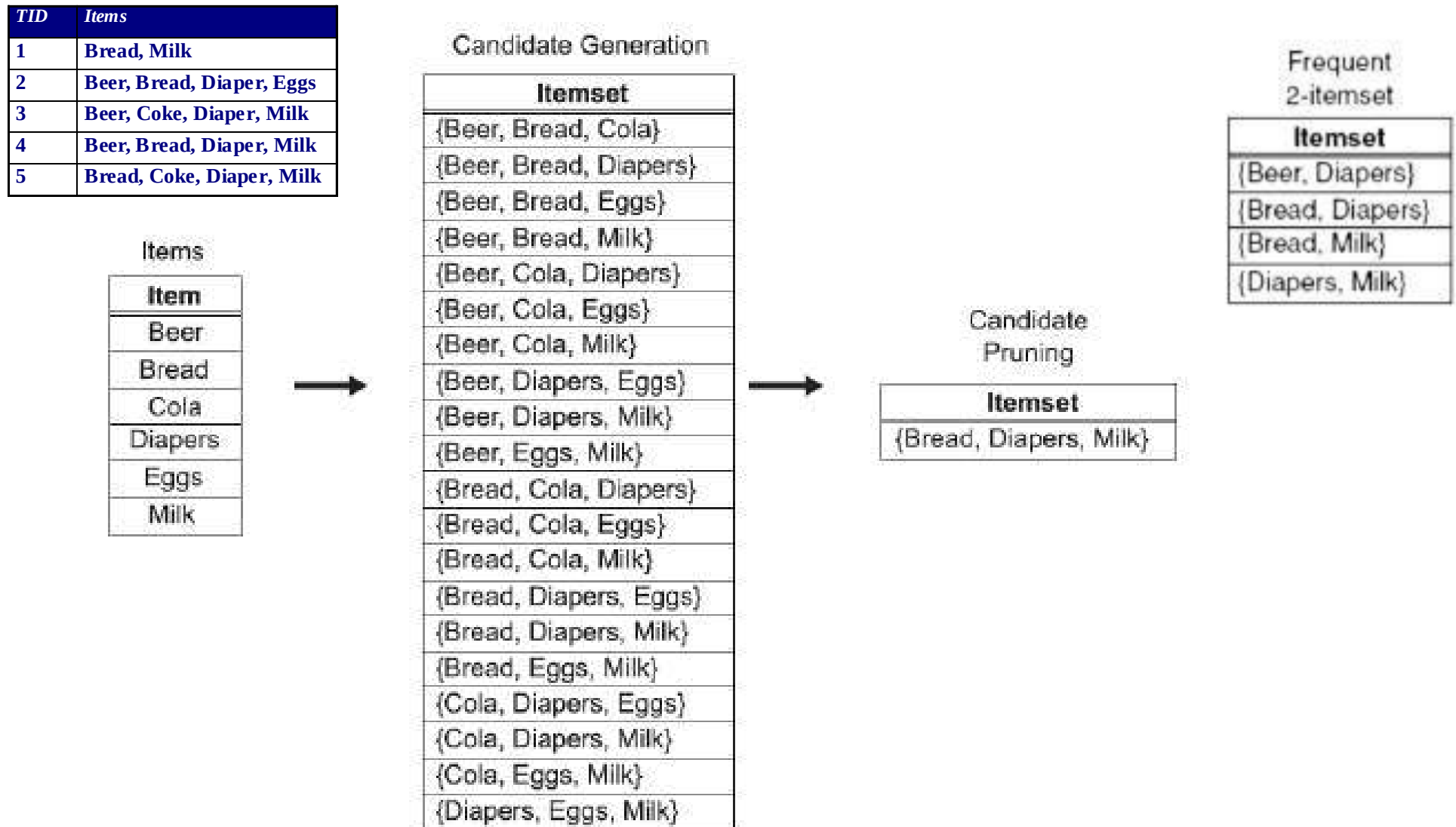


Figure 5.6. A brute-force method for generating candidate 3-itemsets.

Candidate Generation: Merge Fk-1 and F1 itemsets

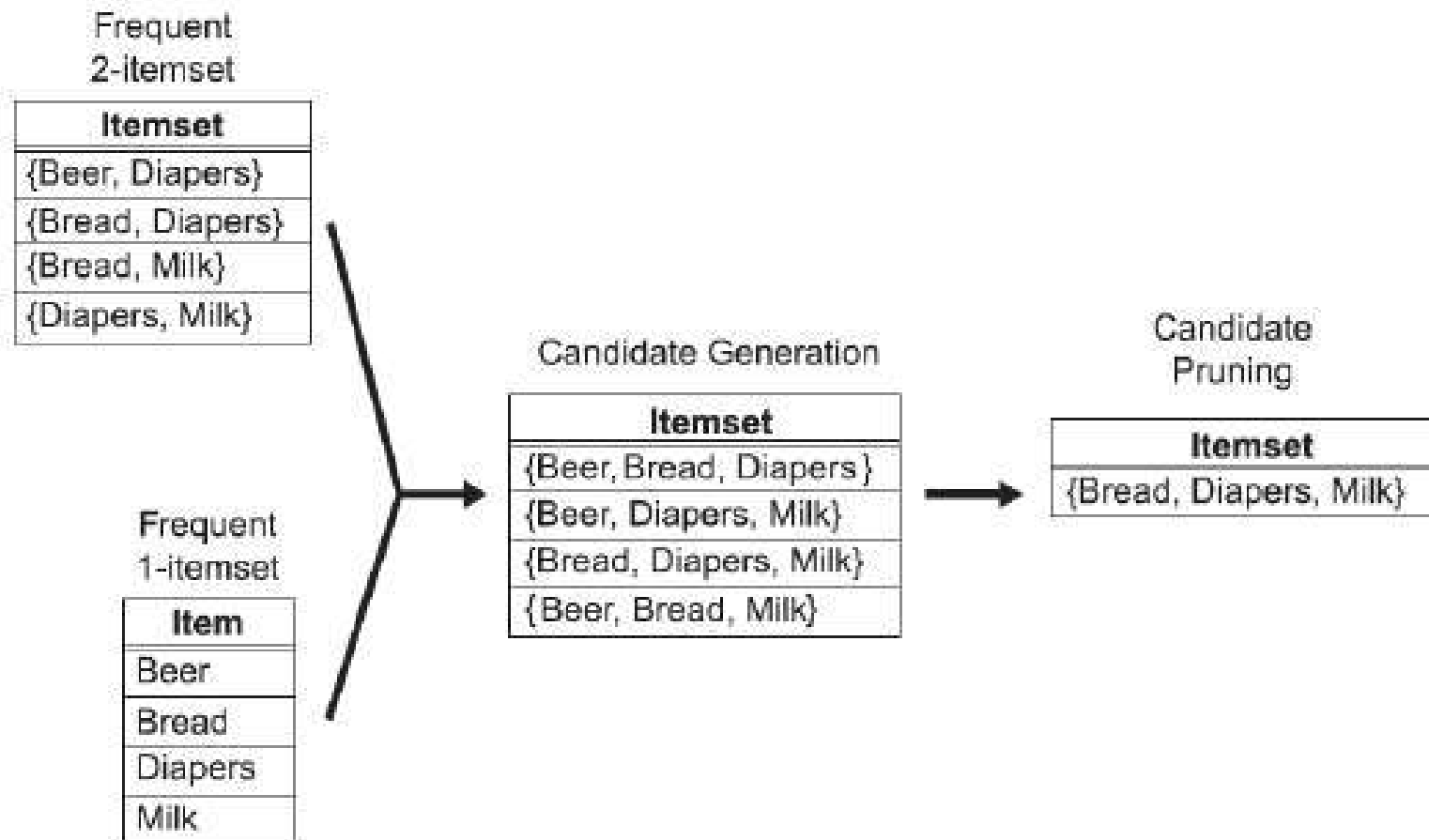


Figure 5.7. Generating and pruning candidate k -itemsets by merging a frequent $(k-1)$ -itemset with a frequent item. Note that some of the candidates are unnecessary because their subsets are infrequent.

Candidate Generation: Fk-1 x Fk-1 Method

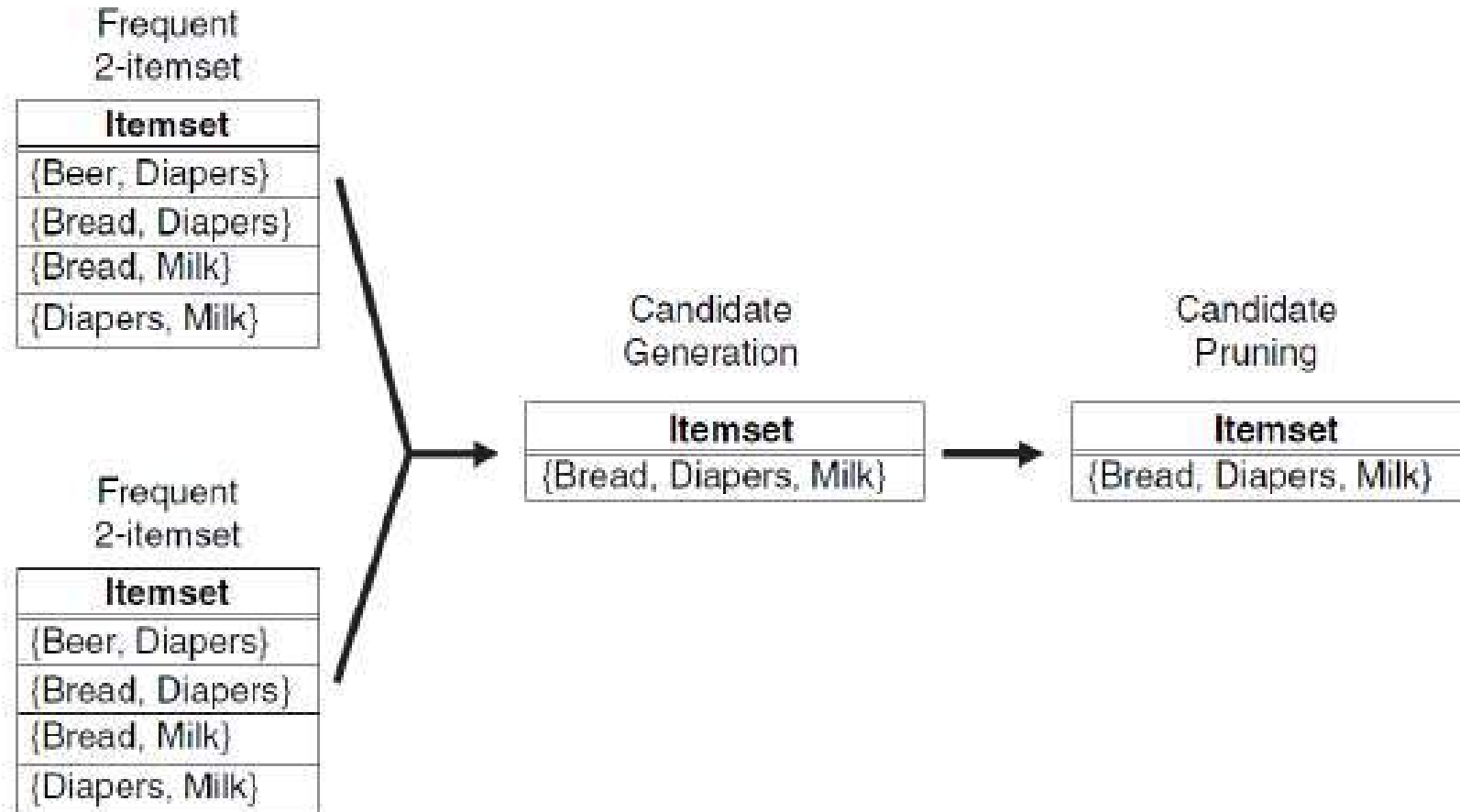


Figure 5.8. Generating and pruning candidate k -itemsets by merging pairs of frequent $(k-1)$ -itemsets.

The Apriori Algorithm: Example

TID	List of Items
T100	I1, I2, I5
T100	I2, I4
T100	I2, I3
T100	I1, I2, I4
T100	I1, I3
T100	I2, I3
T100	I1, I3
T100	I1, I2, I3, I5
T100	I1, I2, I3

- Consider a database, D , consisting of 9 transactions.
- Suppose min. support count required is 2 (i.e. $\text{min_sup} = 2/9 = 22\%$)
- Let minimum confidence required is 70%.
- We have to first find out the frequent itemset using Apriori algorithm.
- Then, Association rules will be generated using min. support & min. confidence.

Step 1: Generating 1-itemset Frequent Pattern

Scan D for
count of each
candidate

Itemset	Sup.Count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

C_1

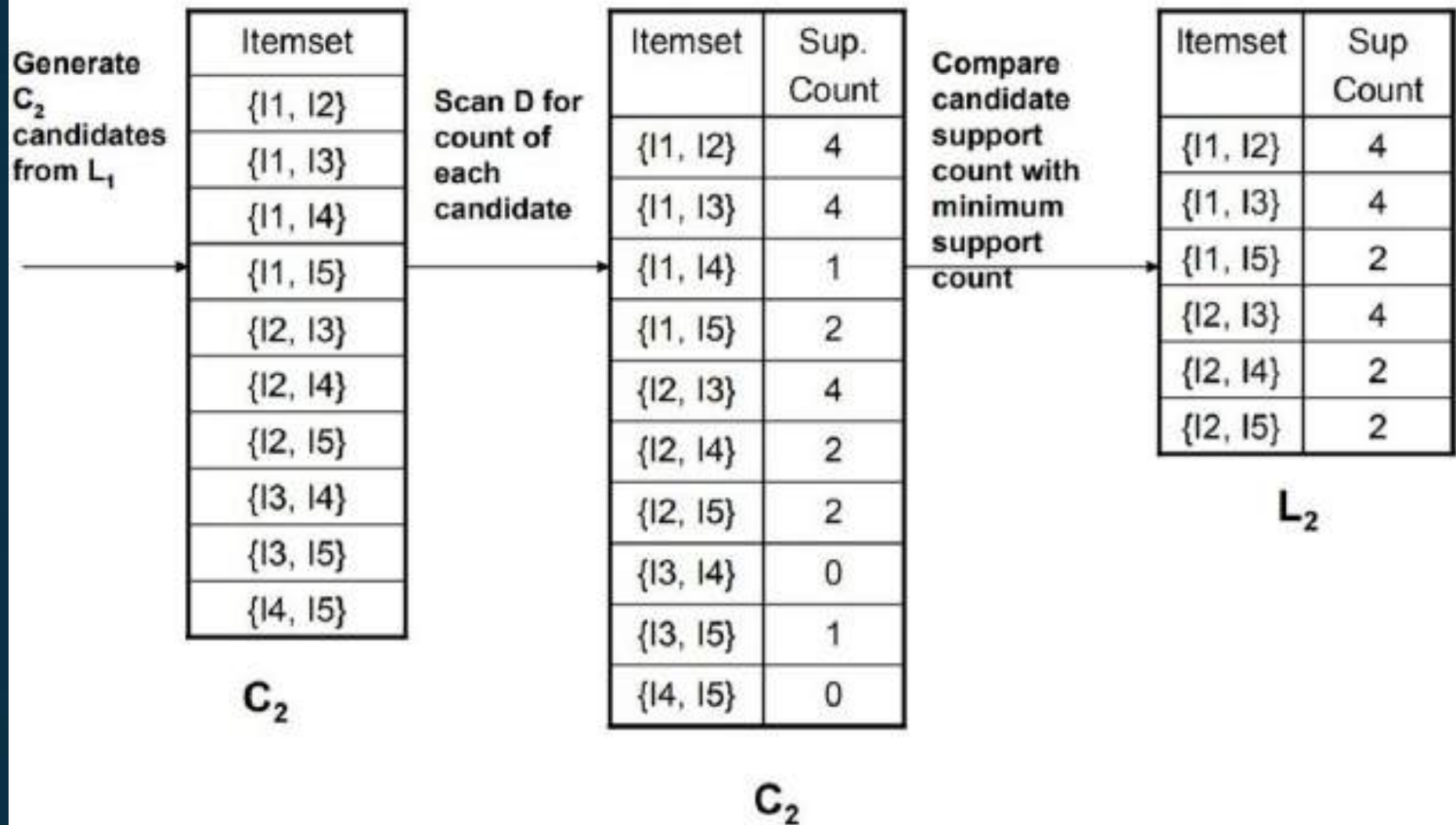
Compare candidate
support count with
minimum support
count

Itemset	Sup.Count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

L_1

- The set of frequent 1-itemsets, L_1 , consists of the candidate 1-itemsets satisfying minimum support.
- In the first iteration of the algorithm, each item is a member of the set of candidate.

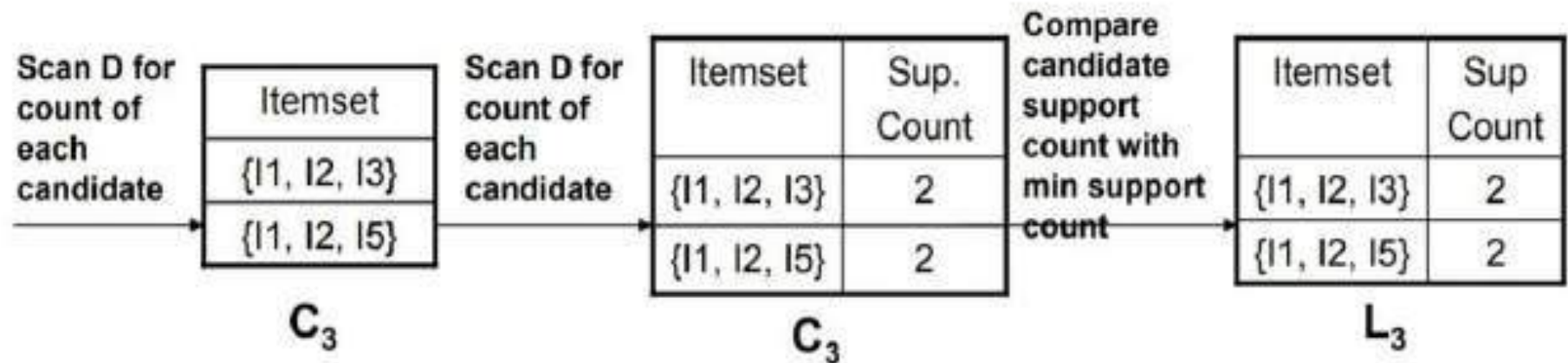
Step 2: Generating 2-itemset Frequent Pattern



Step 2: Generating 2-itemset Frequent Pattern

- To discover the set of frequent 2-itemsets, L_2 , the algorithm uses $L_1 \text{ Join } L_1$ to generate a candidate set of 2-itemsets, C_2 .
- Next, the transactions in D are scanned and the support count for each candidate itemset in C_2 is accumulated (as shown in the middle table).
- The set of frequent 2-itemsets, L_2 , is then determined, consisting of those candidate 2-itemsets in C_2 having minimum support.
- **Note:** We haven't used Apriori Property yet.

Step 3: Generating 3-itemset Frequent Pattern



- The generation of the set of candidate 3-itemsets, C_3 , involves use of the Apriori Property.
- In order to find C_3 , we compute $L_2 \text{ Join } L_2$.
- $C_3 = L_2 \text{ Join } L_2 = \{\{I1, I2, I3\}, \{I1, I2, I5\}, \{I1, I3, I5\}, \{I2, I3, I4\}, \{I2, I3, I5\}, \{I2, I4, I5\}\}$.
- Now, Join step is complete and Prune step will be used to reduce the size of C_3 . Prune step helps to avoid heavy computation due to large C_k .

Step 3: Generating 3-itemset Frequent Pattern

- Based on the **Apriori property** that all subsets of a frequent itemset must also be frequent, we can determine that four latter candidates cannot possibly be frequent. How ?
- For example , lets take **{I1, I2, I3}**. The 2-item subsets of it are {I1, I2}, {I1, I3} & {I2, I3}. Since all 2-item subsets of {I1, I2, I3} are members of L_2 , We will keep {I1, I2, I3} in C_3 .
- Lets take another example of **{I2, I3, I5}** which shows how the pruning is performed. The 2-item subsets are {I2, I3}, {I2, I5} & {I3,I5}.
- BUT, {I3, I5} is not a member of L_2 and hence it is not frequent **violating Apriori Property**. Thus We will have to remove {I2, I3, I5} from C_3 .
- Therefore, $C_3 = \{\{I1, I2, I3\}, \{I1, I2, I5\}\}$ after checking for all members of **result of Join operation for Pruning**.
- Now, the transactions in D are scanned in order to determine L_3 , **consisting of those candidates 3-itemsets in C_3 having minimum support**.

Step 4: Generating 4-itemset Frequent Pattern

- The algorithm uses $L_3 \text{ Join } L_3$ to generate a candidate set of 4-itemsets, C_4 . Although the join results in $\{\{I1, I2, I3, I5\}\}$, this itemset is pruned since its subset $\{\{I2, I3, I5\}\}$ is not frequent.
- Thus, $C_4 = \varnothing$, and algorithm terminates, having found all of the frequent items. This completes our Apriori Algorithm.
- What's Next ?
These frequent itemsets will be used to generate strong association rules (where strong association rules satisfy both minimum support & minimum confidence).

Step 5: Generating Association Rules from Frequent Itemsets

Procedure:

- For each frequent itemset I , generate all nonempty subsets of I .
- For every nonempty subset s of I , output the rule " $s \rightarrow (I-s)$ " if $\text{support_count}(I) / \text{support_count}(s) \geq \text{min_conf}$ where min_conf is minimum confidence threshold.

Back To Example:

We had $L = \{\{I1\}, \{I2\}, \{I3\}, \{I4\}, \{I5\}, \{I1,I2\}, \{I1,I3\}, \{I1,I5\}, \{I2,I3\}, \{I2,I4\}, \{I2,I5\}, \{I1,I2,I3\}, \{I1,I2,I5\}\}$.

- Lets take $I = \{I1,I2,I5\}$.
- Its all nonempty subsets are $\{I1,I2\}, \{I1,I5\}, \{I2,I5\}, \{I1\}, \{I2\}, \{I5\}$.

Step 5: Generating Association Rules from Frequent Itemsets

- Let minimum confidence threshold is , say 70%.
- The resulting association rules are shown below, each listed with its confidence.
 - R1: $I1 \wedge I2 \rightarrow I5$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I1, I2\} = 2/4 = 50\%$
 - R1 is Rejected.
 - R2: $I1 \wedge I5 \rightarrow I2$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I1, I5\} = 2/2 = 100\%$
 - R2 is Selected.
 - R3: $I2 \wedge I5 \rightarrow I1$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I2, I5\} = 2/2 = 100\%$
 - R3 is Selected.

Step 5: Generating Association Rules from Frequent Itemsets

- R4: $I1 \rightarrow I2 \wedge I5$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I1\} = 2/6 = 33\%$
 - R4 is Rejected.
- R5: $I2 \rightarrow I1 \wedge I5$
 - Confidence = $sc\{I1, I2, I5\} / \{I2\} = 2/7 = 29\%$
 - R5 is Rejected.
- R6: $I5 \rightarrow I1 \wedge I2$
 - Confidence = $sc\{I1, I2, I5\} / \{I5\} = 2/2 = 100\%$
 - R6 is Selected.

In this way, We have found three strong association rules.

Frequent itemset generation part of the Apriori algorithm has two important characteristics

- **Level-wise algorithm**

It traverses the itemset lattice one level at a time, from frequent 1-itemsets to the maximum size of frequent itemset

- **Employs a generate-and-test strategy for finding frequent itemset**

- At each iteration, new candidate itemsets are generated from the frequent itemsets found in the previous iteration. The support for each candidate is then counted and tested against the minsup threshold

Exercise:

Consider following transactions:

Transaction	Items
T1	<u>I1</u> , <u>I2</u> , I3
T2	<u>I2</u> , <u>I3</u> , I4
T3	<u>I4</u> , <u>I5</u>
T4	<u>I1</u> , <u>I2</u> , I4
T5	<u>I1</u> , <u>I2</u> , I3, I5
T6	<u>I1</u> , <u>I2</u> , I3, I4

Generate frequent item sets Using Apriori Algorithm. Also generate association rules from it.

Given: Minimum support=3, Minimum Confidence= 60%

FP Growth Algorithm

Apriori: uses a generate-and-test approach – generates candidate itemsets and tests if they are frequent

- Generation of candidate itemsets is expensive (in both space and time)
- Support counting is expensive

FP-Growth: allows frequent itemset discovery without candidate itemset generation. Two step approach:

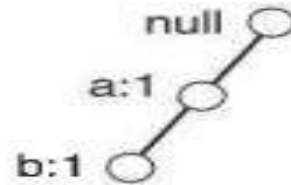
Step 1: Build a compact data structure called the FP-tree

Built using 2 passes over the data-set.

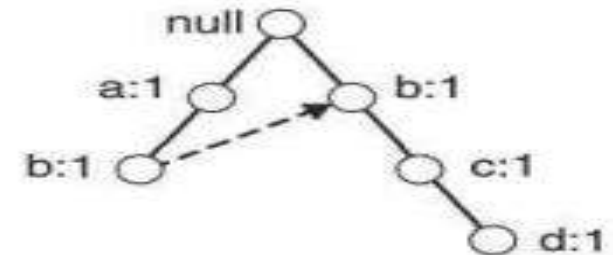
Step 2: Extracts frequent itemsets directly from the FP-tree

Construction of FP Tree

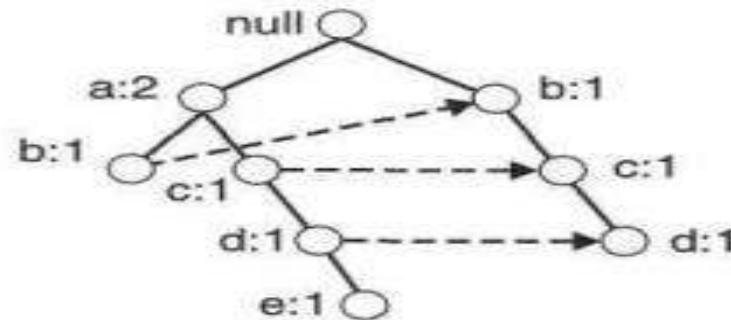
Transaction Data Set	
TID	Items
1	{a,b}
2	{b,c,d}
3	{a,c,d,e}
4	{a,d,e}
5	{a,b,c}
6	{a,b,c,d}
7	{a}
8	{a,b,c}
9	{a,b,d}
10	{b,c,e}



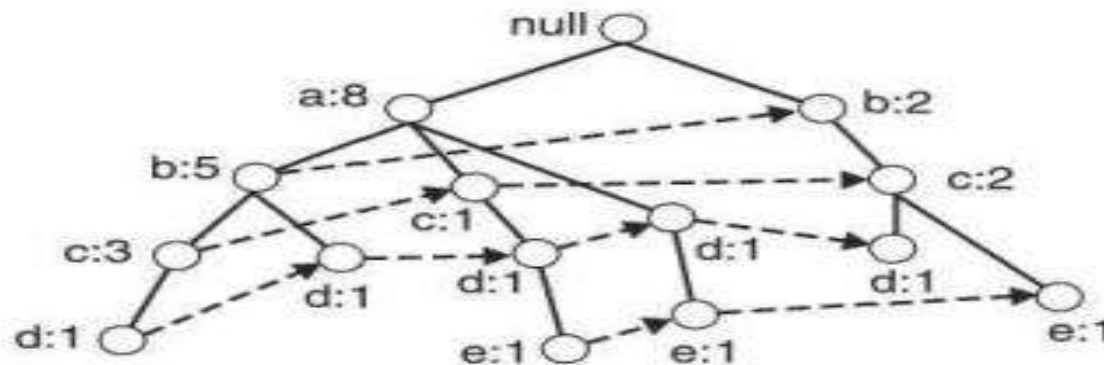
(i) After reading TID=1



(ii) After reading TID=2



(iii) After reading TID=3



(iv) After reading TID=10

Example 2:

Transaction ID	Items
T1	{ <u>E</u> ,K,M,N,O,Y}
T2	{ <u>D</u> ,E,K,N,O,Y}
T3	{A, <u>E</u> ,K,M}
T4	{ <u>C</u> ,K,M,U,Y}
T5	{ <u>C</u> ,E,I,K,O,O}

- The frequency of each individual item is computed:

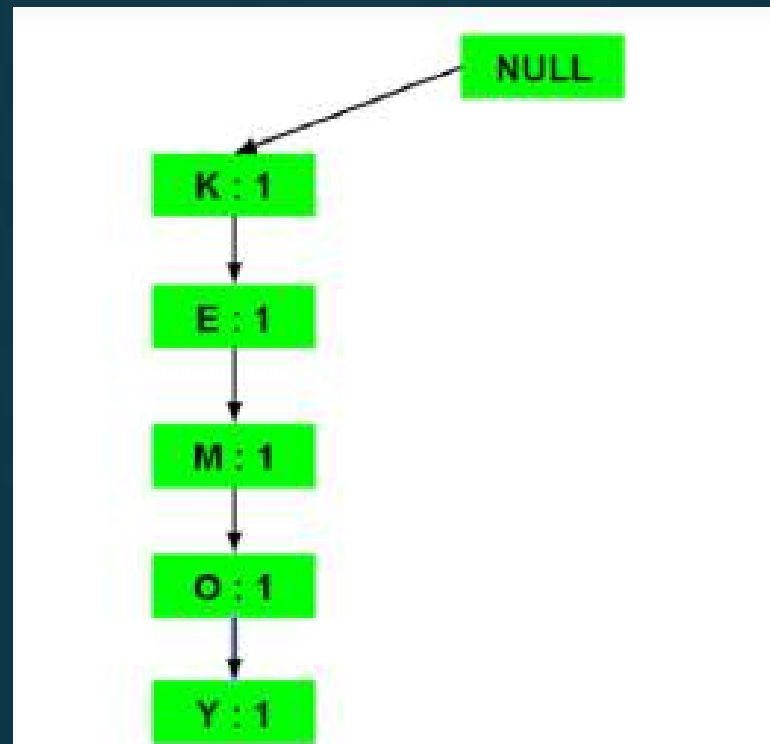
Item	Frequency
A	1
C	2
D	1
E	4
I	1
K	5
M	3
N	2
O	3
U	1
Y	3

- Let the minimum support be 3.
- A Frequent Pattern set is built which will contain all the elements whose frequency is greater than or equal to the minimum support.
- These elements are stored in descending order of their respective frequencies. After insertion of the relevant items, the set L looks like this:-
- $L = \{K : 5, E : 4, M : 3, O : 3, Y : 3\}$

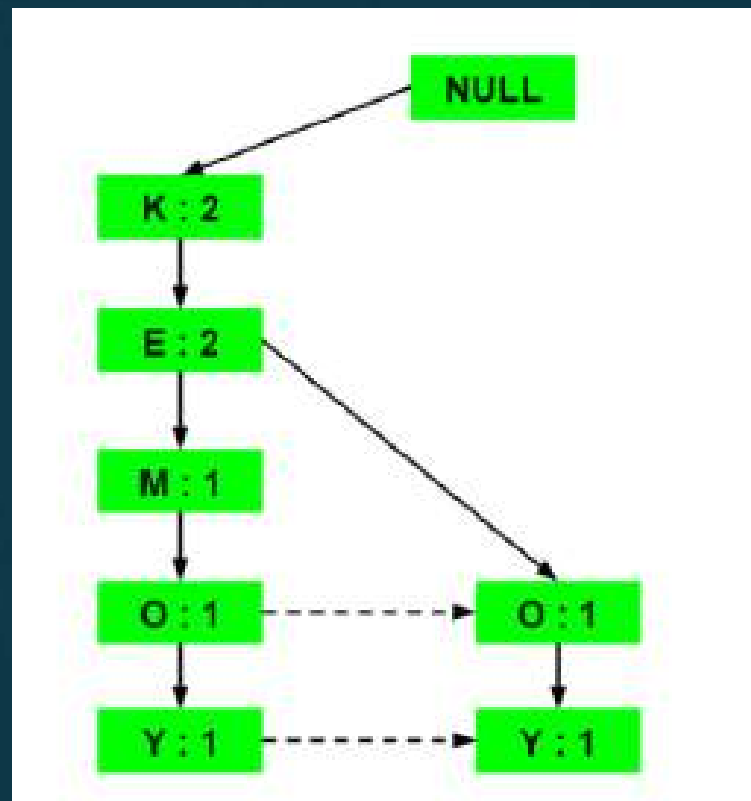
- Now, for each transaction, the respective Ordered-Item set is built.

Transaction ID	Items	Ordered-Item Set
T1	{ <u>E</u> ,K,M,N,O,Y}	{ <u>K</u> ,E,M,O,Y}
T2	{ <u>D</u> ,E,K,N,O,Y}	{ <u>K</u> ,E,O,Y}
T3	{ <u>A</u> ,E,K,M}	{ <u>K</u> ,E,M}
T4	{ <u>C</u> ,K,M,U,Y}	{ <u>K</u> ,M,Y}
T5	{ <u>C</u> ,E,I,K,O,O}	{ <u>K</u> ,E,O}

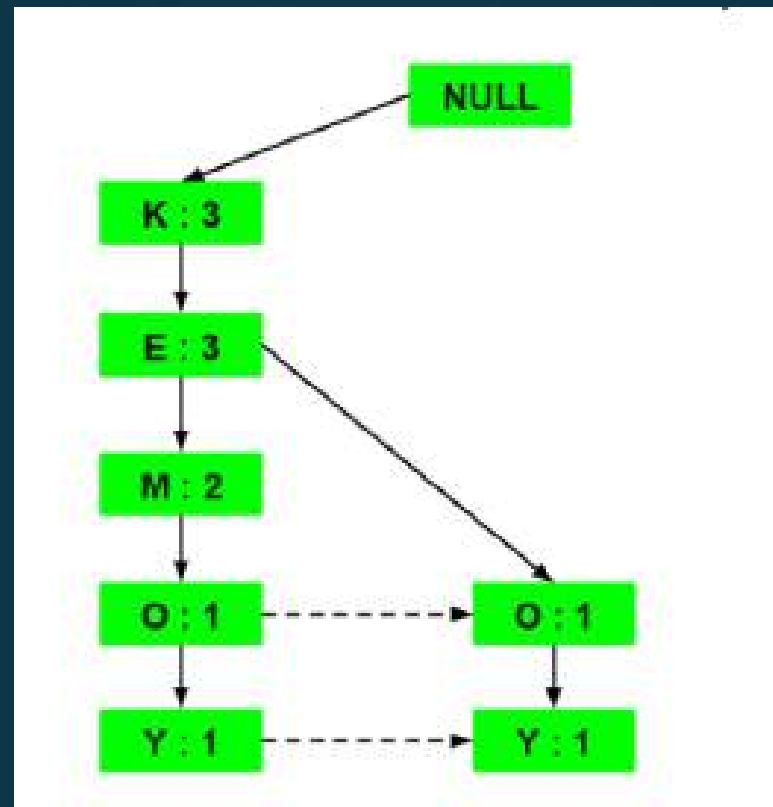
- ▶ a) Inserting the set {K, E, M, O, Y}:



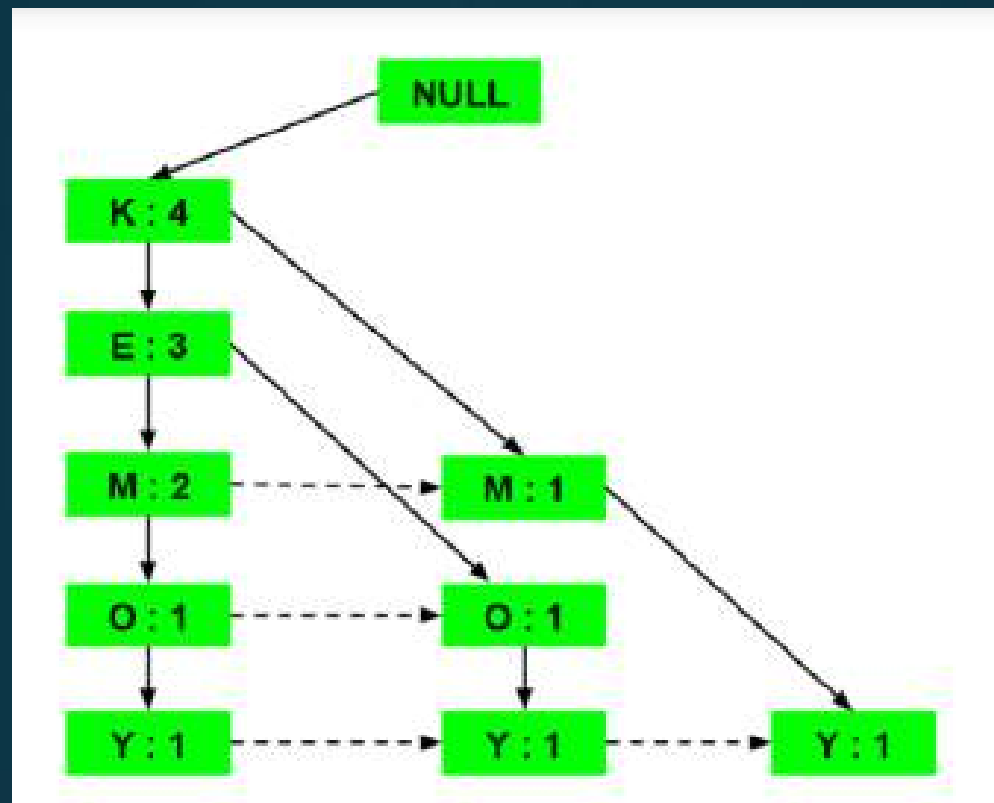
- b) Inserting the set {K, E, O, Y}:



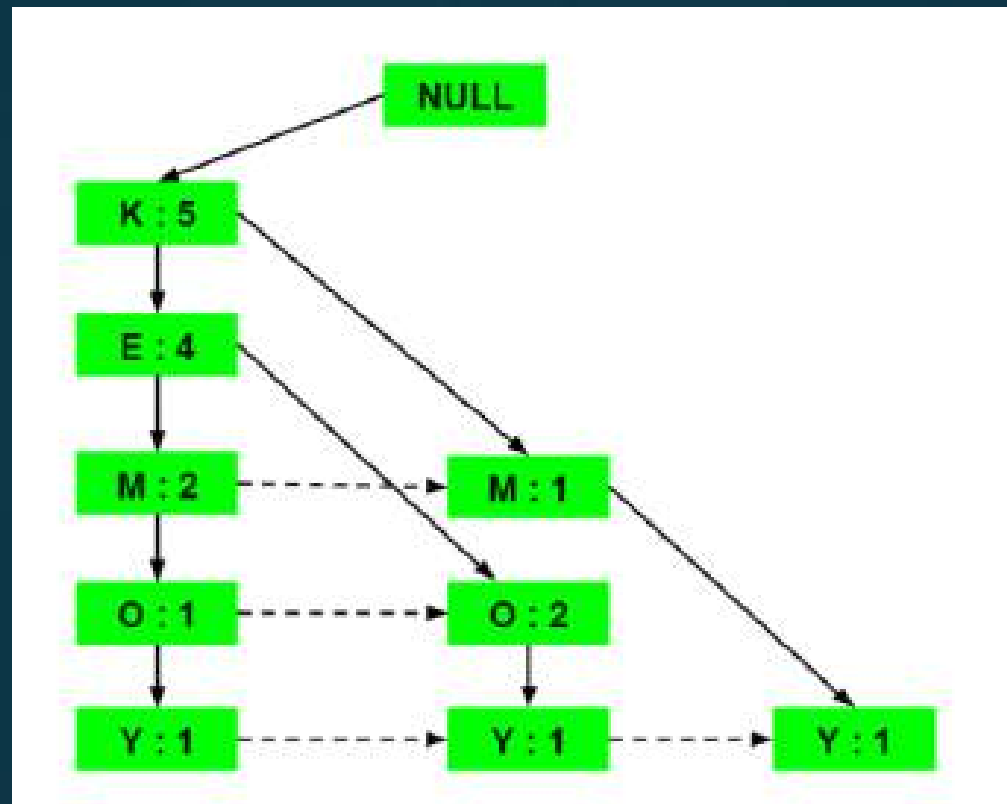
► c) Inserting the set {K, E, M}



- ▶ d) Inserting the set {K, M, Y}:



- e) Inserting the set {K, E, O}:



- Now, for each item, the **Conditional Pattern Base** is computed which is path labels of all the paths which lead to any node of the given item in the frequent-pattern tree.

Items	Conditional Pattern Base
Y	$\{\{\underline{K}, E, M, O : 1\}, \{K, E, O : 1\}, \{K, M : 1\}\}$
O	$\{\{\underline{K}, E, M : 1\}, \{K, E : 2\}\}$
M	$\{\{\underline{K}, E : 2\}, \{K : 1\}\}$
E	$\{\underline{K} : 4\}$
K	

- Now for each item the Conditional Frequent Pattern Tree is built. It is done by taking the set of elements which is common in all the paths in the Conditional Pattern Base of that item and calculating it's support count by summing the support counts of all the paths in the Conditional Pattern Base

Items	Conditional Pattern Base	Conditional Frequent Pattern Tree
Y	$\{\{K, E, M, O : 1\}, \{K, E, O : 1\}, \{K, M : 1\}\}$	$\{K : 3\}$
O	$\{\{K, E, M : 1\}, \{K, E : 2\}\}$	$\{K, E : 3\}$
M	$\{\{K, E : 2\}, \{K : 1\}\}$	$\{K : 3\}$
E	$\{K : 4\}$	$\{K : 4\}$
K		

Items	Frequent Pattern Generated
Y	$\{<K, Y : 3>\}$
O	$\{<K, O : 3>, <E, O : 3>, <E, K, O : 3>\}$
M	$\{<K, M : 3>\}$
E	$\{<E, K : 3>\}$
K	

- From the Conditional Frequent Pattern tree, the **Frequent Pattern rules** are generated by pairing the items of the Conditional Frequent Pattern Tree set to the corresponding to the item

Items	Frequent Pattern Generated
Y	{< <u>K</u> ,Y : 3>}
O	{< <u>K</u> ,O : 3>, <E,O : 3>, <E,K,O : 3>}
M	{< <u>K</u> ,M : 3>}
E	{< <u>E</u> ,K : 3>}
K	

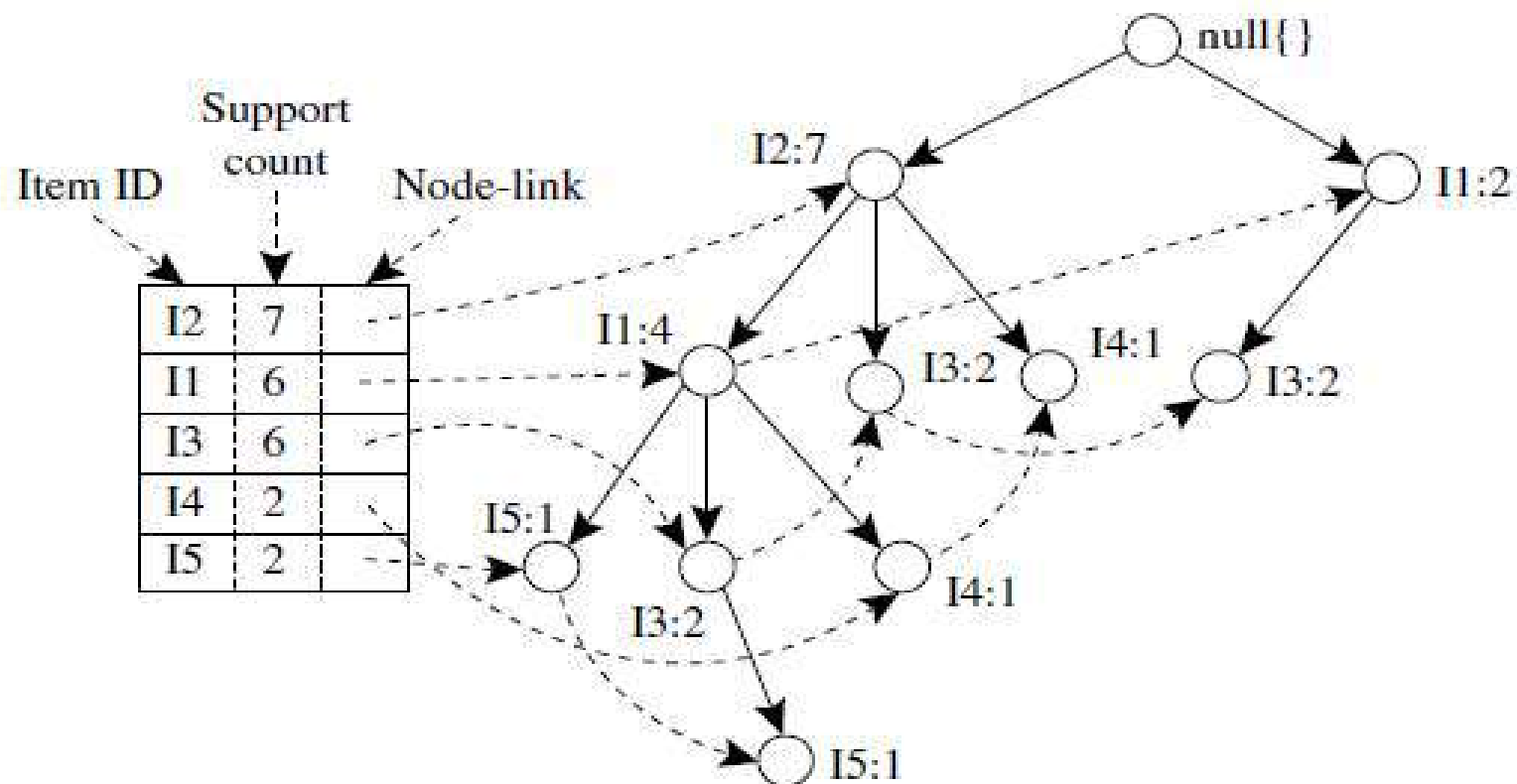
FP-Tree Representation

- Compressed representation of the input data
- Constructed by reading the data set one transaction at a time and mapping each transaction onto a path in the FP-tree
- As different transactions can have several items in common, their paths may overlap
- more the paths overlap with one another, the more compression we can achieve
- If the size of the F.P-tree is small enough to fit into main memory, this will allow us to extract frequent itemsets directly from the structure in memory instead of making repeated passes over the data stored on disk

–

Transactional Data for an *AllElectronics* Branch

<i>TID</i>	<i>List of item_IDs</i>
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3



Mining the FP-Tree by Creating Conditional (Sub-)Pattern Bases

Item	Conditional Pattern Base	Conditional FP-tree	Frequent Patterns Generated
I5	{{I2, I1: 1}, {I2, I1, I3: 1}}	$\langle I2: 2, I1: 2 \rangle$	{I2, I5: 2}, {I1, I5: 2}, {I2, I1, I5: 2}
I4	{{I2, I1: 1}, {I2: 1}}	$\langle I2: 2 \rangle$	{I2, I4: 2}
I3	{{I2, I1: 2}, {I2: 2}, {I1: 2}}	$\langle I2: 4, I1: 2 \rangle, \langle I1: 2 \rangle$	{I2, I3: 4}, {I1, I3: 4}, {I2, I1, I3: 2}
I1	{{I2: 4}}	$\langle I2: 4 \rangle$	{I2, I1: 4}

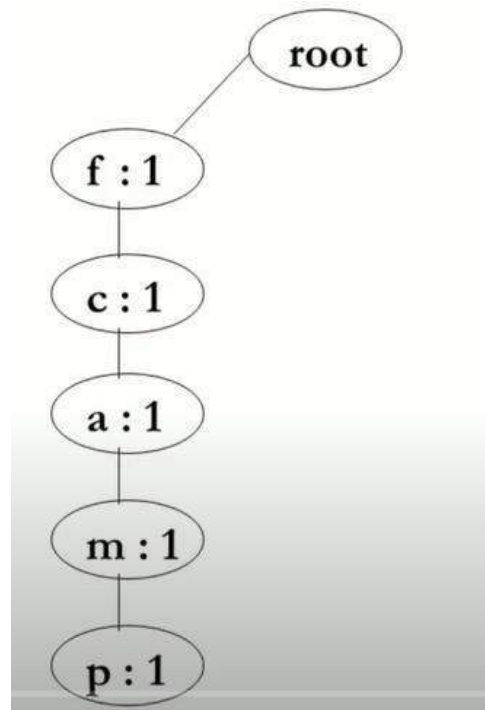
TID	Items Bought
100	<i>f, a, c, d, g, i, m, p</i>
200	<i>a, b, c, f, l, m, o</i>
300	<i>b, f, h, j, o</i>
400	<i>b, c, k, s, p</i>
500	<i>a, f, c, e, l, p, m, n</i>

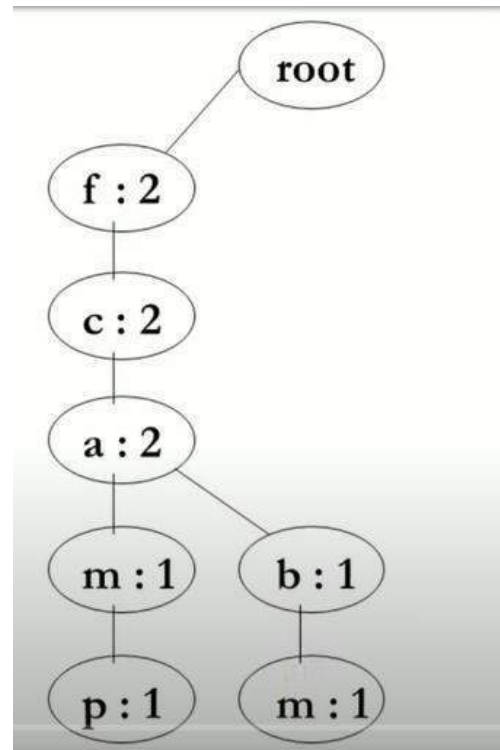
Minimum support count=3

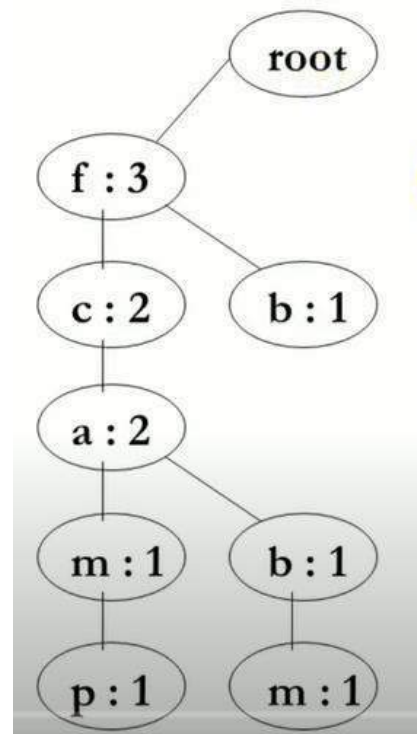
TID	Items Bought	Item	Frequency	Item	Frequency
100	<i>f, a, c, d, g, i, m, p</i>	a	3	j	1
200	<i>a, b, c, f, l, m, o</i>	b	3	k	1
300	<i>b, f, h, j, o</i>	c	4	l	2
400	<i>b, c, k, s, p</i>	d	1	m	3
500	<i>a, f, c, e, l, p, m, n</i>	e	1	n	1
Minimum Support - 3		f	4	o	2
		g	1	p	3
		h	1	s	1
		i	1		

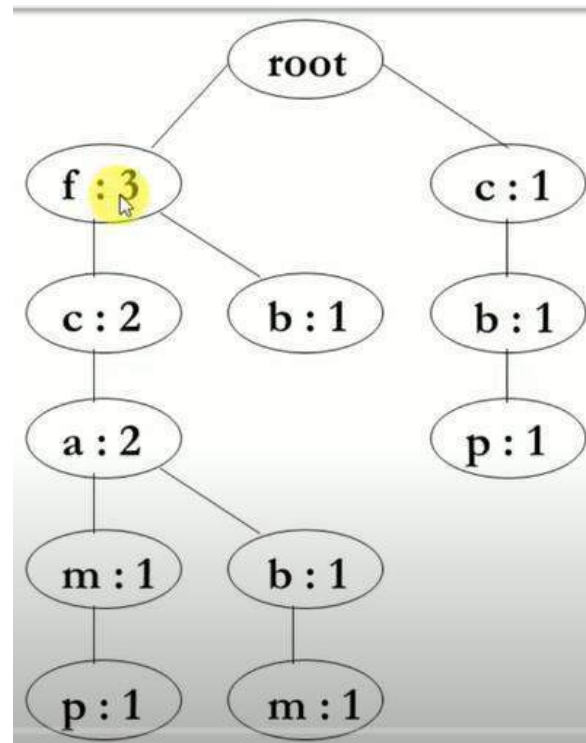
$L = \{ (f:4), (c:4), (a:3), (b:3), (m:3), (p:3) \}$

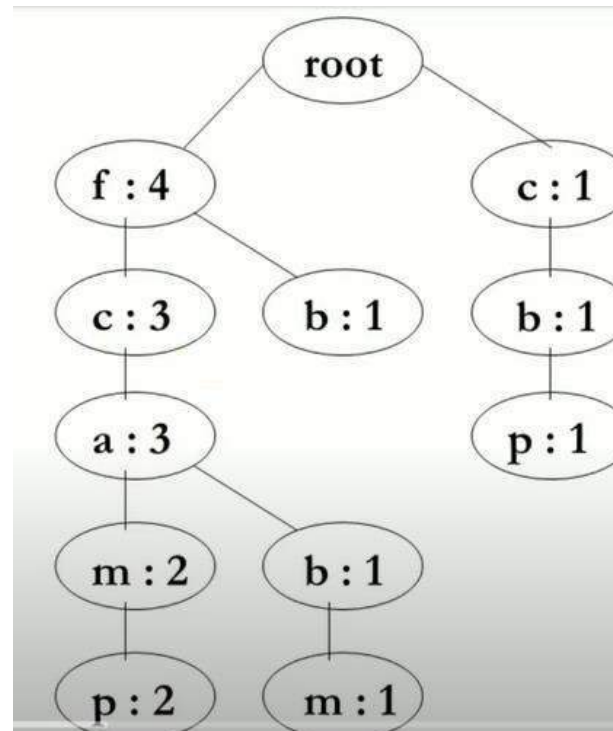
TID	Items Bought	(Ordered) Frequent Items
100	<i>f, a, c, d, g, i, m, p</i>	<i>f, c, a, m, p</i>
200	<i>a, b, c, f, l, m, o</i>	<i>f, c, a, b, m</i>
300	<i>b, f, h, j, o</i>	<i>f, b</i>
400	<i>b, c, k, s, p</i>	<i>c, b, p</i>
500	<i>a, f, c, e, l, p, m, n</i>	<i>f, c, a, m, p</i>











Item	Conditional Pattern Base
p	$\{\{f, c, a, m : 2\}, \{c, b : 1\}\}$
m	$\{\{f, c, a : 2\}, \{f, c, a, b : 1\}\}$
b	$\{\{f, c, a : 1\}, \{f : 1\}, \{c : 1\}\}$
a	$\{\{f, c : 3\}\}$
c	$\{\{f : 3\}\}$
f	Φ

Item	Conditional Pattern Base	Conditional FP-Tree
p	$\{\{f, c, a, m : 2\}, \{c, b : 1\}\}$	$\{c : 3\}$
m	$\{\{f, c, a : 2\}, \{f, c, a, b : 1\}\}$	$\{f, c, a : 3\}$
b	$\{\{f, c, a : 1\}, \{f : 1\}, \{c : 1\}\}$	Φ
a	$\{\{f, c : 3\}\}$	$\{f, c : 3\}$
c	$\{\{f : 3\}\}$	$\{f : 3\}$
f	Φ	Φ

Item	Conditional Pattern Base	Conditional FP-Tree	Frequent Patterns Generated
p	{ {f, c, a, m : 2}, {c, b : 1} }	{c : 3}	{<c, p : 3>}
m	{ {f, c, a : 2}, {f, c, a, b : 1} }	{f, c, a : 3}	{ <f, m : 3>, <c, m : 3> <a, m : 3>, <f, c, m : 3> <f, a, m : 3>, <c, a, m : 3> }
b	{ {f, c, a : 1}, {f : 1}, {c : 1} }	Φ	{}
a	{ {f, c : 3} }	{f, c : 3}	{<f, a : 3>, <c, a : 3>, <f, c, a : 3>}
c	{ {f : 3} }	{f : 3}	{ <f, c : 3> }
f	Φ	Φ	{}

Apriori Algorithm-example:

Transactions List

1	Milk	Egg	Bread	Butter
2	Milk	Butter	Egg	Ketchup
3	Bread	Butter	Ketchup	
4	Milk	Bread	Butter	
5	Bread	Butter	Cookies	
6	Milk	Bread	Butter	Cookies
7	Milk	Cookies		
8	Milk	Bread	Butter	
9	Bread	Butter	Egg	Cookies
10	Milk	Butter	Bread	
11	Milk	Bread	Butter	
12	Milk	Bread	Cookies	Ketchup

Minimum support=33%

Minimum confidence=50%

1-item Sets	Frequency
Milk	9
Bread	10
Butter	10
Egg	3
Ketchup	3
Cookies	5

Frequent 1-item Sets	Frequency
Milk	9
Bread	10
Butter	10
Cookies	5

2-item Sets	Frequency
Milk, Bread	7
Milk, Butter	7
Milk, Cookies	3
Bread, Butter	9
Butter, Cookies	3
Bread, Cookies	4

Frequent 2-item Sets	Frequency
Milk, Bread	7
Milk, Butter	7
Bread, Butter	9
Bread, Cookies	4

3-item Sets	Frequency
Milk, Bread, Butter	6
Milk, Bread, Cookies	1
Bread, Butter, Cookies	3
Milk, Butter, Cookies	2

Frequent 3-item Sets	Frequency
Milk, Bread, Butter	6

- Frequent 3-Item Set = $I \Rightarrow \{Milk, Bread, Butter\}$
- Non-Empty subset are
 - $\{\{Milk\}, \{Bread\}, \{Butter\}, \{Milk, Bread\}, \{Milk, Butter\}, \{Bread, Butter\}\}$

- Rule 1: {Milk} $\rightarrow$ {Bread, Butter} {S=50%, C=66.67%}
 - Support = $6/12 = 50\%$
 - Confidence = $\text{Support}(\text{Milk, Bread, Butter}) / \text{Support}(\text{Milk}) = \frac{6/12}{9/12} = 6/9 = 66.67\% > 60\%$
 - Valid
- Rule 2: {Bread} $\rightarrow$ {Milk, Butter} {S=50%, C=60%}
 - Support = $6/12 = 50\%$
 - Confidence = $\text{Support}(\text{Milk, Bread, Butter}) / \text{Support}(\text{Bread}) = 6/10 = 60\% \geq 60\%$
 - Valid
- Rule 3: {Butter} $\rightarrow$ {Milk, Bread} {S=50%, C=60%}
 - Support = $6/12 = 50\%$
 - Confidence = $\text{Support}(\text{Milk, Bread, Butter}) / \text{Support}(\text{Butter}) = 6/10 = 60\% \geq 60\%$
 - Valid
- Rule 4: {Milk, Bread} $\rightarrow$ {Butter} {S=50%, C=85.7%}
 - Support = $6/12 = 50\%$
 - Confidence = $\text{Support}(\text{Milk, Bread, Butter}) / \text{Support}(\text{Milk, Bread}) = 6/7 = 85.7\% > 60\%$
 - Valid
- Rule 5: {Milk, Butter} $\rightarrow$ {Bread} {S=50%, C=85.7%}
 - Support = $6/12 = 50\%$
 - Confidence = $\text{Support}(\text{Milk, Bread, Butter}) / \text{Support}(\text{Milk, Butter}) = 6/7 = 85.7\% \geq 60\%$
 - Valid
- Rule 6: {Bread, Butter} $\rightarrow$ {Milk} {S=50%, C=66.67%}
 - Support = $6/12 = 50\%$
 - Confidence = $\text{Support}(\text{Milk, Bread, Butter}) / \text{Support}(\text{Bread, Butter}) = 6/9 = 66.67\% \geq 60\%$
 - Valid



CLASSIFICATION

Bayesian Classifier



- In some cases, the relationship between the attribute set and the class variable is non-deterministic
- **Bayesian Classifiers:** an approach for modelling probabilistic relationships between the attribute set and the class variable

Bayes theorem

Let X and Y be a pair of random variables

Joint probability, $P(X = x, Y = y)$

- Probability that variable X will take on the value x and variable Y will take on the value y

Conditional probability, $P(Y = y \mid X = x)$

- Probability that a random variable will take on a particular value given that the outcome for another random variable is known
- Probability that the variable Y will take on the value y , given that the variable X is observed to have the value x

Bayesian theorem

The joint and conditional probabilities for X and Y are related in the following way:

$$P(X,Y) = P(Y | X) \times P(X) = P(X | Y) \times P(Y)$$

Rearranging the last two expression, we get a formula, known as **the Bayes theorem**:

$$P(Y | X) = \frac{P(X | Y) \times P(Y)}{P(X)}$$

- 
- Consider football game between two rival teams: Team 0 and Team 1
 - Suppose Team 0 wins 65% of the time and Team 1 wins the remaining matches
 - Among the games won by Team 0, only 30% of them come from playing on Team 1's football field
 - On the other hand, 75% of the victories for Team 1 are obtained while playing at home
 - **If Team 1 is to host the next match between the two teams, which team will most likely emerge as the winner?**

- Let X be the random variable that represents the team hosting the match
- Y be the random variable that represents the winner of the match
- Both X and Y can take on values from the set $\{0,1\}$
- Probability Team 0 wins is $P(Y = 0) = 0.65$
- Probability Team 1 wins is $P(Y = 1) = 1 - P(Y = 0) = 0.35$
- Probability Team 1 hosted the match and Team 0 won is $P(X = 1 \mid Y = 0) = 0.3$
- Probability Team 1 hosted the match it won is $P(X = 1 \mid Y = 1) = 0.75$

Our objective is to compute $P(Y = 1 \mid X = 1)$, which is the conditional probability that Team 1 wins the next

$$\begin{aligned}
 P(Y = 1|X = 1) &= \frac{P(X = 1|Y = 1) \times P(Y = 1)}{P(X = 1)} \\
 &= \frac{P(X = 1|Y = 1) \times P(Y = 1)}{P(X = 1, Y = 1) + P(X = 1, Y = 0)} \\
 &= \frac{P(X = 1|Y = 1) \times P(Y = 1)}{P(X = 1|Y = 1)P(Y = 1) + P(X = 1|Y = 0)P(Y = 0)} \\
 &= \frac{0.75 \times 0.35}{0.75 \times 0.35 + 0.3 \times 0.65} \\
 &= 0.5738,
 \end{aligned}$$

$$P(X) = \sum_{i=1}^k P(X, Y_i) = \sum_{i=1}^k P(X|Y_i)P(Y_i).$$

- $P(Y = 0 | X = 1) = 1 - P(Y = 1 | X = 1) = 0.4262$
- Since $P(Y = 1 | X = 1) > P(Y = 0 | X = 1)$, Team 1 has a better chance than Team 0 of winning the next match

Using the Bayes Theorem for Classification

- Formalizing the classification problem from a statistical perspective:
- Let X denote the attribute set and Y denote the class variable
- If the class variable has a non-deterministic relationship with the attributes, then we can treat X and Y as random variables and capture their relationship probabilistically using **$P(Y | X)$**
- This conditional probability is also known as the **Posterior probability for Y**

- 
- During the training phase, we need to learn the posterior probabilities $P(Y | X)$ for every combination of X and Y based on information gathered from the training data
 - By knowing these probabilities, a test record X' can be classified by finding the class Y' that maximizes the posterior probability, **$P(Y' | X')$**

	binary	categorical	continuous	class
Tid	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

- **Example:**

- A test record is given with the following attribute set: $X = (\text{Home Owner} = \text{No}, \text{Marital Status} = \text{Married}, \text{Annual Income} = \$120\text{K})$
- To classify the record, we need to compute the posterior probabilities $P(\text{Yes} \mid X)$ and $P(\text{No} \mid X)$ based on information available in the training data
- If $P(\text{Yes} \mid X) > P(\text{No} \mid X)$, then the record is classified as Yes; otherwise, it is classified as No

- 
- Estimating the posterior probabilities accurately for every possible combination of class label and attribute value is a difficult problem because it requires a very large training set, even for a moderate number of attributes
 - The Bayes theorem is useful because it allows us to express the posterior probability in terms of the prior probability $P(Y)$, the class-conditional probability $P(X|Y)$, and the evidence, $P(X)$:

$$P(Y | X) = \frac{P(X | Y) \times P(Y)}{P(X)}$$

- 
- When comparing the posterior probabilities for different values of Y , the denominator term, $P(X)$, is always constant, and thus, can be ignored
 - Prior probability $P(Y)$ can be easily estimated from the training set by computing the fraction of training records that belong to each class

Evaluating the performance of classifiers

Holdout Method:

- The original data with labeled examples is partitioned into two disjoint sets- training and test sets
- Classification model is induced from training set
- Its performance is evaluated on the test set
- The accuracy of the classifier can be estimated based on the accuracy of the induced model on the test set

Limitations:

- Fewer labeled examples are available for training
- Model may be highly dependent on the composition of training and test sets
- Training and test set are not independent of each other

Evaluating the performance of classifiers

Random Subsampling:

- Holdout method can be repeated several times to improve the estimation of classifier's performance
- This approach is known as random subsampling

Limitations:

- Suffers from the problems associated with holdout method
- It also has no control over the number of times each record is used for testing and training

Evaluating the performance of classifiers

Cross Validation:

- Each record is used the same number of times for training and exactly once for testing
- K fold cross validation segments the data into k equal sized partitions
- During each run, one of the partition is chosen for testing, rest of them are used for training
- Procedure is repeated k times so that each partition is used for testing exactly once
- Total error is found by summing up the errors for all k runs

- 
- Leave one out approach: special case where $k=N$, the size of the data
 - Each test set contains only one record
 - Advantages: Utilizes as much as data as possible for training
 - Test sets are mutually exclusive and effectively cover the entire data set
 - Drawback: computationally expensive to repeat the procedure N times
 - Since each test set contains only one record, the variance of the estimated performance metric tends to be high



CLASSIFICATION

Nearest-Neighbor classifiers



Eager Learners

- Designed to learn a model that maps the input attributes to the class label as soon as the training data becomes available
- Examples: decision tree classifiers, rule based classifiers



Lazy learners

- Delay the process of modelling the training data until it is needed to classify the test examples
- Example: Rote classifier - memorizes the entire training data and performs classification only if the attributes of a test instance match one of the training examples exactly
- Drawback: some test records may not be classified because they do not match any training example

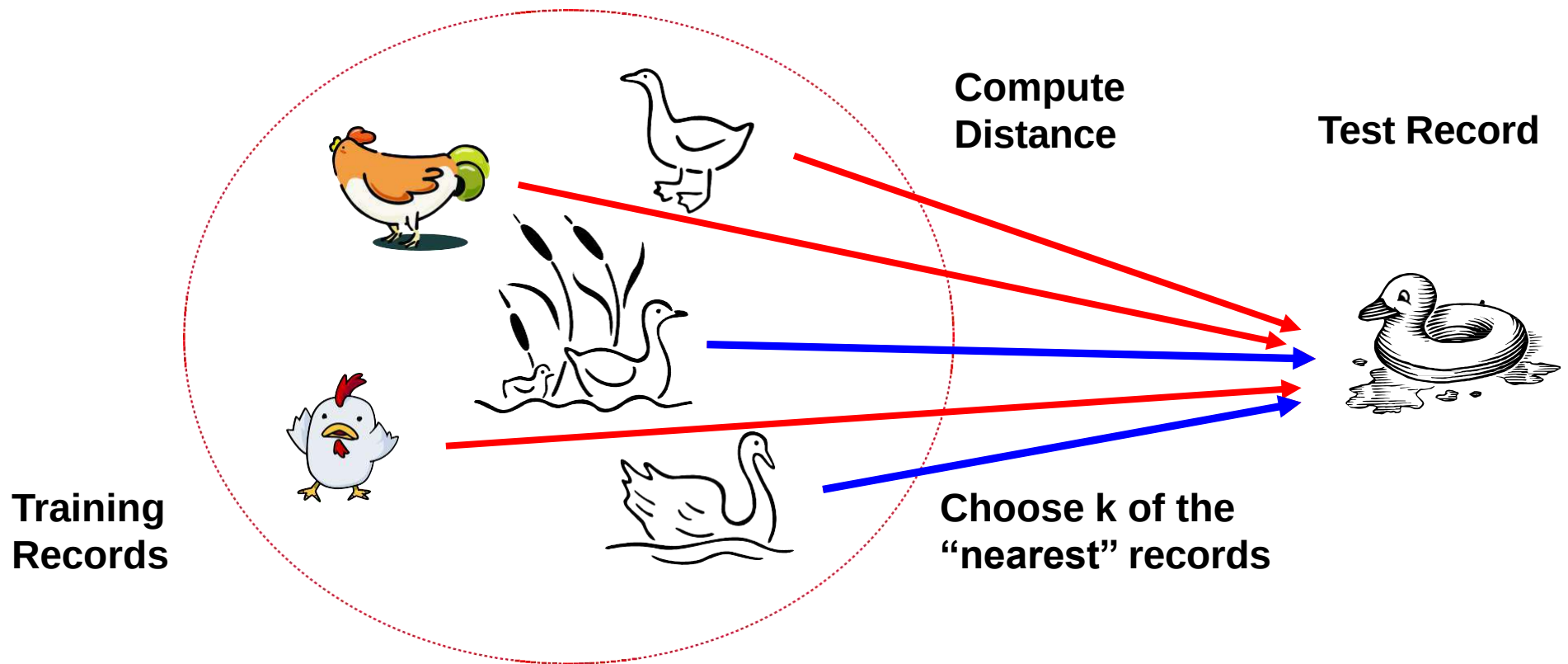


Solution

- Find all the training examples that are relatively similar to the attributes of the test example
- These **nearest neighbours**, can be used to determine the class label of the test example

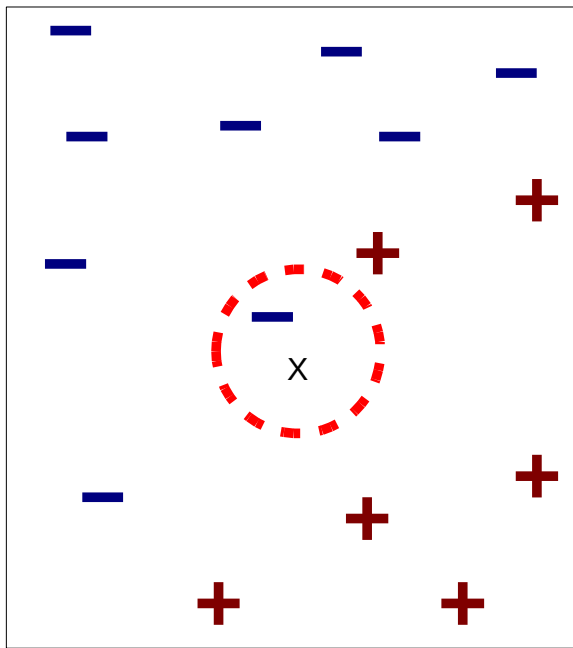
Basic idea:

- If it walks like a duck, quacks like a duck, if it looks like duck, then it's probably a duck

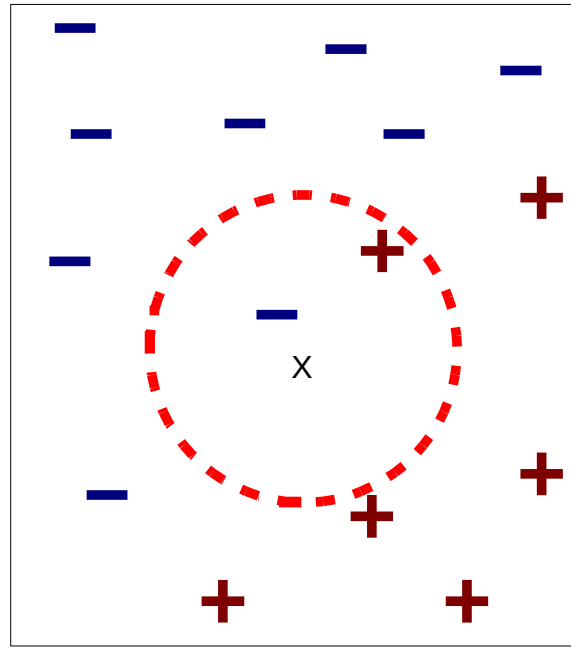


- 
- A nearest neighbor classifier represents each example as a data point in a d -dimensional space, where d is the number of attributes
 - Given a test example, we compute its proximity to the rest of the data points in the training set, using one of the proximity measure
 - K -nearest neighbors of a given example z refer to the k points that are closest to z

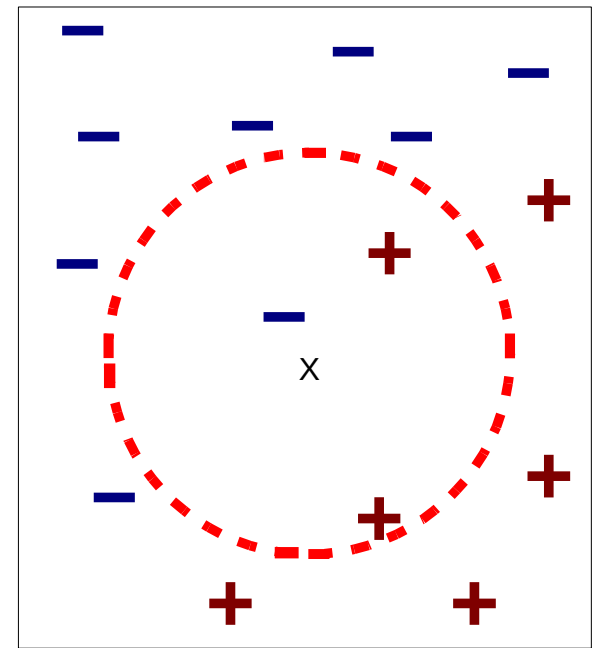
- 
- Data point is classified based on the class labels of its neighbors
 - In the case where the neighbors have more than one label, the data point is assigned to the majority class of its nearest neighbors



(a) 1-nearest neighbor



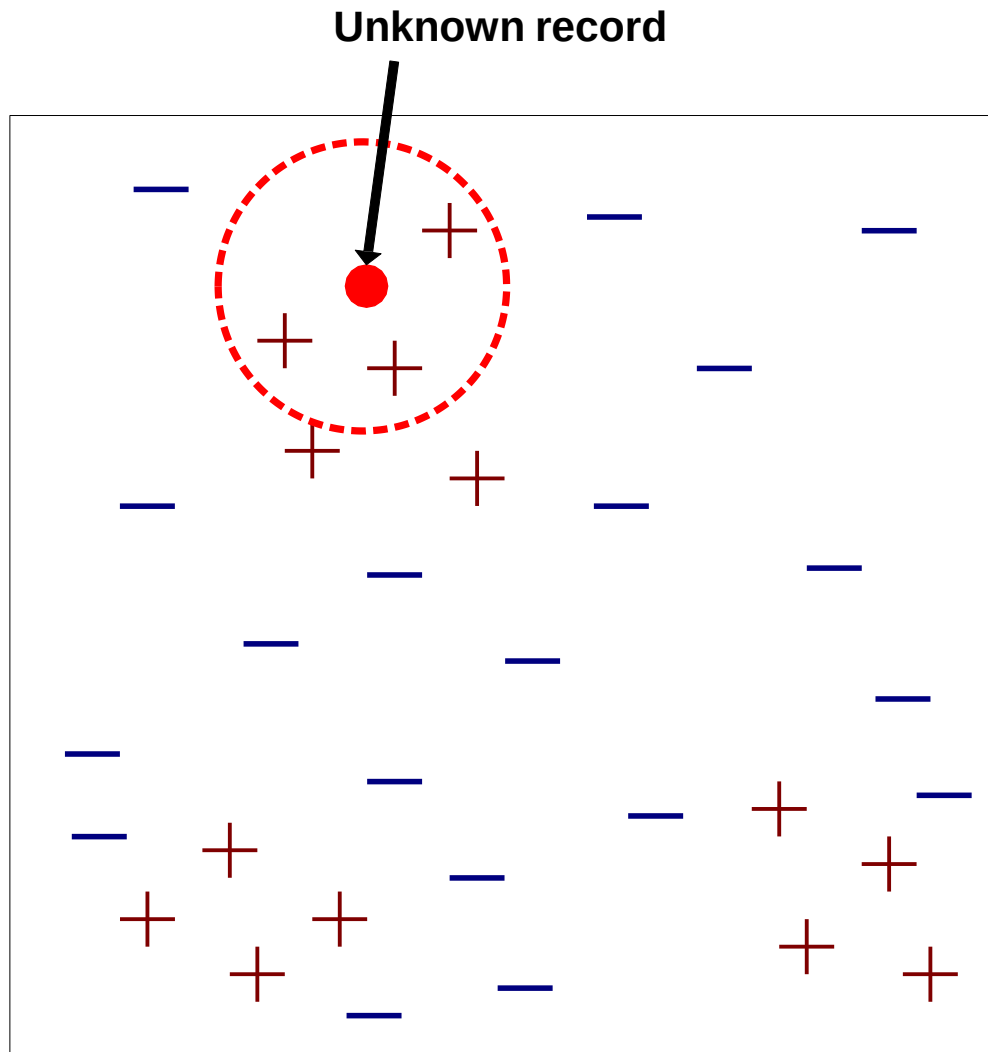
(b) 2-nearest neighbor



(c) 3-nearest neighbor

K-nearest neighbors of a record x are data points that have the k smallest distances to x

- If the number of nearest neighbors is one, data point is a negative example. Therefore the data point is assigned to the negative class
- If the number of nearest neighbors is three, then the neighborhood contains two positive examples and one negative example
- Using the majority voting scheme, the data point is assigned to the positive class
- In the case where there is a tie between the classes, we may randomly choose one of them to classify the data point Choose the right value for k



- Requires three things
 - The set of labeled records
 - Distance Metric to compute distance between records
 - The value of k , the number of nearest neighbors to retrieve
- To classify an unknown record:
 - Compute distance to other training records
 - Identify k nearest neighbors
 - Use class labels of nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)

Algorithm

- Algorithm computes the distance (or similarity) between each test example $z = (\mathbf{x}', y')$ training examples $(\mathbf{x}, y)$ belongs D to determine its nearest-neighbor list, D_z
- Such computation can be costly if the number of training examples is large

- 
- 1: Let k be the number of nearest neighbors and D be the set of training examples.
 - 2: **for** each test example $z = (\mathbf{x}', y')$ **do**
 - 3: Compute $d(\mathbf{x}', \mathbf{x})$, the distance between z and every example, $(\mathbf{x}, y) \in D$.
 - 4: Select $D_z \subseteq D$, the set of k closest training examples to z .
 - 5: $y' = \operatorname{argmax}_v \sum_{(\mathbf{x}_i, y_i) \in D_z} I(v = y_i)$
 - 6: **end for**

- Once the nearest-neighbor list is obtained, the test example is classified based on the majority class of its nearest neighbors

$$\text{Majority Voting: } y' = \operatorname{argmax}_v \sum_{(\mathbf{x}_i, y_i) \in D_z} I(v = y_i),$$

- Where:
- v is a class label
- y_i is the class label for one of the nearest neighbors
- I is an indicator function that returns the value 1 if its argument is true and 0 otherwise

- In the majority voting approach, every neighbor has the same impact on the classification which makes the algorithm sensitive to the choice of k
- To reduce impact of k , weight the influence of each nearest x_i according to its distance: $w = 1/d^2$
- As a result, training examples that are located far away from z have a weaker impact the classification compared to those that are located close to z
- Using the distance weight voting scheme, the class label can be determined as:

$$y' = \underset{v}{\operatorname{argmax}} \sum_{(x_i, y_i) \in D_z} w_i \times I(v = y_i)$$

Characteristics of Nearest Neighbor Classifier

- NN classification is part of instance based learning, which uses specific training instances to make predictions without having to maintain model derived from data
- Do not require model building
- Predictions are based on local information
- Can produce wrong predictions unless the appropriate proximity measure and data preprocessing steps are taken
- Can produce arbitrary shaped decision boundaries

CLASSIFICATION

INTRODUCTION

- Task of assigning objects to one of several predefined categories
- Input data for a classification task is a collection of records

Given a collection of records (training set):

- Each record is by characterized by a tuple (x,y) where x is the attribute set and y is the class label
- x : attribute, predictor, independent variable, input
- y : class, response, dependent variable, output

INTRODUCTION

- Example: Sample data set used for classifying vertebrates
- Attribute set includes properties of a vertebrate such as its body temperature, skin cover, method of reproduction, ability to fly, and ability to live in water
- Class label must be a discrete attribute

Task	Attribute set, x	Class label, y
Categorizing email messages	Features extracted from email message header and content	spam or non-spam
Cataloging galaxies	Features extracted from telescope images	Elliptical, spiral, or irregular-shaped galaxies

Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hibernates	Class Label
human	warm-blooded	hair	yes	no	no	yes	no	mammal
python	cold-blooded	scales	no	no	no	no	yes	reptile
salmon	cold-blooded	scales	no	yes	no	no	no	fish
whale	warm-blooded	hair	yes	yes	no	no	no	mammal
frog	cold-blooded	none	no	semi	no	yes	yes	amphibian
komodo dragon	cold-blooded	scales	no	no	no	yes	no	reptile
bat	warm-blooded	hair	yes	no	yes	yes	yes	mammal
pigeon	warm-blooded	feathers	no	no	yes	yes	no	bird
cat	warm-blooded	fur	yes	no	no	yes	no	mammal
leopard	cold-blooded	scales	yes	yes	no	no	no	fish
shark								
turtle	cold-blooded	scales	no	semi	no	yes	no	reptile
penguin	warm-blooded	feathers	no	semi	no	yes	no	bird
porcupine	warm-blooded	quills	yes	no	no	yes	yes	mammal
eel	cold-blooded	scales	no	yes	no	no	no	fish
salamander	cold-blooded	none	no	semi	no	yes	yes	amphibian

- Task of learning a target function f that maps each attribute set x to one of the predefined class labels y
- Target function is also known informally as a **classification model**

Classification model is useful for the following purposes:

- **Descriptive Modeling**

- It can serve as an explanatory tool to distinguish between objects of different classes

- **Predictive Modeling**

- It can be used to predict the class label of unknown records
- It can be treated as a black box that automatically assigns a class label when presented with the attribute set of an unknown record

Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hibernates	Class Label
gila monster	cold-blooded	scales	no	no	no	yes	yes	?

General Approach to Solving a Classification Problem

- A classification technique (or classifier) is a systematic approach to building classification models from an input data set
- Examples: Decision tree classifiers, Rule-based classifiers, Neural Networks, Support Vector Machines, Naive Bayes classifiers
- Each technique employs a learning algorithm to identify a model that best fits the relationship between the attribute set and class label of the input data

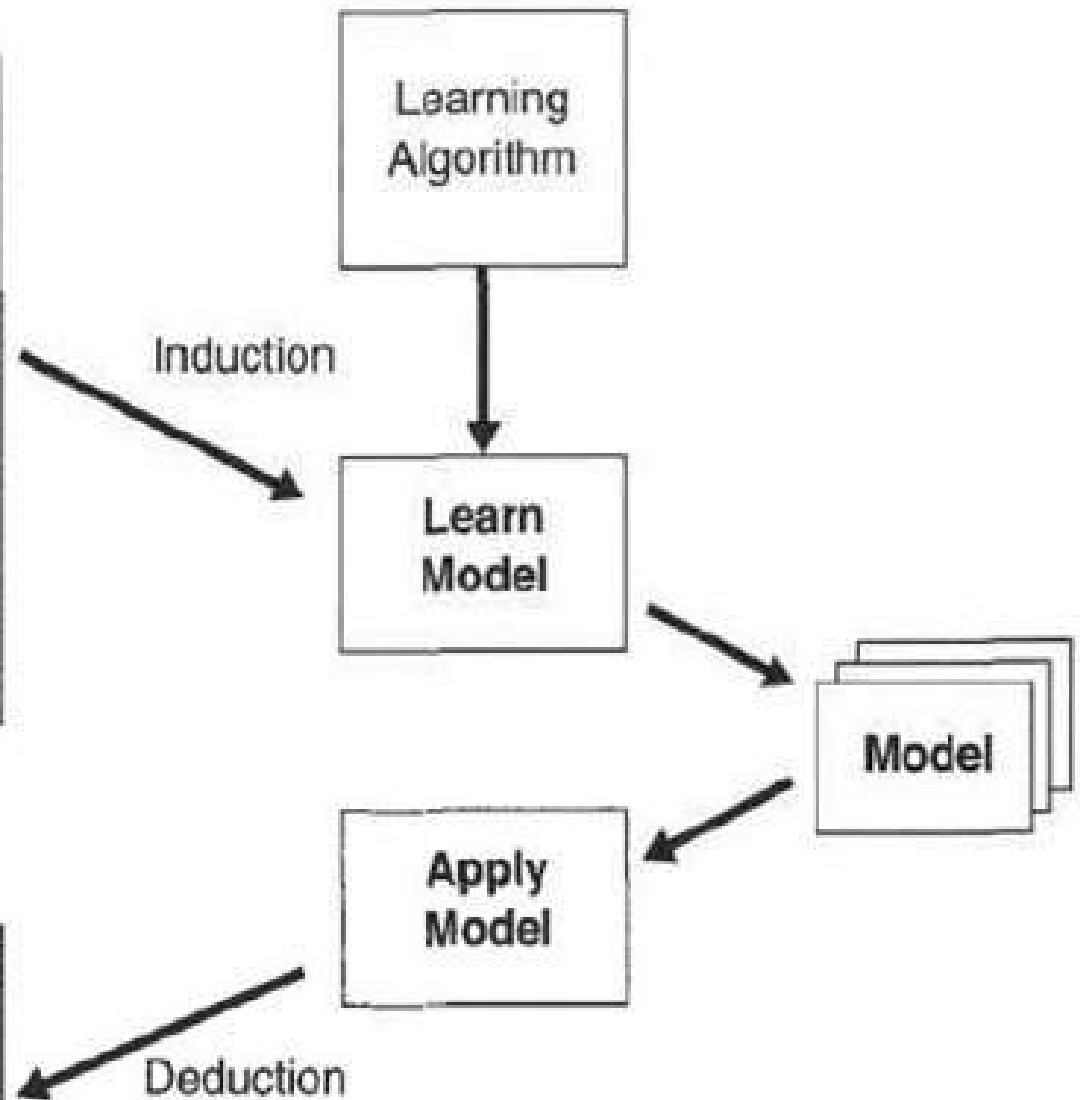
- The model generated by a learning algorithm should both fit the input data well and correctly predict the class labels of records it has never seen before
- Key objective of the learning algorithm is to build models with good generalization capability (models that accurately predict the class labels of previously unknown records)
- First, a training set consisting of records whose class labels are known must be provided
- The training set is used to build a classification model which is subsequently applied to the test set, which consists of records with unknown class labels

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Test Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?



- Evaluation of the performance of a classification model is based on the counts of test records correctly and incorrectly predicted by the model
- Counts are tabulated in a table known as a **Confusion matrix**

		Predicted Class	
		<i>Class = 1</i>	<i>Class = 0</i>
Actual Class	<i>Class = 1</i>	f_{11}	f_{10}
	<i>Class = 0</i>	f_{01}	f_{00}

- Each entry f_{ij} in this table denotes the number of records from class i predicted to be of class j
- f_{01} is the number of records from class 0 incorrectly predicted as class 1
- Total number of correct predictions made by the model is $(f_{11} + f_{00})$
- Total number of incorrect predictions is $(f_{01} + f_{10})$

Performance metrics

- Accuracy

=No. of correct predictions / Total No. of predictions

$$= (f_{11} + f_{00}) / (f_{11} + f_{10} + f_{01} + f_{00})$$

- Error rate

=No. of wrong predictions / Total No. of predictions

$$= (f_{01} + f_{10}) / (f_{11} + f_{10} + f_{01} + f_{00})$$

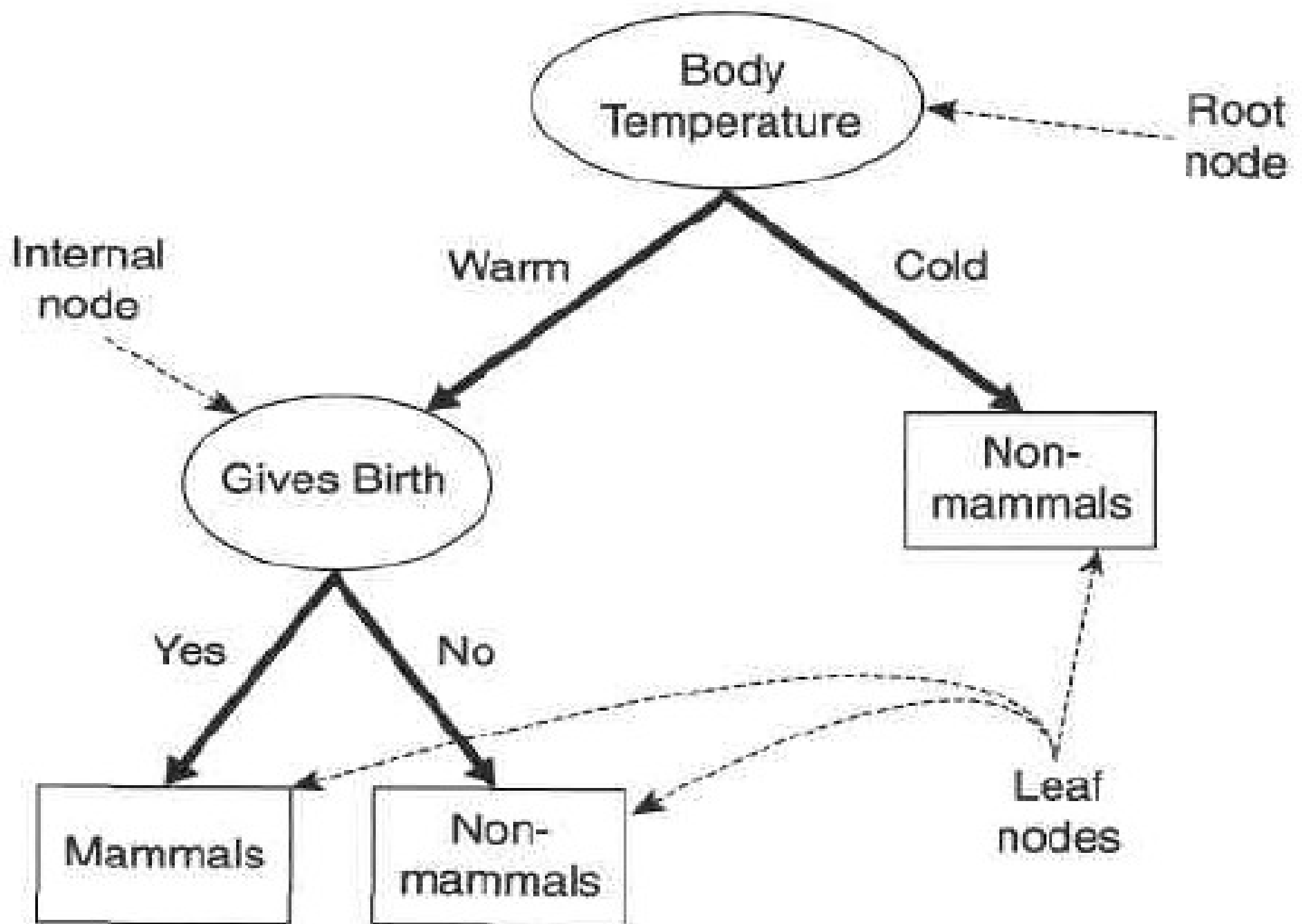
Decision Tree Classifier

How a Decision Tree Works:

- Solves a classification problem by asking a series of carefully crafted questions about the attributes of the test record
- Each time we receive an answer, a follow-up question is asked until we reach a conclusion about the class label of the record
- Series of questions and their possible answers can be organized in the form of a decision tree
- Hierarchical structure consisting of nodes and directed edge

Three types of nodes:

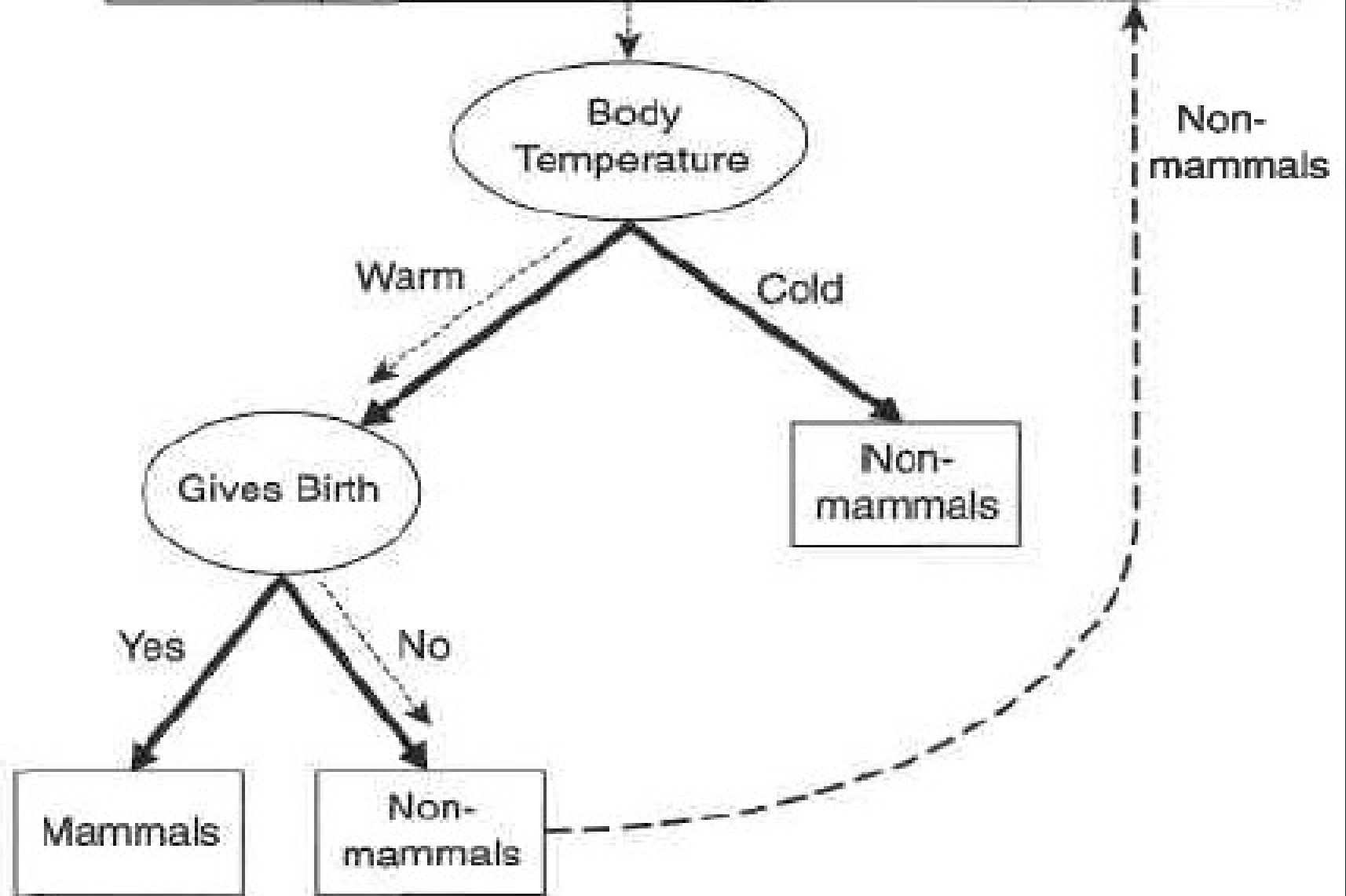
- **Root node** that has no incoming edges and zero or more outgoing edge
- **Internal nodes** each of which has exactly one incoming edge and two or more outgoing edge
- **Leaf or terminal nodes**, each of which has exactly one incoming edge and no outgoing edge
- Each leaf node is assigned a *class label*
- Non-terminal nodes, which include the root and other internal nodes, contain *attribute test conditions* to separate records that have different characteristics



- Classifying a test record is straightforward once a decision tree has been constructed
- Starting from the root node, apply the test condition to the record and follow the appropriate branch based on the outcome of the test
- This will lead either to another internal node, for which a new test condition is applied, or to a leaf node
- The class label associated with the leaf node is then assigned to the record

Unlabeled
data

Name	Body temperature	Gives Birth	...	Class
Flamingo	Warm	No	...	?



How to Build a Decision Tree:

- Finding the optimal tree is computationally infeasible because of the exponential size of the search space
- Efficient algorithms have been developed to induce a reasonably accurate decision tree in a reasonable amount of time

Hunt's Algorithm:

- A decision tree is grown in a recursive fashion by partitioning the training records into successively purer subsets
- Let D_t be the set of training records that are associated with node t
- $y = \{y_1, y_2, \dots, y_c\}$ be the class labels

Recursive definition of Hunt's algorithm

- Step 1: If all the records in D_t belong to the same class y_t , then t is a leaf node labelled as y_t
- Step 2: If D_t contains records that belong to more than one class, an attribute test condition is selected to partition the records into smaller subsets
- A child node is created for each outcome of the test condition and the records in D_t are distributed to the children based on the outcomes
- The algorithm is then recursively applied to each child node

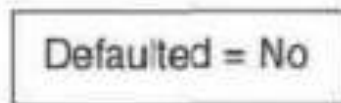
binary

categorical

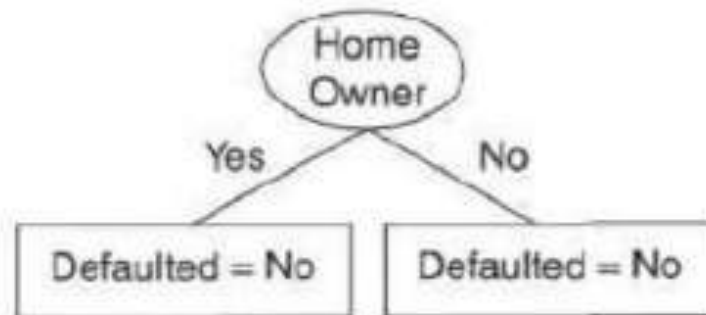
continuous

class

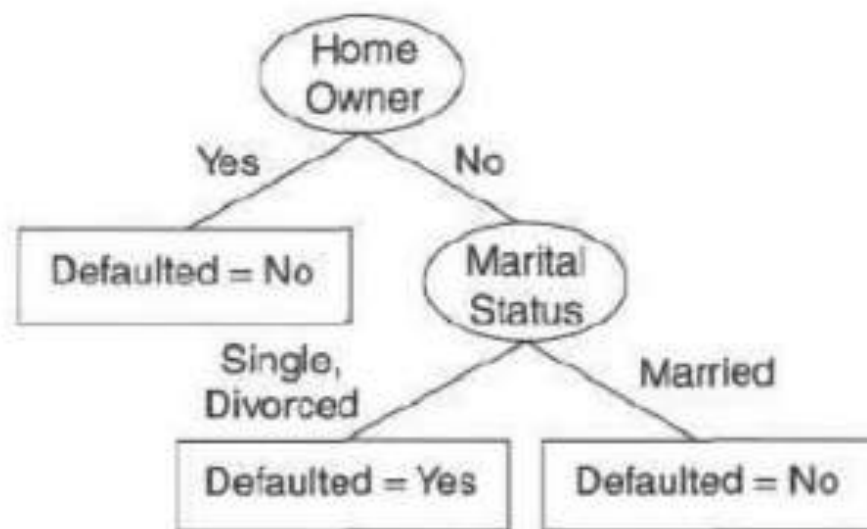
Tid	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



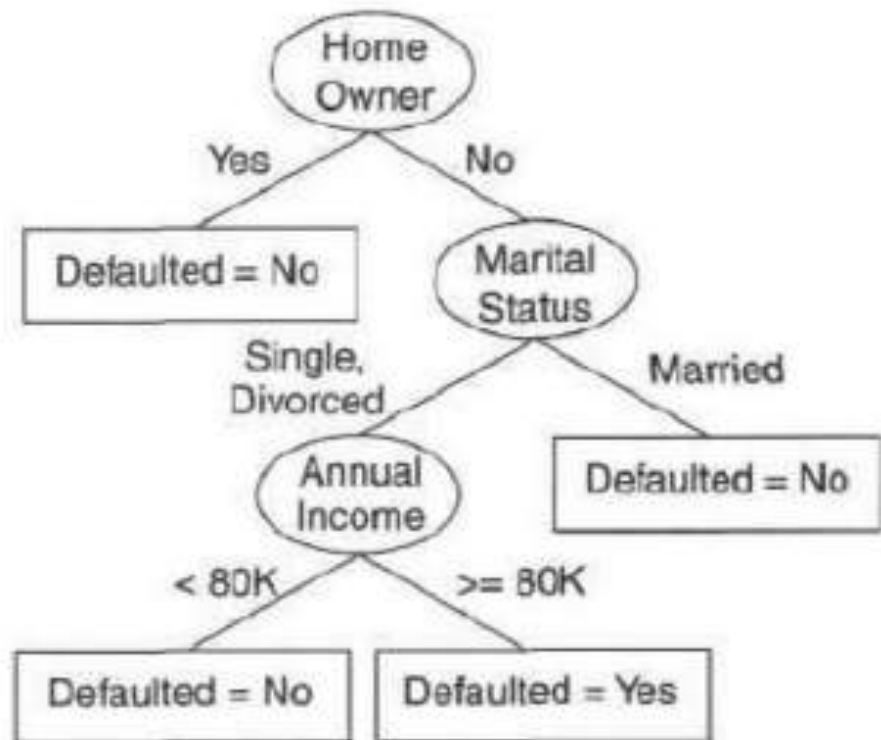
(a)



(b)



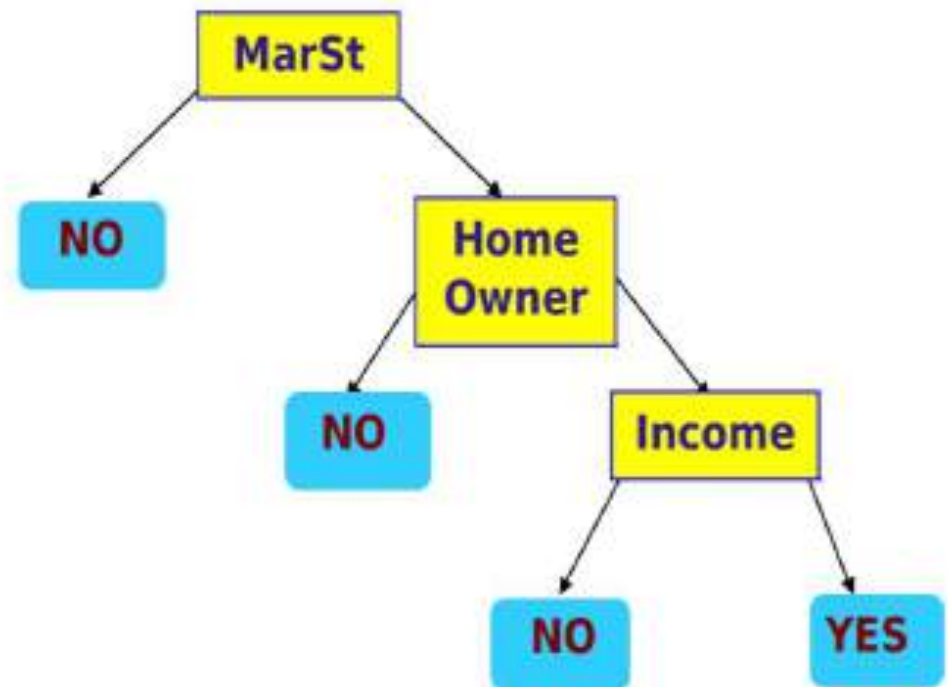
(c)



(d)

categorical
categorical
continuous
class

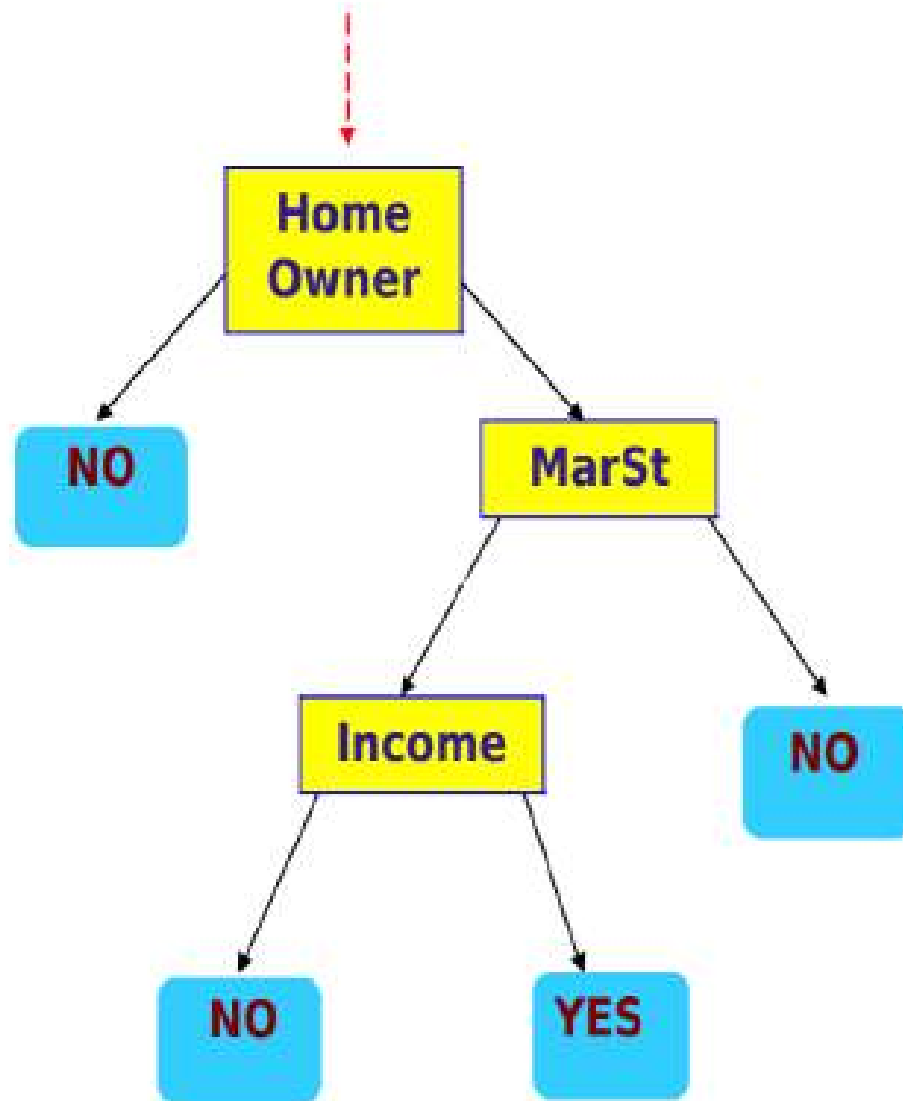
ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



There could be more than one tree that fits the same data!

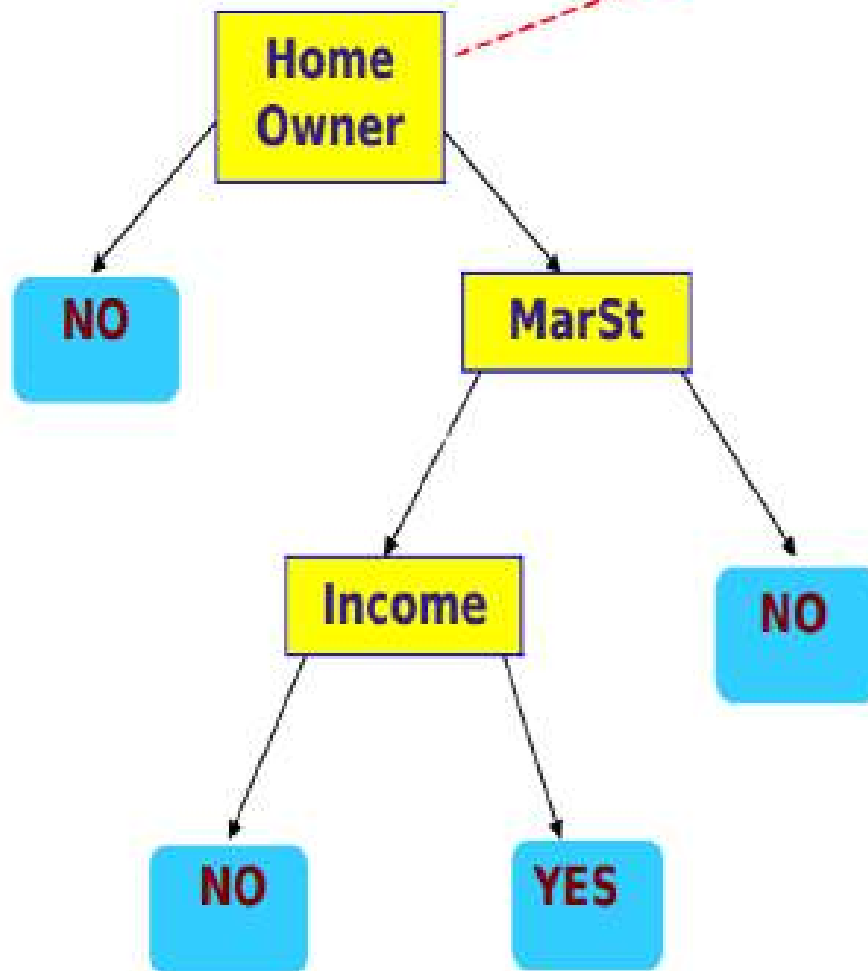
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



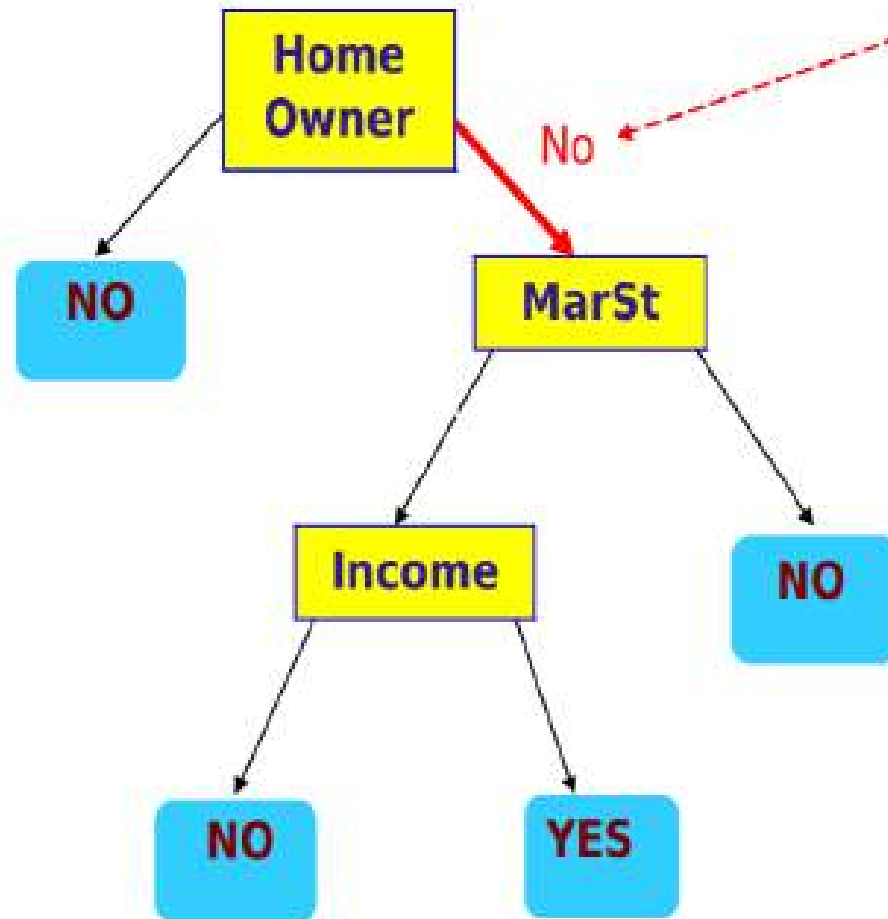
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



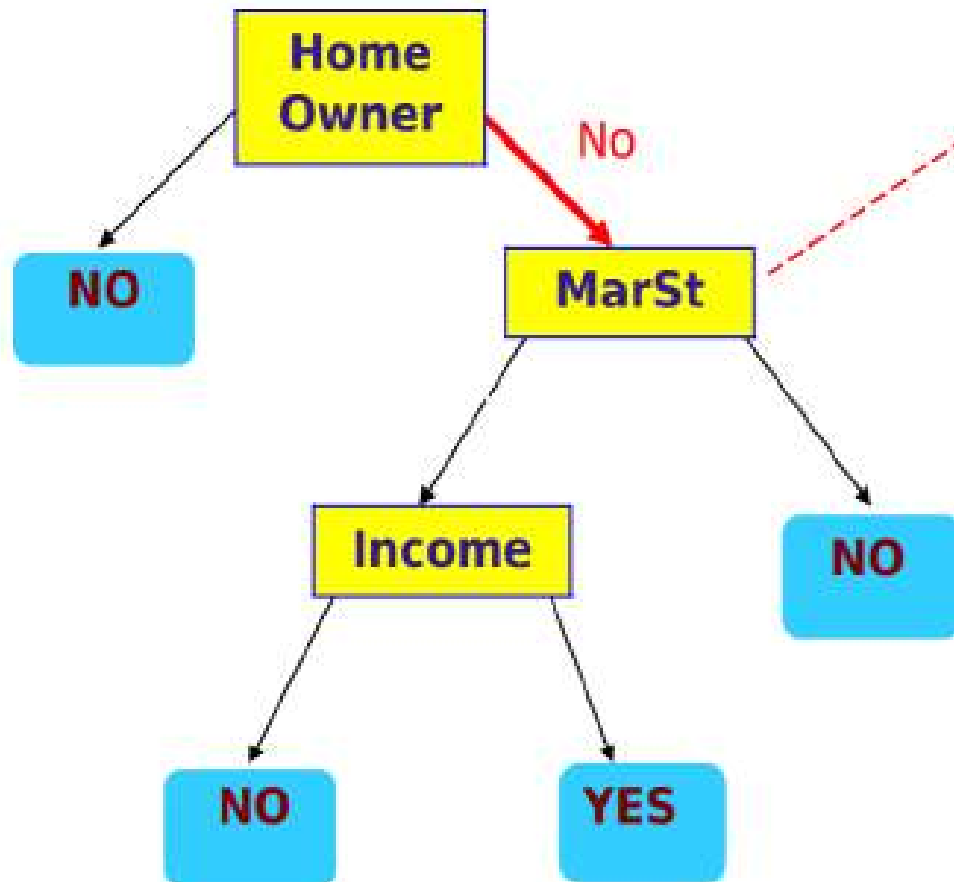
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



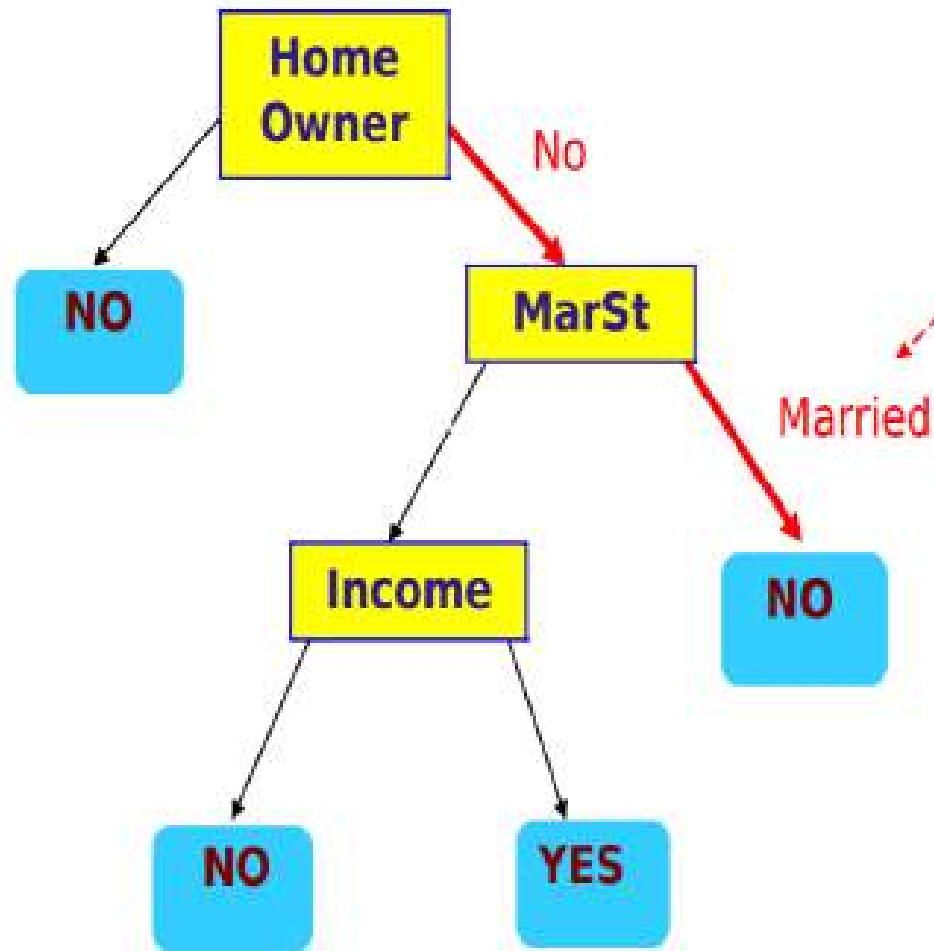
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



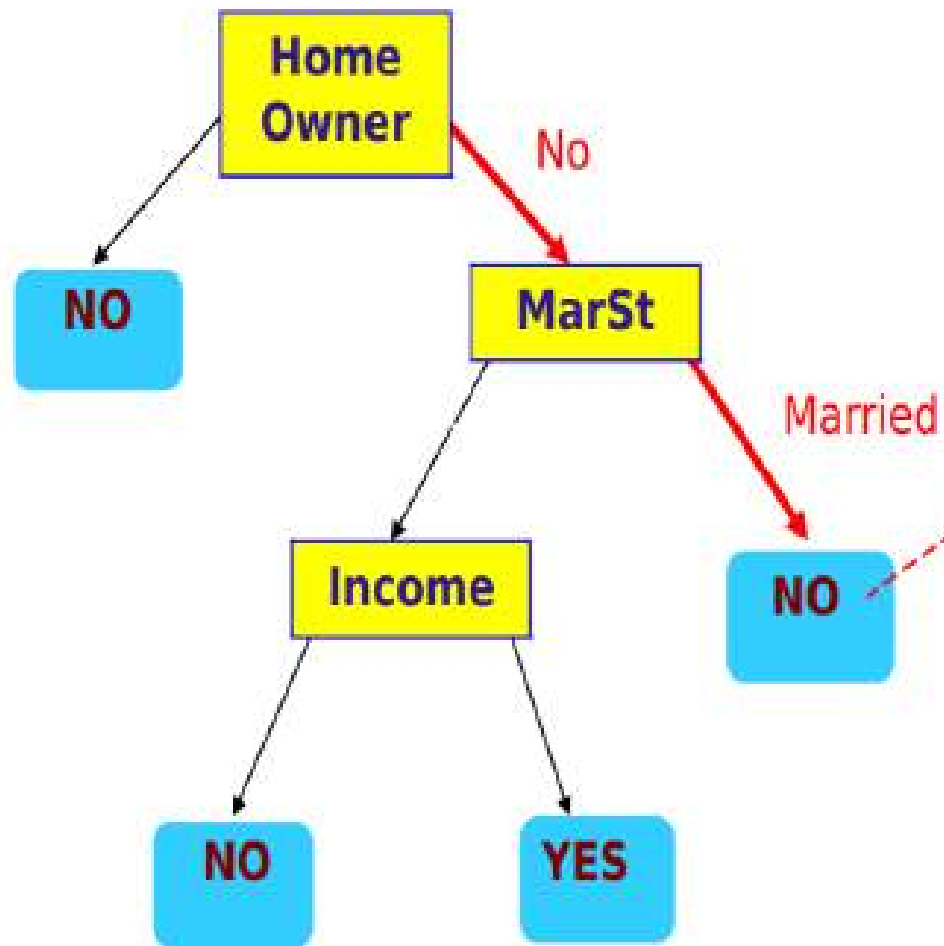
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



- Hunt's algorithm will work if every combination of attribute values is present in the training data and each combination has a unique class label
- These assumptions are too stringent for use in most practical situations

- Additional conditions are needed to handle the following cases:
- It is possible for some of the child nodes created in Step 2 to be empty; i.e., there are no records associated with these nodes. In this case the node is declared a leaf node with the same class label as the majority class of training records associated with its parent node
- In Step 2, if all the records associated with D_i have identical attribute values (except for the class label), then it is not possible to split these records any further. In this case, the node is declared a leaf node with the same class label as the majority class of training records associated with this node

- Additional conditions are needed to handle the following cases:
- It is possible for some of the child nodes created in Step 2 to be empty; i.e., there are no records associated with these nodes. In this case the node is declared a leaf node with the same class label as the majority class of training records associated with its parent node
- In Step 2, if all the records associated with D_i have identical attribute values (except for the class label), then it is not possible to split these records any further. In this case, the node is declared a leaf node with the same class label as the majority class of training records associated with this node

Design Issues of Decision Tree Induction

- | How should training records be split?
 - Method for expressing test condition
 - ◆ depending on attribute types
 - Measure for evaluating the goodness of a test condition
- | How should the splitting procedure stop?
 - Stop splitting if all the records belong to the same class or have identical attribute values
 - Early termination

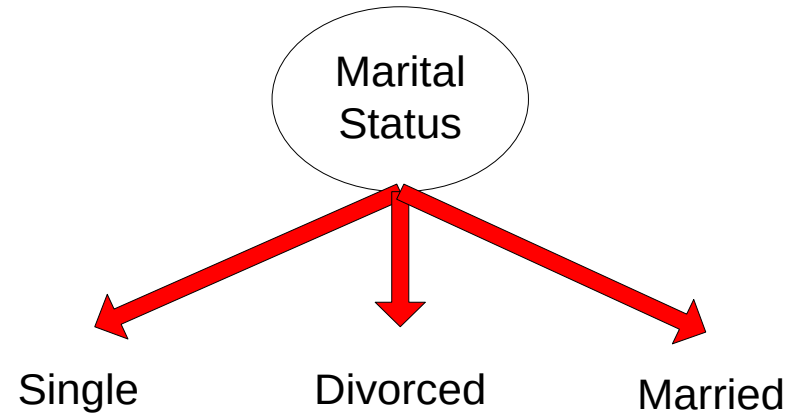
Methods for Expressing Test Conditions

- | Depends on attribute types
 - Binary
 - Nominal
 - Ordinal
 - Continuous

Test Condition for Nominal Attributes

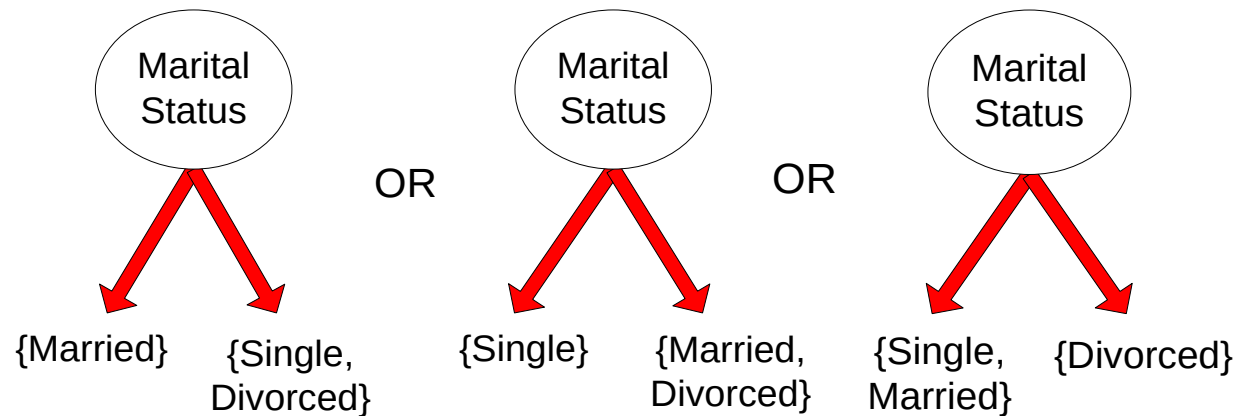
□ Multi-way split:

- Use as many partitions as distinct values.



□ Binary split:

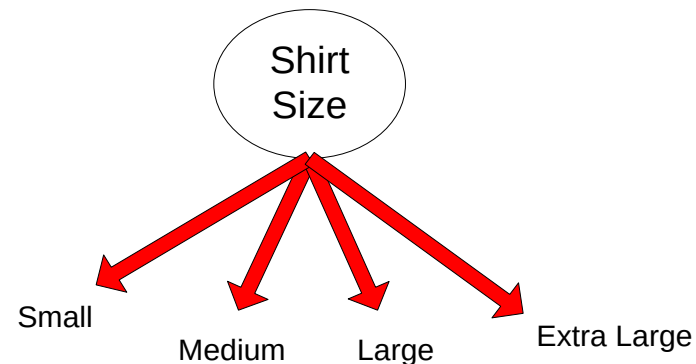
- Divides values into two subsets



Test Condition for Ordinal Attributes

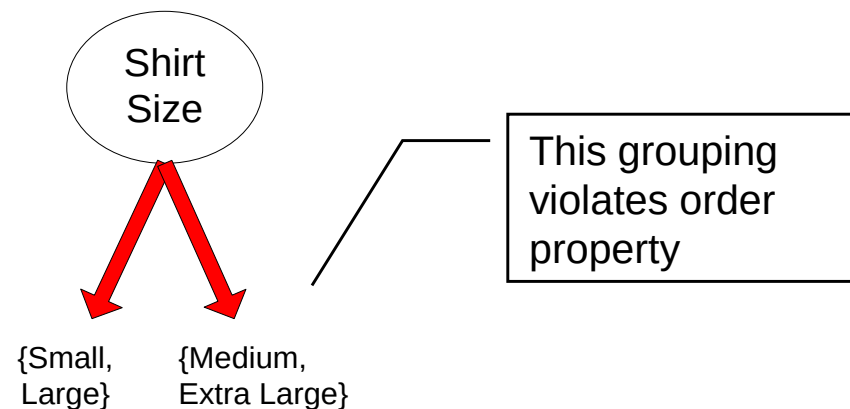
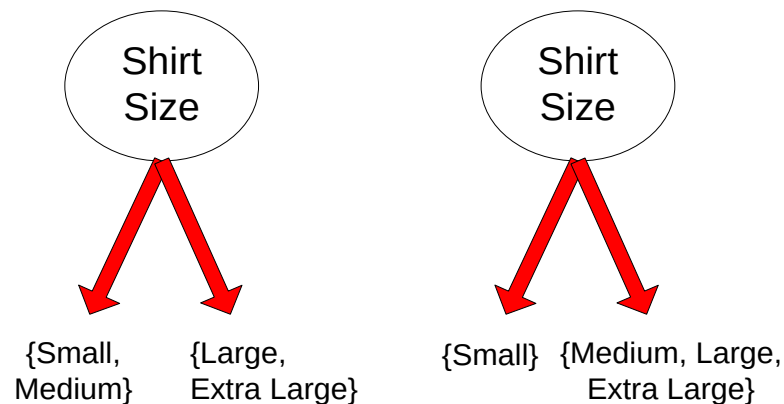
| Multi-way split:

- Use as many partitions as distinct values

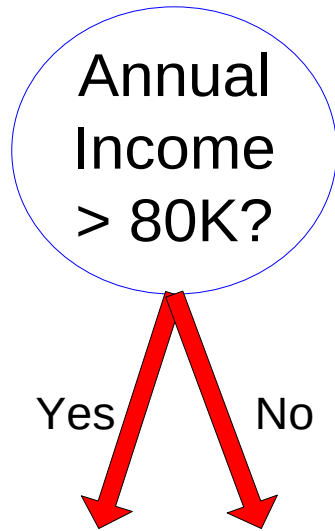


| Binary split:

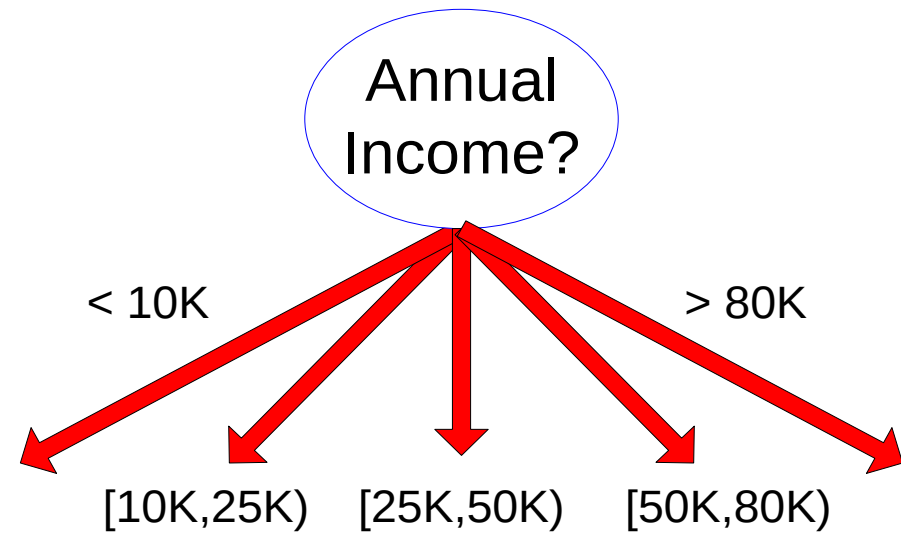
- Divides values into two subsets
- Preserve order property among attribute values



Test Condition for Continuous Attributes



(i) Binary split

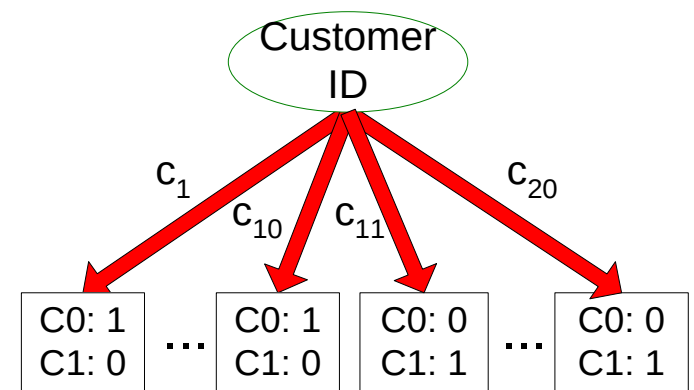
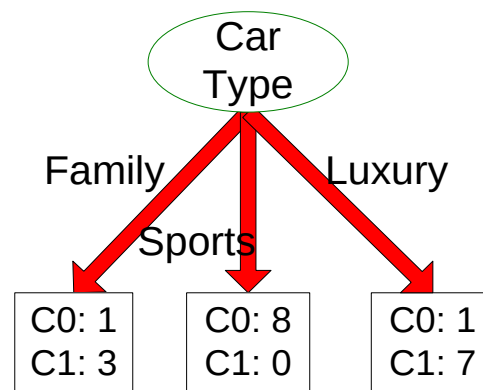
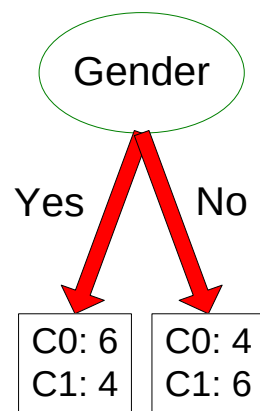


(ii) Multi-way split

How to determine the Best Split

Before Splitting: 10 records of class 0,
10 records of class 1

Customer Id	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1



Which test condition is the best?

How to determine the Best Split

- | Greedy approach:
 - Nodes with **pur**er class distribution are preferred
- | Need a measure of node impurity:

C0: 5
C1: 5

High degree of impurity

C0: 9
C1: 1

Low degree of impurity

Measures of Node Impurity

| Gini Index

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

Where $p_i(t)$ is the frequency of class i at node t , and c is the total number of classes

| Entropy

$$Entropy = - \sum_{i=0}^{c-1} p_i(t) \log_2 p_i(t)$$

| Misclassification error

$$Classification\ error = 1 - \max[p_i(t)]$$

Finding the Best Split

1. Compute impurity measure (P) before splitting
2. Compute impurity measure (M) after splitting
 - | Compute impurity measure of each child node
 - | M is the weighted impurity of child nodes
3. Choose the attribute test condition that produces the highest information gain

$$\text{Gain} = P - M$$

or equivalently, lowest impurity measure after splitting (M)

Gini Index

Gini Index

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

Where $p_l(t)$ is the frequency of class l at node t , and c is the total number of classes

- Lower the Gini index, higher the homogeneity of nodes

Gini Index-Example

Instance Number	X	Y	Z	Class
1	1	1	1	A
2	1	1	0	A
3	0	0	1	B
4	1	0	0	B

The frequencies of the two output classes are given as follows:

- A = 2 (Instances 1,2)
- B = 2 (Instances 3,4)

Gini Index-Example

Attribute 'X':

- There are two possible values of X, i.e., 1, 0
- For $X = 1$, there are 3 instances namely instance 1, 2 and 4.
- The first two instances are labelled as class A and the third instance, i.e, record number 4 is labelled as class B.
- For $X = 0$, there is only 1 instance, i.e, instance number 3 which is labelled as class B.
- Computing the Gini Index for each case,
- $G(X1) = 1 - (2/3)^2 - (1/3)^2 = 0.44444$
- $G(X0) = 1 - (0/1)^2 - (1/1)^2 = 0$

Gini Index-Example

- Weighted Gini Index=probability for X having value 1 * $G(X1)$ +
probability for X having value 0 * $G(X0)$
- Probability for X having value 1= (Number of instances for X having value 1/Total number of instances) = $3/4$
- Probability for X having value 0 = (Number of instances for X having value 0/Total number of instances) = $1/4$
- Weighted Gini Index for two subtrees = $(3/4) G(X1) + (1/4) G(X0)$
- = $0.333333+0 = 0.333333$

Gini Index-Example

Attribute Y:

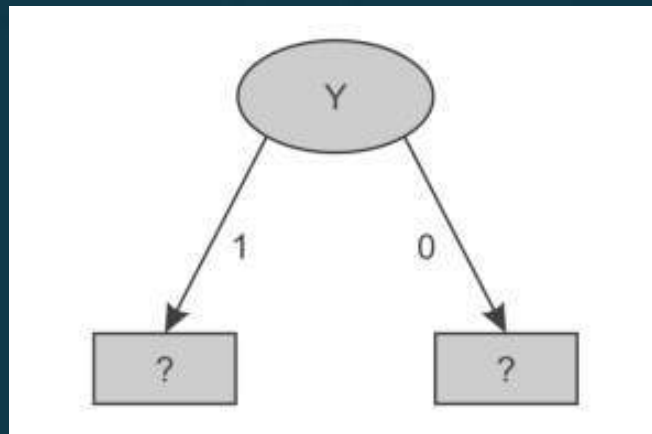
- $G(Y1) = 1 - (2/2)^2 - (0/2)^2 = 0$
- $G(Y0) = 1 - (0/2)^2 - (2/2)^2 = 0$
- Weighted Gini Index = $(2/4) G(Y1) + (2/4) G(Y0)$
 $= 0 + 0 = 0$

Attribute 'Z'

- $G(Z1) = 1 - (1/2)^2 - (1/2)^2 = 0.5$
- $G(Z0) = 1 - (1/2)^2 - (1/2)^2 = 0.5$
- Weighted Gini Index = $(2/4) G(Z1) + (2/4) G(Z0)$
 $= 0.25 + 0.25 = 0.50$

Gini Index-Example

- Since the minimum weighted Gini Index is provided by the attribute 'Y' it is used for the split



- There are two possible values for attribute Y, i.e., 1 or 0, and so the dataset will be split into two subsets based on distinct values of Y attribute

Gini Index-Example

Dataset for Y '1'			
Instance Number	X	Z	Class
1	1	1	A
2	1	0	A

- When Y is 1, we get homogeneous class label, A i.e least impurity, that implies When Y is 1, class label is A
- (This can be confirmed by computing Gini index of data set i.e $1-(2/2)^2-(0/2)^2=0$)

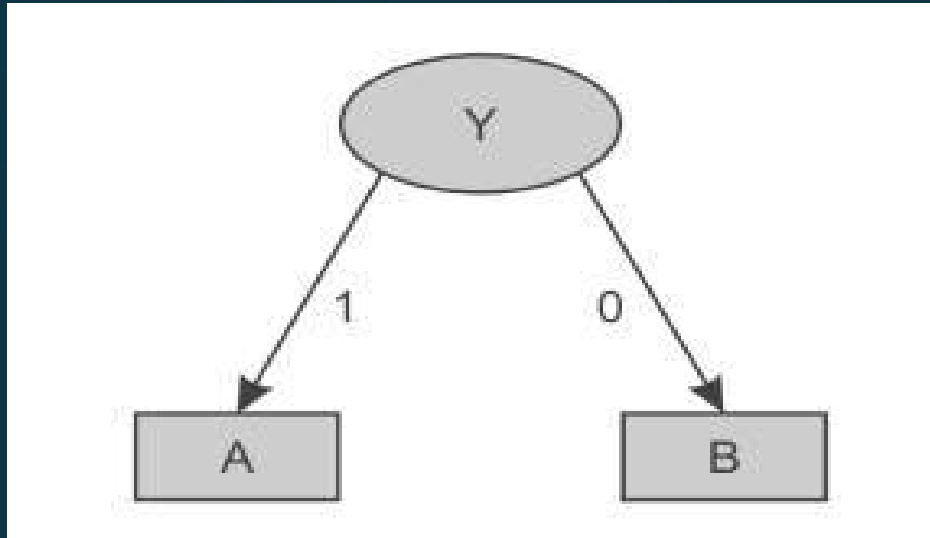
Gini Index-Example

Dataset for Y '0'

Instance Number	X	Z	Class
3	0	1	B
4	1	0	B

- Here, for Y having value 0, both records are classified as class 'B' irrespective of value of X and Z.
- (This can be confirmed by computing Gini index of data set i.e $1-(0/2)^2-(2/2)^2=0$)

Gini Index-Example



Example 2:

Instance Number	Outlook	Temperature	Humidity	Windy	Play
1	sunny	hot	high	false	No
2	sunny	hot	high	true	No
3	overcast	hot	high	false	Yes
4	rainy	mild	high	false	Yes
5	rainy	cool	normal	false	Yes
6	rainy	cool	normal	true	No
7	overcast	cool	normal	true	Yes
8	sunny	mild	high	false	No
9	sunny	cool	normal	false	Yes
10	rainy	mild	normal	false	Yes
11	sunny	mild	normal	true	Yes
12	overcast	mild	high	true	Yes
13	overcast	hot	normal	false	Yes
14	rainy	mild	high	true	No

Gini Index-Example

Outlook	Temperature	Humidity	Windy	Play
sunny = 5	hot = 4	high = 7	true = 6	yes = 9
overcast = 4	mild = 6	normal = 7	false = 8	no = 5
rainy = 5	cool = 4			

Gini Index-Example

- Attribute 'Outlook'

$$G(\text{Sunny}) = G(S) = 1 - (2/5)^2 - (3/5)^2 = 0.48$$

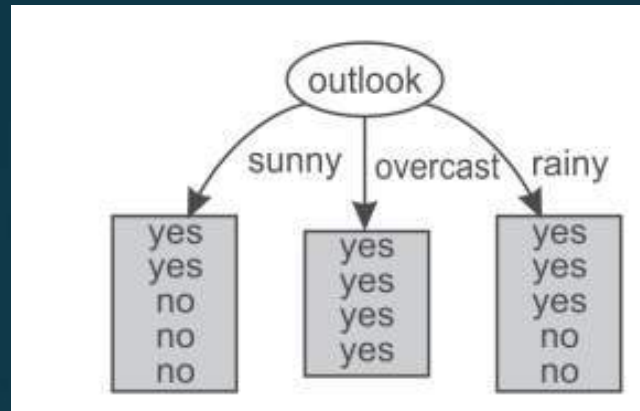
$$G(\text{Overcast}) = G(O) = 1 - (4/4)^2 - (0/4)^2 = 0$$

$$G(\text{Rainy}) = G(R) = 1 - (3/5)^2 - (2/5)^2 = 0.48$$

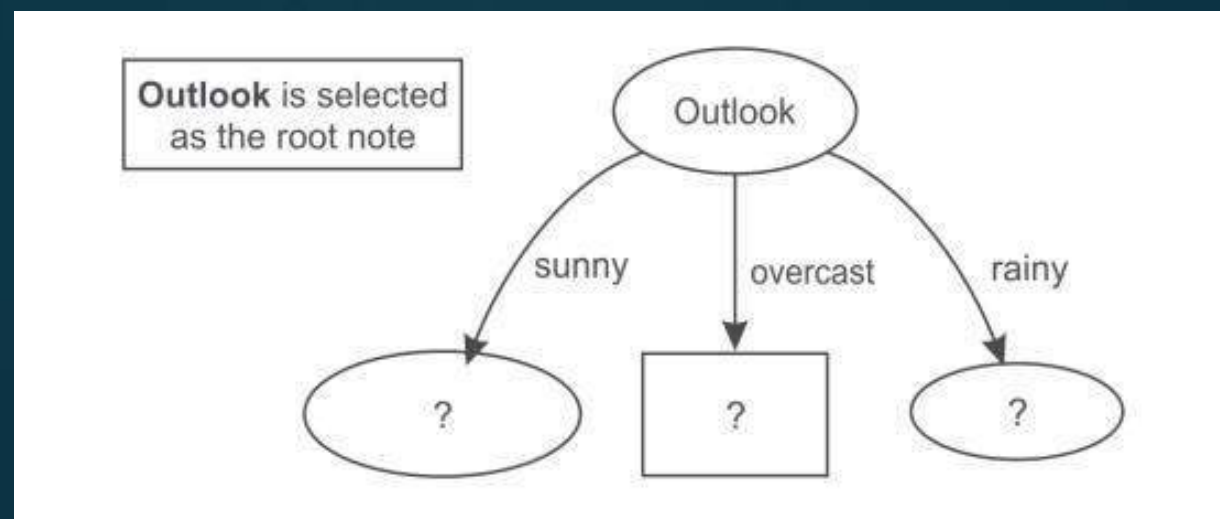
- Weighted Gini Index= $(5/14) G(S) + (4/14) G(O) + (5/14) G(R)$
 $= 0.171428 + 0 + 0.171428$
 $= 0.342857$

Potential Split attribute	Weighted Gini Index
Outlook	0.342857
Temperature	0.44028
Humidity	0.3673439
Windy	0.45918

Gini Index-Example



- For Outlook, as there are three possible values, i.e., sunny, overcast and rainy, the dataset will be split into three subsets based on distinct values of the Outlook attribute



Gini Index-Example

Dataset for Outlook 'Sunny'

Instance Number	Temperature	Humidity	Windy	Play
1	hot	high	false	No
2	hot	high	true	No
3	mild	high	false	No
4	cool	normal	false	Yes
5	mild	normal	true	Yes

- Attribute 'Temperature'

$$G(\text{Hot}) = G(H) = 1 - (0/2)^2 - (2/2)^2 = 0$$

$$G(\text{Mild}) = G(M) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

$$G(\text{Cool}) = G(C) = 1 - (1/1)^2 - (0/1)^2 = 0$$

- Weighted Gini Index = $(2/5)G(H) + (1/5)G(M) + (2/5)G(C) = 0 + 0.1 + 0 = 0.1$

- Potential split Attribute

Temperature

Humidity

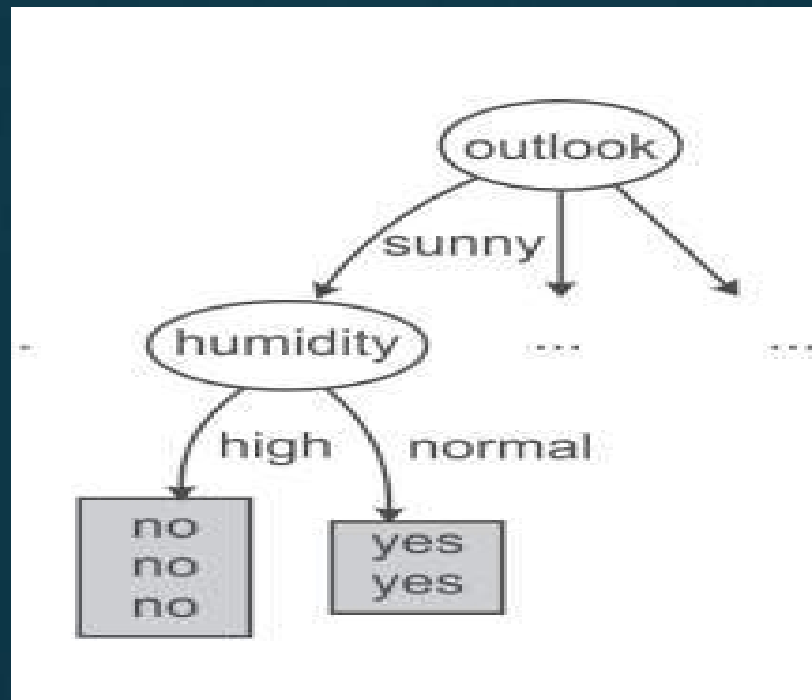
Wind

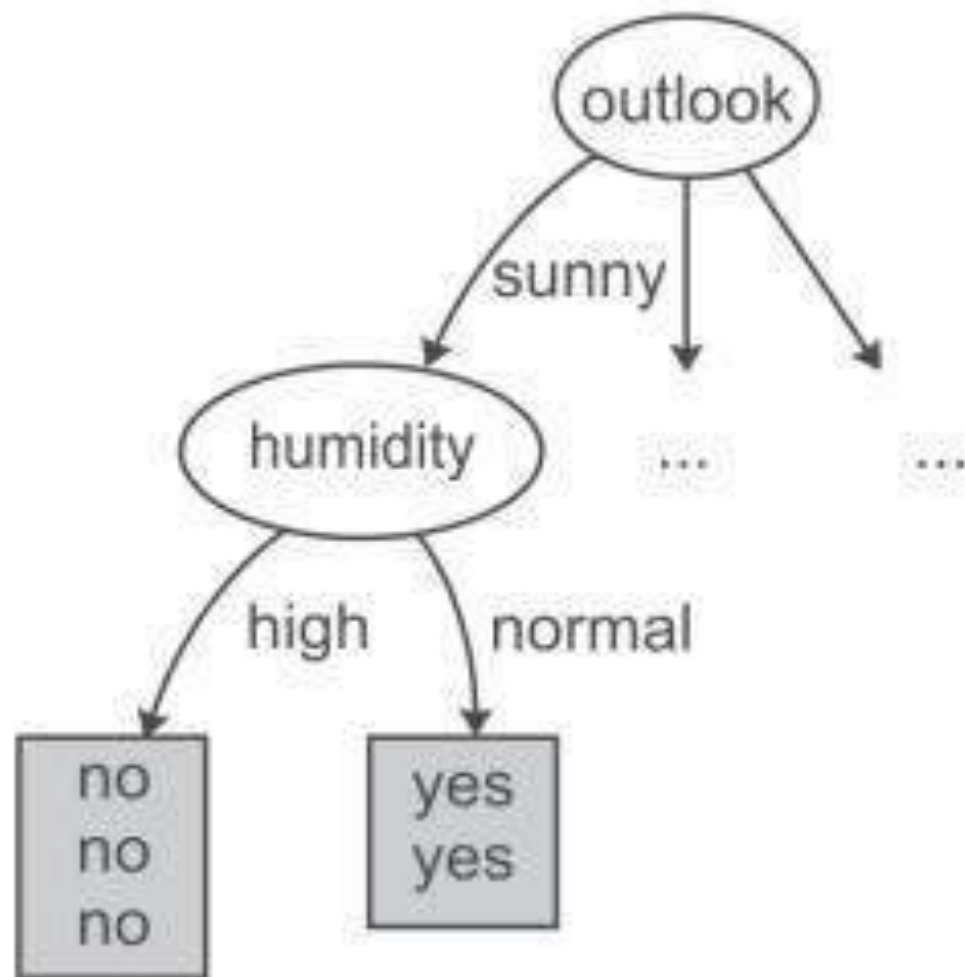
Weighted Gini index

0.1

0

0.466





Dataset for Humidity 'High'

<i>Instance Number</i>	<i>Temperature</i>	<i>Windy</i>	<i>Play</i>
1	Hot	false	No
2	Hot	true	No
3	Mild	false	No

We can clearly see that all records with high humidity are classified as play having 'No' value. On the other hand, all records with normal humidity value are classified as play having 'Yes' value.

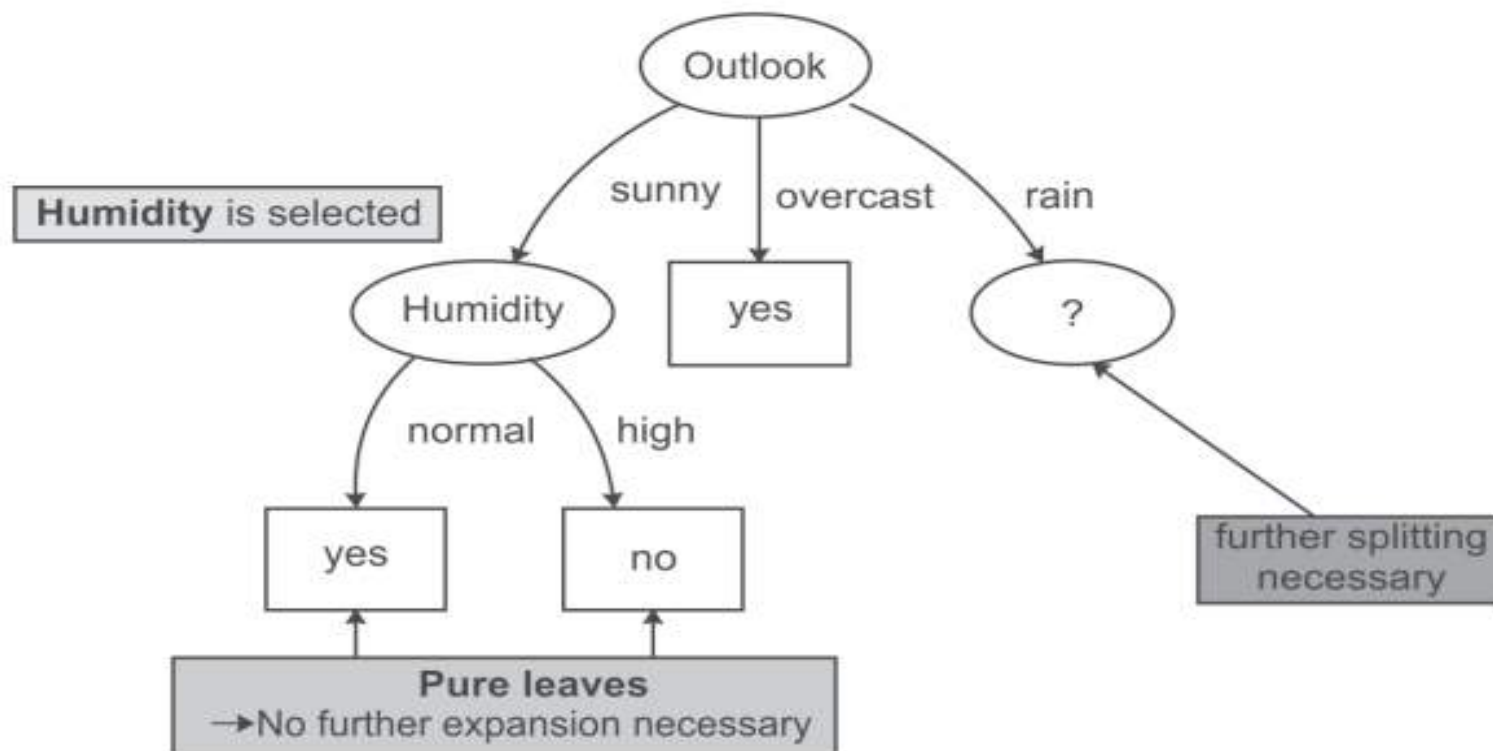
Dataset for Humidity 'Normal'

<i>Instance Number</i>	<i>Temperature</i>	<i>Windy</i>	<i>Play</i>
1	cool	false	Yes
2	mild	true	Yes

Dataset for Outlook 'Overcast'

Instance Number	Temperature	Humidity	Windy	Play
1	hot	high	false	Yes
2	cool	normal	true	Yes
3	mild	high	true	Yes

This dataset consists of 3 records. For outlook overcast all records belong to the 'Yes' class only.



Dataset for Outlook 'Rainy'

Instance Number	Temperature	Humidity	Windy	Play
1	Mild	High	False	Yes
2	Cool	Normal	False	Yes
3	Cool	Normal	True	No
4	Mild	Normal	False	Yes
5	Mild	High	True	No

- Potential split Attribute

Temperature

Humidity

Wind

Weighted Gini index

0.4666

0.4666

0

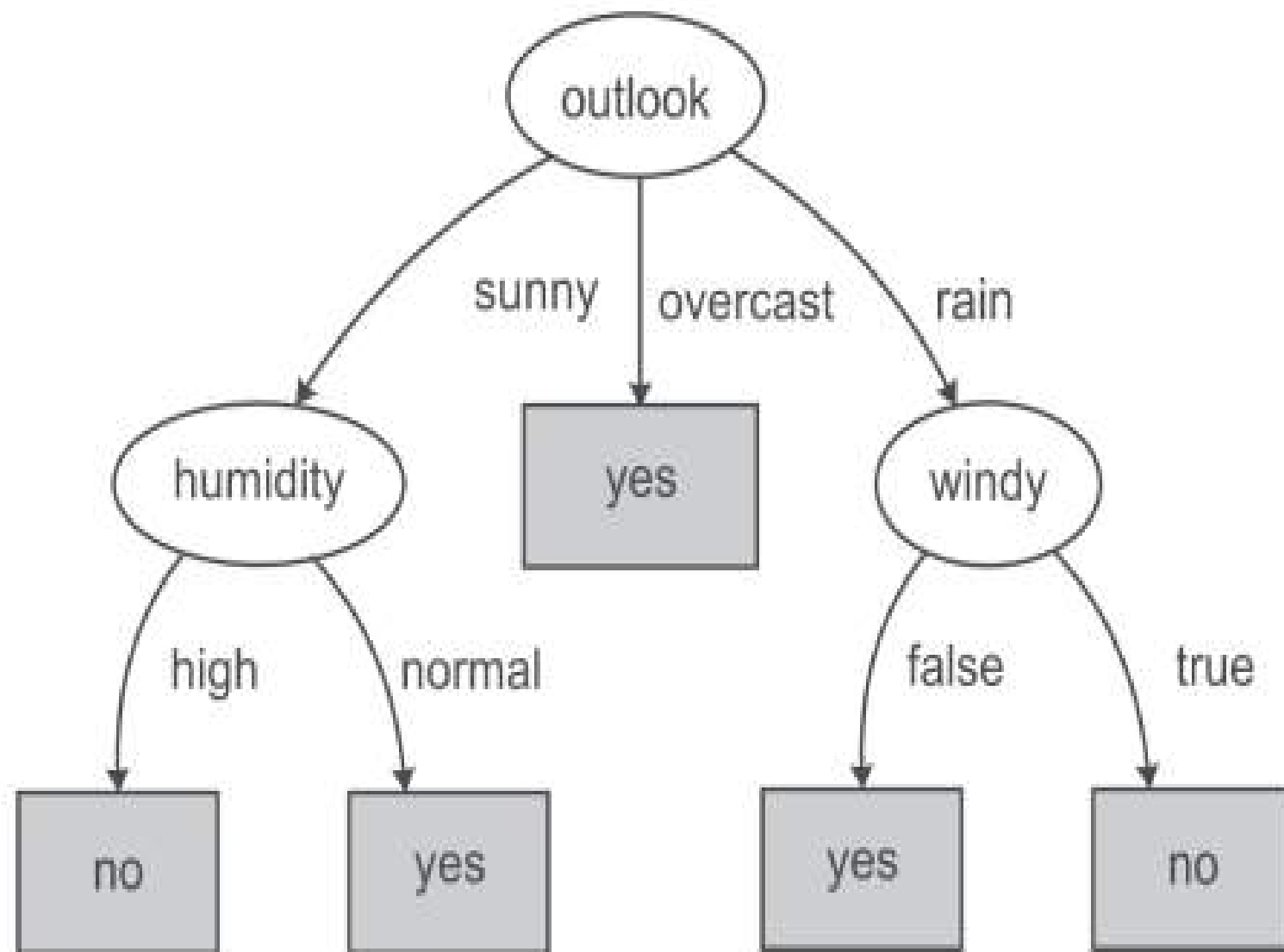
Dataset for Windy 'TRUE'

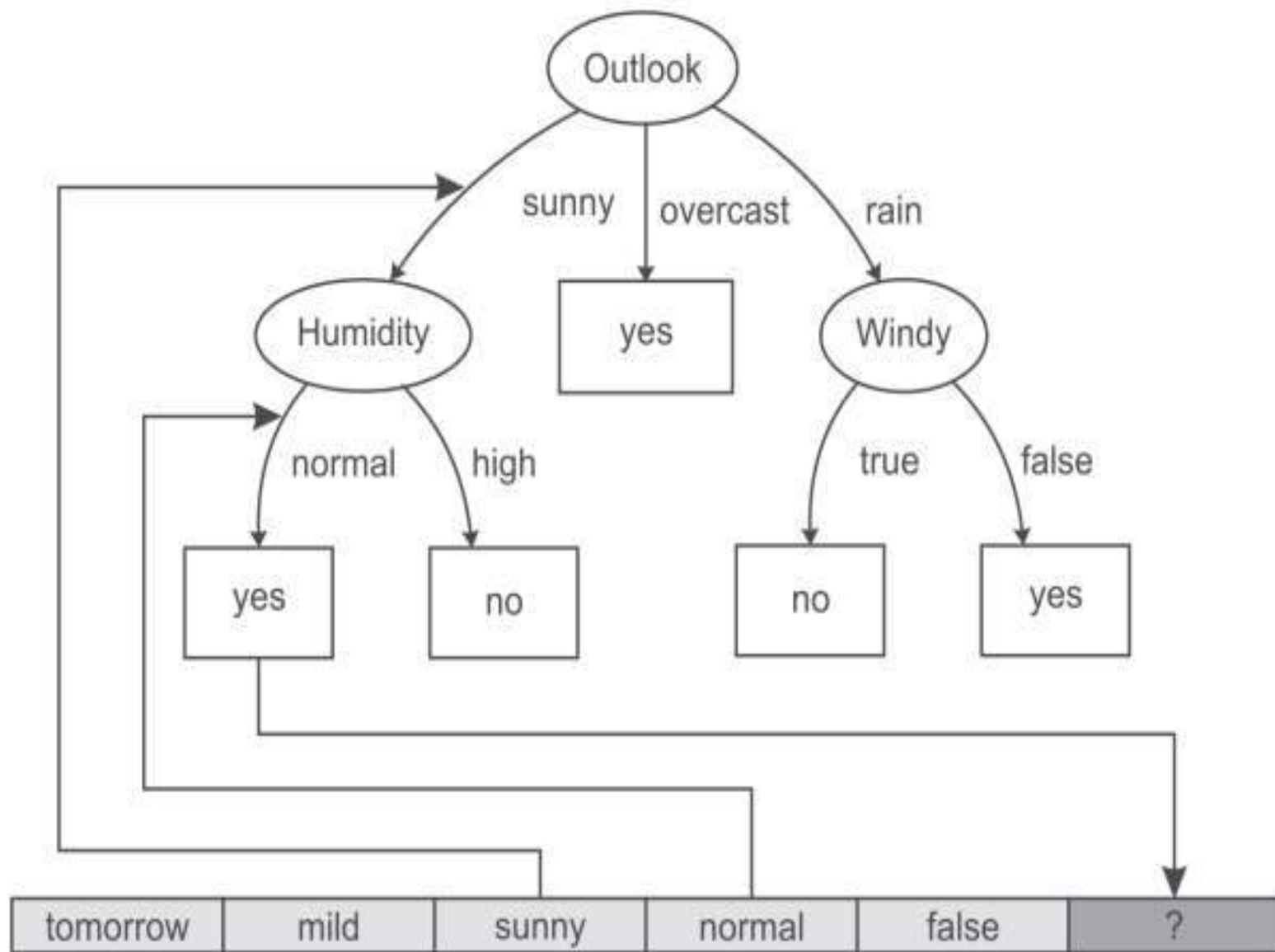
<i>Instance Number</i>	<i>Temperature</i>	<i>Humidity</i>	<i>Play</i>
1	cool	normal	No
2	mild	high	No

It is clear from the data that whenever it is windy the play is not held. Play is held whenever conditions are otherwise.

Dataset for Windy 'FALSE'

<i>Instance Number</i>	<i>Temperature</i>	<i>Humidity</i>	<i>Play</i>
1	mild	high	Yes
2	cool	normal	Yes
3	mild	normal	Yes





Best split using Entropy and Information gain

Instance Number	Outlook	Temperature	Humidity	Windy	Play
1	sunny	hot	high	false	No
2	sunny	hot	high	true	No
3	overcast	hot	high	false	Yes
4	rainy	mild	high	false	Yes
5	rainy	cool	normal	false	Yes
6	rainy	cool	normal	true	No
7	overcast	cool	normal	true	Yes
8	sunny	mild	high	false	No
9	sunny	cool	normal	false	Yes
10	rainy	mild	normal	false	Yes
11	sunny	mild	normal	true	Yes
12	overcast	mild	high	true	Yes
13	overcast	hot	normal	false	Yes
14	rainy	mild	high	true	No

Outlook	Temp.	Humidity	Windy	Play
sunny = 5	hot = 4	high = 7	true = 6	yes = 9
overcast = 4	mild = 6	normal = 7	false = 8	no = 5
rainy = 5	cool = 4			

- In the dataset, there are 14 samples and two classes for target attribute 'Play', i.e., **Yes** or **No**. The frequencies of these two classes are:
Yes = 9 No = 5
- Entropy of the whole dataset on the basis of whether play is held or not is given by:
- $E = - \text{probability for play being held} * \log(\text{probability for play being held}) - \text{probability for play not being held} * \log(\text{probability for play not being held})$

- Probability for play having value Yes = (Number of instances for Play is Yes/Total number of instances) = $9/14$
- Probability for play having value No = (Number of instances for Play is No/Total number of instances) = $5/14$
- Therefore, $E = - (9/14) \log (9/14) - (5/14) \log (5/14) = 0.9435142$

- Now Let us consider each attribute one by one as split attributes and calculate the information for each attribute

Attribute 'Outlook':

- $E(\text{Sunny}) = E(S) = - (2/5) \log(2/5) - (3/5) \log(3/5) = 0.97428$
- $E(\text{Overcast}) = E(O) = - (4/4) \log(4/4) - (0/4) \log(0/4) = 0$
- $E(\text{Rainy}) = E(R) = - (3/5) \log(3/5) - (2/5) \log(2/5) = 0.97428$
- Total Entropy for these three sub-trees = probability for outlook Sunny * E(S) + probability for outlook Overcast * E(O) + probability for outlook Rainy * E(R)
- $= (5/14) E(S) + (4/14) E(O) + (5/14) E(R)$
- $= 0.3479586 + 0.3479586$
- $= 0.695917$

Attribute 'Temperature'

- $E(\text{Hot}) = E(H) = - (2/4) \log(2/4) - (2/4) \log(2/4) = 1.003433$
- $E(\text{Mild}) = E(M) = - (4/6) \log(4/6) - (2/6) \log(2/6) = 0.9214486$
- $E(\text{Cool}) = E(C) = - (3/4) \log(3/4) - (1/4) \log(1/4) = 0.814063501$
- Total Entropy for these three sub-trees = $(4/14) E(H) + (6/14) E(M) + (4/14) E(C)$
- $= 0.2866951429 + 0.3949065429 + 0.23258957 = 0.9141912558$

Attribute 'Humidity'

- $E(\text{High}) = E(H) = - (3/7) \log(3/7) - (4/7) \log(4/7) = 0.98861$
- $E(\text{Normal}) = E(N) = - (6/7) \log(6/7) - (1/7) \log(1/7) = 0.593704$
- Total Entropy for the two sub-trees = $(7/14) E(H) + (7/14) E(N)$
 $= 0.494305 + 0.29685 = 0.791157$

Attribute 'Windy'

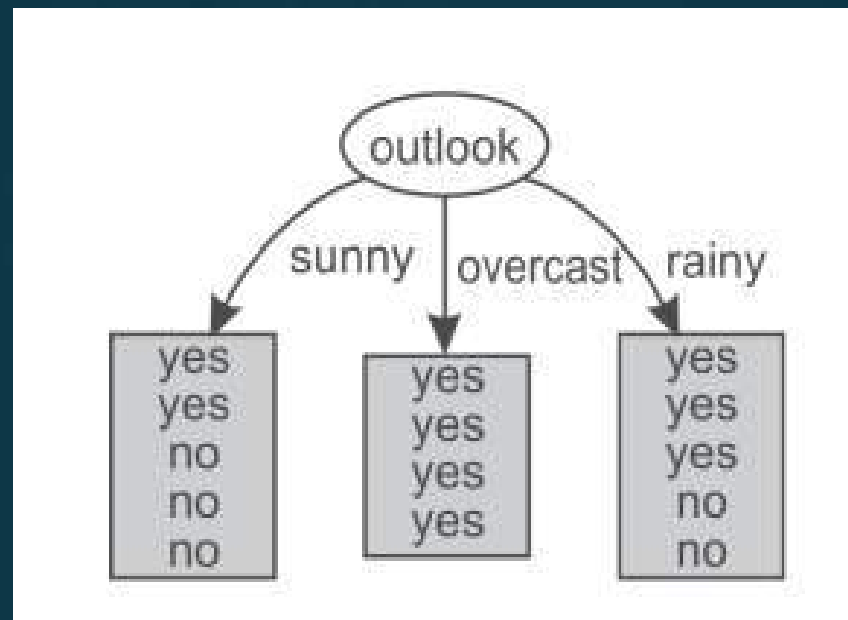
- $E(\text{True}) = E(T) = - (3/6) \log(3/6) - (3/6) \log(3/6) = 1.003433$
- $E(\text{False}) = E(F) = - (6/8) \log(6/8) - (2/8) \log(2/8) = 0.81406$
- Total information for the two sub-trees = $(6/14) E(T) + (8/14) E(F)$
 $= 0.4300427 + 0.465179 = 0.89522$

- The information gain can now be computed:

Potential Split attribute	Information before split	Information after split	Information gain
Outlook	0.9435	0.6959	0.2476
Temperature	0.9435	0.9142	0.0293
Humidity	0.9435	0.7912	0.15234
Windy	0.9435	0.8952	0.0483

- From the above table, it is clear that the largest information gain is provided by the attribute 'Outlook' so it is used for the split

- For Outlook, as there are three possible values, i.e., Sunny, Overcast and Rain, the dataset will be split into three subsets based on distinct values of the Outlook attribute



Dataset for Outlook 'Sunny'

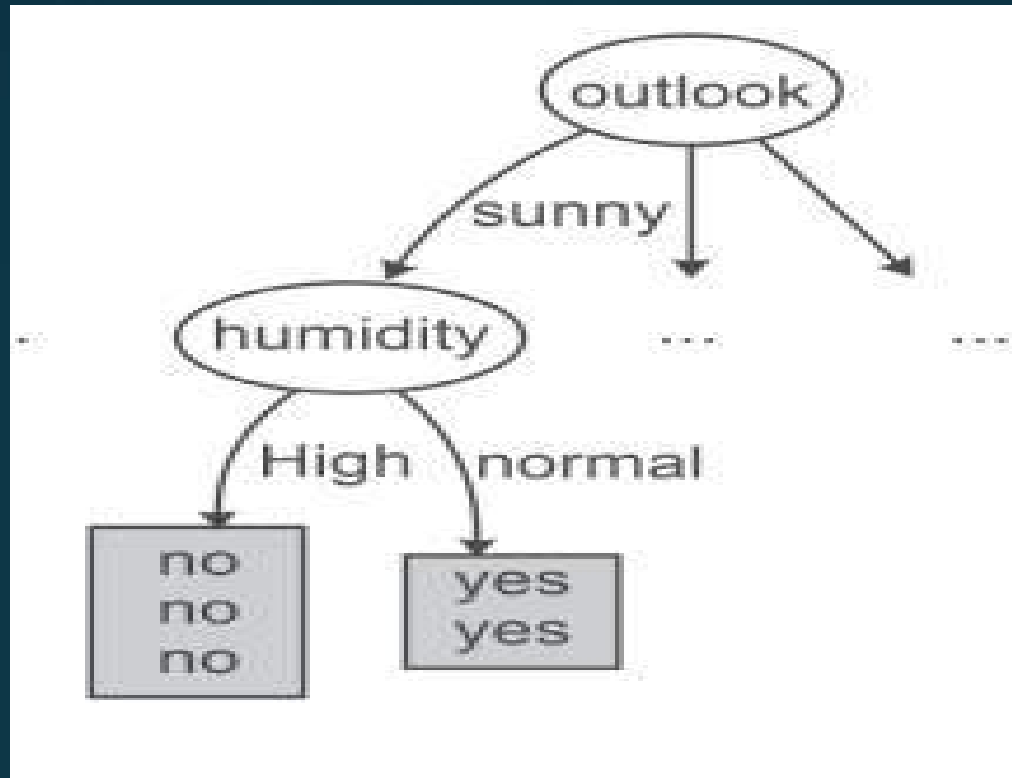
Instance Number	Temperature	Humidity	Windy	Play
1	Hot	high	false	No
2	Hot	high	true	No
3	Mild	high	false	No
4	Cool	normal	false	Yes
5	Mild	normal	true	Yes

- Entropy of the whole dataset on the basis of whether play is held or not is given by $I = - \text{probability for Play Yes} * \log (\text{probability for Play Yes}) - \text{probability for Play No} * \log (\text{probability for Play No})$
- $E = (-2/5) \log (2/5) - (3/5) \log(3/5) = 0.97$

- Let us consider each attribute one by one as a split attribute and calculate the Entropy for each attribute.

Potential Split attribute	Information before split	Information after split	Information gain
Temperature	0.97	0.40137	0.56863
Humidity	0.97	0	0.97
Windy	0.97	0.954239	0.015761

- The largest information gain is provided by the attribute 'Humidity' and it is used for the split



- There are two possible values of humidity so data is split into two parts, i.e. humidity 'high' and humidity 'low'

Dataset for Humidity 'High'

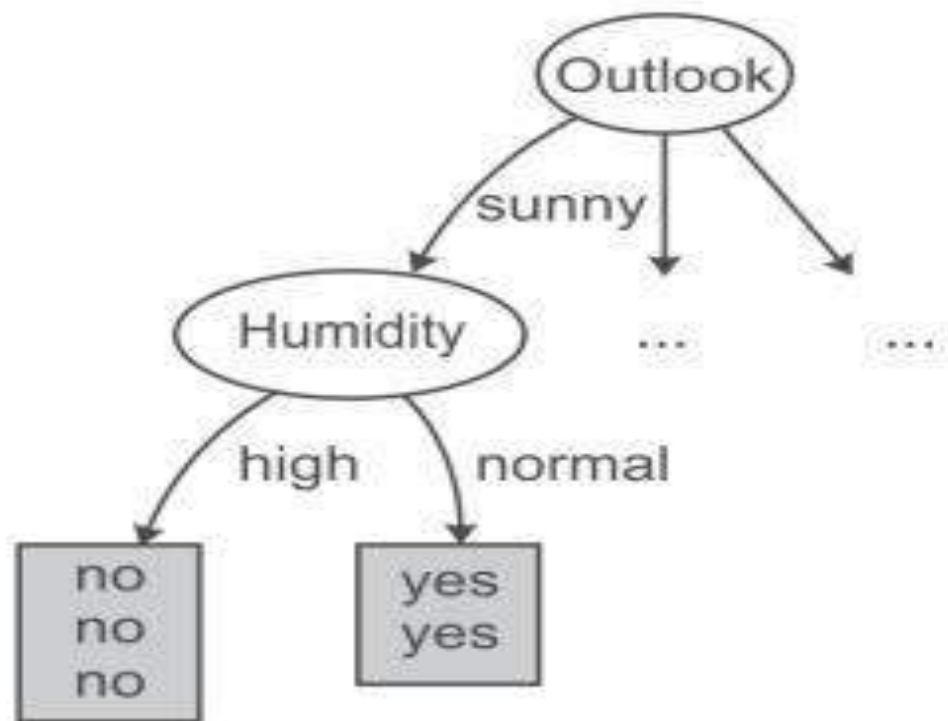
Instance Number	Temperature	Windy	Play
1	hot	false	No
2	hot	true	No
3	mild	false	No

- As the dataset represents the same class 'No' for all the records, therefore for Humidity value 'High' the output class is always 'No'

Dataset for Humidity 'Normal'

Instance No	Temperature	Windy	Play
1	cool	false	Yes
2	mild	true	Yes

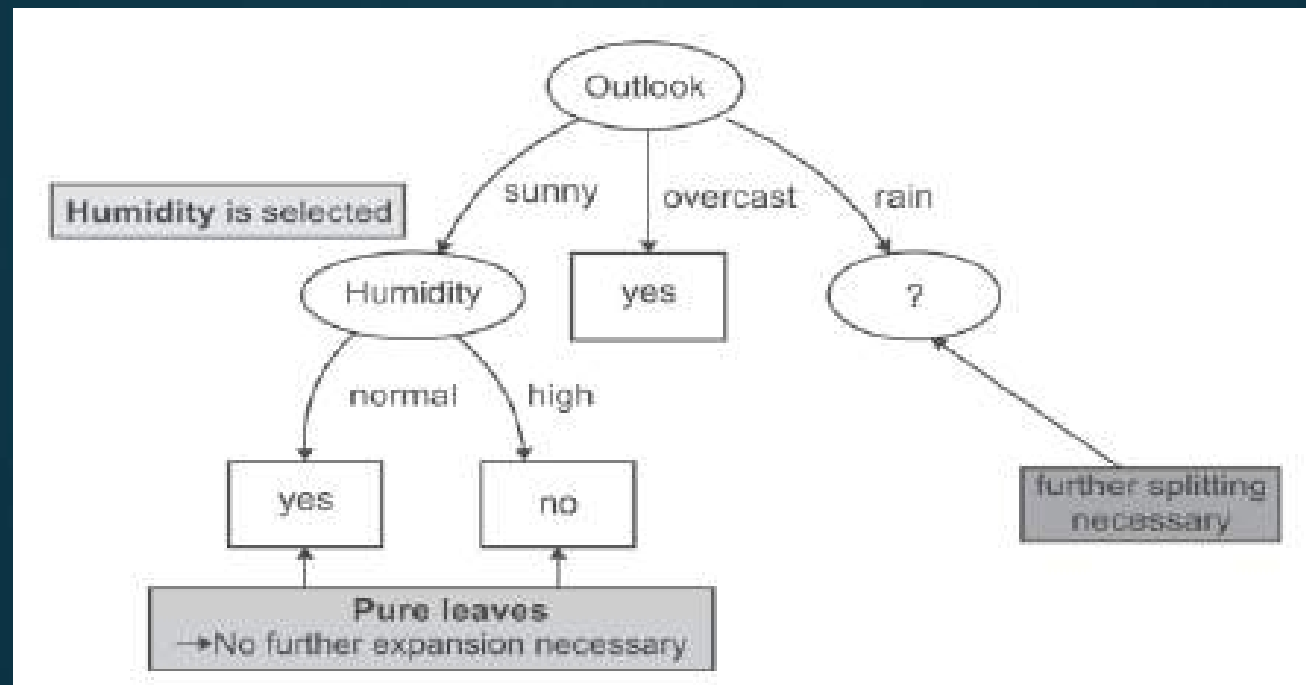
- When humidity's value is 'Normal' the output class is always 'Yes'



- Dataset for Outlook 'Overcast'

Instance Number	Temperature	Humidity	Windy	Play
1	hot	high	false	Yes
2	cool	normal	true	Yes
3	mild	high	true	Yes

- For outlook Overcast, the output class is always 'Yes'.
- it has been concluded that if the outlook is 'Overcast' then play is always held



- Dataset for Outlook 'Rainy'

Instance Number	Temperature	Humidity	Windy	Play
1	Mild	High	False	Yes
2	Cool	Normal	False	Yes
3	Cool	Normal	True	No
4	Mild	Normal	False	Yes
5	Mild	High	True	No

- Information of the whole dataset= $(-3/5) \log (3/5) - (2/5) \log (2/5) = 0.97428$

- Let us consider each attribute one by one as split attributes and calculate the information provided for each attribute.

Potential Split attribute	Information before split	Information after split	Information gain
Temperature	0.97428	0.954297	0.019987
Humidity	0.97428	0.9512	0.02308
Windy	0.97428	0	0.97428

- Hence, the largest information gain is provided by the attribute 'Windy' and this attribute is used for the split.

Dataset for Windy 'TRUE'

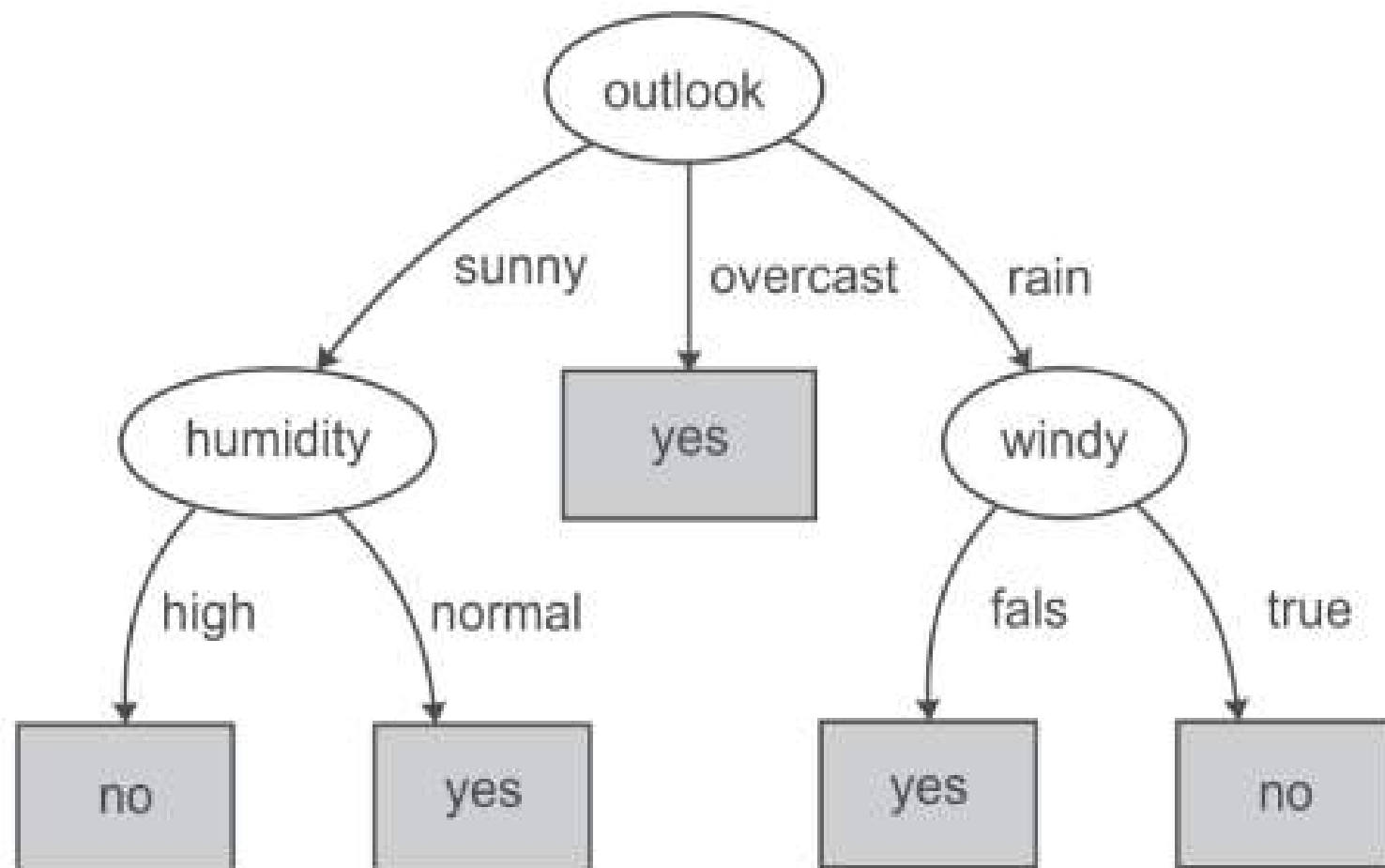
Instance Number	Temperature	Humidity	Play
1	cool	normal	No
2	mild	high	No

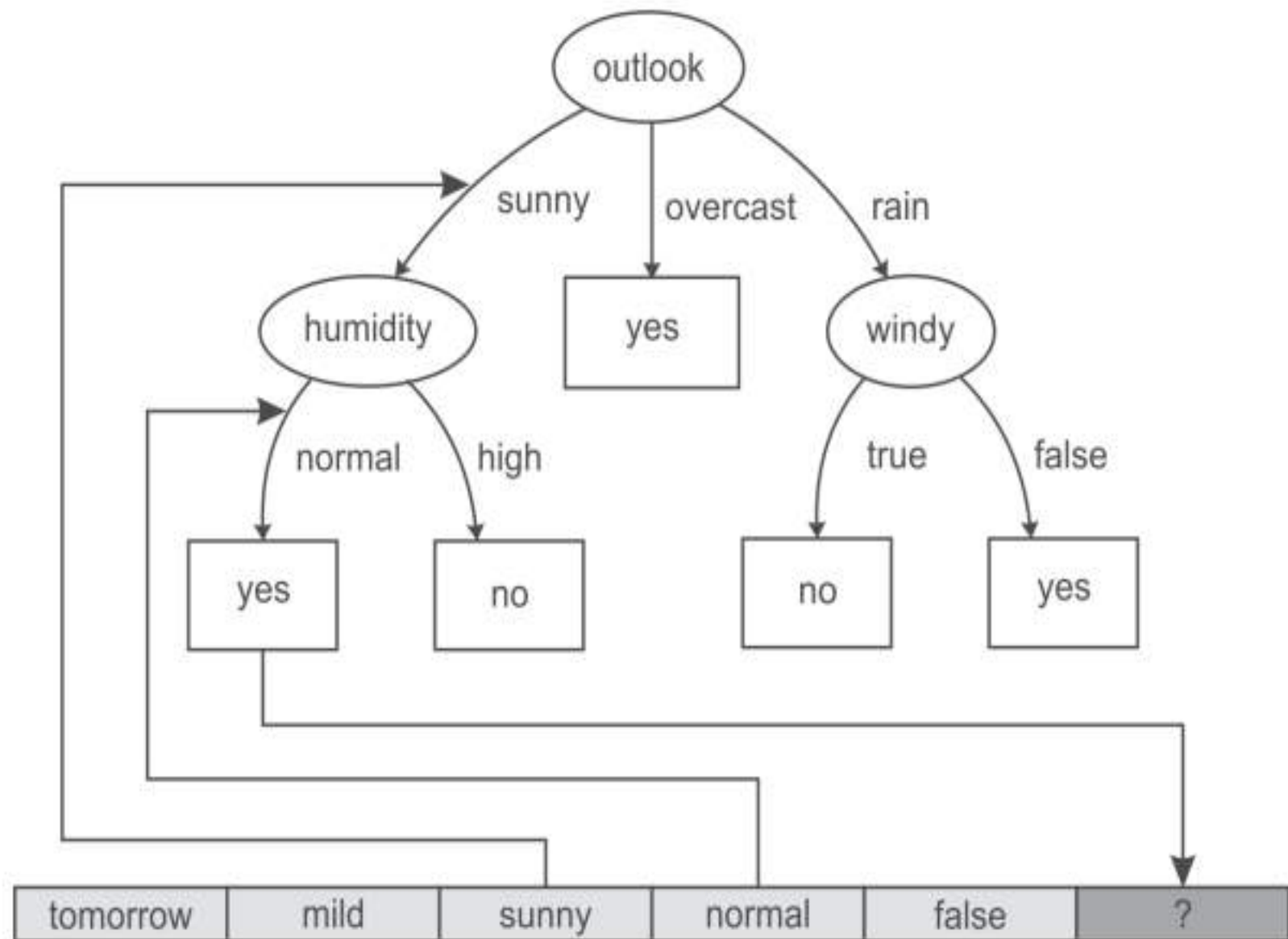
- From above table it is clear that for Windy value 'TRUE', the output class is always 'No'

Dataset for Windy 'FALSE'

Instance Number	Temperature	Humidity	Play
1	Mild	High	Yes
2	Cool	Normal	Yes
3	Mild	Normal	Yes

- Also for Windy value 'FALSE', the output class is always 'Yes'





Naive Bayes classifier

A Naive Bayes classifier is a probabilistic machine learning model that's used for classification task. The crux of the classifier is based on the Bayes theorem.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Using Bayes theorem, we can find the probability of A happening, given that B has occurred. Here, B is the evidence and A is the hypothesis.

The assumption made here is that the predictors/features are independent. That is presence of one particular feature does not affect the other. Hence it is called naive.

Where,

P(A|B) is Posterior probability: Probability of hypothesis A on the observed event B.

P(B|A) is Likelihood probability: Probability of the evidence given that the probability of a hypothesis is true.

P(A) is Prior Probability: Probability of hypothesis before observing the evidence.

P(B) is Marginal Probability: Probability of Evidence.

If we consider Bayes Theorem in another representation,

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)}$$

The variable y is the class variable, Variable X represent the parameters/features.

Bayesian equation to calculate the posterior probability for each class. The class with the highest posterior probability is the outcome of the prediction.

Example for applying Bayes theorem on data set with single feature:

here add---

Suppose we have a dataset of weather conditions and corresponding target variable "Play". So using this dataset we need to decide that whether we should play or not on a particular day according to the weather conditions.

Problem: If the weather is sunny, then the Player should play or not?

	Outlook	Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

Frequency table for the Weather Conditions:

Weather	Yes	No
Overcast	5	0
Rainy	2	2
Sunny	3	2
Total	10	5

Probabilities/likelihood table:

Weather	No	Yes	
Overcast	0	5	$5/14 = 0.35$
Rainy	2	2	$4/14 = 0.29$
Sunny	2	3	$5/14 = 0.35$
All	$4/14 = 0.29$	$10/14 = 0.71$	

Applying Bayes'theorem:

$$P(\text{Sunny} | \text{Yes}) = 3/10 = 0.3$$

$$P(\text{Sunny} | \text{NO}) = 2/4 = 0.5$$

$$P(\text{Sunny}) = 0.35$$

$$P(\text{Sunny}) = 0.35$$

$$P(\text{Yes}) = 0.71$$

$$P(\text{No}) = 0.29$$

$$P(\text{Yes} | \text{Sunny}) = 0.3 * 0.71 / 0.35 = 0.60$$

$$P(\text{No} | \text{Sunny}) = 0.5 * 0.29 / 0.35 = 0.41$$

Therefore $P(\text{Yes} | \text{Sunny}) > P(\text{No} | \text{Sunny})$

Hence on a Sunny day, Player will play the game.

Let us now consider data set with multiple features in a feature /attribute set X:

$$X = (x_1, x_2, x_3, \dots, x_n)$$

By substituting for X, we get,

$$P(y|x_1, \dots, x_n) = \frac{P(x_1|y)P(x_2|y)\dots P(x_n|y)P(y)}{P(x_1)P(x_2)\dots P(x_n)}$$

For all entries in the dataset, the denominator does not change, it remain static.

Therefore, the denominator can be removed and a proportionality can be introduced.

$$P(y|x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

We need to find the class **y** with maximum probability.

$$y = \operatorname{argmax}_y P(y) \prod_{i=1}^n P(x_i|y)$$

The posterior probability $P(y|X)$ can be calculated by first, creating a Frequency Table for each attribute against the target. Then, molding the frequency tables to Likelihood Tables and finally, use the Naïve Bayesian equation to calculate the posterior probability for each class. The class with the highest posterior probability is the outcome of the prediction.

Example2:

Example No.	Color	Type	Origin	Stolen?
1	Red	Sports	Domestic	Yes
2	Red	Sports	Domestic	No
3	Red	Sports	Domestic	Yes
4	Yellow	Sports	Domestic	No
5	Yellow	Sports	Imported	Yes
6	Yellow	SUV	Imported	No
7	Yellow	SUV	Imported	Yes
8	Yellow	SUV	Domestic	No
9	Red	SUV	Imported	No
10	Red	Sports	Imported	Yes

Concerning our dataset, the concept of assumptions made by the algorithm can be understood as:

- We assume that no pair of features are dependent. For example, the color being 'Red' has nothing to do with the Type or the Origin of the car. Hence, the features are assumed to be Independent.

Secondly, each feature is given the same influence(or importance). For example, knowing the only Color and Type alone can't predict the outcome perfectly. So none of the attributes are irrelevant and assumed to be contributing Equally to the outcome.

Here in our dataset, we need to classify whether the car is stolen, given the features of the car.

Frequency Table		Stolen?	
	Color	Yes	No
	Red	3	2
	Yellow	2	3

⇒

Likelihood Table		Stolen?	
	Color	P(Yes)	P(No)
	Red	3/5	2/5
	Yellow	2/5	3/5

Frequency and Likelihood tables of 'Color'

Frequency Table		Stolen?	
	Type	Yes	No
	Sports	4	2
	SUV	1	3

⇒

Likelihood Table		Stolen?	
	Type	P(Yes)	P(No)
	Sports	4/5	2/5
	SUV	1/5	3/5

Frequency and Likelihood tables of 'Type'

aa

Frequency Table		Stolen?	
	Origin	Yes	No
	Domestic	2	3
	Imported	3	2

⇒

Likelihood Table		Stolen?	
	Origin	P(Yes)	P(No)
	Domestic	2/5	3/5
	Imported	3/5	2/5

Frequency and Likelihood tables of 'Origin'

Given test record:

Color	Type	Origin	Stolen
Red	SUV	Domestic	?

Applying Naïve Bayes classifier:

$$\begin{aligned}P(\text{Yes} | X) &= P(\text{Red} | \text{Yes}) * P(\text{SUV} | \text{Yes}) * P(\text{Domestic} | \text{Yes}) * P(\text{Yes}) \\&= \frac{3}{5} * \frac{1}{5} * \frac{2}{5} * 1 \\&= 0.048\end{aligned}$$

$$\begin{aligned}P(\text{No} | X) &= P(\text{Red} | \text{No}) * P(\text{SUV} | \text{No}) * P(\text{Domestic} | \text{No}) * P(\text{No}) \\&= \frac{2}{5} * \frac{3}{5} * \frac{3}{5} * 1 \\&= 0.144\end{aligned}$$

Since $0.144 > 0.048$, Which means given the features RED SUV and Domestic, our example gets classified as 'NO' the car is not stolen.

CLASSIFICATION-

Rule Based Classifier

- Technique for classifying records using a collection of "if ... then..." rules

Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hibernates	Class Label
human	warm-blooded	hair	yes	no	no	yes	no	mammal
python	cold-blooded	scales	no	no	no	no	yes	reptile
salmon	cold-blooded	scales	no	yes	no	no	no	fish
whale	warm-blooded	hair	yes	yes	no	no	no	mammal
frog	cold-blooded	none	no	semi	no	yes	yes	amphibian
komodo dragon	cold-blooded	scales	no	no	no	yes	no	reptile
bat	warm-blooded	hair	yes	no	yes	yes	yes	mammal
pigeon	warm-blooded	feathers	no	no	yes	yes	no	bird
cat	warm-blooded	fur	yes	no	no	yes	no	mammal
leopard	cold-blooded	scales	yes	yes	no	no	no	fish
shark								
turtle	cold-blooded	scales	no	semi	no	yes	no	reptile
penguin	warm-blooded	feathers	no	semi	no	yes	no	bird
porcupine	warm-blooded	quills	yes	no	no	yes	yes	mammal
eel	cold-blooded	scales	no	yes	no	no	no	fish
salamander	cold-blooded	none	no	semi	no	yes	yes	amphibian

$r_1:$	$(\text{Gives Birth} = \text{no}) \wedge (\text{Aerial Creature} = \text{yes}) \longrightarrow \text{Birds}$
$r_2:$	$(\text{Gives Birth} = \text{no}) \wedge (\text{Aquatic Creature} = \text{yes}) \longrightarrow \text{Fishes}$
$r_3:$	$(\text{Gives Birth} = \text{yes}) \wedge (\text{Body Temperature} = \text{warm-blooded}) \longrightarrow \text{Mammals}$
$r_4:$	$(\text{Gives Birth} = \text{no}) \wedge (\text{Aerial Creature} = \text{no}) \longrightarrow \text{Reptiles}$
$r_5:$	$(\text{Aquatic Creature} = \text{semi}) \longrightarrow \text{Amphibians}$

- The rules for the model are represented in a disjunctive normal form
- R is known as the rule set
- r_i 's are the classification rules or disjuncts

- Left-hand side of the rule is called the **antecedent** or **precondition**
- It contains a conjunction of attribute tests:

$$Condition_i = (A_1 \text{ op } v_1) \wedge (A_2 \text{ op } v_2) \wedge \dots (A_k \text{ op } v_k)$$

- (A_i, v_i) is an attribute-value pair
 - op is a logical operator chosen from the set $\{=, \neq, <, >, \geq, \leq\}$
 - Each attribute test $(A_i \text{ op } v_i)$ is known as a conjunct
-
- Right-hand side of the rule is called the rule **consequent**, which contains the predicted class y_i

- A rule r covers a record x if the precondition of r matches the attributes of x

Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hibernates
hawk	warm-blooded	feather	no	no	yes	yes	no
grizzly bear	warm-blooded	fur	yes	no	no	yes	yes

- **$r1: (\text{Give Birth} = \text{no}) \wedge (\text{Aerial Creature} = \text{yes}) \rightarrow \text{Birds}$**
- $r1$ covers first record but does not cover the second

- The quality of a classification rule can be evaluated using measures **Coverage** and **Accuracy**
- Given a data set D and a classification rule $r : A \rightarrow y$
- **Coverage:**
 - Fraction of records in D that trigger the rule r (Fraction of records that satisfy the antecedent of a rule)
- **Accuracy**
 - Fraction of records triggered by r whose class labels are equal to y (Fraction of records that satisfy the antecedent that also satisfy the consequent of a rule)

$$\text{Coverage}(r) = \frac{|A|}{|D|}$$

$$\text{Accuracy}(r) = \frac{|A \cap y|}{|A|},$$

- $|A|$ is the number of records that satisfy the rule antecedent
- $|A \cap y|$ the number of records that satisfy both the antecedent and consequent
- Example:

$(\text{Gives Birth} = \text{yes}) \wedge (\text{Body Temperature} = \text{warm-blooded}) \longrightarrow \text{Mammals}$

- Coverage of 33% since five of the fifteen records support the rule antecedent
- Accuracy is 100% because all five vertebrates covered by the rule are mammals

How a Rule-Based Classifier Works

Rule based classifier classifies a test record based on the rule triggered by the record

R1: (Give Birth = no) $\wedge$ (Can Fly = yes) $\rightarrow$ Birds

R2: (Give Birth = no) $\wedge$ (Live in Water = yes) $\rightarrow$ Fishes

R3: (Give Birth = yes) $\wedge$ (Blood Type = warm) $\rightarrow$ Mammals

R4: (Give Birth = no) $\wedge$ (Can Fly = no) $\rightarrow$ Reptiles

R5: (Live in Water = sometimes) $\rightarrow$ Amphibians

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
lemur	warm	yes	no	no	?
turtle	cold	no	no	sometimes	?
dogfish shark	cold	yes	no	yes	?

A lemur triggers rule R3, so it is classified as a mammal

A turtle triggers both R4 and R5

A dogfish shark triggers none of the rules

Characteristics of Rule Sets

Mutually Exclusive Rules

- Rules in a rule set R are mutually exclusive if no two rules in R are triggered by the same record
- Property ensures that every record is covered by at most one rule in R

Exhaustive Rules

- A rule set R has exhaustive coverage if there is a rule for each combination of attribute values
- This property ensures that every record is covered by at least one rule in R

If the Rule set is not exhaustive

A record may not trigger any rules

Solution?

default rule $r_d : () \longrightarrow y_d$ must be added to cover the remaining cases

- A default rule has an empty antecedent and is triggered when all other rules have failed
- y_d is known as the default class and is typically assigned to the majority class of training records not covered by the existing rules

- **If the rule set is not mutually exclusive**
- record can be covered by several rules, some of which may predict conflicting classes
- Solution?
 - Ordered rule set
 - Unordered rule set

Ordered Rules

- Rules in a rule set are ordered in decreasing order of their priority(based on accuracy, coverage etc)
- An ordered rule set is also known as a decision list
- When a test record is presented, it is classified by the highest-ranked rule that covers the record
- This avoids the problem of having conflicting classes predicted by multiple classification rules

Ordered Rule Set

- When a test record is presented to the classifier
 - It is assigned to the class label of the highest ranked rule it has triggered
 - If none of the rules fired, it is assigned to the default class



R1: (Give Birth = no) $\wedge$ (Can Fly = yes) $\rightarrow$ Birds
R2: (Give Birth = no) $\wedge$ (Live in Water = yes) $\rightarrow$ Fishes
R3: (Give Birth = yes) $\wedge$ (Blood Type = warm) $\rightarrow$ Mammals
R4: (Give Birth = no) $\wedge$ (Can Fly = no) $\rightarrow$ Reptiles
R5: (Live in Water = sometimes) $\rightarrow$ Amphibians

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
turtle	cold	no	no	sometimes	?

Unordered Rules

- Approach allows a test record to trigger multiple classification rules
- It then considers the consequent of each rule as a vote for a particular class
- Votes are then tallied to determine the class label of the test record
- The record is usually assigned to the class that receives the highest number of votes

Rule-Ordering Schemes

- Rule ordering can be implemented on a rule-by-rule basis or on a class-by-class basis

Rule-Based Ordering Scheme

- This approach orders the individual rules by some rule quality measure
- This ordering scheme ensures that every test record is classified by the "best" rule covering it
- A potential drawback of this scheme is that lower-ranked rules are much harder to interpret because they assume the negation of the rules preceding them

Class-Based Ordering Scheme

- In this approach, rules that belong to the same class appear together in the rule set R
- The rules are then collectively sorted on the basis of their class information
- The relative ordering among the rules from the same class is not important

Rule-Based Ordering

(Skin Cover=feathers, Aerial Creature=yes)
==> Birds

(Body temperature=warm-blooded,
Gives Birth=yes) ==> Mammals

(Body temperature=warm-blooded,
Gives Birth=no) ==> Birds

(Aquatic Creature=semi)) ==> Amphibians

(Skin Cover=scales, Aquatic Creature=no)
==> Reptiles

(Skin Cover=scales, Aquatic Creature=yes)
==> Fishes

(Skin Cover=none) ==> Amphibians

Class-Based Ordering

(Skin Cover=feathers, Aerial Creature=yes)
==> Birds

(Body temperature=warm-blooded,
Gives Birth=no) ==> Birds

(Body temperature=warm-blooded,
Gives Birth=yes) ==> Mammals

(Aquatic Creature=semi)) ==> Amphibians

(Skin Cover=none) ==> Amphibians

(Skin Cover=scales, Aquatic Creature=no)
==> Reptiles

(Skin Cover=scales, Aquatic Creature=yes)
==> Fishes

How to build a rule based classifier

- To build a rule-based classifier, we need to extract a set of rules that identifies key relationships between the attributes of a data set and the class label
- Methods for extracting classification rules:
 - Direct methods
 - Indirect methods

• Direct methods

- Extract classification rules directly from data
- Partition the attribute space into smaller subspaces so that all the records that belong to a subspace can be classified using a single classification rule

- Sequential covering algorithm is often used to extract rules directly from data
- Algorithm extracts the rules one class at a time for data sets that contain more than two classes
- Criterion for deciding which class should be generated first depends on a number of factors, such as the class prevalence or the cost of misclassifying records from a given class

Sequential covering algorithm

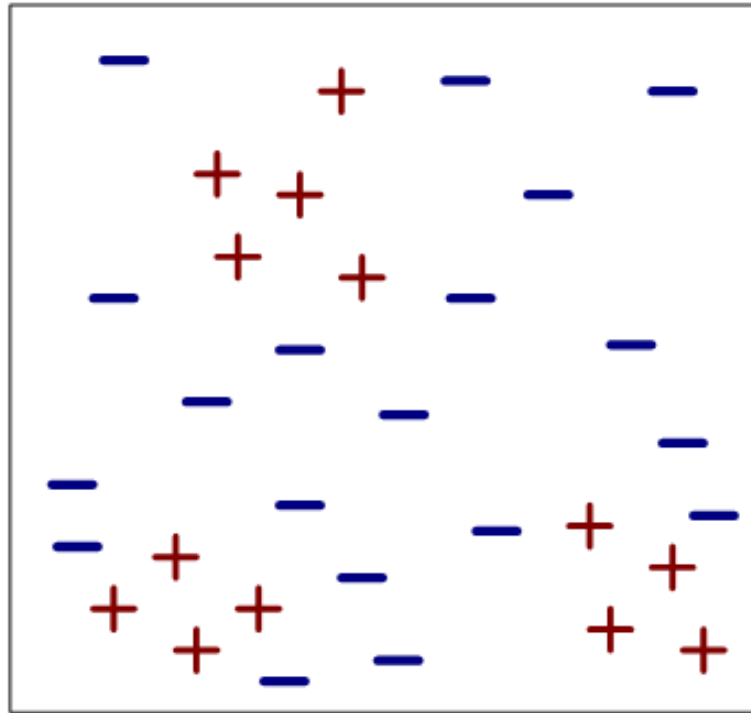
- Algorithm starts with an empty decision list R
- Learn-One-Rule function is then used to extract the best rule for class y that covers the current set of training records
- During rule extraction, all training records for class y are considered to be positive examples, while those that belong to other classes are considered to be negative examples
- A rule is desirable if it covers most of the positive examples and none (or very few) of the negative examples

- Once such a rule is found, the training records covered by the rule are eliminated
- The new rule is added to the bottom of the decision list R
- This procedure is repeated until the stopping criterion is met
- The algorithm then proceeds to generate rules for the next class

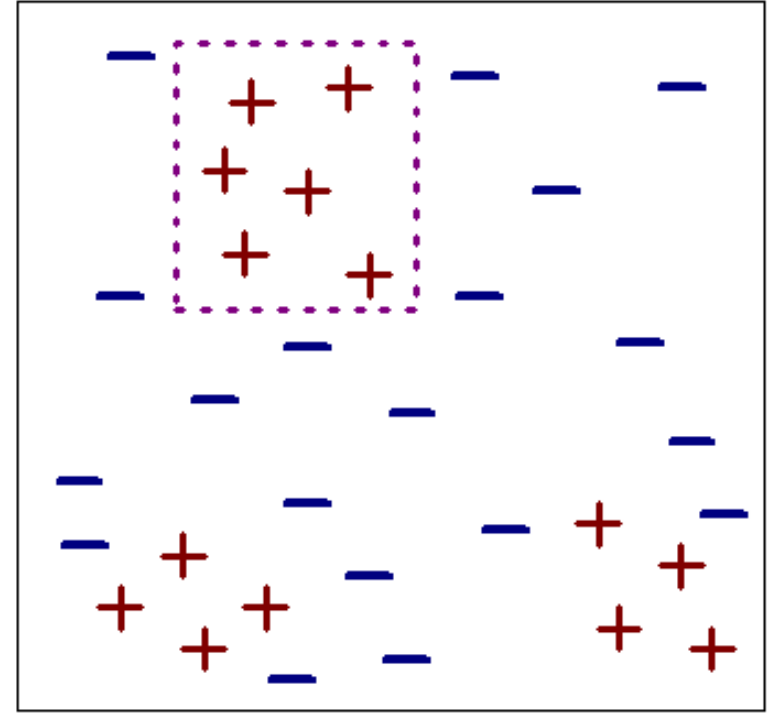
Algorithm 5.1 Sequential covering algorithm.

- 1: Let E be the training records and A be the set of attribute-value pairs, $\{(A_j, v_j)\}$.
 - 2: Let Y_o be an ordered set of classes $\{y_1, y_2, \dots, y_k\}$.
 - 3: Let $R = \{ \}$ be the initial rule list.
 - 4: **for** each class $y \in Y_o - \{y_k\}$ **do**
 - 5: **while** stopping condition is not met **do**
 - 6: $r \leftarrow \text{Learn-One-Rule}(E, A, y)$.
 - 7: Remove training records from E that are covered by r .
 - 8: Add r to the bottom of the rule list: $R \longrightarrow R \vee r$.
 - 9: **end while**
 - 10: **end for**
 - 11: Insert the default rule, $\{ \} \longrightarrow y_k$, to the bottom of the rule list R .
-

Example of Sequential Covering

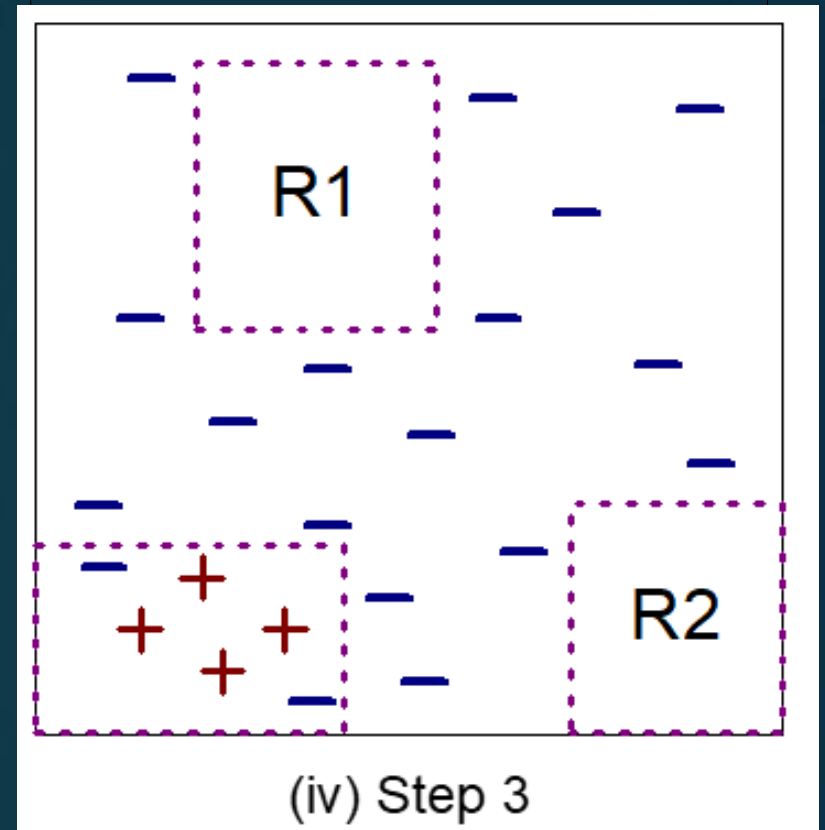
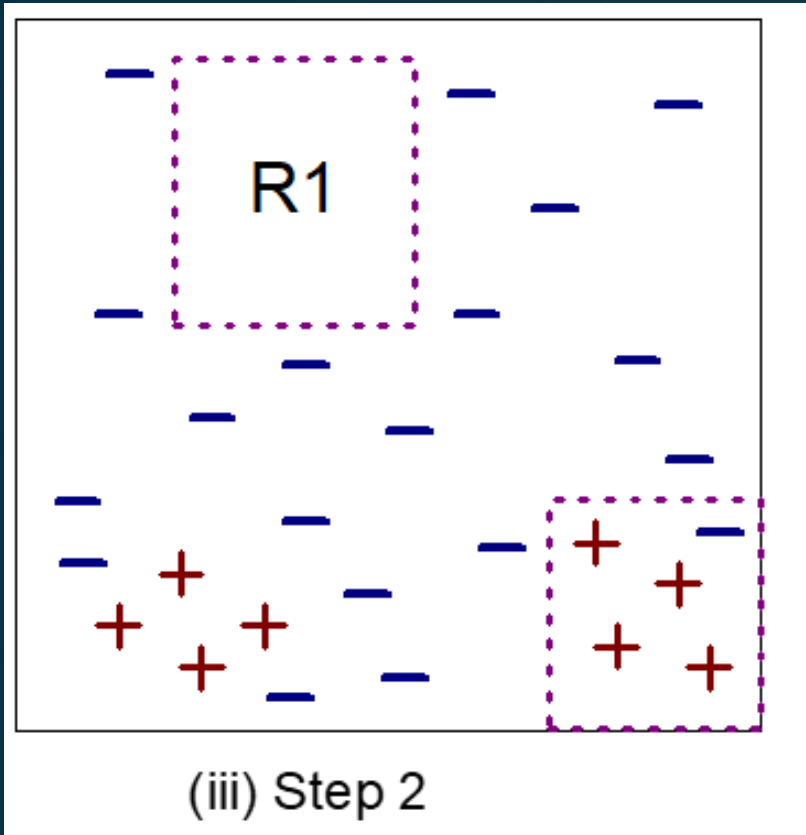


(i) Original Data



(ii) Step 1

Example of Sequential Covering



- Rule R1, whose coverage is extracted first because it covers the largest fraction of positive example
- All the training records covered by R1 are subsequently removed and the algorithm proceeds to look for the next best rule which is R2

Learn-one-Rule function

- Objective is to extract a classification rule that covers many of the positive examples and none (or very few) of the negative examples in the training set
- However, finding an optimal rule is computationally expensive given the exponential size of the search space

Learn-one-Rule function

- Function addresses the exponential search problem by growing the rules in a greedy fashion
- It generates an initial rule r and keeps refining the rule until a certain stopping criterion is met
- The rule is then pruned to improve its generalization error

Rule-Growing Strategy

- Common strategies for growing a classification rule:
 - General-to-specific
 - Specific-to-general

- General-to-specific strategy

- An initial rule $r: \{ \} \rightarrow y$ is created, where the left-hand side is an empty set and the right-hand side contains the target class
- Rule has poor quality because it covers all the examples in the training set
- New conjuncts are subsequently added to improve the rule's quality

- Specific-to-general strategy

- One of the positive examples is randomly chosen as the initial seed for the rule-growing process
- During the refinement step, the rule is generalized by removing one of its conjuncts so that it can cover more positive examples

Characteristics of rule based classifier

- Expressiveness of a rule set is almost equivalent to that of a decision tree
- Generally used to produce descriptive models that are easier to interpret, but gives comparable performance to the decision tree classifier
- The class based ordering approach adopted by many rule based classifiers is well suited for handling data sets with imbalanced class distributions

CLUSTER ANALYSIS

CLUSTER ANALYSIS

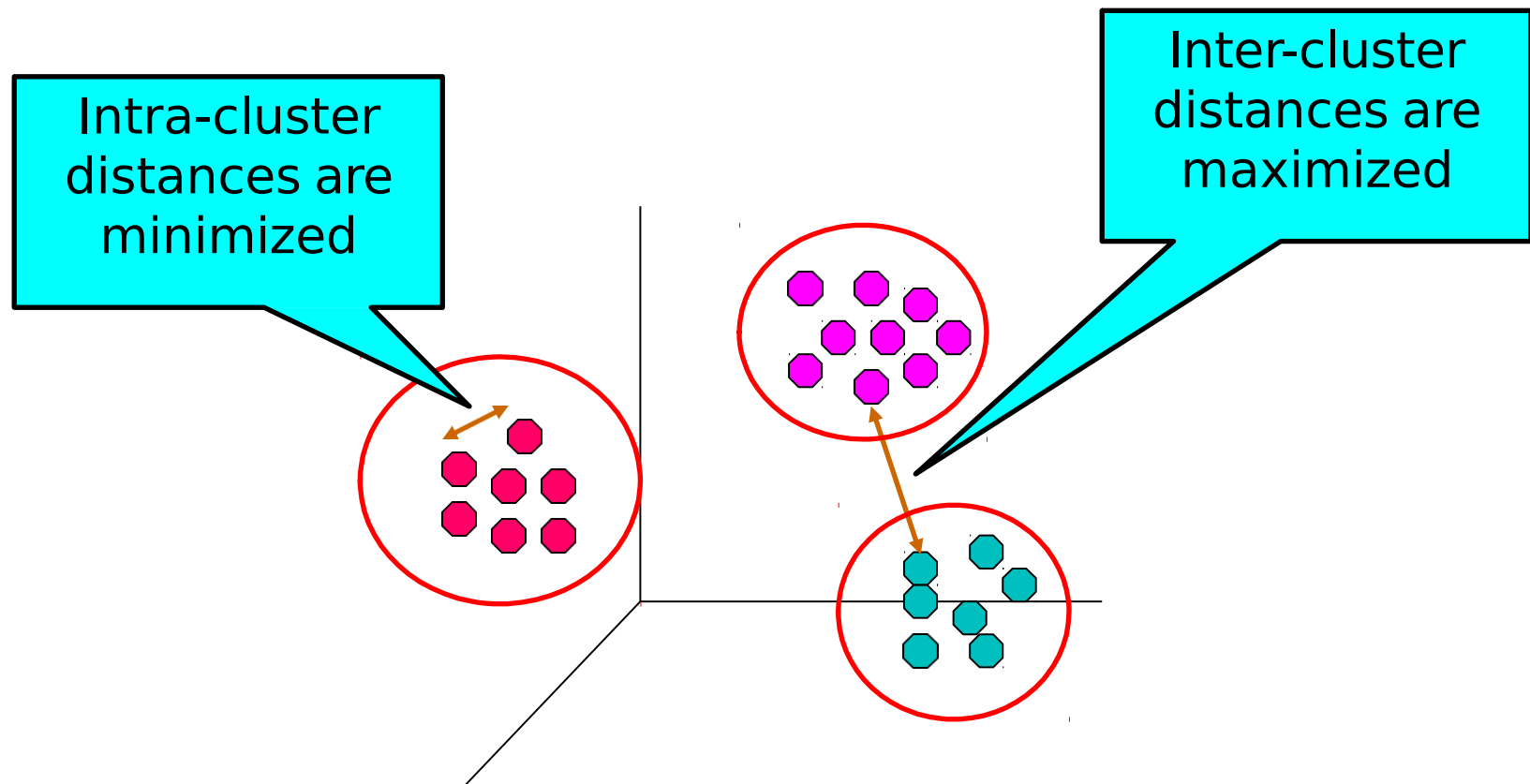
- Divides data into groups (clusters) that are meaningful, useful or both
- Clustering for Understanding
 - Conceptually meaningful groups of objects that share common characteristics
- Clustering for Utility
 - Summarization-Reduce the size of large data sets

What is Cluster Analysis?

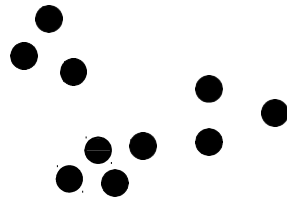
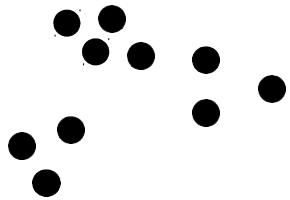
- Groups data objects based only on information found in the data that describes the objects and their relationships
- **Goal** : objects within a group be similar (or related) to one another and different from (or unrelated to) the objects in other groups
- The greater the similarity (or homogeneity) within a group and the greater the difference between groups, the better or more distinct the clustering

What is Cluster Analysis?

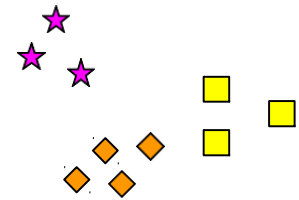
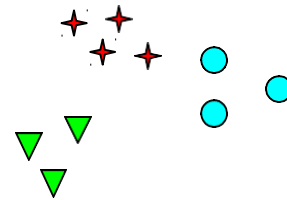
- Finding groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in other groups



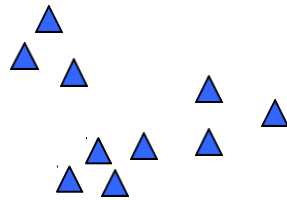
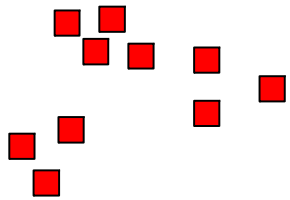
Notion of a Cluster can be Ambiguous



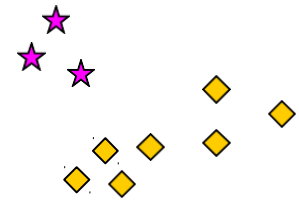
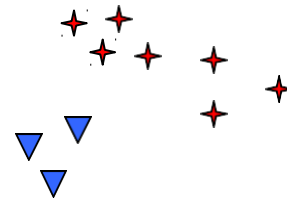
How many clusters?



Six Clusters



Two Clusters

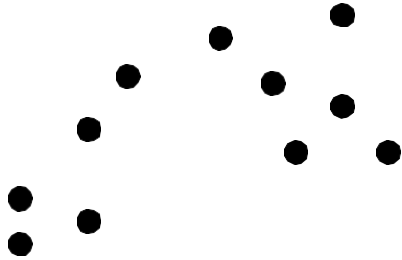


Four Clusters

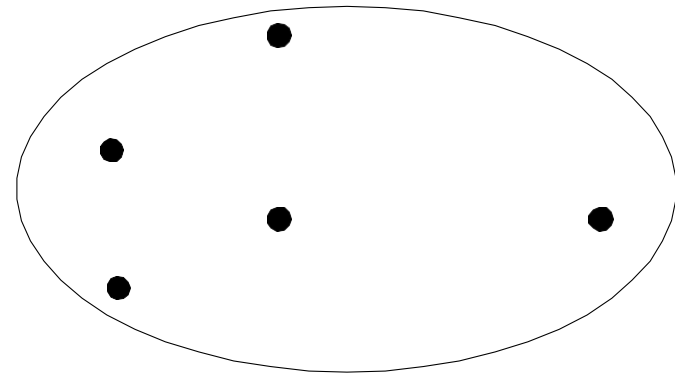
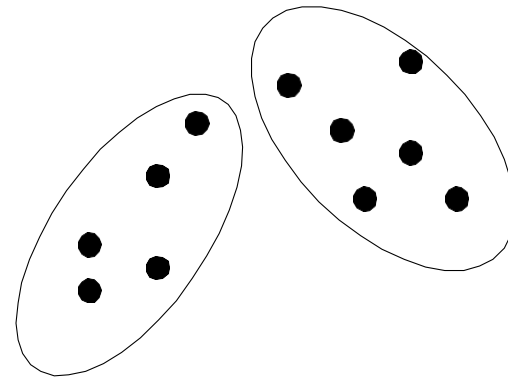
Different Types of Clusterings

- An entire collection of clusters is commonly referred to as a clustering
- **Hierarchical versus Partitional**
- Partitional clustering is simply a division of the set of data objects into non-overlapping subsets (clusters) such that each data object is in exactly one subset
- If we permit clusters to have subclusters, then we obtain a hierarchical clustering- set of nested clusters that are organized as a tree
- Each node (cluster) in the tree (except for the leaf nodes) is the union of its children (subclusters), and the root of the tree is the cluster containing all the objects

Partitional Clustering

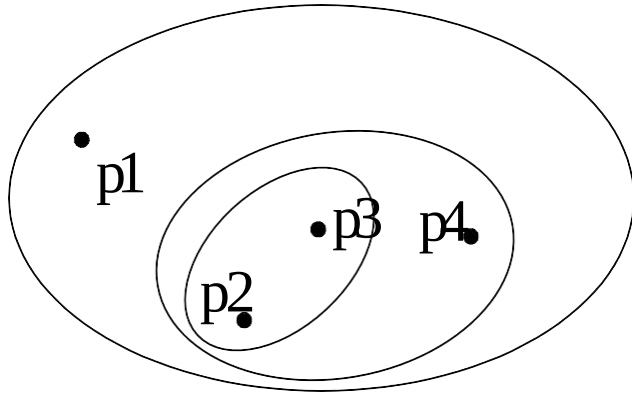


Original Points

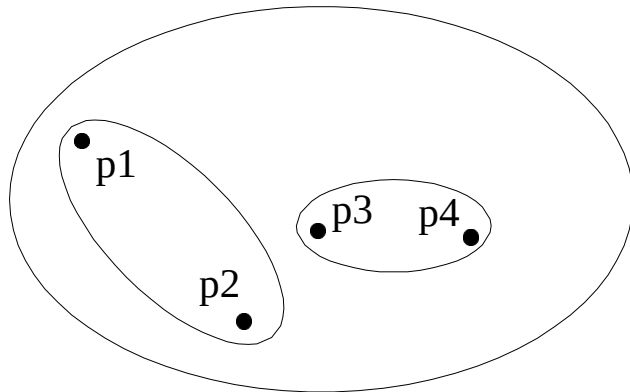


A Partitional Clustering

Hierarchical Clustering



Traditional Hierarchical Clustering



Non-traditional Hierarchical Clustering

Exclusive versus Overlapping versus Fuzzy

- **Exclusive**

- They assign each object to a single cluster

- **Overlapping or non-exclusive clustering**

- An object can simultaneously belong to more than one group (class)
- Also often used when an object is "between" two or more clusters and could reasonably be assigned to any of these cluster

■ Fuzzy clustering

- In a fuzzy clustering, every object belongs to every cluster with a membership weight that is between 0 (absolutely doesn't belong) and 1 (absolutely belongs)
- Sum of the weights for each object must equal 1
- Probabilistic clustering techniques compute the probability with which each point belongs to each cluster, and these probabilities must also sum to 1
- Appropriate for avoiding the arbitrariness of assigning an object to only one cluster when it may be close to several
- Often converted to an exclusive clustering by assigning each object to the cluster in which its membership weight or probability is highest

Complete versus Partial

- A complete clustering assigns every object to a cluster, whereas a partial clustering does not
- Motivation for a partial clustering is that some objects in a data set
 - may not belong to well-defined group
- Many times objects in the data set may represent noise, outliers, or "uninteresting background."
- Example: some newspaper stories may share a common theme, such as global warming, while other stories are more generic or one-of-a-kind. Thus, to find the important topics in last month's stories, we may want to search only for clusters of documents that are tightly related by a common theme

Types of Clusters

- Well-separated clusters
- Prototype-based clusters
- Contiguous clusters
- Density-based clusters
- Property or Conceptual

Types of Clusters: Well-Separated

□ Well-Separated Clusters:

- A cluster is a set of points such that any point in a cluster is closer (or more similar) to every other point in the cluster than to any point not in the cluster.



3 well-separated clusters

Prototype-Based clusters

- A cluster is a set of objects in which each object is closer (more similar) to the prototype that defines the cluster than to the prototype of any other cluster
- For data with continuous attributes, the prototype of a cluster is often a centroid, i.e., the average (mean) of all the points in the cluster
- When a centroid is not meaningful, such as when the data has categorical attributes, the prototype is often a medoid, i.e., the most representative point of a cluster.

Types of Clusters: Center-Based

□ Center-based

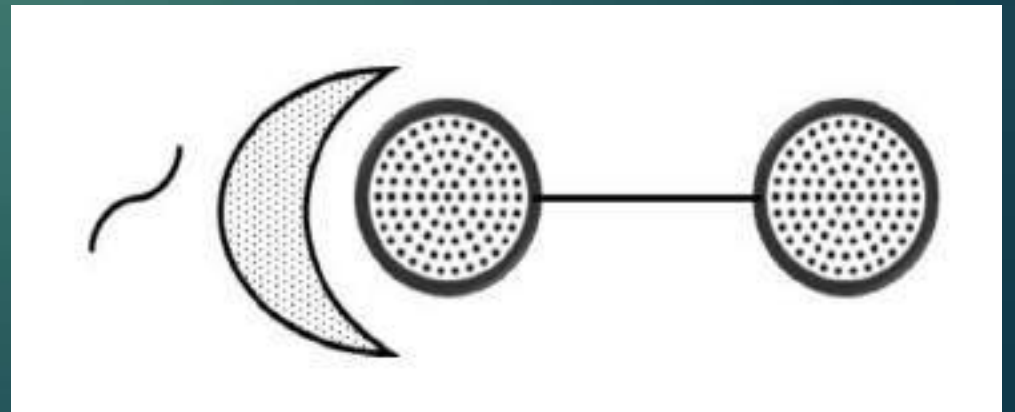
- A cluster is a set of objects such that an object in a cluster is closer (more similar) to the “center” of a cluster, than to the center of any other cluster
- The center of a cluster is often a **centroid**, the average of all the points in the cluster, or a **medoid**, the most “representative” point of a cluster



4 center-based clusters

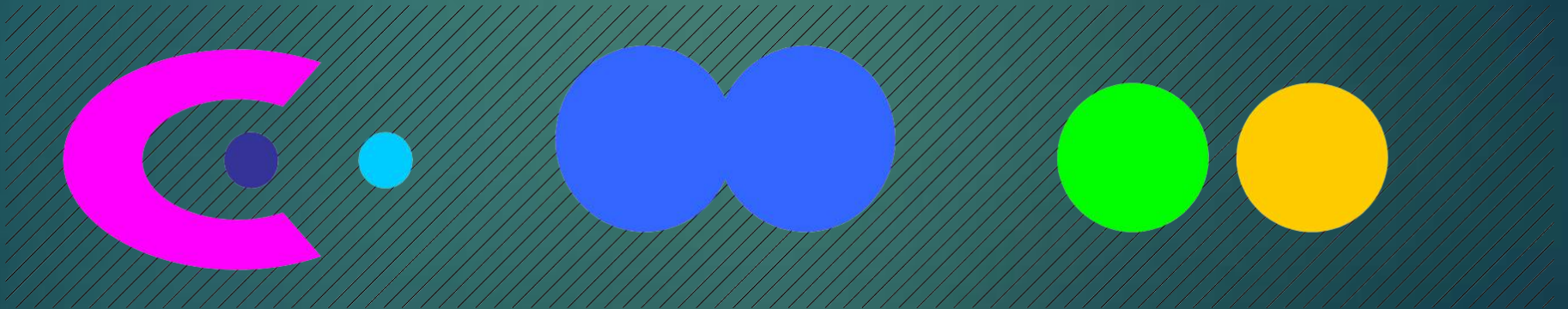
Graph-Based clusters

- If the data is represented as a graph, then a cluster can be defined as a connected component; i.e., a group of objects that are connected to one another, but that have no connection to objects outside the group
- Example: contiguity-based clusters, where two objects are connected only if they are within a specified distance of each other
- This implies that each object in a contiguity-based cluster is closer to some other object in the cluster than to any point in a different cluster



Density-Based clusters

- A cluster is a dense region of objects that is surrounded by a region of low density
- Density-based definition of a cluster is often employed when the clusters are irregular or intertwined, and when noise and outliers are present



Shared-Property (Conceptual) Clusters

- We can define a cluster as a set of objects that share some property

Types of Clusters: Conceptual Clusters

- Finds clusters that share some common property or represent a particular concept.



2 Overlapping Circles

CLUSTER ANALYSIS

Clustering Algorithms & Validation

Clustering Algorithms

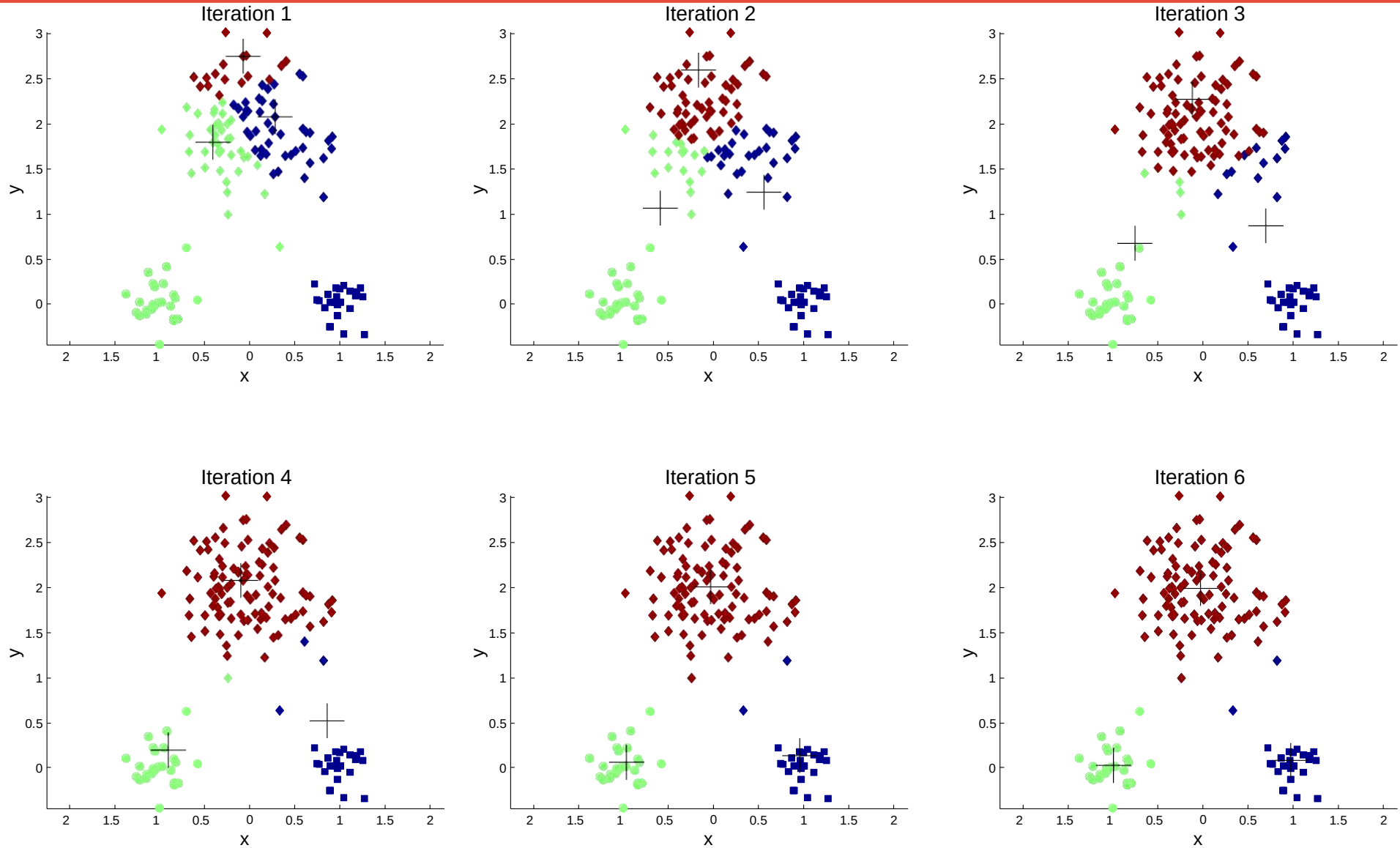
- K-means clustering
- Hierarchical clustering
- Density-based clustering

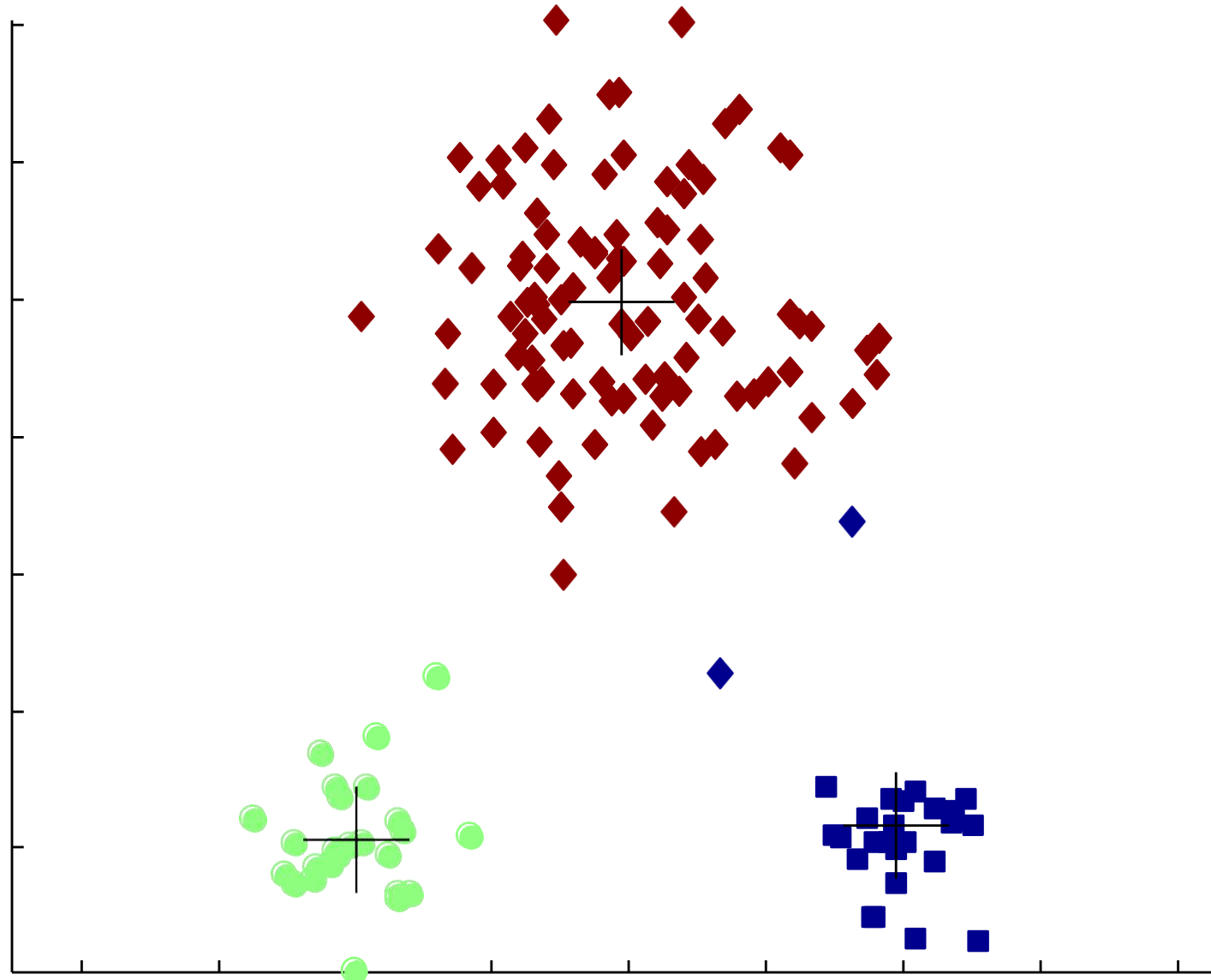
The Basic K-means Algorithm

- Choose K initial centroids, where k is a user-specified parameter, namely, the number of clusters desired
- Each point is then assigned to the closest centroid, and each collection of points assigned to a centroid is a cluster
- Centroid of each cluster is then updated based on the points assigned to the cluster
- Repeat the assignment and update steps until no point changes clusters, or equivalently, until the centroids remain the same

- 
- 1: Select K points as the initial centroids.
 - 2: **repeat**
 - 3: Form K clusters by assigning all points to the closest centroid.
 - 4: Recompute the centroid of each cluster.
 - 5: **until** The centroids don't change

Example of K-means Clustering





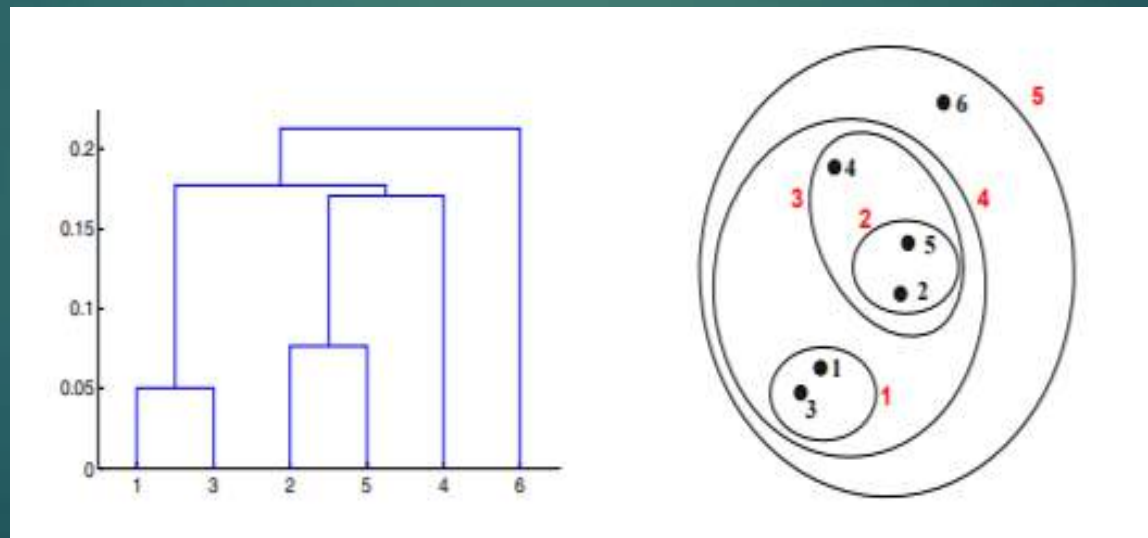
- 
- For some combinations of proximity functions and types of centroids, k means always converges to a solution;
 - K-means reaches a state in which no points are shifting from one cluster to another, and hence, the centroids don't change

Agglomerative Hierarchical Clustering

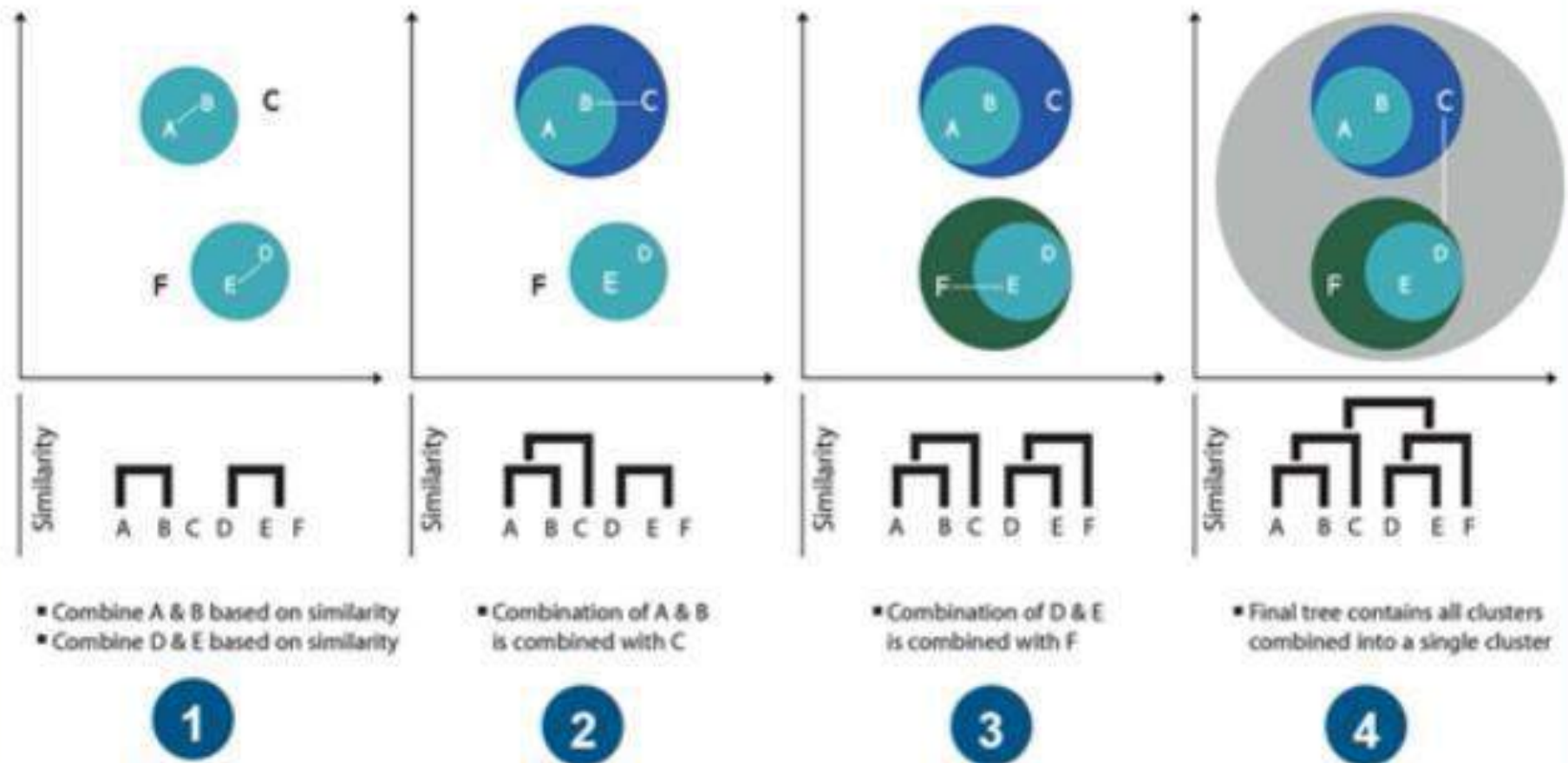
Agglomerative:

- Start with the points as individual clusters
- At each step, merge the closest pair of clusters
- This requires defining a notion of cluster proximity

- A hierarchical clustering is often displayed graphically using a tree-like diagram called a **dendrogram**
- Displays both the cluster-subcluster relationships and the order in which the clusters were merged (agglomerative view)



Hierarchical Clustering



Basic Agglomerative Hierarchical Clustering Algorithm

- Start with individual points as clusters, successively merge the two closest clusters until only one cluster remains

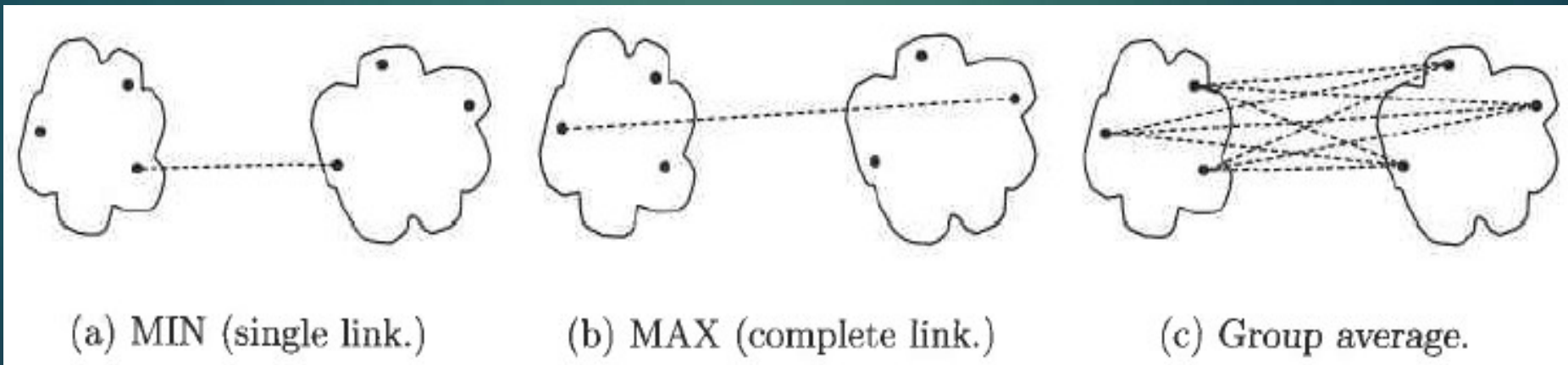
Algorithm 8.3 Basic agglomerative hierarchical clustering algorithm.

- 1: Compute the proximity matrix, if necessary.
- 2: **repeat**
- 3: Merge the closest two clusters.
- 4: Update the proximity matrix to reflect the proximity between the new cluster and the original clusters.
- 5: **until** Only one cluster remains.

Defining Proximity between Clusters

- Definition of cluster proximity differentiates the various agglomerative hierarchical techniques
- **Graph-based definitions of cluster proximity**
- **MIN** defines cluster proximity as the proximity between the closest two points that are in different clusters,
- Graph terms: the shortest edge between two nodes in different subsets of nodes
- **MAX** takes the proximity between the farthest two points in different clusters to be the cluster proximity,
- Graph terms: the longest edge between two nodes in different subsets of nodes

- **Group average** technique defines cluster proximity to be the average pair wise proximities (average length of edges) of all pairs of points from different clusters

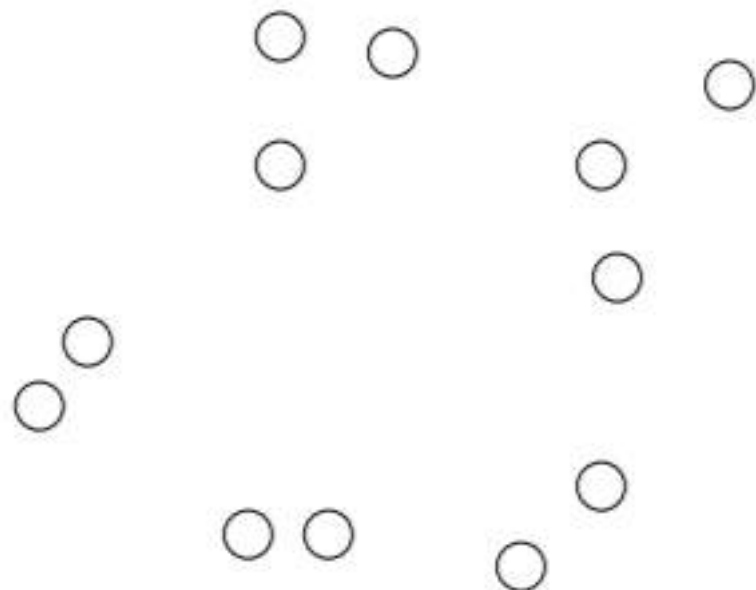


Defining Proximity between Clusters

- **Prototype-based view**
- When using centroids, the cluster proximity is commonly defined as the proximity between cluster centroids

Starting Situation

- Start with clusters of individual points and a proximity matrix



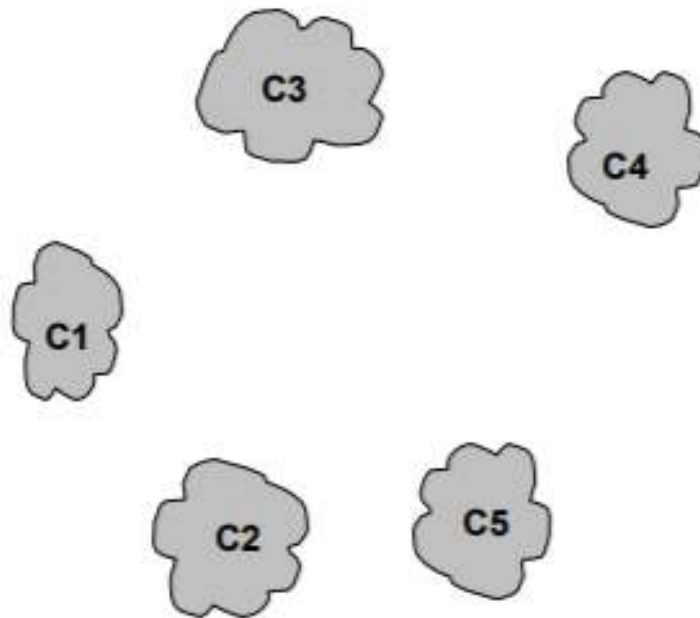
	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix



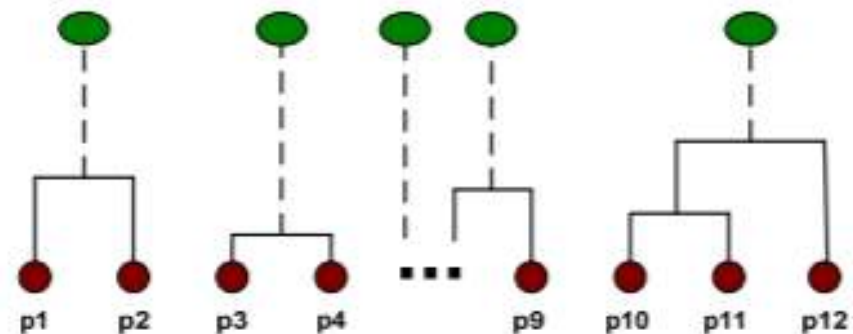
Intermediate Situation

- After some merging steps, we have some clusters



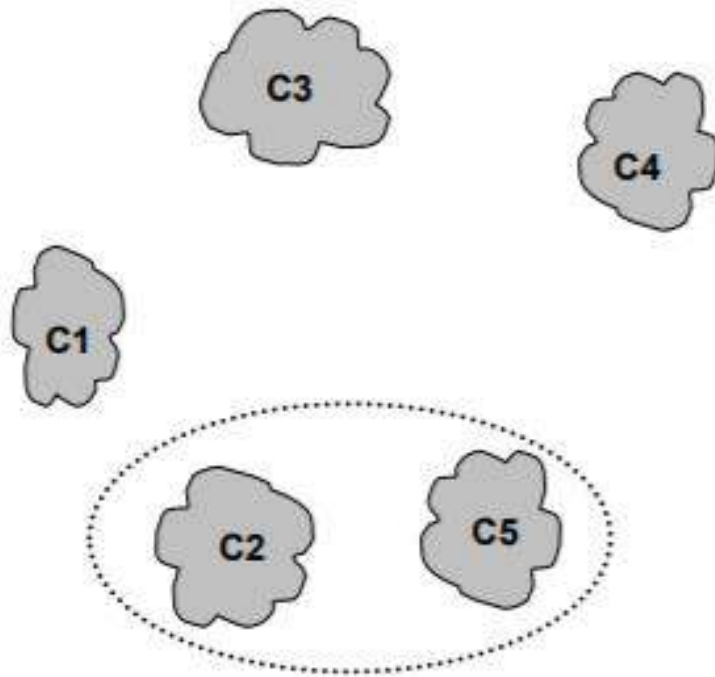
	C1	C2	C3	C4	C5
C1					
C2					
C3					
C4					
C5					

Proximity Matrix



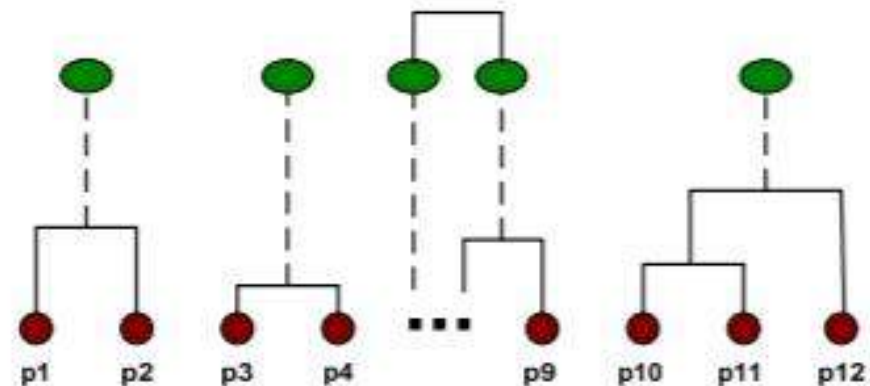
Intermediate Situation

- We want to merge the two closest clusters (C2 and C5) and update the proximity matrix.



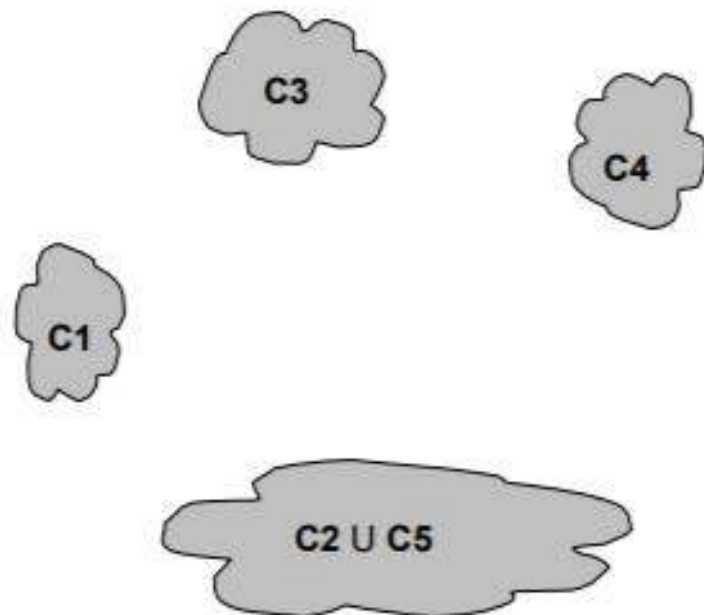
	C1	C2	C3	C4	C5
C1					
C2					
C3					
C4					
C5					

Proximity Matrix



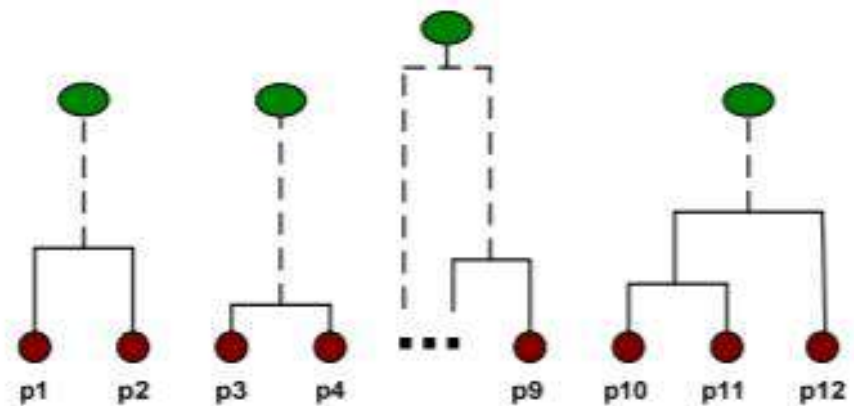
After Merging

- The question is “How do we update the proximity matrix?”

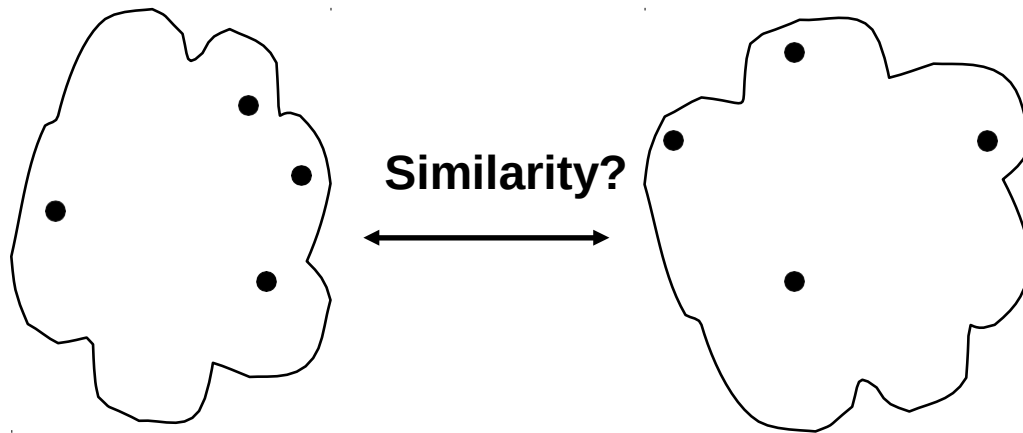


	C1	$C2 \cup C5$	C3	C4
C1		?		
$C2 \cup C5$	?	?	?	?
C3		?		
C4		?		

Proximity Matrix



How to Define Inter-Cluster Distance

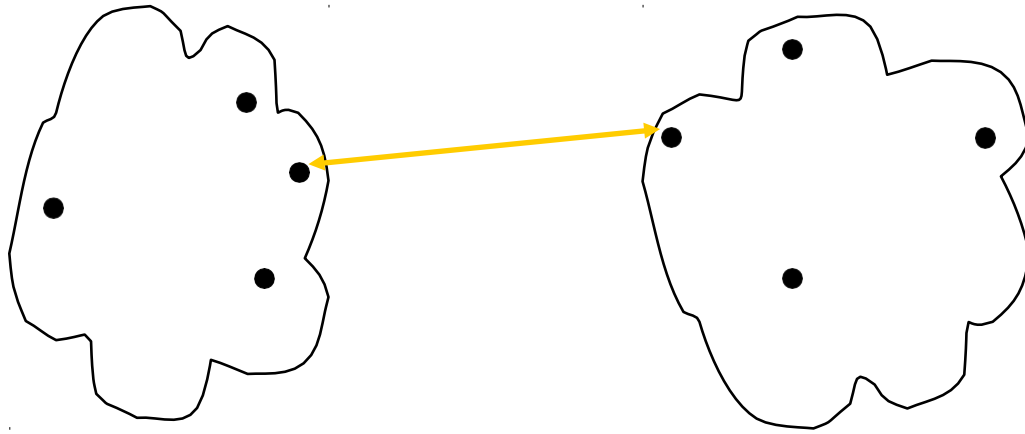


- ☐ MIN
- ☐ MAX
- ☐ Group Average
- ☐ Distance Between Centroids
- ☐ Other methods driven by an objective function
 - Ward's Method uses squared error

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

· **Proximity Matrix**

How to Define Inter-Cluster Similarity

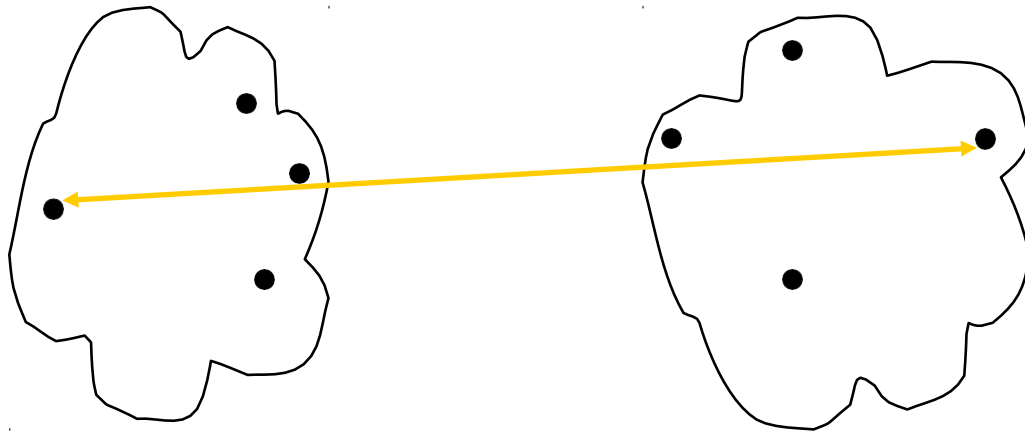


- ☐ MIN
- ☐ MAX
- ☐ Group Average
- ☐ Distance Between Centroids
- ☐ Other methods driven by an objective function
 - Ward's Method uses squared error

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						

· **Proximity Matrix**

How to Define Inter-Cluster Similarity

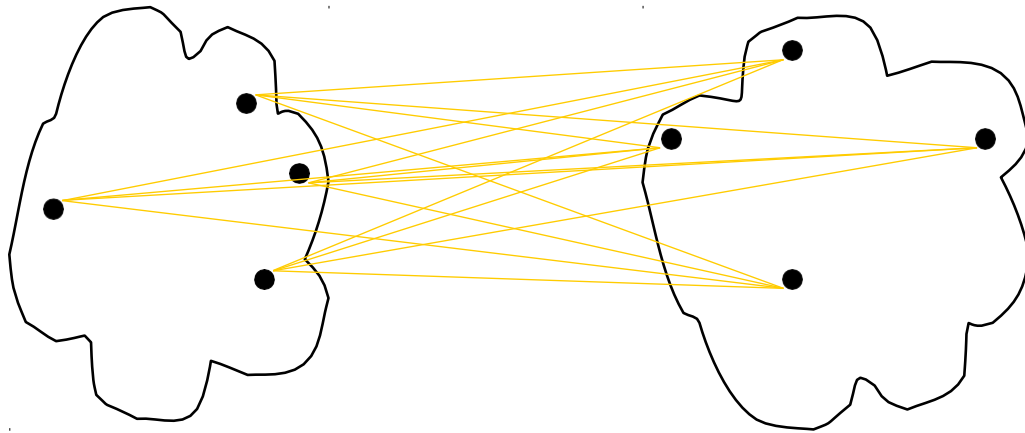


- ☐ MIN
- ☐ MAX
- ☐ Group Average
- ☐ Distance Between Centroids
- ☐ Other methods driven by an objective function
 - Ward's Method uses squared error

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						

· **Proximity Matrix**

How to Define Inter-Cluster Similarity

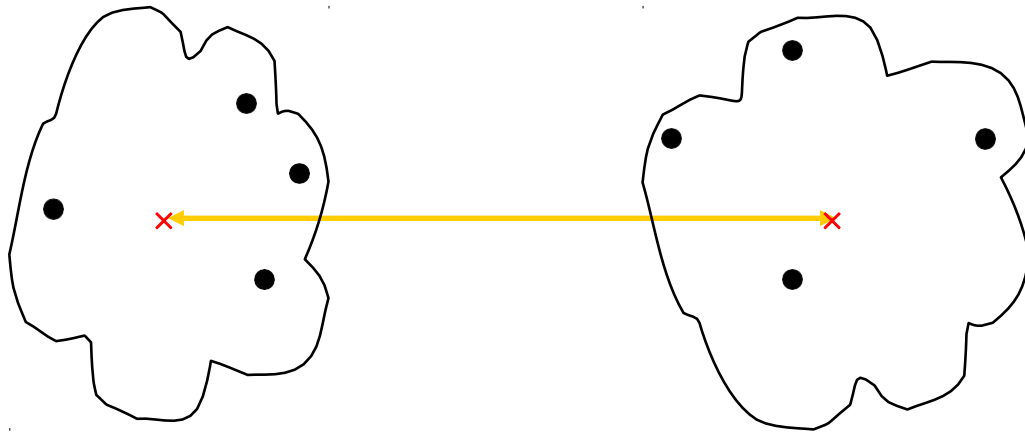


- ☐ MIN
- ☐ MAX
- ☐ **Group Average**
- ☐ Distance Between Centroids
- ☐ Other methods driven by an objective function
 - Ward's Method uses squared error

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						

· **Proximity Matrix**

How to Define Inter-Cluster Similarity



- ☐ MIN
- ☐ MAX
- ☐ Group Average
- ☐ Distance Between Centroids
- ☐ Other methods driven by an objective function
 - Ward's Method uses squared error

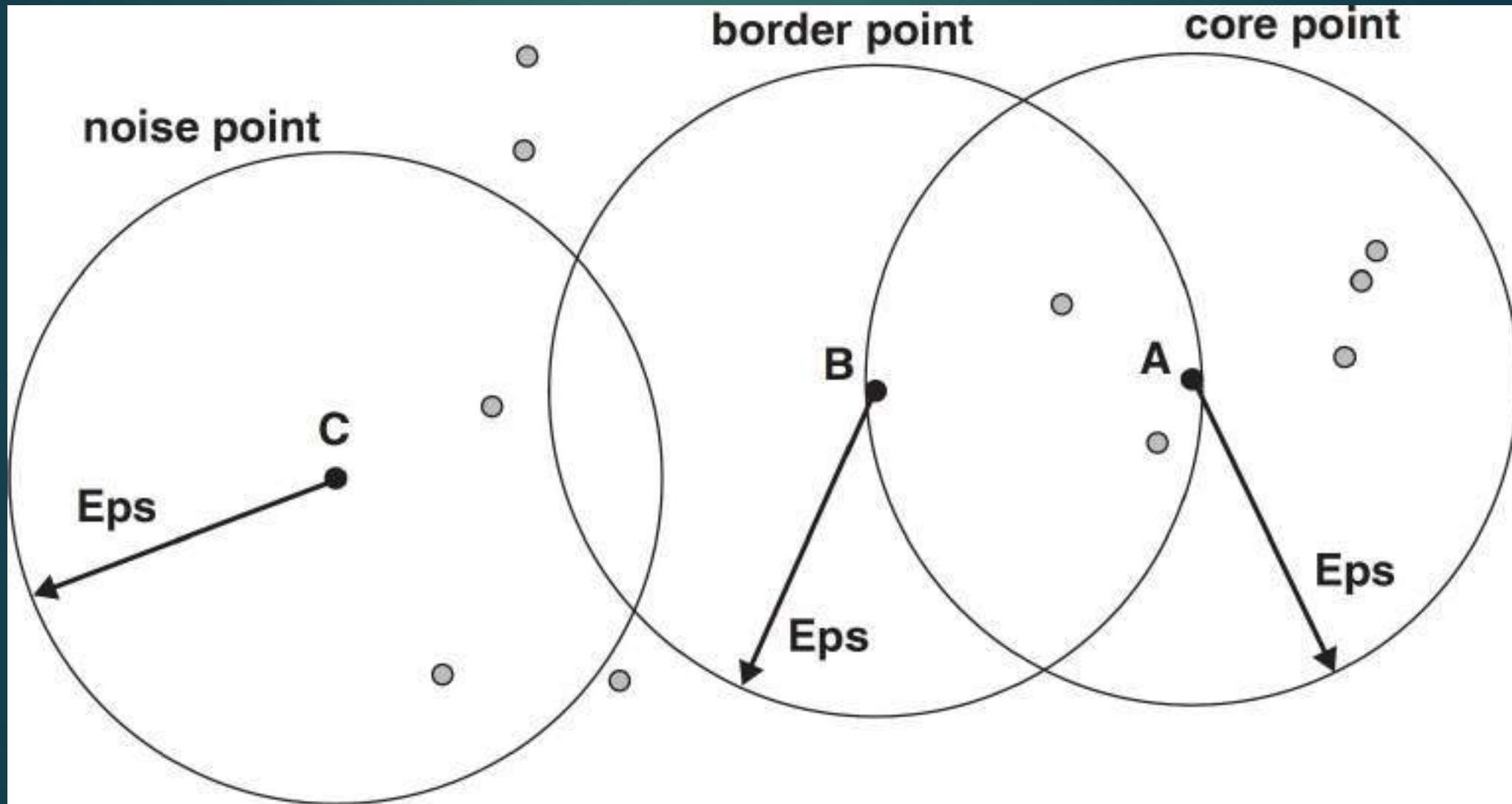
	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

· **Proximity Matrix**

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) Algorithm

- DBSCAN is a density-based algorithm
Density = number of points within a specified radius (Eps)
- A point is a **core point** if it has at least a specified number of points (MinPts) within Eps
- These are points that are at the interior of a cluster
Counts the point itself
- A **border point** is not a core point, but is in the neighborhood of a core point
- A **noise point** is any point that is not a core point or a border point

Core, Border, and Noise Points



DBSCAN Algorithm

- Eliminate noise points
- Perform clustering on the remaining points

Algorithm 7.5 DBSCAN algorithm.

- 1: Label all points as core, border, or noise points.
 - 2: Eliminate noise points.
 - 3: Put an edge between all core points within a distance Eps of each other.
 - 4: Make each group of connected core points into a separate cluster.
 - 5: Assign each border point to one of the clusters of its associated core points.
-

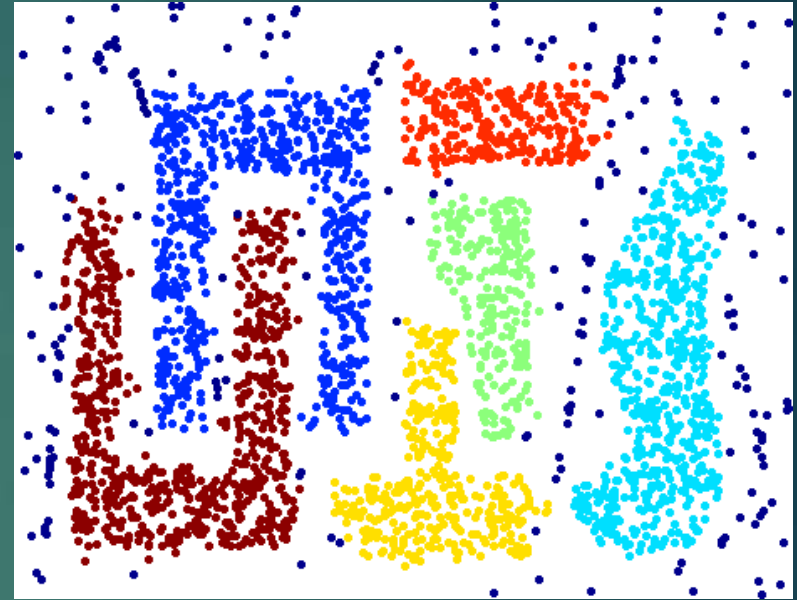
DBSCAN algorithm

1. Label all points as core, border, or noise points
2. Eliminate noise points
3. Put the edge between all core points that are within ϵ of each other
4. Make each group of connected core points into a separate cluster
5. Assign each border point to one of the clusters of its associated core points

When DBSCAN Works Well



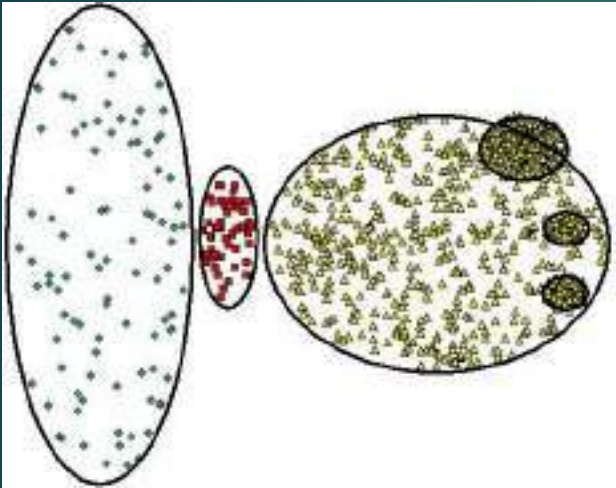
Original Points



Clusters

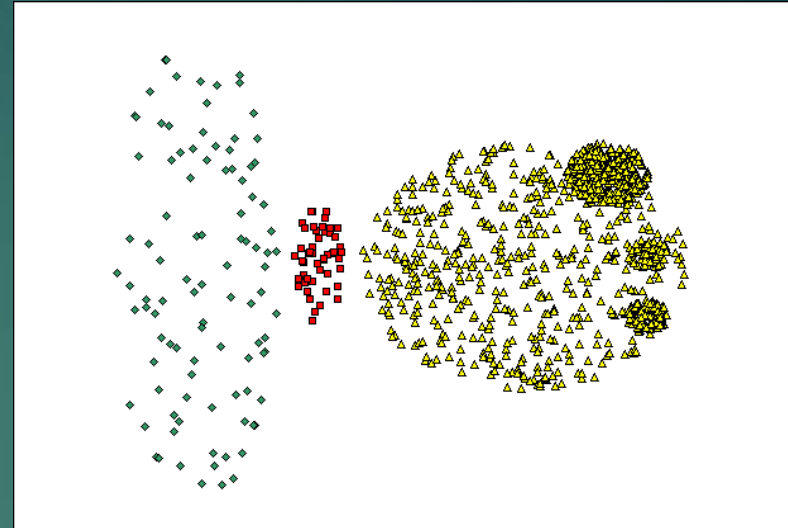
- Resistant to Noise
- Can handle clusters of different shapes and sizes

When DBSCAN Does NOT Work Well

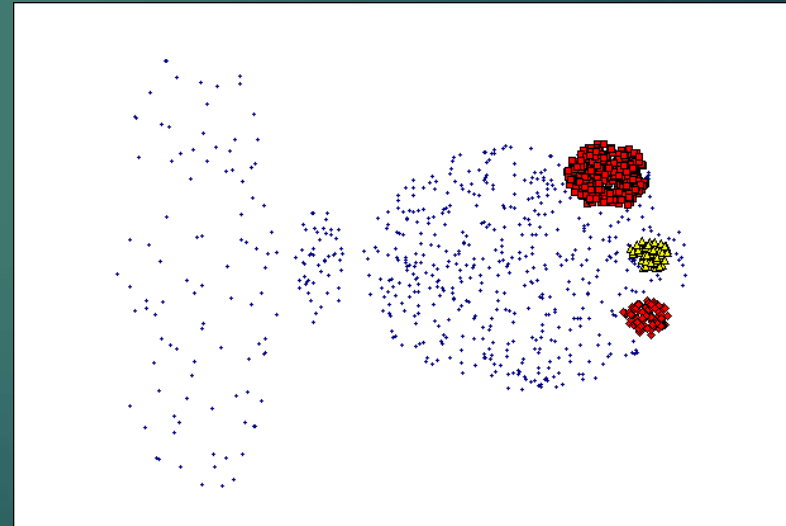


Original Points

- Varying densities
- High-dimensional data



(MinPts=4, Eps=9.75).



(MinPts=4, Eps=9.92)

Cluster Evaluation

Evaluation measures applied to judge various aspects of cluster validity

Unsupervised:

- Measures the goodness of a clustering structure without respect to external information
- e.g.: measures of cluster compactness, isolation etc.



Supervised:

- Measures the extent to which the clustering structure discovered by a clustering algorithm matches some external structure
- External information about data, typically will be in the form of externally derived class labels for the data objects
- Measure the degree of correspondence between the cluster labels and the class labels



Approach 1:

- Use measures from classification, such as entropy, purity, and the F-measure
- These measures evaluate the extent to which a cluster contains objects of a single class

•Approach 2:

- Related to the similarity measures for binary data
- These measure the extent to which two objects that are in the same class are in the same cluster and vice versa

Classification-Oriented Measures of Cluster Validity

Entropy:

- The degree to which each cluster consists of objects of a single class
- For each cluster, the class distribution of the data is calculated first

- For cluster j we compute p_{ij} , the probability that a member of cluster i belongs to class j as $p_{ij} = m_{ij} / m_i$,
 - (m_i is the number of objects in cluster i , and m_{ij} is the number of objects of class j in cluster i)
- Entropy of each cluster i is calculated using formula
$$e_i = - \sum_{j=1}^L p_{ij} \log_2 p_{ij} \quad (L \text{ is the number of classes})$$
- Total entropy for a set of clusters is calculated as the sum of the entropies of each cluster weighted by the size of each cluster
$$e = \sum_{i=1}^K \frac{m_i}{m} e_i$$
- (K is the number of clusters and m is the total number of data points)

Purity:

- Purity of cluster i is
- Overall purity of a clustering :

$$p_i = \max_j p_{ij}$$

$$purity = \sum_{i=1}^K \frac{m_i}{m} p_i$$

Precision:

- The fraction of a cluster that consists of objects of a specified class
- The precision of cluster i with respect to class j is
 $precision(i,j) = p_{ij}$

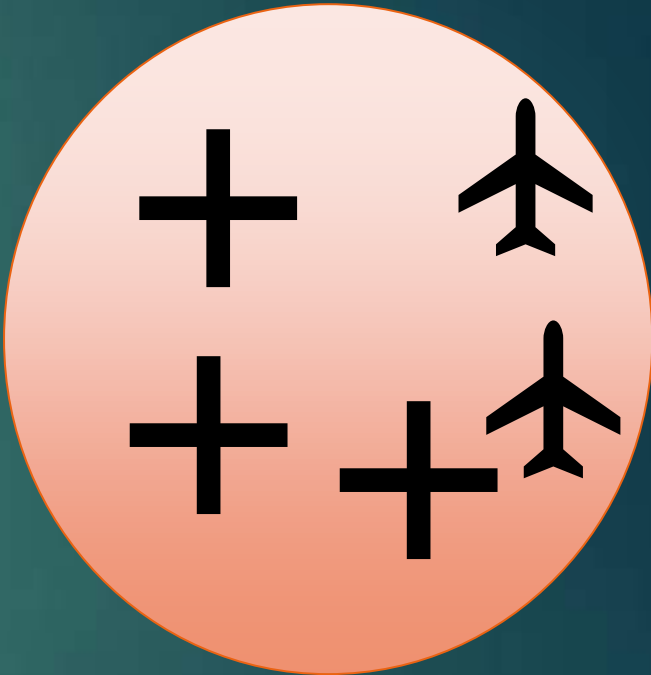
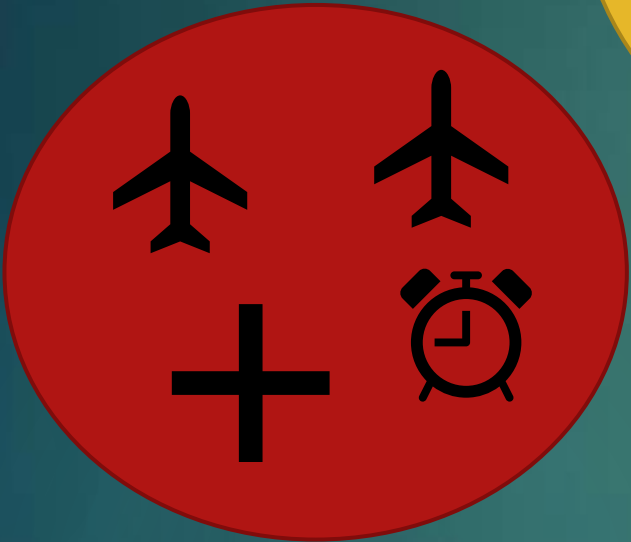
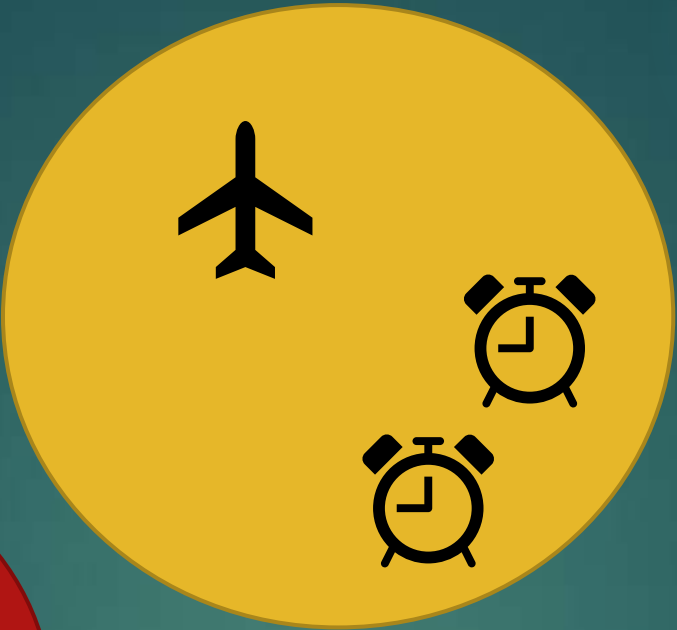
Recall:

- The extent to which a cluster contains all objects of a specified class
- The recall of cluster i with respect to class j is
 $\text{recall}(i,j) = m_{ij} / m_j$, (m_j is the number of objects in class j)

F-measure:

- A combination of both precision and recall that measures the extent to which a cluster contains only objects of a particular class and all objects of that class
- F-measure of cluster i with respect to class j is

$$F(i,j) = (2 \times \text{precision}(i,j) \times \text{recall}(i,j)) / (\text{precision}(i,j) + \text{recall}(i,j))$$



Cluster 1:

Finding P_{ij} :

$$P_{11} = \frac{m_{11}}{m_1}$$

$$P_{11} = \frac{2}{4}$$

$$P_{12} = \frac{m_{12}}{m_1} = \frac{1}{4}$$

$$P_{13} = \frac{m_{13}}{m_1} = \frac{1}{4}$$

Entropy of cluster 1

$$e_1 = -\frac{2}{4} \log\left(\frac{2}{4}\right) - \frac{1}{4} \log\left(\frac{1}{4}\right) - \frac{1}{4} \log\left(\frac{1}{4}\right)$$
$$= 1.5$$

Cluster 2:

$$P_{21} = \frac{m_{21}}{m_2} = \frac{1}{3}$$

$$P_{22} = \frac{m_{22}}{m_2} = \frac{2}{3}$$

$$P_{23} = \frac{m_{23}}{m_2} = \frac{0}{3}$$

Entropy of cluster 2:

$$e_2 = -\frac{1}{3} \log\left(\frac{1}{3}\right) - \frac{2}{3} \log\left(\frac{2}{3}\right) - 0 \log\left(\frac{0}{3}\right)$$
$$= 0.9183$$

Cluster 3:

$$P_{31} = \frac{m_{31}}{m_3} = \frac{2}{5}$$

$$P_{32} = \frac{m_{32}}{m_3} = \frac{0}{5}$$

$$P_{33} = \frac{m_{33}}{m_3} = \frac{3}{5}$$

Entropy of cluster 3:

$$e_3 = -\frac{2}{5} \log\left(\frac{2}{5}\right) - \frac{0}{5} \log\left(\frac{0}{5}\right) - \frac{3}{5} \log\left(\frac{3}{5}\right)$$
$$= 0.9710$$

The total entropy = $\frac{4}{12} \times 1.5 + \frac{3}{12} \times 0.9183 + \frac{5}{12} \times 0.9710$

$$= \underline{\underline{1.1345}}$$

Purity:

$$\text{Purity of cluster 1: } P_1 = \max\left(\frac{2}{4}, \frac{1}{4}, \frac{1}{4}\right) \\ = \frac{2}{4}$$

$$\text{Purity of cluster 2: } P_2 = \frac{2}{3}$$

$$\text{Purity of cluster 3: } P_3 = \frac{3}{5}$$

$$\text{Overall purity of a clustering: } P_{\text{purity}} = \sum_{i=1}^K \frac{m_i}{m} P_i \\ = \frac{4}{12} \times \frac{2}{4} + \frac{3}{12} \times \frac{2}{3} + \frac{5}{12} \times \frac{3}{5} \\ = 0.5833$$

Precision:

$$\text{Precision of cluster } i, \text{ w.r.t class } j \text{ precision } (i, j) = P_{ij} = \frac{m_{ij}}{m_i}$$

$$\text{Precision of cluster 1, w.r.t class 1, } P_{11} = \frac{2}{4}$$

Recall:

Recall of cluster 1 w.r.t class 1

$$\text{Recall } (1, 1) = \frac{m_{11}}{m_1} = \frac{2}{4} ; \text{ Recall } (1, 2) = \frac{m_{12}}{m_2} = \frac{1}{1}, \text{ Recall } (1, 3) = \frac{1}{1}$$



Cluster	Enter- tainment	Financial	Foreign	Metro	National	Sports	Entropy	Purity
1	3	5	40	506	96	27	1.2270	0.7474
2	4	7	280	29	39	2	1.1472	0.7756
3	1	1	1	7	4	671	0.1813	0.9796
4	10	162	3	119	73	2	1.7487	0.4390
5	331	22	5	70	13	23	1.3976	0.7134
6	5	358	12	212	48	13	1.5523	0.5525
Total	354	555	341	943	273	738	1.1450	0.7203

- Consider cluster 1 and the Metro class
- The precision is $506/677=0.75$, recall is $506/943=0.26$,
- F value is 0.39

Similarity-Oriented Measures of Cluster Validity

- Based on the premise that any two objects that are in the same cluster should be in the same class and vice versa

Involves the comparison of two matrices:

Ideal cluster similarity matrix

- has a 1 in the ij th entry if two objects, i and j are in the same cluster and 0 otherwise

Ideal class similarity matrix

- has a 1 in the ij th entry if two objects i and j belong to the same class, and a 0 otherwise

- Consider five data points p_1, p_2, p_3, p_4, p_5 ;
- two clusters, $C_1 = \{p_1, p_2, p_3\}$ and $C_2 = \{p_4, p_5\}$, and
- two classes $L_1 = \{p_1, p_2\}$ and $L_2 = \{p_3, p_4, p_5\}$

Table 8.10. Ideal cluster similarity matrix.

Point	p1	p2	p3	p4	p5
p1	1	1	1	0	0
p2	1	1	1	0	0
p3	1	1	1	0	0
p4	0	0	0	1	1
p5	0	0	0	1	1

Table 8.11. Ideal class similarity matrix.

Point	p1	p2	p3	p4	p5
p1	1	1	0	0	0
p2	1	1	0	0	0
p3	0	0	1	1	1
p4	0	0	1	1	1
p5	0	0	1	1	1

- We need to compute the following quantities for all pairs of distinct objects
- f_{00} =no of pairs of objects having a different class and a different cluster
- f_{01} =no of pairs of objects having a different class and the same cluster
- f_{10} =no of pairs of objects having the same class and a different cluster
- f_{11} =no of pairs of objects having the same class and the same cluster
- Most frequently used similarity-oriented cluster validity measures:

$$\text{Rand statistic} = \frac{f_{00} + f_{11}}{f_{00} + f_{01} + f_{10} + f_{11}}$$

$$\text{Jaccard coefficient} = \frac{f_{11}}{f_{01} + f_{10} + f_{11}}$$

- 
- $f_{00}=4$
 - $f_{01}=2$
 - $f_{10}=2$
 - $f_{11}=2$
 - Rand statistic = $(2 + 4) / 10 = 0.6$
 - Jaccard coefficient = $2 / (2 + 2 + 2) = 0.33$